BarberBook

MVP Specification

Single-Barber Branded Booking Platform

Version 1.0 · March 2026 · MVP Ready

9 Phases · 17,600+ Lines · 50+ Code Files

Document Overview

This document is the complete product, engineering, and technical specification for the BarberBook MVP — a single-barber branded booking micro-platform. It covers all nine phases from product definition through to live deployment and serves as the single source of truth for the entire build.

Phase	Title	Key Deliverables
Phase 1	Product Definition	Scope · Goals · Features
Phase 2	System Architecture	Modules · Routes · Security
Phase 3	Database Design	Schema · RLS · Triggers
Phase 4	Screen Plan	22 Screens · User Actions
Phase 5	API & Backend Plan	Endpoints · Logic · Flows
Phase 6	Implementation Roadmap	Milestones · Risks · Launch
Phase 7	Code Scaffold	50+ Files · Full TypeScript
Phase 8	Test Plan	43 Tests · E2E · Unit · CI
Phase 9	Documentation	Setup · Deploy · ADRs

Tech Stack

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router) + TypeScript	Pages, routing, SSR/SSG
Styling	Tailwind CSS	Utility-first responsive UI
Database	Supabase (PostgreSQL)	All data storage + RLS
Auth	Supabase Auth	Barber + Admin authentication
Storage	Supabase Storage	Barber photos, service images
Payments	Stripe Checkout	Deposit collection
Email	Resend	Confirmations + reminders
Hosting	Vercel	Frontend + API + Cron jobs

Phase 1

Product Definition

Scope · Goals · Features

Phase 1: Product Definition

BarberBook — Single-Barber Branded Booking Micro-Platform

Document Version: 1.0

Date: March 30, 2026

Author: Product Management

Status: Draft — MVP Specification

Table of Contents

1. [Product Summary](#1-product-summary)
2. [Problem Statement](#2-problem-statement)
3. [Target Users](#3-target-users)
4. [Scope Boundaries](#4-scope-boundaries)
5. [Success Criteria](#5-success-criteria)
6. [Feature Prioritization Table](#6-feature-prioritization-table)
7. [Core User Journey Summary](#7-core-user-journey-summary)

---

1. Product Summary

BarberBook is a lightweight, branded booking micro-platform built for the independent barber — a single-operator or solo chair professional who needs a polished, self-owned booking presence without the overhead, marketplace noise, or subscription cost of general-purpose scheduling tools. Each barber receives a dedicated public booking page at `/b/:barberSlug` that they fully control: their name, their services, their availability, their brand. Customers visit that link directly — no account creation, no app download, no marketplace login — and book an appointment in under 60 seconds by entering their name, phone number, and email. Service options are presented as grouped packages (e.g., Hair Only, Hair + Beard, Kids Cut) rather than an overwhelming individual-service catalogue, keeping the booking flow fast and unambiguous. A deposit is collected at time of booking via Stripe to reduce no-shows, and an automated confirmation email plus a pre-appointment reminder are dispatched to the customer. The barber manages their entire operation — availability, service groups, client records, and upcoming bookings — through a clean authenticated dashboard. A platform admin can

provision new barber accounts, configure global settings, and monitor system health. BarberBook's core differentiation is simplicity and ownership: the barber gets their own URL, their own brand, and a friction-free booking experience for their clients — with none of the compromises that come from living inside someone else's marketplace.

---

2. Problem Statement

Independent barbers represent one of the largest segments of solo service professionals, yet the tools available to them are built for either large salons with complex multi-staff workflows or generic appointment businesses that treat every service the same. The result is a persistent mismatch between what barbers need and what existing platforms deliver.

Problem 1 — No-shows are rampant and costly.

Without upfront deposit collection, a significant portion of bookings go unfilled by customers who simply forget or change their minds without cancelling. Independent barbers with no-deposit policies routinely report no-show rates of 15–30% — dead chair time with no compensation and no ability to backfill at short notice. General booking tools either don't support deposits natively or make configuration so complex that barbers skip it entirely.

Problem 2 — Existing platforms extract clients into a marketplace.

Tools like Booksy, StyleSeat, and Vagaro are marketplaces first and booking tools second. When a customer books through them, they are exposed to competing barbers, prompted to create a platform account, and effectively become a platform customer rather than the barber's customer. The barber loses control of their client relationship and effectively pays to build someone else's audience.

Problem 3 — Friction for the customer is too high.

Most booking platforms require the customer to create an account or download an app before they can complete a booking. This friction is particularly damaging for walk-in or referral customers who are acting on an impulse — every additional step results in drop-off. Independent barbers need a booking experience as frictionless as sending a DM, without the chaos of actually managing bookings through DMs.

Problem 4 — Service catalogue complexity doesn't match how barbers actually work.

Enterprise booking tools assume a granular, itemized service catalogue (haircut, beard trim, line-up, shampoo — each as a separate bookable item with individual durations and prices). Independent barbers typically offer a small number of bundled service tiers. Forcing them to model their business as an itemized catalogue creates setup friction and confuses customers at booking time.

Problem 5 — Barbers lack a professional, ownable booking presence.

A barber operating on Instagram or through a generic scheduling link has no professional booking page they can point clients to. The booking URL is generic, unbranded, and shared with thousands of other businesses on the same platform. BarberBook gives each barber a permanent, branded URL (`/b/their-name`) that they can print on cards, share on social, and build a client base around — an asset they actually own.

Problem 6 — Reminder and communication workflows are manual.

Many independent barbers send appointment reminders by hand — texting clients individually the night before. This is time-consuming, easy to forget, and unprofessional at scale. Existing tools that automate reminders are either expensive add-ons or locked behind higher subscription tiers that aren't economically justified for a single operator.

Problem 7 — Pricing and subscription models penalize small operators.

Marketplace platforms charge transaction fees, monthly subscription fees, or both — often structured in ways that are proportionally more burdensome for a solo operator with lower volume than for a busy multi-staff salon. An independent barber doing 80 cuts a month does not need enterprise features; they need a tool sized and priced for their actual operation.

---

3. Target Users

3.1 Customer (Unauthenticated, Public Booking Flow)

Who they are:

The customer is an existing or new client of a specific independent barber. They arrive at the barber's public booking page via a shared link — from the barber's Instagram bio, a Google Maps listing, a business card QR code, or a direct message. They may be a loyal regular or a first-time visitor who received a referral. They span a wide age range and varying levels of digital comfort, which makes friction in the booking flow a critical failure point.

What they need:

- To see the barber's available time slots at a glance
- To select a service group and understand what is included
- To book an appointment in under 60 seconds without creating an account
- To receive immediate confirmation that their booking was accepted
- To receive a timely reminder before their appointment so they don't forget

Technical profile:

No account. No login. No app download. The customer interacts exclusively through three public routes: the barber's profile page (`/b/:barberSlug`), the booking form (`/b/:barberSlug/book`), and the post-booking confirmation screen (`/b/:barberSlug/confirmation`). They provide their name, phone number, and email address as part of the booking form and pay a deposit via Stripe. Their session is ephemeral — no persistent state is stored on the client side beyond the confirmation. All subsequent communication is via email.

Pain points solved:

No marketplace account required. No competing barbers shown alongside their barber. No app to download. A clean, fast, mobile-first booking experience that mirrors the ease of a DM without the chaos.

3.2 Barber (Authenticated, Dashboard User)

Who they are:

The barber is an independent, self-employed hair professional — typically a solo operator working in their own space, renting a chair in a barber shop, or operating a mobile/home studio. They are the sole "product" on their BarberBook instance. They are not a manager of other barbers; they manage only their own schedule, services, and clients. Their technical literacy ranges from moderately comfortable smartphone users to digitally native operators who actively manage their own social media presence.

What they need:

- A clear, real-time view of their upcoming bookings and daily schedule
- The ability to set and modify their working hours and availability
- Control over the service groups and pricing they offer
- Access to a client list with contact history so they can build ongoing relationships
- Confidence that no-show risk is mitigated through deposit requirements
- The ability to personalize their public-facing booking page (name, bio, profile image)
- Timely notifications when a new booking is made or cancelled

Technical profile:

Authenticated via email/password (or magic link) through Supabase Auth. Operates exclusively within the `/barber/*` route namespace:

Route	Purpose
`/barber/login`	Authentication entry point
`/barber/dashboard`	Overview of upcoming bookings, quick stats
`/barber/bookings`	Full booking list with status management
`/barber/clients`	Client directory with booking history
`/barber/services`	Define and edit service groups, pricing, duration
`/barber/settings`	Profile, availability, notification preferences

The barber's account is provisioned by the Admin. They do not self-register. Their slug (the URL identifier for their public page) is assigned at account creation.

Pain points solved:

Owns their client relationship directly. Has a professional booking URL they control. Deposit collection is built in and requires no third-party configuration. Reminders are automated. Dashboard is purpose-built for a solo operator — no multi-staff complexity, no irrelevant features.

3.3 Admin (Authenticated, Platform Operator)

Who they are:

The Admin is the operator or technical owner of the BarberBook platform — in the MVP context, this is likely the founding team member or developer responsible for onboarding barbers onto the platform, managing the system's health, and configuring global settings. In a scaled scenario, the Admin role could be shared across a small operations team. The Admin does not

take bookings and does not interact with the public booking flow.

What they need:

- The ability to create, configure, and manage barber accounts
- Visibility into platform-wide activity: bookings volume, deposit collection, system errors
- Control over global configuration parameters (e.g., platform name, default deposit amount, notification settings)
- Monitoring tools to detect system anomalies or failed transactions

Technical profile:

Authenticated via a separate, secured entry point. Operates within the `/admin/*` route namespace:

Route	Purpose
`/admin/login`	Admin authentication entry point
`/admin/dashboard`	Platform-level overview and key metrics
`/admin/barbers`	Create, view, edit, and deactivate barber accounts
`/admin/config`	Global platform configuration (deposit amounts, branding, email settings)
`/admin/monitoring`	System health, failed jobs, transaction logs

Admin accounts are not self-service — they are provisioned directly in the database or via a secure seed process during platform setup. There is no public admin registration flow.

Pain points solved:

Centralized control over the platform without requiring direct database access for routine operations. Ability to onboard new barbers quickly. Visibility into platform health and financial activity without relying entirely on Stripe and Supabase dashboards.

---

4. Scope Boundaries

The following table defines the explicit boundaries of the MVP. Anything listed under "This IS NOT" is formally out of scope for the initial release and should not be designed, built, or implied in the MVP user experience.

#	This IS (In Scope for MVP)	This IS NOT (Out of Scope for MVP)
1	A single, dedicated public booking page per barber at `/b/:barberSlug`	A multi-barber marketplace or directory where customers browse multiple barbers
2	Grouped service tiers (e.g., Hair Only, Hair + Beard, Kids Cut) with fixed pricing and duration	A granular, individually configurable service catalogue with add-ons and modifiers

#	This IS (In Scope for MVP)	This IS NOT (Out of Scope for MVP)
3	Unauthenticated customer booking via name, phone, and email — no account required	Customer accounts, login, saved booking history, or loyalty profiles
4	Deposit collection at time of booking via Stripe	Full pre-payment or post-service payment collection via BarberBook
5	Automated email confirmation immediately after booking	Native SMS confirmation (SMS infrastructure is abstracted for future release)
6	Automated pre-appointment email reminder	Real-time push notifications or in-app notification center
7	Barber dashboard for managing availability, bookings, service groups, client list, and settings	Multi-staff scheduling, team calendars, or role-based access within a barber account
8	Admin panel for provisioning barber accounts and viewing platform metrics	Self-service barber registration or automated onboarding flow
9	Stripe-based deposit collection with configurable deposit amount	Refund processing UI (refunds handled directly in Stripe dashboard for MVP)
10	Mobile-first responsive web application (Next.js / Tailwind CSS)	Native iOS or Android application
11	Basic client directory (name, phone, email, booking history) within barber dashboard	CRM features: client notes, tags, marketing lists, or campaign tools
12	Barber profile page with name, bio, and profile photo	Portfolio gallery, video content, or social media feed embedding
13	Booking cancellation with configurable policy display	Automated rescheduling, waitlists, or cancellation credit workflows
14	Vercel deployment with Supabase backend	Self-hosted deployment options or white-label infrastructure
15	Single timezone per barber (configured at setup)	Multi-timezone support or automatic customer timezone detection

5. Success Criteria

The following criteria define the conditions under which the MVP is considered successfully launched and performing as intended. Each criterion is measurable and should be assessed

within the first 30–60 days of live operation.

#	Success Criterion	Measurement Method	Target
SC-1	**Booking completion rate** — Customers who land on `/b/:barberSlug/book` complete the booking and reach `/b/:barberSlug/confirmation`	Funnel analytics (page-view to confirmation event ratio)	≥ 70% completion rate
SC-2	**Deposit collection success rate** — Stripe deposit transactions that are initiated and succeed without payment error	Stripe dashboard / webhook event log	≥ 95% of initiated payments succeed
SC-3	**Email delivery rate** — Confirmation and reminder emails successfully delivered to customer inboxes (not bounced or marked spam)	Email provider delivery metrics	≥ 98% delivery rate on confirmation emails
SC-4	**No-show reduction** — Reduction in unfilled appointment slots compared to barber's pre-BarberBook baseline	Barber-reported data or booking cancellation rate vs. completion rate in dashboard	≥ 30% reduction in reported no-shows
SC-5	**Barber onboarding time** — Time from Admin creating a barber account to the barber's public booking page being live and bookable	Measured from account creation timestamp to first live booking received	≤ 30 minutes for a non-technical barber following setup guide
SC-6	**Booking form time-to-complete** — Time a customer takes from landing on the booking page to reaching confirmation	Session duration analytics on the `/book` route	≤ 90 seconds median time-to-complete on mobile
SC-7	**System uptime** — Platform availability during business hours (8 AM – 10 PM local time)	Vercel / uptime monitoring	≥ 99.5% uptime in first 60 days

#	Success Criterion	Measurement Method	Target
SC-8	**Barber dashboard adoption** — Barbers who log into their dashboard at least once per day during a week they have active bookings	Authentication event logs	≥ 80% of active barbers logging in daily on booking days

---

6. Feature Prioritization Table

Priority definitions:

- P0 — Must Have (MVP Blocker): The product cannot launch without this feature. Core to the value proposition.
- P1 — Should Have (MVP): Significantly improves the MVP experience. Build before or immediately at launch.
- P2 — Nice to Have (Post-MVP): Valuable but not required for initial launch. Schedule for a future release.

Feature	User Type	Priority	Notes
Public barber booking page (`/b/:barberSlug`)	Customer	P0	Core entry point; must display barber info, service groups, and available slots
Booking form with name / phone / email capture	Customer	P0	No login required; fields validated before Stripe step
Service group selection (Hair Only, Hair+Beard, Kids Cut, etc.)	Customer	P0	Grouped display — not individual line items
Date and time slot selection (based on barber availability)	Customer	P0	Real-time availability from Supabase; prevents double-booking
Stripe deposit collection at booking	Customer	P0	Must be configurable per barber; failed payment blocks booking confirmation
Booking confirmation page (`/b/:barberSlug/confirmation`)	Customer	P0	Shows booking details, deposit confirmation, next steps
Automated booking confirmation email to customer	Customer	P0	Sent immediately on booking creation; includes appointment details

Feature	User Type	Priority	Notes
Automated pre-appointment reminder email to customer	Customer	P0	Sent configurable hours before appointment (default: 24h)
Barber login (`/barber/login`)	Barber	P0	Supabase Auth; email/password or magic link
Barber dashboard (`/barber/dashboard`)	Barber	P0	Today's bookings, upcoming week summary, quick action links
Barber booking management (`/barber/bookings`)	Barber	P0	List all bookings; filter by date/status; mark as completed or no-show
Barber availability / hours configuration	Barber	P0	Set working days and hours; block-out specific dates
Service group management (`/barber/services`)	Barber	P0	Create, edit, and deactivate service groups with price and duration
Barber profile settings (`/barber/settings`)	Barber	P0	Name, bio, profile photo, slug (read-only post-setup), notification prefs
Admin login (`/admin/login`)	Admin	P0	Separate auth entry; not shared with barber login
Admin barber account management (`/admin/barbers`)	Admin	P0	Create, view, edit, deactivate barber accounts; assign slug
Admin global config (`/admin/config`)	Admin	P0	Default deposit amount, platform email sender, reminder timing
Client directory (`/barber/clients`)	Barber	P1	Auto-populated from bookings; shows name, phone, email, booking count, last visit
Booking cancellation (customer-facing link in email)	Customer	P1	One-click cancel from confirmation email; enforces cancellation policy display
Barber notification on new booking	Barber	P1	Email alert to barber when a new booking is received

Feature	User Type	Priority	Notes
Barber notification on cancellation	Barber	P1	Email alert to barber when a customer cancels
Admin platform monitoring dashboard (`/admin/monitoring`)	Admin	P1	Booking volume, deposit totals, failed webhook/email events
Mobile-optimized booking UI (Tailwind, responsive)	Customer	P1	Majority of customers book on mobile; must be fully functional on small screens
Configurable deposit amount per barber	Barber / Admin	P1	Admin sets platform default; barber can override within allowed range
Booking status management (confirmed / completed / no-show / cancelled)	Barber	P1	Manual status updates from barber dashboard; drives reporting
Basic booking analytics (bookings per week, revenue from deposits)	Barber	P1	Summary stats on barber dashboard; not a full analytics suite
Repeat customer detection (return client flag in client directory)	Barber	P2	Flag clients with more than one booking; useful for relationship management
SMS reminder to customer (via Twilio or equivalent)	Customer	P2	Abstracted for MVP; email reminder is the MVP delivery mechanism
Customer self-rescheduling (via link in confirmation email)	Customer	P2	Post-MVP; requires slot-swap logic and deposit transfer handling
Waitlist / standby booking for full slots	Customer	P2	Post-MVP; requires queue management logic
Barber-side block booking / personal time blocks	Barber	P2	Manual override of availability for non-booking time
Client notes and tagging (CRM features)	Barber	P2	Post-MVP; extends client directory into lightweight CRM
Custom barber page color scheme / brand theming	Barber	P2	Post-MVP; default branded theme ships at MVP

Feature	User Type	Priority	Notes
Portfolio / gallery on public barber page	Barber	P2	Post-MVP; image upload and display for work showcase
Multi-barber account support (chair rental group)	Barber / Admin	P2	Out of MVP scope; single barber per platform instance
Admin-level Stripe revenue reconciliation view	Admin	P2	Post-MVP; use Stripe dashboard directly for MVP
Customer loyalty / rewards system	Customer	P2	Future product feature; not in MVP
Review / rating system for barber	Customer	P2	Future feature; reputational infrastructure is post-MVP
Native iOS / Android app	Customer / Barber	P2	Post-MVP; MVP is web-only, mobile-responsive
Barber self-registration / self-onboarding	Barber	P2	MVP requires Admin provisioning; self-serve onboarding is post-MVP
In-app messaging between barber and customer	Customer / Barber	P2	Future; email is the communication channel for MVP

---

7. Core User Journey Summary

7.1 Customer Journey

A customer learns about BarberBook through their barber — a link in an Instagram bio, a QR code on a business card, or a shared URL in a text message. They tap the link and arrive at the barber's branded public page (`/b/:barberSlug`), where they immediately see the barber's name, photo, and a clear list of available time slots organized by service group. The page loads fast and works entirely within their mobile browser — there is no login prompt, no app store redirect, and no marketplace interface competing for their attention. They select a service group (for example, "Hair + Beard — $35 deposit / 60 min"), pick an available date and time, and are taken to the booking form where they enter their name, phone number, and email. After reviewing their selection, they are prompted to pay the deposit through a Stripe-powered payment screen. On successful payment, they land on the confirmation page (`/b/:barberSlug/confirmation`) which summarizes their appointment and informs them that a confirmation email is on its way. Within minutes, an email arrives with all their appointment details. The morning before their appointment, they receive a reminder email. If they need to cancel, a link in either email takes them to a cancellation flow where the policy is clearly

displayed. From first link tap to confirmation, the entire experience takes under 90 seconds and requires no account, no password, and no download.

7.2 Barber Journey

The barber's relationship with BarberBook begins when the Admin provisions their account and sends them their login credentials and their unique public booking URL. They log in at `/barber/login` and are taken to their dashboard, where the first thing they see is a prompt to complete their setup: add a profile photo, write a short bio, confirm their working hours, and review the pre-configured service groups. Once setup is complete, their public booking page is live and bookable. From that point, the barber's daily use of BarberBook is anchored to the dashboard: they check their upcoming appointments each morning, see which clients are coming in, and manage any same-day changes from the bookings view (`/barber/bookings`). When a new booking is made, they receive an email notification. When the barber wants to adjust their hours — take a day off, extend evening availability, or add a new service group — they do so from the settings and services sections without needing to contact anyone. Over time, the client directory (`/barber/clients`) becomes a growing record of their customer base: names, contact details, visit history, and booking frequency. The barber never has to chase a client for their number or scroll through Instagram DMs to recall a conversation. Their business is organized, their deposits are collected automatically, and their clients have a professional booking experience that reflects well on the barber's brand.

7.3 Admin Journey

The Admin's primary interaction with BarberBook happens at the boundaries: onboarding new barbers and monitoring the platform's health. When a new barber needs to be added, the Admin logs in at `/admin/login` and navigates to the barber management panel (`/admin/barbers`), where they create a new barber account by entering the barber's name, email, and assigning a URL slug. The system sends the barber their login credentials and their public booking link. The Admin then optionally configures any barber-specific settings that differ from platform defaults — such as a custom deposit amount or a different reminder lead time. Day-to-day, the Admin's dashboard (`/admin/dashboard`) provides a live pulse on the platform: how many bookings were made this week, how much deposit revenue has been collected across all barbers, and whether any system events (failed emails, failed Stripe webhooks) require attention. The monitoring panel (`/admin/monitoring`) surfaces errors and failed jobs so the Admin can investigate and resolve issues without needing to dig directly into Supabase logs or Stripe's event stream for routine operational checks. The Admin operates entirely within the `/admin/*` namespace and has no interaction with the customer-facing booking flow — their role is to ensure the platform runs cleanly and that every barber on the system has everything they need to operate.

End of Phase 1: Product Definition

Document metadata

- Platform: BarberBook MVP

- Stack: Next.js · Supabase · Stripe · Vercel · Tailwind CSS
- Routes covered: /b/:barberSlug, /b/:barberSlug/book, /b/:barberSlug/confirmation, /barber/, /admin/
- Next phase: Phase 2 — Data Model & API Specification

Phase 2

System Architecture

Modules · Routes · Security

BarberBook MVP — Phase 2: System Architecture

Document Version: 1.0

Date: March 30, 2026

Author: Solution Architect

Project: BarberBook — Single-Barber Branded Booking Micro-Platform

Status: Draft for Review

Table of Contents

1. [Architecture Overview](#1-architecture-overview)
2. [Module Breakdown](#2-module-breakdown)
3. [Route Structure](#3-route-structure)
4. [Role / Permission Model](#4-role--permission-model)
5. [Integration Points](#5-integration-points)
6. [Security & Privacy Basics](#6-security--privacy-basics)
7. [Data Flow Diagrams](#7-data-flow-diagrams)
8. [Folder Structure Rationale](#8-folder-structure-rationale)

---

1. Architecture Overview

1.1 Summary

BarberBook is deployed as a Next.js monorepo hosted on Vercel, with Supabase serving as the unified backend (PostgreSQL database, authentication, and file storage). The architecture follows a three-layer model:

Layer	Technology	Role
Presentation	Next.js 14 (App Router) + Tailwind CSS	Server-side rendering, client components, routing

Layer	Technology	Role
API	Next.js Route Handlers (Edge + Node.js)	Business logic, data validation, integration orchestration
Backend Services	Supabase (PostgreSQL + Auth + Storage)	Persistence, access control, asset hosting

The system serves three distinct audiences from a single codebase:

- Customers — unauthenticated visitors who book appointments via a barber's public slug URL
- Barbers — authenticated operators who manage their schedule, clients, and services
- Admins — platform super-users who onboard barbers and monitor system health

1.2 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐ │ CLIENT LAYER │
│ │ │ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐ │ │ │ Customer Browser │ │
Barber Browser │ │ Admin Browser │ │ │ │ (unauthenticated)│ │ (JWT session) │ │ (JWT session) │ │
│ └──────────┬───────────┘ └──────────┬───────────┘ └──────────┬───────────┘ │
└────────────┼────────────────────────┼────────────────────────┼──────────────────────┘ │ │ │ │ HTTPS /
TLS (all traffic) │ ▼ ▼ ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐ │ VERCEL EDGE
NETWORK │ │ │ ┌──────────────────────────────────────────────────────────────────────────────────┐ │ │
Next.js Application (App Router) │ │ │ │ │ │ │ ┌──────────────────┐ ┌──────────────────┐
┌──────────────────────────────────┐ │ │ │ │ Public Routes │ Barber Routes │ Admin Routes │ │ │ │ │
/b/:slug │ /barber/* │ /admin/* │ │ │ │ │ /b/:slug/book │ Middleware-gated │
Middleware-gated │ │ │ │ │ /b/:slug/confirm│ (JWT required) │ (Admin role req.) │ │ │ │ │
└──────────┬───────────┘ └──────────┬───────────┘ └──────────┬───────────┘ │ │ │ │ │ │ │ │ │ │ │
└────────────┼────────────────────────┼────────────────────────┼──────────────────────┘ │ │ │ │ ▼ │ │ │ │ ┌──────────────────────┐ │ │
│ │ API Route Layer │ │ │ │ │ /api/public/* │ ← No auth required │ │ │ │ /api/barber/* │ ←
Barber JWT required │ │ │ │ /api/admin/* │ ← Admin JWT required │ │ │ │ /api/webhooks/* │ ←
Stripe signature verified │ │ │ └──────────┬───────────┘ │ │ └──────────────────────┘ │
└──────────────────────────────────────────────────────────────────────────────────────────┘ │ └──────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────────────────────────────┘ │
┌──────────┼──────────────────────────────────┐ │ │ ▼ ▼ ▼ ┌──────────────────┐
┌──────────────────────────────┐ ┌──────────────────┐ │ Supabase Auth │ Supabase PostgreSQL (DB) │ │
Stripe │ │ │ │ │ Payments │ │ - JWT issuance │ │ - bookings │ │ │ - Session mgmt │ - 
barber_profiles │ - Checkout │ - Role claims │ - services │ - Webhooks │ │ │ - 
availability_slots │ - Deposits │ └──────────────────┘ │ - clients │ └──────────────────┘ │ - 
payments │ ┌──────────────────┐ │ - notifications │ ┌──────────────────┐ │ Supabase │ - audit_logs │
│ Resend │ Storage │ │ │ (Email) │ │ │ Row-Level Security on │ │ │ - barber photos│ │
every table │ - Booking │ - service imgs │ │ │ confirm. │ - CDN delivery │
└──────────────────────────────┘ │ - Reminders │ └──────────────────┘ └──────────────────┘
```

1.3 Key Architectural Decisions

Decision	Choice	Rationale
Framework	Next.js 14 App Router	Server components reduce client bundle; built-in API routes eliminate a separate Express server
Hosting	Vercel	Zero-config deploys, edge middleware, preview URLs per branch

Decision	Choice	Rationale
Database	Supabase PostgreSQL	RLS enforces data isolation at the DB layer; managed scaling; real-time capabilities available if needed later
Auth	Supabase Auth	JWT-based, integrates natively with RLS policies; eliminates a separate auth service
Payments	Stripe	Industry standard, PCI-compliant, deposit/hold flows supported
Email	Resend (abstracted)	Simple API, generous free tier, swappable via abstraction layer
CSS	Tailwind CSS	Utility-first, co-located with components, no CSS-in-JS runtime overhead

---

2. Module Breakdown

The backend is logically divided into 9 modules. Each module corresponds to a domain of responsibility and maps to one or more database tables, API route groups, and service files.

2.1 Barber Profile Module

Responsibility: Manages the public-facing identity of each barber — their branded page, bio, photo, and contact details. This is the entry point for the entire customer experience.

Key Functions:

Function	Description
`getBarberBySlug(slug)`	Fetches a barber's public profile by their URL slug. Used on public-facing pages.
`updateBarberProfile(barberId, data)`	Updates bio, display name, phone, social links, and booking settings.
`uploadBarberPhoto(barberId, file)`	Uploads a profile photo to Supabase Storage, returns a CDN URL.
`getBarberSettings(barberId)`	Retrieves private barber settings (deposit %, cancellation policy).
`validateSlugUniqueness(slug)`	Ensures a slug is not already taken before saving.

Dependencies:

- `supabase/storage` — photo upload and CDN delivery
- `supabase/db` — `barber_profiles` table (read/write)
- Admin Setup Module — slug is provisioned by admin at onboarding

Database Table: `barber_profiles`

```
barber_profiles ├── id (uuid, PK) ├── slug (text, unique) ├── display_name (text) ├── bio (text) ├── photo_url (text) ├── phone (text) ├── email (text) ├── timezone (text) ├── deposit_percentage (numeric) ├── cancellation_policy (text) ├── stripe_account_id (text) ├── is_active (boolean) └── created_at (timestamptz)
```

2.2 Service Module

Responsibility: Defines the menu of services a barber offers — haircuts, beard trims, color treatments, etc. Each service has a duration, price, and optional deposit requirement. Services are displayed publicly and drive slot duration calculations in the Availability Engine.

Key Functions:

Function	Description
`listServices(barberId)`	Returns all active services for a given barber, ordered by display order.
`createService(barberId, data)`	Creates a new service entry with name, duration, price, and deposit flag.
`updateService(serviceId, data)`	Edits service details; validates that duration is a multiple of the barber's slot interval.
`deleteService(serviceId)`	Soft-deletes a service (sets `is_active = false`) to preserve historical booking records.
`reorderServices(barberId, orderedIds)`	Updates `display_order` for drag-and-drop reordering on the barber dashboard.

Dependencies:

- `supabase/db` — `services` table
- Availability Engine — service duration determines how many consecutive slots to block
- Booking Engine — selected service is required to create a booking

Database Table: `services`

```
services ├── id (uuid, PK) ├── barber_id (uuid, FK → barber_profiles.id) ├── name (text) ├── description (text) ├── duration_minutes (integer) ├── price (numeric) ├── requires_deposit (boolean) ├── deposit_amount (numeric) ├── display_order (integer) ├── is_active (boolean) └── created_at (timestamptz)
```

2.3 Availability Engine

Responsibility: The core scheduling brain of BarberBook. It translates the barber's working hours and existing bookings into a real-time list of bookable time slots. It handles recurring weekly schedules, date-specific overrides (vacations, special hours), and blocks out time around confirmed bookings.

Key Functions:

Function	Description
`getAvailableSlots(barberId, date, durationMinutes)`	Returns an array of available start times for a given date and service duration. Core algorithm.
`setWeeklySchedule(barberId, schedule)`	Saves the barber's recurring weekly availability (e.g., Mon–Fri 9am–6pm).
`addDateOverride(barberId, date, type, hours?)`	Adds a date-specific override: `CLOSED` (vacation/holiday) or `CUSTOM_HOURS`.
`removeDateOverride(barberId, date)`	Removes a previously set override, reverting that date to the weekly schedule.
`isSlotAvailable(barberId, startAt, durationMinutes)`	Atomic check used by the Booking Engine before confirming a booking. Prevents double-booking.
`blockSlot(barberId, startAt, endAt, reason)`	Manually blocks a time slot (barber-initiated).

Slot Calculation Algorithm:

```
1. Load weekly_schedule for barberId (open/close times for the target day-of-week) 2. Apply any
date_override for the target date (skip if CLOSED; use custom hours if CUSTOM_HOURS) 3. Generate
candidate slots: startTime += slotInterval (15 min) until endTime - serviceDuration 4. For each
candidate slot: a. Query bookings where start_at < slot.end AND end_at > slot.start (overlap
check) b. Query manual_blocks with same overlap condition c. If no overlaps → slot is available
5. Return array of available ISO8601 datetime strings
```

Dependencies:

- supabase/db — weeklyschedules, dateoverrides, bookings, manual_blocks tables
- Booking Engine — calls `isSlotAvailable` as a pre-condition before writing a booking

Database Tables:

```
weekly_schedules ├── id (uuid, PK) ├── barber_id (uuid, FK) ├── day_of_week (integer, 0=Sun …
6=Sat) ├── open_time (time) └── close_time (time) date_overrides ├── id (uuid, PK) ├── barber_id
(uuid, FK) ├── date (date) ├── type (enum: CLOSED | CUSTOM_HOURS) ├── open_time (time, nullable)
└── close_time (time, nullable) manual_blocks ├── id (uuid, PK) ├── barber_id (uuid, FK) ├──
start_at (timestamptz) ├── end_at (timestamptz) └── reason (text)
```

2.4 Booking Engine

Responsibility: Orchestrates the full booking lifecycle — from a customer submitting a booking request through to confirmation, rescheduling, and cancellation. It coordinates with the Availability Engine (slot validation), Payment Engine (deposit collection), CRM Module (client record upsert), and Notification Engine (confirmation messages).

Key Functions:

Function	Description
`createBooking(payload)`	Validates slot availability, creates the booking record, triggers payment if deposit required, and enqueues confirmation notifications.
`confirmBooking(bookingId)`	Called after successful payment; transitions status from `PENDING_PAYMENT` → `CONFIRMED`.
`cancelBooking(bookingId, cancelledBy, reason)`	Cancels a booking; triggers refund logic if deposit was paid.
`rescheduleBooking(bookingId, newStartAt)`	Moves a booking to a new slot after re-validating availability.
`getBookingsForBarber(barberId, filters)`	Returns paginated bookings for the barber dashboard with date, status, and client filters.
`getBookingByToken(token)`	Fetches a single booking by its unique customer-facing token (for confirmation page).

Booking Status State Machine:

```
PENDING_PAYMENT ──(payment success)──► CONFIRMED ──(barber/customer cancel)──► CANCELLED │ │
└──(payment timeout / failure)──► EXPIRED └──(barber marks complete)──► COMPLETED
```

Dependencies:

- Availability Engine — `isSlotAvailable()` called atomically before write
- Payment Engine — initiates Stripe Checkout Session if deposit required
- CRM Module — `upsertClient()` called on every new booking
- Notification Engine — triggers confirmation and reminder jobs
- `supabase/db` — `bookings` table

Database Table: `bookings`

```
bookings ├── id (uuid, PK) ├── barber_id (uuid, FK → barber_profiles.id) ├── service_id (uuid, FK
→ services.id) ├── client_id (uuid, FK → clients.id) ├── start_at (timestamptz) ├── end_at
(timestamptz) ├── status (enum: PENDING_PAYMENT | CONFIRMED | CANCELLED | COMPLETED | EXPIRED)
├── customer_token (text, unique) ← used in confirmation URL ├── notes (text) ├──
cancellation_reason (text) ├── cancelled_by (enum: CUSTOMER | BARBER | SYSTEM) └── created_at
(timestamptz)
```

2.5 Payment Engine (Stripe)

Responsibility: Handles deposit collection at booking time via Stripe Checkout. Manages payment session creation, webhook event processing, and refund initiation on cancellation. BarberBook collects a configurable deposit percentage defined per barber.

Key Functions:

Function	Description
`createCheckoutSession(bookingId, amount, barberId)`	Creates a Stripe Checkout Session for the deposit amount. Returns session URL for redirect.
`handleWebhookEvent(event)`	Processes `checkout.session.completed` and `payment_intent.payment_failed` events from Stripe.
`issueRefund(paymentId, amount?)`	Initiates a full or partial refund via Stripe API on booking cancellation.
`getPaymentRecord(bookingId)`	Fetches the stored payment record for a booking (used in admin reporting).
`verifyWebhookSignature(rawBody, signature)`	Validates that incoming webhooks are genuinely from Stripe using the webhook secret.

Payment Flow:

```
Customer submits booking form │ ▼ Booking Engine creates booking (status: PENDING_PAYMENT) │ ▼
Payment Engine creates Stripe Checkout Session │ ▼ Customer redirected to Stripe-hosted payment
page │ ┌──────┴──────┐ │ │ Success Failure / Abandon │ │ ▼ ▼ Stripe sends Booking stays
PENDING_PAYMENT webhook event → expires after TTL (24h) │ ▼ handleWebhookEvent → confirmBooking()
│ ▼ Notification Engine sends confirmation email
```

Dependencies:

- Booking Engine — called during `createBooking`; calls `confirmBooking` on webhook
- `stripe` Node SDK
- `supabase/db` — `payments` table

Database Table: `payments`

```
payments ├── id (uuid, PK) ├── booking_id (uuid, FK → bookings.id) ├── stripe_session_id (text)
├── stripe_payment_intent_id (text) ├── amount (numeric) ├── currency (text, default 'cad') ├──
status (enum: PENDING | SUCCEEDED | FAILED | REFUNDED) ├── refund_amount (numeric) └── created_at
(timestamptz)
```

2.6 CRM Module

Responsibility: Maintains a lightweight client roster for each barber. Every booking automatically creates or updates a client record based on email address. Barbers can view client history, add private notes, and track visit frequency. No customer-facing login is required.

Key Functions:

Function	Description
`upsertClient(barberId, email, name, phone)`	Finds an existing client by email+barberId or creates a new record. Called on every booking.
`getClients(barberId, filters)`	Returns paginated client list with last-visit date, booking count, and revenue total.

Function	Description
`getClientHistory(clientId)`	Returns all bookings for a client, with service names, dates, and amounts.
`addClientNote(clientId, note)`	Appends a private barber note to the client record.
`updateClientTags(clientId, tags)`	Updates tags (e.g., "VIP", "Regulars") for segmentation.

Dependencies:

- Booking Engine — `upsertClient` is called within `createBooking`
- `supabase/db` — `clients`, `client_notes` tables
- Reporting Module — client data used in revenue and visit reports

Database Tables:

```
clients ├── id (uuid, PK) ├── barber_id (uuid, FK → barber_profiles.id) ├── email (text) ├── full_name (text) ├── phone (text) ├── tags (text[]) ├── total_visits (integer) ├── total_spent (numeric) ├── last_visit_at (timestamptz) └── created_at (timestamptz) client_notes ├── id (uuid, PK) ├── client_id (uuid, FK → clients.id) ├── note (text) └── created_at (timestamptz)
```

2.7 Reminder / Notification Engine

Responsibility: Sends transactional emails at defined trigger points in the booking lifecycle: booking confirmation, 24-hour reminder, and cancellation notice. Designed with an abstraction layer over the email provider so that Resend can be swapped for SendGrid or Postmark without changing business logic.

Key Functions:

Function	Description
`sendBookingConfirmation(booking)`	Sends confirmation email to the customer with booking details and a cancellation link.
`sendBarberNotification(booking)`	Notifies the barber of a new booking via email.
`sendReminder(booking)`	Sends a 24-hour-before reminder to the customer. Scheduled via a cron job.
`sendCancellationNotice(booking, cancelledBy)`	Notifies both parties of a cancellation with the appropriate messaging.
`scheduleReminders()`	Cron job (runs hourly via Vercel Cron) — queries bookings starting in ~24h and dispatches reminders.

Email Templates:

Template	Recipient	Trigger
`booking_confirmation`	Customer	Booking confirmed (post-payment)
`new_booking_alert`	Barber	Any new confirmed booking
`appointment_reminder`	Customer	24h before start_at
`cancellation_customer`	Customer	Booking cancelled (any actor)
`cancellation_barber`	Barber	Customer-initiated cancellation

Dependencies:

- Booking Engine — triggers `sendBookingConfirmation`, `sendCancellationNotice`
- Vercel Cron — schedules `scheduleReminders()`
- `resend` SDK (abstracted behind `EmailProvider` interface)
- `supabase/db` — `notification_logs` table for idempotency tracking

Database Table: `notification_logs`

```
notification_logs ├── id (uuid, PK) ├── booking_id (uuid, FK) ├── type (enum: CONFIRMATION |
REMINDER | CANCELLATION | BARBER_ALERT) ├── recipient_email (text) ├── sent_at (timestamptz) └──
status (enum: SENT | FAILED)
```

2.8 Reporting Module

Responsibility: Provides the barber with business insights — revenue summaries, booking counts, popular services, and client retention metrics. All data is computed from existing tables (no separate analytics database). Reports are read-only and scoped strictly to the authenticated barber's data.

Key Functions:

Function	Description
`getRevenueSummary(barberId, period)`	Returns total revenue and deposit collected for a given period (day/week/month/year).
`getBookingStats(barberId, period)`	Returns counts by status (confirmed, cancelled, completed) for the period.
`getTopServices(barberId, period)`	Returns services ranked by booking count and revenue generated.
`getClientRetentionMetrics(barberId)`	Returns new vs. returning client breakdown.
`getUpcomingBookings(barberId, days)`	Returns bookings for the next N days — used for the dashboard quick-view widget.

Dependencies:

- `supabase/db` — read-only queries on `bookings`, `payments`, `services`, `clients`
- No external services required

2.9 Admin Setup Module

Responsibility: Platform-level administration. Admins create and configure barber accounts, assign slugs, manage platform-wide settings, and monitor system health. This module is never exposed to customers or barbers.

Key Functions:

Function	Description
`createBarberAccount(data)`	Creates a Supabase Auth user for the barber, provisions a `barber_profiles` record with slug.
`deactivateBarber(barberId)`	Sets `is_active = false`, preventing new bookings. Existing bookings are preserved.
`listAllBarbers(filters)`	Returns all barber accounts with booking counts and last-active timestamps.
`setPlatformConfig(key, value)`	Writes to the `platform_config` table — e.g., default deposit %, global maintenance mode.
`getSystemMetrics()`	Returns platform-wide stats: total bookings, revenue, active barbers.
`viewAuditLog(filters)`	Reads the `audit_logs` table — admin actions, auth events, payment events.

Dependencies:

- Supabase Auth Admin SDK — for creating/deleting auth users
- supabase/db — barberprofiles, platformconfig, audit_logs tables

Database Tables:

```
platform_config ├── key (text, PK) ├── value (jsonb) └── updated_at (timestamptz) audit_logs ├── id (uuid, PK) ├── actor_id (uuid) ├── actor_role (enum: ADMIN | BARBER | SYSTEM) ├── action (text) ├── resource_type (text) ├── resource_id (uuid) ├── metadata (jsonb) └── created_at (timestamptz)
```

3. Route Structure

3.1 Page Routes

Route	Access Level	Purpose	Notes
`/b/:barberSlug`	Public (no auth)	Barber's branded public profile page	Displays photo, bio, services list, and "Book Now" CTA. Server-rendered for SEO. Reads from `barber_profiles` + `services`. Returns 404 if slug not found or barber inactive.
`/b/:barberSlug/book`	Public (no auth)	Multi-step booking flow	Step 1: service selection → Step 2: date/time picker → Step 3: customer details form → Step 4: deposit payment redirect. State held in URL params or session storage.
`/b/:barberSlug/confirmation`	Public (no auth)	Booking confirmation page	Accessed via `?token=<customer_token>`. Displays booking summary, add-to-calendar link, and cancellation option.
`/barber/login`	Public (redirects if authed)	Barber authentication	Email + password sign-in via Supabase Auth. Redirects to `/barber/dashboard` on success.
`/barber/dashboard`	Barber (JWT required)	Barber home screen	Today's schedule, upcoming bookings widget, revenue summary cards.
`/barber/bookings`	Barber (JWT required)	Full booking management view	Calendar + list view, status filters, reschedule and cancel actions.
`/barber/clients`	Barber (JWT required)	Client roster and CRM	Searchable client list, individual client detail with booking history and notes.
`/barber/services`	Barber (JWT required)	Service menu management	Add/edit/delete services, drag-to-reorder, toggle active/inactive.
`/barber/settings`	Barber (JWT required)	Profile and account settings	Edit bio, photo, working hours, date overrides, deposit %, Stripe connect, cancellation policy.

Route	Access Level	Purpose	Notes
`/admin/login`	Public (redirects if authed)	Admin authentication	Separate login form; validates `role = 'admin'` claim in JWT. Redirects to `/admin/dashboard`.
`/admin/dashboard`	Admin (JWT + admin role)	Platform overview	Key metrics: active barbers, total bookings, revenue, error rates.
`/admin/barbers`	Admin (JWT + admin role)	Barber account management	Create, view, activate, deactivate barber accounts. Assign slugs.
`/admin/config`	Admin (JWT + admin role)	Platform configuration	Edit `platform_config` entries: default deposit %, maintenance mode, feature flags.
`/admin/monitoring`	Admin (JWT + admin role)	System health and audit log	Recent errors, webhook event log, notification delivery status, audit trail.

3.2 API Routes

Route	Method(s)	Access Level	Purpose	Notes			
`/api/public/barbers/:slug`	`GET`	Public	Fetch barber profile by slug	Returns public fields only; excludes `stripe_account_id`, `email`.			
`/api/public/barbers/:slug/services`	`GET`	Public	List active services for a barber	Returns `id`, `name`, `duration_minutes`, `price`, `deposit_amount`.			

Route	Method(s)	Access Level	Purpose	Notes			
`/api/public/barbers/:slug/availability`	`GET`	Public	Get available slots for a date + service	Query params: `date`, `serviceId`. Returns array of ISO8601 start times. Rate-limited.			
`/api/public/bookings`	`POST`	Public	Create a new booking	Validates availability, upserts client, initiates payment. Returns `bookingId` and Stripe redirect URL.			
`/api/public/bookings/:token`	`GET`	Public	Get booking by customer token	Used on confirmation page. Returns booking details without PII of other clients.			
`/api/public/bookings/:token/cancel`	`POST`	Public	Customer self-cancel a booking	Validates token, triggers cancellation + refund + notification.			
`/api/barber/profile`	`GET`, `PATCH`	Barber JWT	Get/update own barber profile	PATCH allows bio, photo URL, deposit%, cancellation policy.			

Route	Method(s)	Access Level	Purpose	Notes			
`/api/barber/profile/photo`	`POST`	Barber JWT	Upload profile photo	Multipart form; stores to Supabase Storage, returns CDN URL.			
`/api/barber/schedule`	`GET`, `PUT`	Barber JWT	Get/set weekly availability schedule	Full weekly schedule replaced on `PUT`.			
`/api/barber/schedule/overrides`	`GET`, `POST`, `DELETE`	Barber JWT	Manage date-specific overrides	`POST` to add, `DELETE ?date=YYYY-MM-DD` to remove.			
`/api/barber/bookings`	`GET`	Barber JWT	List own bookings with filters	Query: `status`, `from`, `to`, `clientId`, `page`.			
`/api/barber/bookings/:id/cancel`	`POST`	Barber JWT	Barber cancels a booking	Triggers refund if applicable, sends cancellation notifications.			
`/api/barber/bookings/:id/reschedule`	`POST`	Barber JWT	Reschedule a booking	Body: `newStartAt`. Re-validates availability, sends rescheduled notification.			
`/api/barber/bookings/:id/complete`	`POST`	Barber JWT	Mark a booking as completed	Transitions status to `COMPLETED`.			
`/api/barber/services`	`GET`, `POST`	Barber JWT	List/create services				

Route	Method(s)	Access Level	Purpose	Notes			
`/api/barber/services/:id`	`PATCH`, `DELETE`	Barber JWT	Update/soft-delete a service				
`/api/barber/services/reorder`	`POST`	Barber JWT	Update display order	Body: `{ orderedIds: string[] }`.			
`/api/barber/clients`	`GET`	Barber JWT	List own clients	Query: `search`, `tag`, `page`.			
`/api/barber/clients/:id`	`GET`, `PATCH`	Barber JWT	Get/update client record	PATCH allows `tags` only; other fields are system-managed.			
`/api/barber/clients/:id/notes`	`GET`, `POST`	Barber JWT	List/add client notes				
`/api/barber/reports/revenue`	`GET`	Barber JWT	Revenue summary	Query: `period` (`day`\	`week`\	`month`\	`year`).
`/api/barber/reports/bookings`	`GET`	Barber JWT	Booking stats	Query: `period`.			
`/api/barber/stripe/connect`	`GET`	Barber JWT	Initiate Stripe Connect onboarding	Returns Stripe account link URL.			
`/api/admin/barbers`	`GET`, `POST`	Admin JWT	List/create barber accounts	`POST` provisions Auth user + profile row.			
`/api/admin/barbers/:id`	`GET`, `PATCH`, `DELETE`	Admin JWT	Get/update/deactivate a barber	`DELETE` sets `is_active = false` (soft delete).			
`/api/admin/config`	`GET`, `PUT`	Admin JWT	Read/write platform config	Body is `{ key, value }` pairs.			

Route	Method(s)	Access Level	Purpose	Notes			
`/api/admin/metrics`	`GET`	Admin JWT	Platform-wide metrics	Aggregates across all barbers.			
`/api/admin/audit-log`	`GET`	Admin JWT	Read audit logs	Query: `actorId`, `action`, `from`, `to`, `page`.			
`/api/webhooks/stripe`	`POST`	Stripe signature	Stripe webhook receiver	Validates signature with `STRIPE_WEBHOOK_SECRET`. Processes `checkout.session.completed`, `payment_intent.payment_failed`.			

---

4. Role / Permission Model

4.1 Role Definitions

Role	How Assigned	Session Type
`customer`	Implicit — any unauthenticated visitor	No session; identified by booking token for self-service actions
`barber`	Supabase Auth user with `role = 'barber'` in JWT metadata	JWT session via Supabase Auth
`admin`	Supabase Auth user with `role = 'admin'` in JWT metadata	JWT session via Supabase Auth

4.2 Permissions Matrix

Resource	Action	Customer	Barber	Admin
Barber Profile	Read (public fields)	□	□	□
Barber Profile	Read (private fields: email, Stripe ID)	□	□ Own only	□ All
Barber Profile	Update	□	□ Own only	□ All
Barber Profile	Create / Deactivate	□	□	□
Services	Read (active)	□	□	□
Services	Create / Update / Delete	□	□ Own only	□ All
Availability Slots	Read	□	□	□
Weekly Schedule	Read / Update	□	□ Own only	□ All
Date Overrides	Create / Update / Delete	□	□ Own only	□ All
Bookings	Create	□	□	□
Bookings	Read own (by token)	□	□	□
Bookings	Read all (barber's)	□	□ Own clients	□ All
Bookings	Cancel (self)	□ By token	□ Own only	□ All
Bookings	Reschedule	□	□ Own only	□ All
Bookings	Mark complete	□	□ Own only	□ All
Clients	Create (via booking)	□ Implicit	□	□
Clients	Read	□	□ Own only	□ All
Clients	Update (tags, notes)	□	□ Own only	□ All
Clients	Delete	□	□	□
Payments	Initiate	□	□	□
Payments	Read	□	□ Own bookings	□ All
Payments	Refund	□	□	□
Reports	View own reports	□	□ Own data	□ All
Platform Config	Read	□	□	□

Resource	Action	Customer	Barber	Admin
Platform Config	Update	□	□	□
Audit Logs	Read	□	□	□
Notifications	Receive	□ (email)	□ (email)	□

4.3 RLS Policy Summary

All database-level enforcement is implemented via Supabase Row Level Security policies. The JWT `role` claim and `sub` (user ID) are used in policy expressions:

```
-- Example: Barbers can only read their own bookings CREATE POLICY "barber_own_bookings" ON bookings FOR SELECT USING ( auth.role() = 'barber' AND barber_id = auth.uid() ); -- Example: Customers can read a booking only by its token (no auth required) CREATE POLICY "customer_booking_by_token" ON bookings FOR SELECT USING ( customer_token = current_setting('app.booking_token', true) ); -- Example: Admins have unrestricted read access CREATE POLICY "admin_full_read" ON bookings FOR SELECT USING (auth.role() = 'admin');
```

---

5. Integration Points

5.1 Supabase Auth

Role: Authentication and session management for barbers and admins only. Customers do not create accounts.

Integration Method:

- `@supabase/ssr` package used for server-side session handling in Next.js middleware and Server Components
- JWT tokens are issued on login, stored in secure `httpOnly` cookies
- Custom `role` claim (`barber` or `admin`) is set in the Supabase Auth JWT via a `custom_claims` hook (PostgreSQL function triggered on login)
- Next.js middleware (middleware.ts) intercepts all /barber/ and /admin/ routes, verifies the JWT, and redirects unauthenticated requests to the appropriate login page
- Admin routes additionally verify the `role = 'admin'` claim before allowing access

Environment Variables:

```
NEXT_PUBLIC_SUPABASE_URL NEXT_PUBLIC_SUPABASE_ANON_KEY SUPABASE_SERVICE_ROLE_KEY (server-side only, never exposed to client)
```

5.2 Supabase PostgreSQL

Role: Primary data store for all application entities.

Integration Method:

- Server-side: Supabase client initialized with `servicerolekey` for admin operations (bypasses RLS intentionally)
- Barber-facing API routes: Supabase client initialized with the barber's JWT to enforce RLS automatically
- Public API routes: Supabase client initialized with the `anon` key; RLS policies control what anonymous requests can see
- All queries use the Supabase TypeScript client (`@supabase/supabase-js`). Raw SQL is used only for complex reporting aggregations
- Database migrations managed via Supabase CLI (`supabase/migrations/` folder, versioned in git)

5.3 Supabase Storage

Role: Hosts barber profile photos and service images, served via CDN.

Integration Method:

- Two storage buckets: `barber-photos` (public) and `service-images` (public)
- Uploads performed server-side via the barber's authenticated Supabase client to enforce bucket policies
- Public bucket policies allow anonymous `GET` requests (images are non-sensitive)
- Returned CDN URLs are stored in `barberprofiles.photourl` and `services.image_url`
- File size limit enforced at the API layer (max 5MB for photos) before upload

Bucket Policies:

```
barber-photos: - SELECT: public (anon) - INSERT/UPDATE/DELETE: barber JWT matching folder path
/{barber_id}/* service-images: - SELECT: public (anon) - INSERT/UPDATE/DELETE: barber JWT
matching folder path /{barber_id}/*
```

5.4 Stripe

Role: Handles deposit collection from customers at booking time.

Integration Method:

- Server-side: `stripe` Node.js SDK initialized with `STRIPESECRETKEY`
- Flow: API route creates a Stripe Checkout Session → returns `session.url` to the client → client is redirected to Stripe-hosted payment page
- On payment success: Stripe sends a `checkout.session.completed` webhook to `/api/webhooks/stripe`
- Webhook signature verified using `stripe.webhooks.constructEvent(rawBody, signature, STRIPEWEBHOOKSECRET)` before any processing
- Future: Stripe Connect used to route deposits directly to individual barber Stripe accounts (platform fee model)

Environment Variables:

```
STRIPE_SECRET_KEY STRIPE_PUBLISHABLE_KEY STRIPE_WEBHOOK_SECRET
```

5.5 Email Provider (Resend — Abstracted)

Role: Transactional email delivery for booking confirmations, reminders, and cancellation notices.

Integration Method:

- All email-sending logic is routed through a shared `EmailProvider` interface (`lib/email/provider.ts`)
- The concrete implementation (`lib/email/resend.ts`) is injected at runtime via environment config
- This abstraction means swapping to SendGrid or Postmark requires only a new implementation file — no changes to business logic
- Email templates are React components rendered to HTML via `react-email` package
- Cron-based reminders triggered via a Vercel Cron Job hitting `/api/cron/reminders` (secured with a `CRON_SECRET` header)

Environment Variables:

```
EMAIL_PROVIDER=resend RESEND_API_KEY EMAIL_FROM_ADDRESS=bookings@barberbook.com CRON_SECRET
```

5.6 Vercel

Role: Hosting, deployment, and infrastructure for the Next.js application.

Integration Method:

- Serverless Functions: Next.js API routes deployed as Vercel Serverless Functions (Node.js 20 runtime)
- Edge Middleware: middleware.ts runs on Vercel's Edge Runtime for fast JWT verification without cold starts — protects all /barber/ and /admin/ routes
- Vercel Cron Jobs: Configured in `vercel.json` — runs `/api/cron/reminders` every hour
- Environment Variables: All secrets stored in Vercel Project Settings; automatically injected at build and runtime
- Preview Deployments: Each git branch gets a unique preview URL, enabling QA before merging to `main`
- Build: `next build` runs on Vercel's build infrastructure; static pages pre-rendered at build time where possible

```
// vercel.json { "crons": [ { "path": "/api/cron/reminders", "schedule": "0 * * * *" } ] }
```

---

6. Security & Privacy Basics

6.1 Row Level Security (RLS)

Every table in the Supabase PostgreSQL database has RLS enabled. No table is accessible without an explicit policy matching the request's auth context. Key principles:

- Barber isolation: A barber's JWT can only access rows where `barber_id = auth.uid()`. Cross-barber data leakage is impossible at the database layer.
- Admin override: Admin role has explicit `FOR ALL USING (auth.role() = 'admin')` policies.

- Public access: The `anon` key can only read whitelisted public fields (barber profile public fields, active services, available slots). No PII is readable by anonymous requests.
- Service role: The Supabase `servicerolekey` bypasses RLS and is used only in server-side admin operations and webhook handlers. It is never sent to the client.

6.2 No PII Exposure on Public Routes

Public API routes (`/api/public/*`) are carefully designed to exclude personal information:

Field	Exposed Publicly?	Reason
`barber_profiles.email`	□	Barber contact, not needed publicly
`barber_profiles.phone`	Optional — barber controls	Settings-driven toggle
`barber_profiles.stripe_account_id`	□ Never	Payment credential
`clients.email`	□ Never	Customer PII
`bookings.client_id`	□ Never	Linkable to PII
Booking token lookup	Only own booking	Token acts as auth

6.3 JWT Authentication for Barber / Admin Routes

- All /barber/ page routes and /api/barber/ API routes require a valid Supabase JWT
- All /admin/ page routes and /api/admin/ API routes require a valid Supabase JWT and role = 'admin' claim
- JWT verification happens in Next.js middleware (edge) before the route handler runs — unauthenticated requests are redirected, never served partial data
- JWTs have a 1-hour expiry; Supabase automatically refreshes sessions via `@supabase/ssr` helpers
- Sessions stored in `httpOnly`, `Secure`, `SameSite=Lax` cookies — not accessible to JavaScript

6.4 Input Validation

- All API route handlers validate request bodies using Zod schemas before processing
- Validation errors return `400 Bad Request` with a structured error object — no stack traces exposed
- Booking form inputs are validated both client-side (UX) and server-side (security)
- `barberId` and `serviceId` in booking requests are cross-referenced against the database — no spoofed foreign keys accepted
- File uploads (photos) validated for MIME type (`image/jpeg`, `image/png`, `image/webp`) and size (≤ 5MB) before upload

6.5 Rate Limiting on Booking Endpoints

The booking flow is rate-limited to prevent abuse and slot exhaustion attacks:

Endpoint	Limit	Window	Enforcement
`POST /api/public/bookings`	5 requests	Per IP, per 10 minutes	Vercel Edge middleware
`GET /api/public/barbers/:slug/availability`	30 requests	Per IP, per minute	Vercel Edge middleware
`POST /api/barber/bookings/:id/cancel`	10 requests	Per JWT, per hour	API route handler

Rate limiting state stored in Vercel KV (Redis-compatible) or via the `@upstash/ratelimit` library with Upstash Redis.

6.6 HTTPS Only

- Vercel enforces HTTPS on all custom domains and preview URLs by default — HTTP requests are redirected to HTTPS (301)
- `Strict-Transport-Security` header set with `max-age=31536000; includeSubDomains`
- All third-party integrations (Supabase, Stripe, Resend) communicate over TLS 1.2+
- No sensitive data is ever written to browser `localStorage` — only `httpOnly` cookies

6.7 Stripe Webhook Security

- The `/api/webhooks/stripe` endpoint is excluded from JWT auth middleware (Stripe sends unsigned requests from its servers)
- Instead, every incoming request is validated using `stripe.webhooks.constructEvent()` with the `STRIPEWEBHOOKSECRET` before any database writes occur
- Raw request body (`Buffer`) is preserved for signature verification — `bodyParser` is disabled on this route
- Invalid signatures return `400` immediately with no additional processing
- Idempotency: each webhook event ID is checked against `notification_logs` before processing to prevent double-confirmation

---

7. Data Flow Diagrams

7.1 Customer Booking Flow

```
Customer opens /b/:barberSlug │ ▼ [Public Profile Page] Server Component fetches barber_profiles
+ services from Supabase Renders: photo, bio, service menu, "Book Now" button │ │ Click "Book
Now" ▼ [/b/:barberSlug/book — Step 1: Select Service] Client renders service cards from
pre-fetched data Customer selects a service (duration + price displayed) │ │ Service selected →
advance to Step 2 ▼ [Step 2: Select Date & Time] Client fetches GET
/api/public/barbers/:slug/availability │ Query: { date, serviceId } ▼ Availability Engine: 1.
Loads weekly schedule for selected date's day-of-week 2. Checks for date overrides (closed /
custom hours) 3. Generates candidate slots (15-min intervals) 4. Filters out booked/blocked slots
via DB query 5. Returns available slot array │ │ Returns ISO8601 times → rendered as selectable
buttons │ Customer selects a time slot ▼ [Step 3: Enter Details] Form fields: Full Name, Email,
Phone, Notes Client-side Zod validation │ │ Customer submits form ▼ [Step 4: POST
/api/public/bookings] Server validates input (Zod) Rate limit check (IP-based) │ ├─
isSlotAvailable() → atomic DB check │ If unavailable → 409 Conflict → UI shows "Slot taken, pick
```

```
another" │ ├─ upsertClient(barberId, email, name, phone) │ Finds or creates client record │ ├─
INSERT INTO bookings (status: PENDING_PAYMENT) │ ├─ createCheckoutSession(bookingId,
depositAmount) │ Stripe API call → returns session.url │ └─ Returns { bookingId, checkoutUrl } to
client │ │ Client redirects browser to checkoutUrl (Stripe-hosted) ▼ [Stripe Checkout Page]
Customer enters card details on Stripe's secure page │ ┌────┴────┐ │ │ Payment Payment Fails /
Abandoned ... [truncated for brevity]
```

7.2 Barber Dashboard Data Flow

```
Barber navigates to /barber/dashboard │ ▼ [Next.js Middleware (Edge)] Reads JWT from httpOnly
cookie Verifies signature + expiry Checks role = 'barber' │ ┌────┴────┐ │ │ Valid JWT Invalid /
Expired JWT │ │ ▼ └─► Redirect to /barber/login [Dashboard Server Component] Supabase client
initialized with barber's JWT Parallel data fetches (all scoped to auth.uid() via RLS): │ ├─ GET
upcoming bookings (next 7 days) │ SELECT * FROM bookings │ WHERE barber_id = auth.uid() │ AND
start_at BETWEEN now() AND now() + interval '7 days' │ AND status = 'CONFIRMED' │ ├─ GET revenue
summary (current month) │ SELECT SUM(amount) FROM payments │ JOIN bookings ON bookings.id =
payments.booking_id │ WHERE bookings.barber_id = auth.uid() │ AND payments.created_at >=
date_trunc('month', now()) │ ├─ GET today's bookings (time-blocked schedule) │ SELECT * FROM
bookings WHERE date(start_at) = today │ └─ GET new client count (current month) SELECT COUNT(*)
FROM clients WHERE barber_id = auth.uid() AND created_at >= date_trunc('month', now()) │ ▼
[Dashboard renders with server-fetched data] No client-side loading states for initial data
Interactive actions (cancel, reschedule) trigger Client Components: │ ├─ Cancel booking → POST
/api/barber/bookings/:id/cancel │ Server validates ownership (barber_id = auth.uid()) │ Triggers
refund + cancellation notifications │ └─ Reschedule → POST /api/barber/bookings/:id/reschedule
Server re-validates slot availability Updates booking, sends rescheduled email
```

7.3 Admin Setup Flow (Creating a New Barber)

```
Admin navigates to /admin/barbers │ ▼ [Next.js Middleware (Edge)] Verifies JWT: valid + role =
'admin' (Barber JWT rejected here even if valid) │ ▼ [Admin Barbers Page] Lists all barber
accounts from barber_profiles "Create New Barber" button opens a modal form │ │ Admin fills form:
│ - Display name │ - Email address │ - Desired slug (validated for uniqueness client-side) │ -
Timezone ▼ [POST /api/admin/barbers] Admin JWT verified in route handler │ ├─
validateSlugUniqueness(slug) → SELECT from barber_profiles │ If taken → 409 → "Slug already in
use" error │ ├─ supabase.auth.admin.createUser({ │ email, │ password: temporaryPassword, // sent
to barber via email │ user_metadata: { role: 'barber' } │ }) │ Returns new auth user with UUID │
├─ INSERT INTO barber_profiles ({ │ id: authUser.id, // same UUID │ slug, │ display_name, │
timezone, │ is_active: true │ }) │ ├─ INSERT INTO weekly_schedules (default Mon–Fri 9am–6pm) │ ├─
INSERT INTO audit_logs ({ │ actor_id: admin.uid, │ actor_role: 'ADMIN', │ action:
'CREATE_BARBER', │ resource_type: 'barber_profile', │ resource_id: newBarberId │ }) │ └─ Email
sent to barber: "Welcome to BarberBook — set your password" Contains magic link (Supabase Auth
invite flow) │ ▼ [Barber receives email, clicks link] Sets their own password Redirected to
/barber/dashboard Prompted to complete profile setup: - Upload photo - Write bio ... [truncated
for brevity]
```

---

8. Folder Structure Rationale

8.1 Why Next.js App Router

The Next.js App Router (introduced in Next.js 13, stable in 14) is chosen over the Pages Router for several reasons:

Capability	Impact on BarberBook
React Server Components (RSC)	Public profile pages (`/b/:slug`) and the barber dashboard fetch data directly in Server Components — no client-side loading states, no API round-trips for initial render
Nested layouts	`/barber/*` routes share a persistent sidebar layout component; `/b/:barberSlug/*` routes share a minimal public layout. Both defined once with `layout.tsx`.
Colocation of data fetching	Each Server Component fetches exactly the data it needs — no prop drilling, no global state management library needed
Built-in route handlers	`route.ts` files in the `app/api/` directory replace a separate Express/Fastify server
Streaming	Long-running data fetches (reporting page) can stream incrementally with `<Suspense>` wrappers

8.2 Annotated Folder Structure

```
barberbook/ │ ├── app/ # Next.js App Router root │ ├── b/ # Public customer-facing routes │ │ └── [barberSlug]/ │ │ ├── page.tsx # /b/:slug — Barber profile page │ │ ├── book/ │ │ │ └── page.tsx # /b/:slug/book — Booking flow │ │ └── confirmation/ │ │ └── page.tsx # /b/:slug/confirmation │ │ ├── barber/ # Authenticated barber routes │ │ ├── layout.tsx # Shared barber shell (sidebar, nav) │ │ ├── login/ │ │ │ └── page.tsx │ │ ├── dashboard/ │ │ │ └── page.tsx │ │ ├── bookings/ │ │ │ └── page.tsx │ │ ├── clients/ │ │ │ └── page.tsx │ │ ├── services/ │ │ │ └── page.tsx │ │ └── settings/ │ │ └── page.tsx │ │ ├── admin/ # Authenticated admin routes │ │ ├── layout.tsx # Admin shell layout │ │ ├── login/ │ │ │ └── page.tsx │ │ ├── dashboard/ │ │ │ └── page.tsx │ │ ├── barbers/ │ │ │ └── page.tsx │ │ ├── config/ │ │ │ └── page.tsx │ │ └── monitoring/ │ │ └── page.tsx │ │ └── api/ # API Route Handlers │ ├── public/ │ │ ├── barbers/ │ │ │ └── [slug]/ │ │ │ ├── route.ts # GET /api/public/barbers/:slug │ │ │ ├── services/ │ │ │ │ └── route.ts # GET services │ │ │ └── availability/ │ │ │ └── route.ts # GET available slots │ │ └── bookings/ │ │ ├── route.ts # POST create booking │ │ └── [token]/ │ │ ├── route.ts # GET by token │ │ └── cancel/ │ │ └── route.ts # POST cancel │ │ ├── barber/ # All require Barber JWT │ │ ├── profile/ │ │ │ └── route.ts │ │ ├── schedule/ │ │ │ ├── route.ts ... [truncated for brevity]
```

8.3 Public vs. Authenticated Route Separation

The separation is enforced at three levels simultaneously:

1. Folder structure: Public customer routes live under `app/b/`, barber routes under `app/barber/`, admin routes under `app/admin/`. The folder name itself makes access level visually obvious to any developer.
2. Middleware (`middleware.ts`): Runs on Vercel's Edge Runtime (< 1ms overhead). Matches route patterns and enforces JWT requirements before any page or API handler executes. Unauthenticated requests to protected routes are redirected — no partial data is ever served.

3. Row Level Security: Even if middleware were bypassed (e.g., a direct API call), the Supabase RLS policies enforce data isolation at the database query level. This defense-in-depth means no single point of failure can expose cross-user data.

8.4 Shared Services Organization

The `lib/modules/` directory mirrors the 9-module architecture described in Section 2. Each module file exports pure TypeScript functions that receive a Supabase client as a parameter — this makes modules:

- Testable: Functions can be called with a mock Supabase client in unit tests
- Reusable: The same `getAvailableSlots()` function is called from the public API, the barber dashboard, and the admin monitoring view
- Dependency-injectable: Different Supabase clients (anon, authenticated, service-role) are passed in by the calling API route — the module itself doesn't decide which credentials to use

```
// Example: lib/modules/availability.ts export async function getAvailableSlots( supabase: SupabaseClient, // caller provides the right client barberId: string, date: string, durationMinutes: number ): Promise<string[]> { ... } // Public API route — uses anon client import { createServerClient } from '@/lib/supabase/server' const slots = await getAvailableSlots(createServerClient(), barberId, date, duration) // Barber dashboard — uses authenticated client (RLS enforced) const slots = await getAvailableSlots(createBarberClient(req), barberId, date, duration)
```

End of Phase 2: System Architecture

Document Control

Version	Date	Author	Changes
1.0	March 30, 2026	Solution Architect	Initial draft

Phase 3

Database Design

Schema · RLS · Triggers

Phase 3: Database Design

BarberBook MVP — Technical Specification

Project: BarberBook — Single-Barber Branded Booking Micro-Platform

Phase: 3 of 5

Author Role: Database Architect

Database: PostgreSQL 15+ (hosted via Supabase)

Date: March 30, 2026

Table of Contents

1. [Entity Relationship Explanation](#1-entity-relationship-explanation)
2. [Status ENUMs](#2-status-enums)
3. [Complete SQL Schema](#3-complete-sql-schema)
4. [Row Level Security (RLS) Policies](#4-row-level-security-rls-policies)
5. [Triggers](#5-triggers)
6. [Seed Data](#6-seed-data)

---

1. Entity Relationship Explanation

Overview

BarberBook's data model is designed around a single core actor — the Barber — and all other entities either belong to, extend, or are generated by activity within that barber's platform. The schema supports multi-tenant isolation (each barber sees only their own data) while allowing an admin role to oversee everything. Below is a plain-English description of each entity and its relationships.

1.1 User → Barber (1:1)

User is the authentication and identity record. Every person who can log into BarberBook (whether as an admin or a barber) has exactly one User row. The User stores credentials (`email`, `password_hash`) and a `role` flag that determines what they can do in the system.

Barber is the public profile and configuration layer for a barber practitioner. It extends User with branding information: display name, bio, photo, location, social links, and branding settings (stored as flexible JSON). Each User of role `barber` has exactly one Barber record, linked via `user_id`. The relationship is strictly one-to-one — a barber cannot have multiple profiles, and a User row cannot own more than one Barber profile.

A User is who can log in. A Barber is what clients see and book through.

1.2 Barber → ServiceGroup (1:Many)

A ServiceGroup represents a bookable service offering that a barber provides — for example, "Classic Fade", "Full Cut & Beard", or "Kids Cut". Each ServiceGroup belongs to exactly one Barber and carries pricing details (price, depositype, depositvalue, material_cost), duration, and optional imagery.

A Barber can have many ServiceGroups (their full service menu), but each ServiceGroup can belong to only one Barber. ServiceGroups can be individually toggled active/inactive so a barber can temporarily hide a service without deleting it.

1.3 Barber → AvailabilityRule (1:Many)

AvailabilityRule defines the barber's recurring weekly schedule — the hours they are open for business on each day of the week. Each rule specifies a dayofweek (0 = Sunday through 6 = Saturday), a starttime, and an endtime. A barber can have multiple rules (one per working day), forming their standard weekly calendar.

These rules are the foundation the booking engine reads when generating available time slots. Rules can be toggled inactive to temporarily suspend a day without deletion (e.g., taking Mondays off for a month).

1.4 Barber → BlockedTime (1:Many)

BlockedTime represents ad hoc, non-recurring unavailability on specific calendar dates — a vacation day, a personal appointment, or lunch breaks. Unlike AvailabilityRule, BlockedTime targets a specific `date` rather than a recurring weekday pattern, and can block a full day or just a specific time window.

A Barber can have many BlockedTime entries. The booking engine subtracts blocked windows from AvailabilityRule slots before presenting open times to clients.

1.5 Barber → Client (1:Many)

A Client is a customer who has booked with a specific barber. Clients are barber-scoped — the same real-world person may exist as separate Client records across different barbers on the platform (there is no shared global customer identity in the MVP). Each Client record stores contact details and automatically maintained visit statistics (totalvisits, noshowcount, lastvisit_at) updated by database triggers.

A Barber has many Clients (their clientele), but each Client belongs to exactly one Barber. This enables per-barber CRM features: notes, visit history, and no-show tracking.

1.6 Barber + Client + ServiceGroup → Booking (Many:1 from Each)

Booking is the central transactional entity. It records a specific appointment at a specific date and time. Each Booking connects:

- A Barber (who is performing the service)
- A Client (who has booked the appointment)
- A ServiceGroup (what service is being performed)

All three foreign keys are present on every Booking row, making it the junction that ties the platform's core entities together. A single Booking captures the full context of an appointment: date, start/end time, current status, payment status, deposit amount collected, and any note from the customer.

Many Bookings can exist for the same Barber, Client, or ServiceGroup over time (a returning customer booking repeatedly, or a popular service booked by many clients). This makes it a many-to-one relationship from each of those three entities into the Booking table.

1.7 Booking → PaymentRecord (1:1 or 1:0)

A PaymentRecord captures the financial transaction detail for a booking — specifically the deposit or full payment made through an external payment provider (e.g., Stripe). Not every booking results in a payment (some services may have no deposit required), making this a zero-or-one relationship: a Booking may have zero PaymentRecords (no payment taken) or exactly one (a deposit or full payment was processed).

PaymentRecord stores the provider name, amount, currency, `payment_type`, `status`, and the external reference ID returned by the payment provider for reconciliation.

1.8 Booking / Client → MessageLog (1:Many)

MessageLog records every outbound communication sent by the platform on behalf of a barber — booking confirmations, 24-hour reminders, 1-hour reminders, follow-ups, and custom messages. Each log entry is tied to a Barber (sender context), a Client (recipient), an optional Booking (the appointment the message concerns), a `message_type`, a `channel` (email or SMS), a delivery `status`, and the timestamp it was sent.

A single Booking can generate many MessageLog entries over its lifecycle (confirmation → reminder → follow-up). A Client similarly accumulates many messages across all their bookings over time. MessageLog is append-only — entries are never updated, only inserted.

Entity Relationship Summary Table

Relationship	Cardinality	Foreign Key Location
User → Barber	1:1	`barber.user_id`
Barber → ServiceGroup	1:Many	`service_group.barber_id`
Barber → AvailabilityRule	1:Many	`availability_rule.barber_id`
Barber → BlockedTime	1:Many	`blocked_time.barber_id`
Barber → Client	1:Many	`client.barber_id`
Booking → Barber	Many:1	`booking.barber_id`
Booking → Client	Many:1	`booking.client_id`
Booking → ServiceGroup	Many:1	`booking.service_group_id`
Booking → PaymentRecord	1:0 or 1:1	`payment_record.booking_id`
Booking → MessageLog	1:Many	`message_log.booking_id`
Client → MessageLog	1:Many	`message_log.client_id`

---

2. Status ENUMs

All ENUMs are defined before the table definitions so they can be referenced in `CREATE TABLE` statements. ENUMs enforce valid values at the database level, eliminating the need for application-side check constraints on status columns.

```
-- ============================================================ -- ENUM TYPE DEFINITIONS --
============================================================ -- Lifecycle state of a booking
appointment CREATE TYPE booking_status AS ENUM ( 'booked', -- Initial state: slot reserved, not
yet confirmed 'confirmed', -- Barber has confirmed the appointment 'rescheduled', -- Appointment
was moved to a new date/time 'cancelled', -- Appointment was cancelled by barber or client
'completed', -- Service was rendered; appointment finished 'no_show' -- Client did not appear for
the appointment ); -- Financial state of payment associated with a booking CREATE TYPE
payment_status AS ENUM ( 'pending', -- Payment initiated but not yet settled 'paid', -- Payment
successfully captured 'refunded', -- Payment was reversed/refunded 'failed', -- Payment attempt
failed 'not_required' -- No payment is required for this booking ); -- How a deposit is
calculated for a service CREATE TYPE deposit_type AS ENUM ( 'fixed', -- Deposit is a fixed
currency amount 'percentage', -- Deposit is a percentage of the service price 'none' -- No
deposit required ); -- Communication channel for outbound messages CREATE TYPE message_channel AS
ENUM ( 'email', 'sms' ); -- Category/purpose of an outbound message CREATE TYPE message_type AS
ENUM ( 'confirmation', -- Sent immediately on booking 'reminder_24h', -- Sent 24 hours before
appointment 'reminder_1h', -- Sent 1 hour before appointment 'follow_up', -- Sent after
appointment completion 'custom' -- Ad hoc message sent by barber ); -- Access role for system
users CREATE TYPE user_role AS ENUM ( 'admin', -- Platform administrator with full access
```

```
'barber' -- Individual barber with access to their own data only ); -- Delivery status for a sent
message CREATE TYPE message_delivery_status AS ENUM ( 'queued', -- Message queued for sending
'sent', -- Message dispatched to provider 'delivered', -- Provider confirmed delivery 'failed' --
Delivery failed );
```

---

3. Complete SQL Schema

The schema uses uuid primary keys generated via genrandomuuid() (built into PostgreSQL 13+). All tables include createdat with a default timestamp. Tables with mutable records also include updatedat, which is maintained automatically by triggers defined in Section 5.

The `pgcrypto` and `uuid-ossp` extensions are not required; `genrandomuuid()` is available natively in PostgreSQL 13+ and Supabase.

```
-- ============================================================ -- EXTENSIONS --
============================================================ -- Enable Row Level Security
globally (Supabase does this by default) -- No additional extensions required for UUID generation
in PG 13+ -- ============================================================ -- TABLE: users --
Authentication and identity for all platform users. --
============================================================ CREATE TABLE users ( id UUID PRIMARY
KEY DEFAULT gen_random_uuid(), email TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL, role
user_role NOT NULL DEFAULT 'barber', created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at
TIMESTAMPTZ NOT NULL DEFAULT now() ); -- Index for auth lookups by email CREATE INDEX
idx_users_email ON users (email); CREATE INDEX idx_users_role ON users (role); --
============================================================ -- TABLE: barbers -- Public profile
and configuration for a barber practitioner. -- One barber record per user of role 'barber'. --
============================================================ CREATE TABLE barbers ( id UUID
PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL UNIQUE REFERENCES users(id) ON
DELETE CASCADE, slug TEXT NOT NULL UNIQUE, display_name TEXT NOT NULL, bio TEXT, photo_url TEXT,
phone TEXT, email TEXT, location TEXT, social_links JSONB NOT NULL DEFAULT '{}',
branding_settings JSONB NOT NULL DEFAULT '{}', is_published BOOLEAN NOT NULL DEFAULT false,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT barbers_slug_format CHECK (slug ~ '^[a-z0-9-]+$') ); -- Indexes for profile lookup and
public page routing CREATE INDEX idx_barbers_user_id ON barbers (user_id); CREATE INDEX
idx_barbers_slug ON barbers (slug); CREATE INDEX idx_barbers_is_published ON barbers
(is_published); -- ============================================================ -- TABLE:
service_groups ... [truncated for brevity]
```

---

4. Row Level Security (RLS) Policies

Supabase uses PostgreSQL's built-in Row Level Security. All policies below assume:

- The Supabase `auth.uid()` function returns the UUID of the currently authenticated User.
- A helper function `is_admin()` checks whether the current user has the `admin` role.
- A helper function `ownbarberid()` returns the `barbers.id` for the currently authenticated barber user.

```
-- ============================================================ -- HELPER FUNCTIONS -- Used as
building blocks in RLS policies. -- ============================================================
-- Returns true if the authenticated user has the 'admin' role CREATE OR REPLACE FUNCTION
is_admin() RETURNS BOOLEAN LANGUAGE sql STABLE SECURITY DEFINER AS $$ SELECT EXISTS ( SELECT 1
FROM users WHERE id = auth.uid() AND role = 'admin' ); $$; -- Returns the barber.id owned by the
authenticated user (NULL if not a barber) CREATE OR REPLACE FUNCTION own_barber_id() RETURNS UUID
```

```
LANGUAGE sql STABLE SECURITY DEFINER AS $$ SELECT id FROM barbers WHERE user_id = auth.uid()
LIMIT 1; $$; -- ============================================================ -- ENABLE RLS ON ALL
TABLES -- ============================================================ ALTER TABLE users ENABLE
ROW LEVEL SECURITY; ALTER TABLE barbers ENABLE ROW LEVEL SECURITY; ALTER TABLE service_groups
ENABLE ROW LEVEL SECURITY; ALTER TABLE availability_rules ENABLE ROW LEVEL SECURITY; ALTER TABLE
blocked_times ENABLE ROW LEVEL SECURITY; ALTER TABLE clients ENABLE ROW LEVEL SECURITY; ALTER
TABLE bookings ENABLE ROW LEVEL SECURITY; ALTER TABLE payment_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE message_logs ENABLE ROW LEVEL SECURITY; --
============================================================ -- POLICIES: users --
============================================================ -- Barbers can read and update only
their own user record CREATE POLICY barber_select_own_user ON users FOR SELECT USING (id =
auth.uid()); CREATE POLICY barber_update_own_user ON users FOR UPDATE USING (id = auth.uid()); --
Admins can read and update all user records CREATE POLICY admin_select_all_users ON users ...
[truncated for brevity]
```

---

5. Triggers

5.1 Auto-Update `updated_at` on Any Row Update

A single reusable trigger function is defined once and attached to every table that has an `updated_at` column.

```
-- ============================================================ -- TRIGGER FUNCTION:
set_updated_at -- Automatically sets updated_at = now() before any UPDATE. -- Attach to every
table that has an updated_at column. --
============================================================ CREATE OR REPLACE FUNCTION
set_updated_at() RETURNS TRIGGER LANGUAGE plpgsql AS $$ BEGIN NEW.updated_at = now(); RETURN NEW;
END; $$; -- Attach to all tables with updated_at CREATE TRIGGER trg_users_updated_at BEFORE
UPDATE ON users FOR EACH ROW EXECUTE FUNCTION set_updated_at(); CREATE TRIGGER
trg_barbers_updated_at BEFORE UPDATE ON barbers FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_service_groups_updated_at BEFORE UPDATE ON service_groups FOR EACH ROW EXECUTE
FUNCTION set_updated_at(); CREATE TRIGGER trg_clients_updated_at BEFORE UPDATE ON clients FOR
EACH ROW EXECUTE FUNCTION set_updated_at(); CREATE TRIGGER trg_bookings_updated_at BEFORE UPDATE
ON bookings FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

5.2 Auto-Upsert Client Record After Booking INSERT or UPDATE

When a Booking is created or its status transitions to completed, the Client's totalvisits and lastvisit_at are updated automatically. This keeps Client statistics accurate without requiring the application to make a separate update call.

```
-- ============================================================ -- TRIGGER FUNCTION:
sync_client_visit_stats -- Fires AFTER INSERT or UPDATE on bookings. -- Updates total_visits and
last_visit_at on the related client -- whenever a booking reaches 'completed' status. --
============================================================ CREATE OR REPLACE FUNCTION
sync_client_visit_stats() RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$ BEGIN -- Only
act when the booking status is or becomes 'completed' IF NEW.status = 'completed' AND (TG_OP =
'INSERT' OR OLD.status IS DISTINCT FROM 'completed') THEN UPDATE clients SET total_visits =
total_visits + 1, last_visit_at = now(), updated_at = now() WHERE id = NEW.client_id; END IF;
RETURN NEW; END; $$; CREATE TRIGGER trg_booking_sync_client_visits AFTER INSERT OR UPDATE OF
status ON bookings FOR EACH ROW EXECUTE FUNCTION sync_client_visit_stats();
```

5.3 Increment `noshowcount` When Booking Status Becomes `no_show`

```
-- ============================================================ -- TRIGGER FUNCTION:
increment_client_no_show -- Fires AFTER UPDATE on bookings when status transitions to 'no_show'.
-- Increments no_show_count on the related client record. --
============================================================ CREATE OR REPLACE FUNCTION
increment_client_no_show() RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$ BEGIN -- Only
act on a transition TO 'no_show' (not already no_show) IF NEW.status = 'no_show' AND OLD.status
IS DISTINCT FROM 'no_show' THEN UPDATE clients SET no_show_count = no_show_count + 1, updated_at
= now() WHERE id = NEW.client_id; END IF; RETURN NEW; END; $$; CREATE TRIGGER
trg_booking_increment_no_show AFTER UPDATE OF status ON bookings FOR EACH ROW EXECUTE FUNCTION
increment_client_no_show();
```

5.4 Prevent Double-Booking (Scheduling Conflict Guard)

A safety trigger that rejects an INSERT or UPDATE on bookings if the same barber already has an active (non-cancelled, non-no_show) booking overlapping the proposed time window on the same date.

```
-- ============================================================ -- TRIGGER FUNCTION:
check_booking_conflict -- Fires BEFORE INSERT or UPDATE on bookings. -- Raises an exception if
the barber already has an overlapping -- active booking on the same date. --
============================================================ CREATE OR REPLACE FUNCTION
check_booking_conflict() RETURNS TRIGGER LANGUAGE plpgsql AS $$ DECLARE conflict_count INTEGER;
BEGIN -- Skip conflict check for cancelled or no_show bookings IF NEW.status IN ('cancelled',
'no_show') THEN RETURN NEW; END IF; SELECT COUNT(*) INTO conflict_count FROM bookings WHERE
barber_id = NEW.barber_id AND booking_date = NEW.booking_date AND id != NEW.id AND status NOT IN
('cancelled', 'no_show') -- Overlap condition: new booking starts before existing ends AND ends
after existing starts AND NEW.start_time < end_time AND NEW.end_time > start_time; IF
conflict_count > 0 THEN RAISE EXCEPTION 'Booking conflict: barber % already has an active booking
overlapping %T–%T on %', NEW.barber_id, NEW.start_time, NEW.end_time, NEW.booking_date; END IF;
RETURN NEW; END; $$; CREATE TRIGGER trg_booking_conflict_check BEFORE INSERT OR UPDATE OF
barber_id, booking_date, start_time, end_time, status ON bookings FOR EACH ROW EXECUTE FUNCTION
check_booking_conflict();
```

---

6. Seed Data

These INSERT statements populate the database with a minimal working dataset suitable for development and local testing. All UUIDs are hardcoded so foreign key references are consistent across statements.

> *Note: The `password_hash` values below are bcrypt hashes of the plaintext passwords shown in the comments. Never store or commit real credentials in seed files for production.*

```
-- ============================================================ -- SEED DATA --
Development/testing baseline data for BarberBook MVP --
============================================================ --
------------------------------------------------------------ -- 1. ADMIN USER -- Email:
admin@barberbook.io | Password: AdminPass123! --
------------------------------------------------------------ INSERT INTO users (id, email,
password_hash, role) VALUES ( '00000000-0000-0000-0000-000000000001', 'admin@barberbook.io',
```

```
'$2b$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/LewdBPj/RK.s5uq0G', -- AdminPass123! 'admin' ); --
---------------------------------------------------------------- -- 2. BARBER USER -- Email:
marcus@barberbook.io | Password: BarberPass456! --
---------------------------------------------------------------- INSERT INTO users (id, email,
password_hash, role) VALUES ( '00000000-0000-0000-0000-000000000002', 'marcus@barberbook.io',
'$2b$12$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWFpig6', -- BarberPass456! 'barber' ); --
---------------------------------------------------------------- -- 3. BARBER PROFILE -- Linked to
the barber user above. -- Public booking page: /book/marcus-cuts --
---------------------------------------------------------------- INSERT INTO barbers ( id, user_id,
slug, display_name, bio, photo_url, phone, email, location, social_links, branding_settings,
is_published ) VALUES ( '00000000-0000-0000-0000-000000000010',
'00000000-0000-0000-0000-000000000002', 'marcus-cuts', 'Marcus Carter', 'Award-winning barber
with 10+ years of experience in precision fades, beard styling, and classic cuts. Based in
downtown Toronto.', 'https://cdn.barberbook.io/profiles/marcus-cuts/photo.jpg', ... [truncated
for brevity]
```

Appendix A: Schema Diagram (Entity Listing)

```
┌──────────┐ ┌──────────┐ ┌──────────────┐ │ users │ 1:1 │ barbers │ 1:N │ service_groups│
│──────────│──────►│──────────│──────►│──────────────│ │ id (PK) │ │ id (PK) │ │ id (PK) │ │ email
│ │ user_id │ │ barber_id (FK)│ │ role │ │ slug │ │ title │ └──────────┘ │ branding │ │ price │
└────┬─────┘ └──────┬───────┘ │ │ ┌────────┬─────────┐ │ │ │ │ ▼ ▼ ▼ │ ┌──────────────┐
┌──────────┐ ┌─────┴────┐ │availability │ │blocked_ │ │ clients │ │_rules │ │times │
│──────────│ │──────────│ │──────────│ │ id (PK) │ │ barber_id(FK)│ │barber_id │ │
barber_id │ │ day_of_week │ │date │ └────┬─────┘ └──────────────┘ └──────────┘ │ │ ▼
┌──────────────────┐ │ bookings │ │──────────────────│ │ id (PK) │ │ barber_id (FK) │ │
client_id (FK) │ │ service_group_id (FK) │ │ status │ │ payment_status │ └────────┬──────────┘
│ ┌──────────┬──────────┐ ▼ ▼ ┌──────────────┐ ┌──────────────┐ │ payment_records
│ │ message_logs │ │──────────────│ │──────────────│ │ id (PK) │ │ id (PK) │ │ booking_id
(FK) │ │ booking_id (FK) │ │ barber_id (FK) │ │ client_id (FK) │ │ amount / status │ │
message_type │ └──────────────┘ └──────────────┘
```

Appendix B: Design Decisions & Notes

Decision	Rationale
UUID primary keys	Globally unique across environments; safe for distributed inserts; no sequential enumeration risk
JSONB for `social_links` and `branding_settings`	These fields have variable structure and evolve over time; JSONB avoids schema migrations for new social platforms or branding options
Barber-scoped clients (no global identity)	MVP simplicity; avoids cross-barber data leakage; each barber manages their own clientele
`SECURITY DEFINER` on trigger functions	Trigger functions that UPDATE client stats run as the function owner (bypassing RLS), ensuring they can update client rows even when triggered by a public INSERT on bookings

Decision	Rationale
`ON DELETE RESTRICT` on bookings	Prevents accidental deletion of a barber or service that has booking history; deletions must be intentional
`ON DELETE CASCADE` on service_groups, availability_rules, blocked_times	These are configuration data owned entirely by the barber; deleting the barber should clean up all their settings
No `updated_at` on `payment_records` and `message_logs`	These are append-only audit records; mutations are not expected in normal operation
Partial index on bookings (excluding cancelled/no_show)	Availability conflict checks only need to consider active bookings; the partial index keeps this lookup fast
`is_admin()` as SECURITY DEFINER function	Ensures the role check always reads from the `users` table with elevated privileges, even inside RLS policies

---

End of Phase 3: Database Design

BarberBook MVP Technical Specification

Phase 4

Screen Plan

22 Screens · User Actions

BarberBook MVP — Phase 4: Screen Plan

Document Version: 1.0

Date: March 30, 2026

Author: UX/UI Design — BarberBook Team

Scope: All screens in the BarberBook MVP, covering Customer, Barber, and Admin surfaces.

Table of Contents

1. [Customer Screens](#customer-screens)

- [CS-01 — Barber Landing Page](#cs-01--barber-landing-page)
- [CS-02 — Service Selection (Booking Step 1)](#cs-02--service-selection-booking-step-1)
- [CS-03 — Date & Time Selection (Booking Step 2)](#cs-03--date--time-selection-booking-step-2)
- [CS-04 — Customer Details Form (Booking Step 3)](#cs-04--customer-details-form-booking-step-3)
- [CS-05 — Payment / Checkout (Booking Step 4)](#cs-05--payment--checkout-booking-step-4)
- [CS-06 — Booking Confirmation](#cs-06--booking-confirmation)

2. [Barber Screens](#barber-screens)

- [BS-01 — Barber Login](#bs-01--barber-login)
- [BS-02 — Barber Dashboard](#bs-02--barber-dashboard)
- [BS-03 — Calendar / Bookings Screen](#bs-03--calendar--bookings-screen)
- [BS-04 — Booking Detail Modal / View](#bs-04--booking-detail-modal--view)
- [BS-05 — Clients Screen](#bs-05--clients-screen)
- [BS-06 — Client Detail View](#bs-06--client-detail-view)
- [BS-07 — Services Screen](#bs-07--services-screen)
- [BS-08 — Service Edit / Create Form](#bs-08--service-edit--create-form)
- [BS-09 — Settings Screen](#bs-09--settings-screen)

3. [Admin Screens](#admin-screens)

- [AS-01 — Admin Login](#as-01--admin-login)
- [AS-02 — Admin Dashboard](#as-02--admin-dashboard)
- [AS-03 — Barbers Management](#as-03--barbers-management)

- [AS-04 — Barber Setup / Creation](#as-04--barber-setup--creation)
- [AS-05 — Branding & Config](#as-05--branding--config)
- [AS-06 — Payment & Reminders Config](#as-06--payment--reminders-config)
- [AS-07 — Monitoring Screen](#as-07--monitoring-screen)

4. [Navigation Model](#navigation-model)
5. [Design Principles](#design-principles)
6. [Component Library Notes](#component-library-notes)

Customer Screens

Customer screens form a public-facing, branded micro-site unique to each barber. The URL namespace is `/b/:barberSlug`, where `:barberSlug` is a unique identifier chosen at barber setup. All screens in this section inherit the barber's branding (colors, logo, name) injected at the layout level.

CS-01 — Barber Landing Page

Attribute	Value
Screen ID	CS-01
Screen Name	Barber Landing Page
Route	`/b/:barberSlug`

Purpose

The public-facing entry point for a specific barber's booking site. It establishes brand identity, communicates what the barber offers, and drives customers toward booking. This screen must convert a first-time visitor into a booking initiation within seconds — it is the top of the booking funnel.

User Type

- Anonymous customer (first-time or returning)
- No authentication required

Entry Points

- Direct link shared by the barber (SMS, Instagram bio, business card QR code)
- Search engine results (if the barber's page is indexed)
- Redirect from a confirmation email ("Book again" link)
- Any external marketing campaign link

Key UI Elements

Header / Hero Section

- Barber logo (circular avatar or rectangular logo image)
- Barber business name (H1, prominent)
- Short tagline or bio (1–2 sentences, supplied by barber in settings)
- Primary CTA button — "Book Now" (sticky on scroll)
- Optional hero background image or gradient using barber's brand color

About / Info Strip

- Location or city (no full address required for MVP; optional barber-supplied text)
- Barber's working hours summary (e.g., "Mon–Sat, 9am–7pm")
- Social proof element — total bookings count or a short testimonial (optional, can be hidden if data is empty)

Services Preview Section

- Grid or list of service groups offered by this barber
- Each service card shows: service name, short description, duration, price (or price range)
- "Book" button on each card that pre-selects that service and enters the booking flow at step 2

Footer

- Privacy policy link
- Powered by BarberBook attribution (subtle, small)
- Contact info if barber has provided it

User Actions

- Click "Book Now" → navigates to CS-02 (Service Selection) with no pre-selection
- Click a specific service card's "Book" button → navigates to CS-02 with that service pre-selected
- Scroll to read barber info and services
- Tap social links if present

State Variations

State	Behavior
Loading	Skeleton loaders for hero image, service cards; spinner not used — skeleton is less disruptive
No services configured	Services section shows a single placeholder card with message: "Services coming soon — check back shortly." CTA "Book Now" is hidden to prevent entering a broken flow
Barber inactive / suspended	Full-screen message: "This booking page is currently unavailable." No booking CTA is shown
Barber not found (invalid slug)	404 page with BarberBook branding and a generic "This barber page doesn't exist" message
Loaded, no bio set	Hero section collapses gracefully — no blank space; only logo and name render

Mobile Considerations

- Hero section is full-width and occupies 60–70 vh on mobile — scrolling reveals services below
- "Book Now" CTA becomes a fixed bottom bar on mobile (full-width button, 56 px height) to ensure persistent visibility while scrolling
- Service cards stack vertically in a single column on screens < 640 px
- Barber logo renders at 80 px × 80 px circle on mobile (vs. 120 px on desktop)
- Touch targets for service cards are minimum 48 px height

Data Required

- `GET /api/barbers/:slug` → barber profile: name, slug, logo URL, bio, brandColor, location label, social links, isActive
- `GET /api/barbers/:slug/services` → list of service groups: id, name, description, duration (minutes), price, isActive
- `GET /api/barbers/:slug/hours` → weekly hours summary for display (not full availability — that comes in step 2)

CS-02 — Service Selection (Booking Step 1)

Attribute	Value
Screen ID	CS-02
Screen Name	Service Selection
Route	`/b/:barberSlug/book` (step 1) or initiated inline on CS-01

Purpose

Allows the customer to confirm or select the service they want to book. Even if a service was pre-selected from the landing page, this screen gives the customer a clear overview of all available services and a chance to change their mind before committing to a time slot.

User Type

- Anonymous customer

Entry Points

- "Book Now" button on CS-01 (no pre-selection)
- Service card "Book" button on CS-01 (arrives with service pre-selected and highlighted)
- Browser back button from CS-03 (returns with previously selected service preserved in state)

Key UI Elements

Booking Flow Progress Indicator

- Step bar at top: Step 1 (Service) → Step 2 (Date & Time) → Step 3 (Your Details) → Step 4 (Payment, conditional)
- Current step visually highlighted; completed steps marked with a checkmark

- Barber name + logo shown in a compact sub-header beneath the progress bar

Service List

- Full list of the barber's active service groups
- Each row/card shows: service name, duration badge (e.g., "45 min"), price, optional short description
- Selected service gets a distinct selected state (border highlight, checkmark icon, brand color fill)
- Only one service can be selected at a time (radio-group behavior)

Service Group Sections (if barber uses grouping)

- Services optionally organized under named group headers (e.g., "Cuts", "Beard", "Combos")
- Section headers are non-interactive, purely organizational

Continue CTA

- "Continue" button at the bottom, disabled until a selection is made
- Button label: "Continue — [Service Name]" once selected, to reinforce the user's choice

User Actions

- Tap/click a service card to select it
- Tap "Continue" to proceed to CS-03
- Tap barber logo / name in sub-header to return to CS-01

State Variations

State	Behavior
Pre-selected (arrived from CS-01 card)	Corresponding service card is pre-highlighted; "Continue" button is immediately enabled
No selection made	"Continue" button is disabled; subtle helper text under button: "Select a service to continue"
Loading	Skeleton rows replace service cards during data fetch
No active services	Placeholder message with barber contact info if available; no booking can proceed
Service becomes unavailable mid-flow	On next step load, an inline banner warns the user and returns them to step 1 to reselect

Mobile Considerations

- Service cards are full-width stacked rows on mobile
- Duration and price badges display inline on the right side of each card
- "Continue" button is fixed to the bottom of the viewport on mobile (same sticky bar pattern as CS-01 CTA)
- Progress indicator condenses to numbered dots on very small screens (< 360 px)

Data Required

- `GET /api/barbers/:slug/services` → active service groups with pricing and duration

- Booking session state (from URL query param or in-memory state store): pre-selected service ID if passed from CS-01

CS-03 — Date & Time Selection (Booking Step 2)

Attribute	Value
Screen ID	CS-03
Screen Name	Date & Time Selection
Route	`/b/:barberSlug/book` (step 2)

Purpose

Allows the customer to select a date and then choose from available time slots. Availability is computed based on the barber's working hours, existing bookings, blocked slots, and the duration of the selected service. This is the most data-intensive step in the customer flow.

User Type

- Anonymous customer (with service selection carried forward from step 1)

Entry Points

- "Continue" from CS-02 (Service Selection)
- Browser back from CS-04 (Customer Details), returning to a previously selected date/time

Key UI Elements

Compact Sub-header

- Progress bar (step 2 active)
- Selected service name + duration chip as a reminder of what they're booking

Date Picker / Calendar Strip

- Horizontal scrollable date strip showing the next 30 days
- Each date cell shows: day of week (short), day number
- Unavailable dates (barber closed, fully booked) are visually dimmed and non-tappable
- Selected date is highlighted with brand color
- A "full month calendar" toggle is available for users who prefer a grid view
- Today is always the first option; past dates are not shown

Time Slot Grid

- Rendered after a date is selected
- Slots displayed in a two-column grid (AM / PM split, or simply chronological)
- Each slot shows the start time (e.g., "2:30 PM")
- Unavailable slots are greyed out and non-interactive
- Available slots are tappable; selected slot gets a highlighted state
- Slot duration is implied by the service — end time is shown in smaller text below start time (e.g., "+ 45 min → 3:15 PM")

Timezone Notice

- Small inline note below the time grid: "Times shown in [barber's local timezone]" — avoids confusion for traveling customers

Continue CTA

- Disabled until both date and time are selected
- Label: "Continue" — confirms and moves to step 3

User Actions

- Scroll the date strip to find available dates
- Tap a date to load its time slots
- Toggle to full calendar view for a broader overview
- Tap a time slot to select it
- Tap "Continue" to proceed to CS-04
- Tap back (browser or back link) to return to CS-02

State Variations

State	Behavior
Loading dates	Skeleton date cells in the horizontal strip
Loading time slots	Spinner or skeleton grid beneath the date strip while slots are fetched for the selected date
No available dates (next 30 days fully booked)	Message: "No availability in the next 30 days. Check back soon or contact [barber name] directly."
Date selected, no slots available	Within the time grid: "No openings on this day. Please select a different date." Date strip remains active
Slot booked by someone else after selection	On step 3 → step 4 transition, a conflict check is run. If slot is taken, user is returned to step 2 with an error: "That slot was just taken. Please choose another time."
Returning from step 3 (back navigation)	Previously selected date and slot are restored from session state

Mobile Considerations

- Horizontal date strip is native-scroll friendly — no arrows required; momentum scroll works
- Time slot grid collapses to a single column on narrow screens
- Full calendar view uses a compact 7-column grid; tap area per date is minimum 44 × 44 px
- Timezone notice is deprioritized visually — small muted text below the slot grid

Data Required

- Service data from session state (duration required to calculate slot boundaries)
- `GET /api/barbers/:slug/availability?from=YYYY-MM-DD&to=YYYY-MM-DD&serviceId=...` → array of available dates with available slot times per date
- `GET /api/barbers/:slug/hours` → working hours for calendar rendering (closed day indicators)

CS-04 — Customer Details Form (Booking Step 3)

Attribute	Value
Screen ID	CS-04
Screen Name	Customer Details Form
Route	`/b/:barberSlug/book` (step 3)

Purpose

Collects the minimum customer information needed to complete a booking and send confirmation/reminder communications. The form must be as short as possible — every extra field increases drop-off. MVP collects name, phone, email, and an optional note to the barber.

User Type

- Anonymous customer

Entry Points

- "Continue" from CS-03 (Date & Time Selection)
- Browser back from CS-05 (Payment) if deposit is required

Key UI Elements

Compact Sub-header

- Progress bar (step 3 active)
- Booking summary chip: service name, selected date, selected time — always visible as a reminder

Customer Form Fields

- First Name — text input, required
- Last Name — text input, required
- Phone Number — tel input with country code prefix (default to barber's country), required; used for SMS reminders
- Email Address — email input, required; used for confirmation email and calendar invite
- Note to barber — optional multi-line textarea, max 280 characters; character counter shown below
- All required fields are marked with a subtle asterisk; no verbose "required" text

Terms Acknowledgment

- Checkbox: "I agree to the [Privacy Policy] and understand this booking may require a deposit."
- Links open in a modal or new tab — do not navigate away
- "Continue" is blocked until checkbox is checked

Auto-fill Hint

- Browser autocomplete attributes set correctly (`autocomplete="given-name"`, `family-name`, `tel`, `email`) to allow autofill — reduces friction significantly on mobile

Continue / Submit CTA

- Label: "Continue to Payment" if deposit is configured; "Confirm Booking" if no deposit
- Disabled until all required fields are valid

User Actions

- Fill in form fields
- Check terms checkbox
- Tap "Continue to Payment" → CS-05 (if deposit required)
- Tap "Confirm Booking" → skip CS-05 and go to CS-06 (if no deposit)
- Tap back to return to CS-03

State Variations

State	Behavior
Inline validation	Fields validate on blur (not on every keystroke). Errors shown inline below the field in red: e.g., "Enter a valid email address"
Phone validation	Phone number format validated per country code selection using a lightweight formatter
Submitting (no deposit)	"Confirm Booking" button shows a loading spinner and is disabled during API call
API error on submit	Inline banner: "Something went wrong. Your booking was not saved — please try again." Form state is preserved
Returning from step 4 (back nav)	All form values are restored from session state

Mobile Considerations

- Single-column form layout (no side-by-side fields on mobile)
- First name and last name appear on the same row on tablet and desktop (≥ 768 px) to reduce perceived length
- Keyboard type is set appropriately per field (`type="tel"`, `type="email"`) to trigger correct mobile keyboards
- The booking summary chip stays pinned just below the progress bar — always visible even while the keyboard is open
- "Continue" button is sticky at the bottom of the viewport on mobile to avoid scrolling past the form

Data Required

- Booking session state: service ID, date, time slot — needed to build the final booking payload
- `GET /api/barbers/:slug/config` → to determine if deposit is required and set the CTA label correctly
- No pre-fetch of customer data (anonymous flow)

CS-05 — Payment / Checkout Screen (Booking Step 4)

Attribute	Value
Screen ID	CS-05
Screen Name	Payment / Checkout
Route	`/b/:barberSlug/book` (step 4) — conditional, only rendered if barber has deposit enabled

Purpose

Collects a deposit payment from the customer to secure the booking. Reduces no-shows by requiring a financial commitment. This screen is conditionally rendered — if the barber has not configured a deposit, the flow jumps from CS-04 directly to CS-06.

User Type

- Anonymous customer who has completed steps 1–3

Entry Points

- "Continue to Payment" from CS-04 (Customer Details), only if deposit is configured

Key UI Elements

Compact Sub-header

- Progress bar (step 4 active)
- Booking summary: service name, date/time, customer name

Order Summary Card

- Service name
- Full service price
- Deposit amount due now (e.g., "Deposit: $20.00")
- Remaining balance note: "Remaining $X.XX due at appointment"
- Visual divider between deposit line and total

Payment Form — Stripe Elements

- Card number input (Stripe CardElement embedded — PCI-compliant iframe)
- Expiry date input (part of Stripe CardElement)
- CVC input (part of Stripe CardElement)
- Cardholder name field (custom input above the Stripe element)
- Stripe branding / "Powered by Stripe" badge below the form

Security Indicators

- Lock icon with "Secure payment" label
- SSL badge or text: "Your payment is encrypted"

Pay CTA

- Label: "Pay $[deposit amount] & Confirm Booking"
- Shows a loading spinner during Stripe tokenization and API call

User Actions

- Enter card details
- Tap "Pay & Confirm Booking" to submit payment
- Tap back to return to CS-04 (card details are cleared on return for security; form fields re-enter blank)

State Variations

State	Behavior
Loading Stripe Elements	Skeleton card input area while Stripe JS loads; fall-through message if Stripe fails to load
Processing payment	Pay button disabled, shows spinner; all form inputs disabled
Payment declined	Inline Stripe error shown below card input: e.g., "Your card was declined. Please try a different card." Booking is NOT created
Network error	Banner: "Payment could not be processed. Please check your connection and try again."
Payment successful	Booking record is created in backend; user is redirected to CS-06
Slot conflict on payment	If slot was taken while user was on this screen, error: "This time slot is no longer available. Your card was not charged." User is returned to CS-03

Mobile Considerations

- Stripe CardElement renders responsively within its container — no special treatment needed
- "Pay & Confirm" button is full-width and sticky at the bottom on mobile
- Cardholder name field appears above the Stripe element (no reordering on mobile)
- Order summary card collapses to a compact 2-line chip on very small screens to give more space to the payment form

Data Required

- Booking session state: service, date/time, customer details
- `GET /api/barbers/:slug/config` → deposit amount, currency
- Stripe publishable key (from platform config or per-barber Stripe Connect account)
- `POST /api/bookings` (on successful payment token) → creates booking and charges deposit via Stripe

---

CS-06 — Booking Confirmation Screen

Attribute	Value
Screen ID	CS-06
Screen Name	Booking Confirmation

Attribute	Value
Route	`/b/:barberSlug/confirmation?bookingId=...`

Purpose

Provides the customer with a clear, reassuring confirmation that their booking was successfully created. Delivers all the information they need to remember and keep the appointment, and offers actions to add it to their calendar or share the booking details.

User Type

- Anonymous customer who has just completed the booking flow

Entry Points

- Automatic redirect after a successful booking creation (from CS-04 or CS-05)
- Direct link from a confirmation email ("View your booking" link) — should display a read-only summary even if the session has expired

Key UI Elements

Success Hero

- Large checkmark icon (animated, brand color) — success microanimation
- Heading: "You're booked!" (friendly, confident)
- Subheading: "A confirmation has been sent to [customer email]"

Booking Summary Card

- Barber name + avatar
- Service name
- Date and time (formatted clearly: e.g., "Wednesday, April 8, 2026 at 2:30 PM")
- Duration
- Address or location label (if barber has provided one)
- Deposit paid (if applicable): "Deposit paid: $20.00" + "Balance due at appointment: $X.XX"
- Booking reference ID (short alphanumeric, e.g., "BB-4F7A") for customer's reference

Action Buttons

- "Add to Calendar" — generates and downloads an `.ics` file or opens Google/Apple Calendar link
- "Share Booking" — uses Web Share API on mobile; copies a summary to clipboard on desktop
- "Book Another Appointment" — returns to CS-01 (the barber's landing page) to start a new booking

Cancellation Info

- Brief paragraph: "Need to cancel? [Contact barber name] at [phone/email]" or a self-serve cancel link if that feature is enabled in config

User Actions

- Download / add to calendar
- Share booking details
- Navigate back to barber landing page

- Close tab (common; no forced action)

State Variations

State	Behavior
Arrived via direct link (email)	Fetches booking by `bookingId` from API; renders same summary card in read-only mode
Booking not found	Message: "We couldn't find your booking. Please contact your barber directly."
Loading booking details	Skeleton summary card
Cancellation enabled	"Cancel Booking" link appears beneath the summary card; tapping opens a confirmation dialog before cancelling

Mobile Considerations

- Full-screen success hero fills the viewport on first render
- Booking summary card is scrollable if content overflows
- "Add to Calendar" uses native iOS/Android calendar integration via the Web Share API where supported
- Action buttons stack vertically on mobile, horizontal on desktop
- Booking reference ID is displayed in a monospace chip that is easy to tap and copy

Data Required

- `GET /api/bookings/:bookingId` → full booking record: service, datetime, barber info, customer name, deposit info, cancellation policy
- Booking reference ID for display

Barber Screens

Barber screens are the authenticated management interface used exclusively by the barber (shop owner). Access requires login. All barber screens share a persistent sidebar navigation on desktop and a bottom tab bar on mobile. The barber's own branding (logo, brand color) is reflected in the sidebar header.

BS-01 — Barber Login

Attribute	Value
Screen ID	BS-01
Screen Name	Barber Login
Route	`/barber/login`

Purpose

Authenticates the barber to access their management dashboard. Must be simple and fast — barbers are often logging in quickly between clients on a phone. Supports email/password authentication and optionally magic link (passwordless) for easier mobile access.

User Type

- Barber (account holder)

Entry Points

- Direct URL navigation
- Redirect from any protected `/barber/*` route (with `?redirect=` param preserved)
- "Barber Login" link from the BarberBook marketing site or admin-provided onboarding email

Key UI Elements

BarberBook Logo / Brand Mark

- Centered at top; links to BarberBook marketing site (not the barber's page)

Login Form

- Email address field (required)
- Password field with show/hide toggle (required for password auth)
- "Forgot password?" link — triggers a password reset email
- Login button — primary CTA

Magic Link Option (if enabled)

- Secondary button: "Send me a login link" — sends a one-click email login link

Footer

- "Not a barber? Book an appointment" — links to BarberBook's landing/home page
- Privacy policy link

User Actions

- Enter email and password and submit
- Request a magic link via email
- Click "Forgot password?" to initiate reset

State Variations

State	Behavior
Submitting	Login button disabled, spinner shown
Invalid credentials	Inline error below the form: "Incorrect email or password." No specific field highlighting (avoids username enumeration)
Account suspended	Error: "Your account has been suspended. Contact support."

State	Behavior
Magic link sent	Form replaced by: "Check your email — we sent a login link to [email]."
Already logged in	Redirect to `/barber/dashboard` immediately

Mobile Considerations

- Form is centered in a card on desktop; full-screen flush on mobile with generous padding
- Password field with show/hide is especially important on mobile due to autocorrect interference
- "Login" button is full-width on mobile
- Autofill is supported (`autocomplete="email"` and `autocomplete="current-password"`)

Data Required

- `POST /api/auth/barber/login` → JWT session token on success
- `POST /api/auth/barber/magic-link` → triggers email

BS-02 — Barber Dashboard

Attribute	Value
Screen ID	BS-02
Screen Name	Barber Dashboard
Route	`/barber/dashboard`

Purpose

The first screen a barber sees after logging in. Provides a high-level snapshot of today's schedule, recent activity, key metrics, and quick actions. Designed to answer "What do I need to know right now?" in under 5 seconds.

User Type

- Authenticated barber

Entry Points

- Successful login (redirect from BS-01)
- Clicking the "Dashboard" link in the sidebar
- Root redirect: `/barber` → `/barber/dashboard`

Key UI Elements

Top Bar (mobile) / Sidebar Header (desktop)

- Barber avatar + business name
- Notification bell icon with unread badge
- Quick "Share Booking Link" icon button — copies the public booking URL to clipboard

Today at a Glance Card

- Date: "Today, [Day], [Date]"
- Number of bookings today (e.g., "6 appointments")
- First upcoming appointment details: client name, service, time
- "View today's schedule" → links to BS-03 with today's date pre-selected

Upcoming Bookings List (next 3–5)

- Compact rows: time, client name, service name, duration chip
- Each row tappable → opens BS-04 (Booking Detail)
- "View all bookings" link at the bottom

Stats Summary Cards (current week)

- Total bookings this week
- Revenue this week (if payment integration active)
- New clients this week
- These are small metric tiles, not detailed charts

Quick Actions Panel

- "Block time off" → shortcut to add a blocked slot via a compact modal
- "Copy booking link" → copies `/b/:barberSlug` to clipboard
- "Add service" → jumps to BS-08 (Service Create Form)

Recent Activity Feed

- Last 5–10 events: new booking made, cancellation, client note added, etc.
- Each event shows timestamp and brief description

User Actions

- Tap upcoming booking row → view booking detail
- Tap "View today's schedule" → navigate to calendar screen for today
- Tap "Block time off" → open block-time modal
- Tap "Copy booking link" → copy URL (with toast confirmation)
- Tap stat card → navigate to the relevant detail screen

State Variations

State	Behavior
No bookings today	Today card shows: "No appointments today — enjoy your day!"
No bookings this week	Stats cards show zeros; activity feed shows empty state with prompt to share booking link
Loading	Skeleton cards for each section
First-time login (new barber)	Onboarding banner at top: "Finish setting up your profile → [Add Services] [Set Your Hours]" — dismissible once complete

Mobile Considerations

- Dashboard stacks all cards vertically in a single column

- Stats tiles render in a 2×2 grid on mobile (vs. 4-column row on desktop)
- "Quick Actions" panel becomes a horizontal scrollable chip row on mobile
- Bottom tab bar replaces the sidebar for navigation

Data Required

- `GET /api/barbers/me/bookings?date=today&limit=5` → today's upcoming bookings
- `GET /api/barbers/me/stats?range=this_week` → weekly stats aggregate
- `GET /api/barbers/me/activity?limit=10` → recent activity events

BS-03 — Calendar / Bookings Screen

Attribute	Value
Screen ID	BS-03
Screen Name	Calendar / Bookings Screen
Route	`/barber/bookings`

Purpose

The primary operational screen for the barber — shows all bookings in a calendar or list format. Used daily to review the schedule, find specific bookings, and manage the day's flow. Supports both a visual calendar view (for overview) and a list view (for quick scanning).

User Type

- Authenticated barber

Entry Points

- Sidebar "Bookings" navigation link
- "View today's schedule" from BS-02 Dashboard
- "View all bookings" from BS-02 Dashboard

Key UI Elements

View Toggle

- Tab or segmented control: "Calendar" | "List"
- Persisted in localStorage per barber (remembers last-used view)

Calendar View

- Week view by default (Monday–Sunday)
- Day view selectable via the date header tap
- Time axis on the left (hourly increments)
- Booking blocks rendered as colored cards within their time slots: client name (truncated), service name chip
- Blocked time slots rendered as a hatched/grey band
- Today's column highlighted with brand color background tint
- Navigation arrows (prev/next week or day)

- "Today" button to jump back to current date

List View

- Date-grouped rows, starting from today (or the selected filter date)
- Each booking row: time, client avatar (initials fallback), client name, service, duration, status badge (Confirmed, Pending, Cancelled, No-show)
- Tap any row → BS-04 (Booking Detail)

Filter / Search Bar

- Search by client name or booking reference (in list view only)
- Filter dropdown: All / Confirmed / Pending / Cancelled / No-show
- Date range picker (from → to) to jump to a different period

"+ New Block" Floating Button

- Opens a modal to block a time range (manual block, holiday, etc.)
- Not a new booking — bookings are customer-initiated only in MVP

User Actions

- Switch between calendar and list view
- Navigate to previous/next week or day
- Tap a booking block/row → open BS-04 (Booking Detail)
- Search for a client or booking
- Apply filters
- Block a time slot via the modal

State Variations

State	Behavior
Loading	Calendar grid renders with skeleton booking blocks; list view shows skeleton rows
No bookings in range	Calendar view shows an empty grid (normal behavior — slots simply have no cards); list view shows: "No bookings found for this period."
Cancelled bookings	Shown in list with a strikethrough and "Cancelled" badge; hidden from calendar view by default (toggle available to show them)
Past bookings	In list view, past bookings are visually dimmed; in calendar view, past days are desaturated

Mobile Considerations

- Default to list view on mobile (calendar week view is unusable on small screens)
- Day view is available on mobile in place of week view
- Date navigation uses a horizontally scrollable date strip (same component as CS-03)
- Booking rows in list view are minimum 64 px height for comfortable tapping
- Filter/search bar is collapsed into a filter icon button on mobile — expands as a bottom drawer

Data Required

- `GET /api/barbers/me/bookings?from=...&to=...&status=...` → paginated bookings in range with client, service, and time data
- `GET /api/barbers/me/blocked-slots?from=...&to=...` → blocked time windows for calendar rendering

BS-04 — Booking Detail Modal / View

Attribute	Value
Screen ID	BS-04
Screen Name	Booking Detail
Route	Modal overlay from `/barber/bookings` or `/barber/dashboard`; deep-linkable at `/barber/bookings/:bookingId`

Purpose

Shows the full detail of a single booking and allows the barber to take action on it: confirm, mark as no-show, add a note, or initiate cancellation. This is the barber's primary interaction point with individual appointments.

User Type

- Authenticated barber

Entry Points

- Tapping a booking row or calendar block in BS-03
- Tapping an upcoming booking row in BS-02 Dashboard
- Direct URL `/barber/bookings/:bookingId` (e.g., from a notification link)

Key UI Elements

Booking Header

- Client name (large)
- Booking reference (small, copyable)
- Status badge (Confirmed / Pending / Cancelled / No-show / Completed)
- Date and time (prominent)
- Duration and end time

Service Details Section

- Service name
- Service price (full price)
- Deposit paid (if any)
- Balance due at appointment

Client Info Section

- Phone number (tappable → opens dialer on mobile)
- Email address (tappable → opens mail client)

- Client note to barber (if submitted during booking)
- Link: "View full client profile" → CS-06 (Client Detail)

Barber Notes Section

- Free-text textarea for internal notes (private to barber; never shown to customer)
- Auto-saves on blur

Action Buttons

- Mark as Confirmed — transitions status from Pending to Confirmed (sends confirmation SMS/email)
- Mark as No-show — transitions status; can trigger deposit forfeit if configured
- Cancel Booking — opens a confirmation dialog; triggers cancellation email/SMS
- Reschedule — opens a date/time picker modal to move the booking (generates new confirmation)

User Actions

- Read booking details
- Tap phone number to call client
- Tap email to compose email
- Add/edit barber note
- Mark as confirmed, no-show, or cancel
- Reschedule booking

State Variations

State	Behavior
Pending status	"Mark as Confirmed" is the primary action button (most prominent)
Already confirmed	"Mark as Confirmed" is hidden; status badge shows "Confirmed" in green
Cancelled booking	All action buttons hidden except "View Client Profile"; a read-only cancelled banner is shown
Completed booking	Actions limited to viewing notes; "No-show" action is no longer available
Note saving	"Saving…" micro-text appears next to the notes textarea; replaced by "Saved" checkmark on success

Mobile Considerations

- Renders as a full-screen page on mobile (not a modal — modal overlays don't work well on small screens)
- Renders as a right-side slide-in panel on tablet and desktop (keeping the calendar visible behind it)
- Phone and email tappable links are especially important for mobile — placed prominently
- Action buttons are stacked at the bottom of the page on mobile in a sticky action bar

Data Required

- `GET /api/bookings/:bookingId` → full booking record including service, client, payment, and notes
- `PUT /api/bookings/:bookingId/status` → for status updates
- `PUT /api/bookings/:bookingId/notes` → for saving barber note
- `PUT /api/bookings/:bookingId/reschedule` → for rescheduling

BS-05 — Clients Screen

Attribute	Value
Screen ID	BS-05
Screen Name	Clients Screen
Route	`/barber/clients`

Purpose

Provides the barber with a searchable, filterable list of all clients who have ever booked with them. Serves as a simple CRM — the barber can look up a client, view their history, and add notes. Supports relationship-based barbering (remembering what each client likes).

User Type

- Authenticated barber

Entry Points

- Sidebar "Clients" navigation link
- "View full client profile" link from BS-04 (Booking Detail)

Key UI Elements

Search Bar

- Prominent at top; searches client name or phone number in real time (debounced)
- Placeholder: "Search clients by name or phone…"

Filter Options

- Sort by: Most recent booking | Name A–Z | Total bookings (most loyal)
- Filter by: All clients | New this month | Returning | Flagged (if barber has flagged a client)

Client List

- Each row: client avatar (initials circle with brand color), client name, last booking date, total booking count (e.g., "8 visits")
- Tap row → BS-06 (Client Detail View)
- Subtle dividers between rows

Client Count Summary

- Small text above the list: "X clients total"

Empty State

- First-time state (no bookings yet): "Your clients will appear here once bookings come in. Share your booking link to get started."
- After search/filter with no results: "No clients match your search."

User Actions

- Search for a client by name or phone
- Sort and filter the client list
- Tap a client row to view their detail
- (Future MVP+: export client list — not in MVP)

State Variations

State	Behavior
Loading	Skeleton rows during data fetch
Empty (no clients)	Full empty-state illustration with prompt to share booking link
Search — no results	Filtered empty state with "No clients match" message
Offline	Banner: "You're offline. Client list may be outdated."

Mobile Considerations

- Single-column list; full-width rows
- Search bar is sticky at the top (does not scroll away)
- Sort/filter options are accessible via a bottom sheet triggered by a "Filter" chip
- Client initials avatar is 40 × 40 px on mobile

Data Required

- `GET /api/barbers/me/clients?search=...&sort=...&filter=...` → paginated client list with last booking date and visit count

---

BS-06 — Client Detail View

Attribute	Value
Screen ID	BS-06
Screen Name	Client Detail View
Route	`/barber/clients/:clientId`

Purpose

A complete view of a single client's relationship with the barber. Shows contact info, all past and upcoming bookings, notes, and aggregate stats. Helps the barber build a personal connection and provide better service.

User Type

- Authenticated barber

Entry Points

- Tapping a client row in BS-05 (Clients Screen)
- "View full client profile" link from BS-04 (Booking Detail)

Key UI Elements

Client Header

- Large initials avatar or uploaded photo (if platform supports photo collection in future)
- Client full name (H1)
- Member since date (date of first booking)
- Phone number (tappable → dialer)
- Email address (tappable → mail client)

Stats Row

- Total visits
- Total spend (if payment integration active)
- Last visit date
- Average service (most frequently booked service name)

Barber's Private Notes

- Free-text area for the barber's private notes about this client (e.g., "Prefers tapered fade, sensitive to clippers on neck")
- Auto-saves on blur
- Never visible to the client

Booking History

- Chronological list (most recent first) of all bookings
- Each row: date, service name, status badge, amount paid
- Tap any row → BS-04 (Booking Detail for that booking)
- "Upcoming" section appears at top if the client has future bookings

Quick Action Bar

- "Call [client name]" — opens dialer
- "Text [client name]" — opens SMS app with the barber's booking link pre-filled (optional helper)

User Actions

- Read client info
- Add/edit private notes about the client
- Tap a past booking to view its detail
- Call or text the client directly
- Navigate back to client list

State Variations

State	Behavior
Loading	Skeleton for header and stats row
No past bookings	Booking history section shows: "No past bookings found." (edge case — client records are auto-created on first booking)
Note saving	Same "Saving…" / "Saved" micro-indicator as BS-04

Mobile Considerations

- Stats row renders as a 2×2 grid on mobile (vs. 4-column row on desktop)
- Notes textarea is full-width and expands vertically as the barber types
- Booking history uses the same compact row format as BS-03 list view for consistency

Data Required

- `GET /api/clients/:clientId` → client contact info, creation date, stats
- `GET /api/clients/:clientId/bookings` → full booking history
- `GET /api/clients/:clientId/notes` → barber's private notes (barber-scoped)
- `PUT /api/clients/:clientId/notes` → save notes

BS-07 — Services Screen

Attribute	Value
Screen ID	BS-07
Screen Name	Services Screen
Route	`/barber/services`

Purpose

Allows the barber to manage the menu of services they offer — creating, editing, reordering, and toggling the visibility of service groups. This screen directly controls what customers see on the booking landing page (CS-01, CS-02).

User Type

- Authenticated barber

Entry Points

- Sidebar "Services" navigation link
- "Add service" quick action from BS-02 Dashboard
- Onboarding banner prompt

Key UI Elements

Services List

- Each service is shown as a card: service name (large), duration badge, price, toggle switch (active/inactive)
- Optional: group label if services are organized under categories
- Drag handle on each card for reordering (desktop: drag-and-drop; mobile: up/down arrow buttons)
- "Edit" button on each card → opens BS-08 (Service Edit Form)
- "Delete" icon (with confirmation dialog) on each card

Service Group Headers (if groups are enabled)

- Expandable/collapsible group sections
- "Add service to group" button within each group
- "Add new group" button at the bottom

"Add Service" CTA

- Floating action button (FAB) or top-right button → opens BS-08 in create mode

Active / Inactive Toggle

- Services set to inactive are greyed out and not shown on the public booking page
- Toggle is inline — no page navigation needed to hide a service

User Actions

- Tap "Add Service" → navigate to BS-08 (create mode)
- Tap "Edit" on a service card → navigate to BS-08 (edit mode)
- Toggle service active/inactive
- Reorder services
- Delete a service (confirmation dialog required)

State Variations

State	Behavior
No services yet	Empty state with illustration: "You haven't added any services yet. Add your first service to start accepting bookings." + "Add Service" button
Loading	Skeleton service cards
All services inactive	Warning banner: "All your services are currently hidden. Customers cannot book. Enable at least one service."
Reordering mode (mobile)	Arrow buttons appear; drag handles are hidden on touch devices

Mobile Considerations

- Service cards stack vertically, full-width
- Toggle is on the right side of each card — minimum 44 × 44 px touch area
- Reorder arrows (↑ ↓) replace drag-and-drop on mobile for reliable touch interaction
- "Add Service" FAB is fixed in the bottom-right corner

Data Required

- `GET /api/barbers/me/services` → full list of service groups with all fields and isActive status
- `PATCH /api/services/:serviceId` → for inline toggle / reorder updates

BS-08 — Service Edit / Create Form

Attribute	Value
Screen ID	BS-08
Screen Name	Service Edit / Create Form
Route	`/barber/services/new` (create) or `/barber/services/:serviceId/edit` (edit)

Purpose

A focused form for creating a new service or editing an existing one. Contains all the fields needed to define a service that customers can book. Validates inputs before saving.

User Type

- Authenticated barber

Entry Points

- "Add Service" button or FAB on BS-07 (Services Screen)
- "Edit" button on a service card in BS-07

Key UI Elements

Form Fields

- Service Name — text input, required (max 60 characters, character counter shown)
- Description — optional multi-line textarea (max 140 characters) — shown on customer booking page
- Duration — a select or stepper input, required; options in 15-minute increments (15 min, 30 min, 45 min, 1 hr, 1 hr 15 min, up to 3 hr)
- Price — numeric currency input, required; currency derived from barber's settings
- Service Group / Category — select from existing groups or create a new group inline; optional
- Active — toggle switch (defaults to active for new services)
- Display Order — numeric field (auto-set to last position; barber can override)

Form Header

- "New Service" or "Edit Service — [Name]" as the page title
- Back link to BS-07

Save / Cancel Actions

- "Save Service" button — primary CTA
- "Cancel" — returns to BS-07 without saving
- "Delete Service" — destructive action, shown only in edit mode; triggers a confirmation dialog

User Actions

- Fill in all service details
- Tap "Save Service" to create or update
- Tap "Cancel" to discard changes
- Tap "Delete Service" (edit mode only) to remove the service

State Variations

State	Behavior
Saving	Save button shows spinner; all inputs disabled
Validation errors	Inline errors below fields on attempted save (name required, price must be > 0, duration required)
Saved successfully	Redirect to BS-07; success toast: "Service saved."
Delete confirmation	Modal dialog: "Delete [service name]? Any existing bookings for this service will not be affected." Confirm / Cancel buttons
Edit — pre-filled	All fields pre-populated with existing service data

Mobile Considerations

- Single-column form layout
- Duration stepper renders as a native `<select>` on mobile for system-native picker behavior
- "Save Service" button is sticky at the bottom on mobile
- Character counters update in real time below each limited-length field

Data Required

- `GET /api/services/:serviceId` (edit mode only) → existing service data for pre-fill
- `GET /api/barbers/me/service-groups` → list of existing groups for the group select dropdown
- `POST /api/services` (create) or `PUT /api/services/:serviceId` (update)
- `DELETE /api/services/:serviceId` (delete)

BS-09 — Settings Screen

Attribute	Value
Screen ID	BS-09
Screen Name	Settings Screen
Route	`/barber/settings`

Purpose

Allows the barber to configure all aspects of their profile, availability, notification preferences, and branding. The settings screen is divided into clearly separated sections accessible via an

in-page tab or anchor navigation.

User Type

- Authenticated barber

Entry Points

- Sidebar "Settings" navigation link (often represented by a gear icon)
- Onboarding banner links ("Set your hours" → Settings > Hours)

Key UI Elements

Settings Sections (tab or anchor-nav)

1. Profile

- Business name (text input)
- Barber's own name / display name
- Bio / tagline (max 200 characters — shown on customer landing page)
- Avatar / logo upload (drag-and-drop on desktop, camera/gallery on mobile)
- Location label (free text — e.g., "Toronto, ON" — not a precise address)
- Social media links (Instagram, TikTok — optional, max 2 in MVP)
- Public booking URL display: `barberbook.app/b/[slug]` with copy button

2. Brand Color

- Color picker (hex input + visual swatch picker) to set the brand color used on the customer-facing pages
- Preview: small mockup showing how the color appears on the CTA button and highlights

3. Working Hours

- Weekly availability grid: for each day of the week, a toggle (open/closed) + start time and end time selectors
- Ability to set different hours per day
- "Copy to all weekdays" convenience button

4. Booking Rules

- Minimum advance notice (e.g., "Clients must book at least 2 hours in advance")
- Maximum advance booking window (e.g., "Clients can book up to 60 days ahead")
- Buffer time between appointments (e.g., "15-minute gap between bookings")

5. Notifications

- SMS reminders toggle (requires verified phone)
- Email reminders toggle
- Reminder timing: "Remind clients X hours before appointment" (dropdown: 2, 4, 12, 24, 48 hours)
- Notify barber on new booking: toggle + preferred channel (email / SMS)

6. Account

- Email address (current, with "Change email" flow)
- Password change form
- Logout button
- "Delete account" — destructive, behind a confirmation modal

Save Button

- Per-section save (not a single global save) to avoid losing work across sections

User Actions

- Edit and save any settings section
- Upload avatar/logo image
- Select brand color
- Configure weekly hours
- Toggle notification preferences
- Change email or password
- Log out

State Variations

State	Behavior
Saving	Section save button shows spinner
Saved	Toast: "Settings saved."
Upload in progress	Avatar area shows a circular progress indicator
Hours conflict (end < start)	Inline error: "End time must be after start time."
No phone for SMS	SMS toggle is greyed out with tooltip: "Add a phone number to enable SMS reminders."

Mobile Considerations

- Settings sections are stacked and accessible via a vertical list with anchor links (no tabs on mobile — too cramped)
- Working hours grid collapses to an accordion (each day is an expandable row) on mobile
- Color picker uses a simplified swatch grid on mobile rather than a full color wheel
- Avatar upload triggers the native camera/file picker on mobile

Data Required

- `GET /api/barbers/me/settings` → full barber profile, hours, booking rules, notification preferences
- `PUT /api/barbers/me/settings` → saves any settings section
- `POST /api/barbers/me/logo` → image upload (multipart form)

Admin Screens

Admin screens are a restricted management console used only by the platform operator (the BarberBook team or designated administrators). Admin access is entirely separate from barber access — different auth token scope, different routes, different visual theme (neutral / system palette rather than branded).

AS-01 — Admin Login

Attribute	Value
Screen ID	AS-01
Screen Name	Admin Login
Route	`/admin/login`

Purpose

Authenticates platform administrators with elevated security requirements. Admin login is intentionally not discoverable from the public site — accessed only by direct URL.

User Type

- Platform administrator

Entry Points

- Direct URL navigation to `/admin/login`
- Redirect from any protected `/admin/*` route

Key UI Elements

Minimal Admin Branding

- "BarberBook Admin" wordmark (no public-facing branding)
- Neutral / dark color scheme to visually distinguish from the barber portal

Login Form

- Email address field
- Password field with show/hide toggle
- Two-Factor Authentication (2FA) Code field — appears as a second step after valid email/password (TOTP or SMS code)
- Login button

Security Notice

- "Authorized personnel only. Access is logged." — brief legal deterrent

User Actions

- Enter email and password
- Enter 2FA code (second step)
- Submit login

State Variations

State	Behavior
Step 1: credentials	Email + password form shown

State	Behavior
Step 2: 2FA	Password form replaced by a 6-digit code input with a 30-second countdown and "Resend code" option
Invalid credentials	Generic error: "Login failed. Check your credentials." (no enumeration)
2FA failed	"Incorrect code. Try again." with retry count limit
Account locked	"Too many failed attempts. Contact system admin."
Already logged in	Redirect to `/admin/dashboard`

Mobile Considerations

- Full-screen centered card on all screen sizes
- 2FA code input uses `type="number"` with `inputmode="numeric"` for numeric keyboard on mobile

Data Required

- `POST /api/auth/admin/login` → step 1 credentials check
- `POST /api/auth/admin/2fa/verify` → step 2 TOTP/SMS verification

AS-02 — Admin Dashboard

Attribute	Value
Screen ID	AS-02
Screen Name	Admin Dashboard
Route	`/admin/dashboard`

Purpose

Gives the platform operator a real-time overview of platform health, growth, and activity. Answers: How many barbers are active? How many bookings happened today/this week? Are there any issues?

User Type

- Authenticated platform administrator

Entry Points

- Successful admin login (redirect from AS-01)
- "Dashboard" link in admin sidebar
- Root admin redirect: `/admin` → `/admin/dashboard`

Key UI Elements

Top-Line KPI Tiles (row)

- Total active barbers
- Total bookings this month
- Total platform revenue this month (if payment data is available)
- New barbers this week
- System status indicator (green / amber / red) — links to AS-07 (Monitoring)

Bookings Over Time Chart

- Line chart: daily bookings for the last 30 days
- Hoverable data points showing exact counts
- Simple, clean — no complex analytics

Recent Barber Signups

- Table: barber name, email, signup date, first booking date (or "No bookings yet"), status badge
- Link to full barbers management: AS-03

Recent Bookings Feed

- Last 10–20 bookings platform-wide: booking ID, barber slug, client name (hashed/anonymized for privacy), service, datetime, status

Active Alerts

- Any system-level alerts (see AS-07) surfaced here as dismissible banner cards

User Actions

- Click a KPI tile to navigate to the relevant detail screen
- Click a barber name in the signups table to navigate to their detail (AS-03)
- Dismiss an alert
- Navigate via sidebar to any admin section

State Variations

State	Behavior
Loading	Skeleton KPI tiles and skeleton chart
No barbers yet	KPIs show zeros; "Get started" prompt to create the first barber account
No bookings	Chart is flat/empty; recent bookings feed shows empty state

Mobile Considerations

- KPI tiles wrap to 2×2 grid on mobile
- Chart is hidden on very small screens (< 400 px) — replaced by a simple "X bookings this month" stat
- Admin console is primarily desktop-use; mobile support is functional but not the primary target

Data Required

- `GET /api/admin/stats` → platform-wide KPI aggregates

- `GET /api/admin/bookings/recent?limit=20` → recent bookings feed
- `GET /api/admin/barbers/recent?limit=10` → recent barber signups
- `GET /api/admin/alerts` → active system alerts

AS-03 — Barbers Management

Attribute	Value
Screen ID	AS-03
Screen Name	Barbers Management
Route	`/admin/barbers`

Purpose

The central directory of all barber accounts on the platform. Allows admins to search, view, and manage barber accounts — including suspending, activating, and drilling into individual barber details.

User Type

- Authenticated platform administrator

Entry Points

- Admin sidebar "Barbers" link
- KPI tile on AS-02 Dashboard
- "Recent signups" table on AS-02

Key UI Elements

Summary Strip

- Total barbers, active barbers, suspended barbers — small counts above the table

Search & Filter Bar

- Search by barber name, email, or slug
- Filter: All | Active | Inactive | Suspended | Pending setup

Barbers Table

- Columns: Barber Name | Slug (link to public page) | Email | Status | Signup Date | Total Bookings | Last Active | Actions
- Actions per row: "View" (→ barber detail overlay), "Suspend" / "Activate", "Delete" (with confirmation)
- Rows are sortable by column header click

"+ New Barber" Button

- Top-right; navigates to AS-04 (Barber Setup / Creation)

Pagination

- Standard pagination controls at the bottom; 25 rows per page default

User Actions

- Search and filter barbers
- View a barber's detail
- Suspend or activate a barber account
- Delete a barber account (with confirmation)
- Navigate to create a new barber account

State Variations

State	Behavior
Loading	Skeleton table rows
No barbers	Empty state with "Create your first barber account" CTA
Search — no results	"No barbers match your search."
Suspend confirmation	Inline confirm dialog: "Suspend [name]? Their booking page will become unavailable to customers."

Mobile Considerations

- Table collapses to card list on mobile (each barber is a card with name, status badge, and action buttons)
- Sort and filter accessible via bottom sheet on mobile

Data Required

- `GET /api/admin/barbers?search=...&filter=...&page=...` → paginated barber list with all table fields
- `PATCH /api/admin/barbers/:barberId/status` → suspend/activate
- `DELETE /api/admin/barbers/:barberId` → delete

AS-04 — Barber Setup / Creation

Attribute	Value
Screen ID	AS-04
Screen Name	Barber Setup / Creation
Route	`/admin/barbers/new`

Purpose

Allows an admin to provision a new barber account on the platform. The admin sets up the barber's credentials and basic profile; the barber then completes their full setup (services, hours, branding) via the barber portal on first login.

User Type

- Authenticated platform administrator

Entry Points

- "+ New Barber" button on AS-03 (Barbers Management)

Key UI Elements

Creation Form Fields

- Barber / Business Name — required; becomes the H1 on their public landing page
- Booking Slug — required; auto-generated from business name (editable); validated for uniqueness in real time
- Email Address — required; used for login and notifications
- Temporary Password — auto-generated (shown once, copyable) or admin-set
- Plan / Tier — select from platform's plans (MVP: single plan or "Free trial")
- Send Onboarding Email — checkbox (default checked); sends credentials + setup instructions to the barber

Slug Preview

- Live preview: "Booking page: `barberbook.app/b/[slug]`" updates as the barber types their name or edits the slug
- Green checkmark or red X indicating slug availability (debounced API check)

Save Button

- "Create Barber Account" — primary CTA
- "Cancel" — returns to AS-03

User Actions

- Fill in barber details
- Accept or edit the auto-generated slug
- Choose whether to send an onboarding email
- Create the account

State Variations

State	Behavior
Slug taken	Inline error under slug field: "This slug is already in use. Try [suggestion]."
Slug available	Green checkmark inline
Saving	Button spinner; inputs disabled
Created successfully	Redirect to AS-03 with success toast: "Barber account created. Onboarding email sent."
Email send failed	Account is still created; warning toast: "Account created but onboarding email failed to send. Copy credentials manually."

Mobile Considerations

- Single-column form; full-width inputs
- Slug availability indicator updates visually without a separate "Check" button
- Generated password is displayed in a monospace chip with a copy button

Data Required

- `GET /api/admin/slugs/check?slug=...` → slug availability check
- `POST /api/admin/barbers` → creates the barber account and triggers the onboarding email

---

AS-05 — Branding & Config

Attribute	Value
Screen ID	AS-05
Screen Name	Branding & Config
Route	`/admin/config`

Purpose

Platform-level configuration screen. Controls global defaults for the BarberBook platform — default branding elements, booking rules defaults, feature flags, and platform-wide settings that apply unless overridden at the barber level.

User Type

- Authenticated platform administrator

Entry Points

- Admin sidebar "Config" navigation link

Key UI Elements

Platform Identity Section

- Platform name (e.g., "BarberBook") — used in email footers, legal pages
- Platform logo upload — used in admin UI and system emails
- Support email address — shown in system notifications and error pages
- Platform domain / base URL — for generating booking links

Default Booking Rules

- Default minimum advance notice (hours)
- Default maximum booking window (days)
- Default buffer time between appointments (minutes)
- These are defaults applied to new barber accounts; each barber can override in their own settings

Feature Flags

- Enable/disable magic link login for barbers
- Enable/disable self-serve booking cancellation by customers
- Enable/disable client testimonials / reviews (not in MVP — toggle for future)

- Enable/disable the "Powered by BarberBook" attribution on customer pages

Legal Links

- Privacy policy URL
- Terms of service URL
- These are injected into the customer-facing footer on all booking pages

Save Button

- Platform-wide save; changes take effect immediately for all barbers

User Actions

- Update platform identity settings
- Adjust default booking rules
- Toggle feature flags
- Update legal link URLs
- Save changes

State Variations

State	Behavior
Saving	Save button spinner; form locked
Saved	Toast: "Platform config updated."
Invalid URL (legal links)	Inline error: "Enter a valid URL including https://"

Mobile Considerations

- Settings are stacked in a single column with clear section headers
- Feature flag toggles have minimum 44 px touch target height

Data Required

- `GET /api/admin/config` → current platform configuration
- `PUT /api/admin/config` → save updated config

AS-06 — Payment & Reminders Config

Attribute	Value
Screen ID	AS-06
Screen Name	Payment & Reminders Config
Route	`/admin/config/payments` (payment tab) and `/admin/config/reminders` (reminders tab)

Purpose

Controls the platform-level payment integration settings (Stripe configuration, deposit rules) and reminder/notification infrastructure (SMS provider, email provider, default reminder schedules). These are operator-level settings — not editable by individual barbers except where explicitly enabled.

User Type

- Authenticated platform administrator

Entry Points

- Sub-navigation within AS-05 (Config section) — "Payments" and "Reminders" tabs
- Direct URL navigation

Key UI Elements

Payments Tab

Stripe Integration

- Stripe mode toggle: Test / Live (prominently labeled — switching to Live enables real charges)
- Stripe publishable key input (shown masked except last 4 chars once saved)
- Stripe secret key input (write-only — stored encrypted; never displayed after save)
- Stripe webhook endpoint URL (read-only, auto-generated): for admin to copy into Stripe dashboard
- "Test Connection" button — calls Stripe API to verify keys are valid

Deposit Rules Defaults

- Enable deposits platform-wide toggle
- Default deposit amount (flat amount or percentage toggle)
- Default deposit value (e.g., "$20" or "25%")
- Refund policy label (free text, max 200 characters — shown to customers on the checkout screen)
- Barbers can override deposit settings in their own settings (if platform allows)

Stripe Connect (for per-barber payouts — future; toggle to enable)

- MVP: platform collects all payments; Stripe Connect is disabled by default

Reminders Tab

SMS Provider

- Provider select: Twilio (MVP default)
- Twilio account SID (masked)
- Twilio auth token (write-only)
- Twilio sender phone number
- "Test SMS" button — sends a test message to the admin's phone

Email Provider

- Provider select: SendGrid / AWS SES / SMTP
- API key / SMTP credentials (masked)
- From address (e.g., `noreply@barberbook.app`)
- From display name (e.g., "BarberBook")
- "Send Test Email" button

Default Reminder Schedule

- Customer booking confirmation: immediately on booking (always on)
- Customer reminder: X hours before (default 24h) — toggle + time selector
- Customer same-day reminder: morning of appointment — toggle
- Barber new booking notification: toggle + channel (email / SMS)
- Cancellation notifications: toggle (always recommended on)

User Actions

- Configure Stripe keys and test the connection
- Set deposit rules and refund policy
- Configure SMS and email providers and test them
- Set default reminder timings
- Save each section independently

State Variations

State	Behavior
Test Connection — Stripe	Loading spinner on button; success: "Connected — Stripe keys are valid." Error: "Connection failed — check your keys."
Test SMS	Phone prompt dialog → spinner → "Test message sent to [number]."
Test Email	Spinner → "Test email sent to [admin email]."
Live mode active	Prominent amber banner: "Live mode is active — real charges will be processed."
Saving	Per-section spinner; form locked during save
Keys already saved	Key fields show masked value (e.g., "sk_live_••••••••••••wxyz") with "Replace" button to re-enter

Mobile Considerations

- Tabs collapse to a dropdown or stepper on very small screens
- Masked key fields with "Copy" and "Replace" buttons are full-width on mobile

Data Required

- `GET /api/admin/config/payments` → current payment config (keys masked)
- `PUT /api/admin/config/payments` → save payment config
- `GET /api/admin/config/reminders` → current reminder config
- `PUT /api/admin/config/reminders` → save reminder config
- `POST /api/admin/config/payments/test` → Stripe connection test
- `POST /api/admin/config/reminders/test-sms` → test SMS send
- `POST /api/admin/config/reminders/test-email` → test email send

AS-07 — Monitoring Screen

Attribute	Value
Screen ID	AS-07
Screen Name	Monitoring Screen
Route	`/admin/monitoring`

Purpose

Provides the platform operator with a live health dashboard showing the operational status of all platform services. Surfaces errors, failed jobs, and anomalies so the team can respond proactively before issues affect barbers or customers.

User Type

- Authenticated platform administrator

Entry Points

- Admin sidebar "Monitoring" link
- System status indicator on AS-02 Dashboard (click → navigates here)
- Alert notification links

Key UI Elements

System Status Summary Banner

- Overall status: Operational (green) | Degraded (amber) | Outage (red)
- Last updated timestamp with auto-refresh every 60 seconds

Service Health Grid

- Each critical service shown as a tile with status indicator:
  - API Server — response time, uptime %
  - Database — connection status, query latency
  - Stripe Payments — last successful transaction, any failed webhooks
  - SMS Provider (Twilio) — messages sent today, delivery rate, any errors
  - Email Provider — emails sent today, delivery rate, bounce rate
  - Booking Worker / Queue — jobs queued, jobs failed, avg processing time
  - File Storage — disk usage, any upload errors

Recent Errors Log

- Table of last 50 errors: timestamp, service, error message (truncated), severity (warning / error / critical)
- Tapping a row expands to show full stack trace / context (in an expandable panel)
- "Mark as resolved" action per error

Failed Reminders Log

- Table of SMS/email reminders that failed to send: booking ID, client contact, failure reason, retry count
- "Retry" action per failed reminder
- "Retry all failed" bulk action

Job Queue Status

- Current queue depth for background jobs (reminders, notifications)
- Failed jobs count (with link to failed job details)

Auto-refresh Toggle

- "Auto-refresh: ON / OFF" toggle with refresh interval selector (30s, 60s, 5m)

User Actions

- Review service health status
- Expand an error log entry to see details
- Mark an error as resolved
- Retry a failed reminder
- Retry all failed reminders (bulk)
- Toggle auto-refresh

State Variations

State	Behavior
All systems operational	Green banner; all tiles show green status indicators
One service degraded	Amber banner; affected tile highlighted amber; alert shown on AS-02 dashboard
Critical outage	Red banner; affected tile shows red; full-width alert injected into all admin screens
Loading	Skeleton grid for service tiles
Empty error log	"No errors in the last 24 hours — all systems healthy."

Mobile Considerations

- Service health tiles stack vertically (2-column grid on mobile)
- Error log table collapses to card list on mobile with most important fields visible
- Auto-refresh is off by default on mobile to conserve battery

Data Required

- `GET /api/admin/monitoring/health` → per-service health statuses and metrics
- `GET /api/admin/monitoring/errors?limit=50` → recent error log
- `GET /api/admin/monitoring/failed-reminders` → failed notification log
- `GET /api/admin/monitoring/queue` → job queue stats
- `POST /api/admin/monitoring/reminders/:reminderId/retry` → retry a failed reminder

Navigation Model

Customer Navigation (CS-01 through CS-06)

Customers experience a linear stepper flow — not a navigable app. There is no persistent navigation bar or sidebar; navigation is entirely contextual.

Mechanism	Description
Progress indicator	A step bar (or numbered dots on small screens) is fixed at the top of all booking steps (CS-02 through CS-05). It shows the customer exactly where they are and how many steps remain. Completed steps are clickable for back-navigation.
Back navigation	Each step provides a back button (or relies on browser back) to return to the previous step without losing entered data. Session state is preserved in memory (or sessionStorage) for the duration of the booking flow.
No sidebar / no menu	The customer flow intentionally has no distracting navigation — the only available movement is forward (continue) or backward (back).
Exit points	Only the barber name/logo in the sub-header links back to CS-01 (the barber's landing page).
Post-confirmation	CS-06 (Confirmation) has shallow navigation only: return to landing page, add to calendar, or close the tab. There is no "app shell" to navigate within.

Barber Navigation (BS-01 through BS-09)

Barbers use a persistent sidebar navigation on desktop and a bottom tab bar on mobile.

Element	Desktop	Mobile
Primary nav	Left sidebar (fixed, 240 px wide)	Bottom tab bar (5 tabs)
Sidebar header	Barber avatar + business name + "Go to public page" link	—
Nav items	Dashboard, Bookings, Clients, Services, Settings	Dashboard, Bookings, Clients, Services, Settings
Active state	Filled background pill on active item	Filled icon + label on active tab
Collapse	Sidebar can collapse to icon-only mode (64 px) to give more content space	—
Notification bell	Top-right of the sidebar header	Top-right of the top bar
Top bar (mobile)	—	Fixed top bar with page title and notification icon

The sidebar is always accessible from any barber screen. No route requires more than 2 taps from any other barber route.

Admin Navigation (AS-01 through AS-07)

Admins use a persistent left sidebar — similar to the barber portal but with a distinct visual theme (neutral dark palette) and different nav items.

Element	Description
Sidebar	Fixed, 240 px, always visible on desktop
Nav items	Dashboard, Barbers, Config (expandable to: Config, Payments, Reminders), Monitoring
Sub-nav	Config expands inline to show Config, Payments, and Reminders sub-items
Top bar	Shows current admin user name, "Sign out" button
Mobile	Collapsible hamburger menu (admin console is not optimized for mobile but remains functional)
Breadcrumbs	All pages deeper than 1 level (e.g., `/admin/barbers/new`) show a breadcrumb trail below the top bar

---

Design Principles

1. Mobile-First, Always

Every layout is designed for the smallest supported viewport (360 × 640 px) first, then progressively enhanced for tablet and desktop. Tap targets meet the 44 × 44 px minimum. No interaction requires a hover state to be discoverable or usable.

2. Minimal Friction in the Booking Flow

The customer booking flow (CS-01 through CS-06) must have no dead ends, no unexpected navigation, and no unnecessary steps. Every field that could be removed from customer data collection (CS-04) has been removed. Form autofill is actively supported. The sticky CTA pattern ensures the primary action is always visible without scrolling.

3. Progressive Disclosure

Complex information is revealed only when relevant. The barber dashboard shows a summary; drilling down reveals detail. The admin config is sectioned so an operator only configures what they need. Service cards on the landing page show a summary; the booking flow reveals price and duration in context.

4. Clear, Branded, but Not Noisy

Each barber's public pages are lightly branded (brand color applied to CTAs, avatar, and highlights) but the underlying layout remains clean and consistent. The booking interface is never cluttered with the barber's branding choices — a barber cannot break the UX by selecting a garish color.

5. Error States Are First-Class

Every screen has explicit, human-readable error states. Error messages are written in plain language and always tell the user what went wrong and what to do next. No screen shows a raw API error or a generic "Something went wrong" without context.

6. Confirmation Before Destruction

Any irreversible action (cancel booking, delete service, delete barber account, suspend a barber) requires a confirmation dialog with clear labeling of the consequences. Destructive buttons are styled in a warning color (not the brand color) to signal their nature.

7. Consistent Visual Hierarchy

H1 for page/screen title. H2 for section headers. Body text for content. Caption text for metadata (timestamps, IDs, helper text). This hierarchy is applied consistently across all screens so users always understand their context without reading everything.

8. Speed Perception

Skeleton loaders are used universally in place of spinners for page-level content loads. This makes the interface feel faster even when data is loading. Spinners are used only for short in-button actions (form submission, API calls).

Component Library Notes

The following reusable components are required across the BarberBook screens. They should be built as a shared component library before screen-level development begins.

Layout Components

Component	Used In	Notes
`AppShell`	All barber and admin screens	Contains sidebar + top bar + main content area. Swaps sidebar for bottom tabs on mobile.
`Sidebar`	BS, AS screens	Accepts nav items as props; handles active state; collapsible

Component	Used In	Notes
`BottomTabBar`	BS screens (mobile)	5 tabs; icon + label; active state; notification badge support
`BookingStepBar`	CS-02 through CS-05	Step count prop; current step prop; completed step click handler
`BookingSummaryChip`	CS-03, CS-04, CS-05	Displays service name + date + time in a compact read-only chip

Form Components

Component	Used In	Notes
`TextInput`	All forms	Supports label, placeholder, error state, helper text, required marker, autocomplete
`TextArea`	CS-04, BS-04, BS-06, BS-08, BS-09	Supports character counter, max-length, auto-resize
`PhoneInput`	CS-04	Country code prefix selector + phone number field
`CurrencyInput`	BS-08, AS-06	Numeric input with currency prefix; handles decimal precision
`DurationSelect`	BS-08	Dropdown with 15-minute increments
`TimeRangePicker`	BS-09, AS-05	Start and end time selectors with validation (end > start)
`WeeklyHoursGrid`	BS-09	7-row grid: day toggle + time range per day + "copy to all" button
`ColorPicker`	BS-09, AS-05	Hex input + swatch grid; simplified on mobile
`ImageUpload`	BS-09, AS-04	Drag-and-drop on desktop; native file picker on mobile; preview after upload
`FormSection`	All settings screens	Wraps a group of fields with a section title and a section-level save button
`TermsCheckbox`	CS-04	Checkbox with modal-linked terms text

Display Components

Component	Used In	Notes
`ServiceCard`	CS-01, CS-02, BS-07	Displays service name, duration, price; selected state variant for CS-02
`BookingRow`	BS-02, BS-03, BS-06	Compact booking list item: time, client name, service, status badge
`ClientRow`	BS-05	Client name, initials avatar, last visit, visit count
`StatusBadge`	BS-03, BS-04, BS-06, AS-03	Pill badge: Confirmed (green), Pending (amber), Cancelled (red), No-show (grey), Completed (blue)
`StatTile`	BS-02, AS-02	KPI tile with a label, a large number, and an optional trend indicator
`BookingSummaryCard`	CS-06, BS-04	Detailed booking summary card used in confirmation and detail views
`ServiceHealthTile`	AS-07	Service name, status dot, key metric; green/amber/red variant
`AlertBanner`	AS-02, AS-07, all admin screens	Dismissible banner: info, warning, error variants

Navigation & Overlay Components

Component	Used In	Notes
`ConfirmDialog`	All destructive actions	Modal dialog with title, consequence description, cancel and confirm buttons; destructive button variant
`BottomSheet`	Mobile filter panels (BS-03, BS-05, AS-03)	Swipe-up sheet for filter/sort options on mobile
`SlidePanel`	BS-04 (Booking Detail, desktop)	Right-side slide-in panel for detail views without full navigation
`Toast`	All save/action confirmations	Auto-dismissing notification: success, error, info variants; bottom-center on mobile, top-right on desktop
`SkeletonLoader`	All loading states	Configurable skeleton that matches the shape of the content it replaces (card, row, tile, chart)

Calendar & Scheduling Components

Component	Used In	Notes
`DateStrip`	CS-03, BS-03 (mobile)	Horizontal scrollable date strip; available/unavailable/selected states
`TimeSlotGrid`	CS-03	Grid of time slots; available/unavailable/selected states; end-time label
`WeekCalendar`	BS-03	7-column time-axis calendar; booking blocks; blocked bands
`DayCalendar`	BS-03 (mobile/detail)	Single-column time-axis calendar; same booking block rendering as WeekCalendar

Data Display Components

Component	Used In	Notes
`DataTable`	AS-03 (Barbers), AS-07 (Errors)	Sortable columns, row actions, pagination, skeleton state, empty state
`LineChart`	AS-02	Lightweight line chart (e.g., using Recharts or Chart.js); responsive container
`ActivityFeed`	BS-02	Chronological event list with timestamp, icon, and description
`ErrorLogRow`	AS-07	Expandable error log item: timestamp, severity, message, expand for stack trace

---

End of Phase 4: Screen Plan

Next: Phase 5 — API Design & Endpoint Specification

Phase 5

API & Backend Plan

Endpoints · Logic · Flows

Phase 5: API / Backend Plan

Project: BarberBook — Single-Barber Branded Booking Micro-Platform

Stack: Next.js 14 App Router · Supabase (DB + Auth) · Stripe · Resend

Date: 2026-03-30

---

1. API Route Table

All routes live under `/app/api/`. Public routes require no authentication. Barber routes require a valid Supabase JWT with `role = barber`. Admin routes require `role = admin`. The Stripe webhook verifies its own signature and does not use Supabase auth.

Method	Route	Auth	Purpose
Public			
GET	`/api/public/barbers/[slug]`	None	Fetch barber profile, branding, and active services by slug
GET	`/api/public/services/[slug]`	None	Fetch all active services for a barber (by barber slug)
GET	`/api/public/availability/[slug]`	None	Return available time slots for a given date and service
POST	`/api/public/bookings`	None	Create a new booking (or initiate deposit checkout session)
Barber			
GET	`/api/barber/profile`	barber JWT	Fetch authenticated barber's own profile
PATCH	`/api/barber/profile`	barber JWT	Update barber display name, bio, avatar, slug, branding colors
GET	`/api/barber/services`	barber JWT	List all services belonging to the barber

Method	Route	Auth	Purpose
POST	`/api/barber/services`	barber JWT	Create a new service
PUT	`/api/barber/services/[id]`	barber JWT	Update a service (name, duration, price, deposit, active flag)
DELETE	`/api/barber/services/[id]`	barber JWT	Soft-delete (deactivate) a service
GET	`/api/barber/bookings`	barber JWT	List bookings with optional filters (status, date range, clientId)
GET	`/api/barber/bookings/[id]`	barber JWT	Fetch a single booking with client and service details
PATCH	`/api/barber/bookings/[id]`	barber JWT	Update booking status (confirm, cancel, complete, no_show)
GET	`/api/barber/clients`	barber JWT	List all clients with visit counts and last visit date
GET	`/api/barber/clients/[id]`	barber JWT	Fetch a single client record with notes and booking history
PATCH	`/api/barber/clients/[id]`	barber JWT	Update client notes field
GET	`/api/barber/availability`	barber JWT	Fetch barber's weekly availability rules
PUT	`/api/barber/availability`	barber JWT	Replace availability rules for one or more days of the week
GET	`/api/barber/blocked-times`	barber JWT	List blocked time windows (vacations, breaks)
POST	`/api/barber/blocked-times`	barber JWT	Create a new blocked time window
DELETE	`/api/barber/blocked-times/[id]`	barber JWT	Remove a blocked time window
Admin			
GET	`/api/admin/barbers`	admin JWT	List all barbers on the platform
POST	`/api/admin/barbers`	admin JWT	Provision a new barber account
GET	`/api/admin/barbers/[id]`	admin JWT	Fetch detailed barber record

Method	Route	Auth	Purpose
PATCH	`/api/admin/barbers/[id]`	admin JWT	Update any barber's settings or status
DELETE	`/api/admin/barbers/[id]`	admin JWT	Deactivate a barber account
GET	`/api/admin/config`	admin JWT	Fetch global platform configuration (feature flags, defaults)
PATCH	`/api/admin/config`	admin JWT	Update global platform configuration
Payments			
POST	`/api/payments/create-checkout`	None*	Create a Stripe Checkout session for a deposit; returns `checkoutUrl`
POST	`/api/payments/webhook`	Stripe sig	Receive Stripe events; on `checkout.session.completed` create booking
Notifications			
POST	`/api/notifications/send`	Internal†	Internal route: send email/SMS via NotificationService abstraction

** `create-checkout` is called from the public booking flow with a short-lived `pendingBookingToken` in the body — no Supabase JWT required.*

† `/api/notifications/send` is called only by server-side code (cron/queue); it validates a shared `INTERNALAPISECRET` header rather than a Supabase JWT.

---

2. Request/Response Shapes

GET `/api/public/barbers/[slug]`

Auth: None

URL params: `slug` — the barber's unique URL slug (e.g. `jamal-cuts`)

Response 200:

```
interface GetBarberResponse { id: string; slug: string; displayName: string; bio: string | null;
avatarUrl: string | null; branding: { primaryColor: string; // hex, e.g. "#1A1A2E" accentColor:
string; logoUrl: string | null; }; services: Array<{ id: string; name: string; description:
string | null; durationMinutes: number; price: number; // cents depositAmount: number; // cents;
0 = no deposit required isActive: boolean; }>; }
```

Error responses:

```
// 404 — slug not found or barber inactive { "error": "BARBER_NOT_FOUND", "message": "No active barber with that slug." }
```

GET `/api/public/availability/[slug]`

Auth: None

URL params: `slug` — barber slug

Query params:

```
?date=2026-04-15&serviceId=svc_abc123
```

Response 200:

```
interface GetAvailabilityResponse { date: string; // ISO date, e.g. "2026-04-15" serviceId: string; durationMinutes: number; slots: Array<{ startTime: string; // ISO datetime, e.g. "2026-04-15T09:00:00" endTime: string; // ISO datetime available: boolean; // always true in this response (unavailable slots are omitted) }>; }
```

Error responses:

```
// 400 — missing or invalid query params { "error": "INVALID_PARAMS", "message": "date must be a valid ISO date (YYYY-MM-DD) and serviceId is required." } // 404 — barber or service not found { "error": "SERVICE_NOT_FOUND", "message": "Service does not belong to this barber or is inactive." } // 422 — date is in the past { "error": "DATE_IN_PAST", "message": "Cannot query availability for past dates." }
```

POST `/api/public/bookings`

Auth: None

Content-Type: `application/json`

Request body:

```
interface CreateBookingRequest { barberSlug: string; serviceId: string; startTime: string; // ISO datetime, e.g. "2026-04-15T10:00:00" client: { name: string; email: string; phone?: string; // E.164 format, e.g. "+16135550101" }; notes?: string; // client-supplied notes for the barber }
```

Response 200 — no deposit required (booking created immediately):

```
interface CreateBookingResponseDirect { status: "confirmed"; booking: { id: string; confirmationCode: string; // short human-readable code, e.g. "BBK-7X4Q" startTime: string; endTime: string; service: { name: string; durationMinutes: number; price: number }; barber: { displayName: string; avatarUrl: string | null }; }; }
```

Response 200 — deposit required (Stripe redirect):

```
interface CreateBookingResponseDeposit { status: "deposit_required"; checkoutUrl: string; // Stripe Checkout Session URL expiresAt: string; // ISO datetime; slot is held until this time }
```

Error responses:

```
// 400 — validation failure (Zod) { "error": "VALIDATION_ERROR", "details": { "field": "message" } } // 409 — slot no longer available { "error": "SLOT_UNAVAILABLE", "message": "That time slot is no longer available." } // 422 — service inactive or barber not found { "error": "SERVICE_INACTIVE", "message": "This service is no longer offered." }
```

POST `/api/payments/create-checkout`

Auth: None (validated via `pendingBookingToken`)

Content-Type: `application/json`

Request body:

```
interface CreateCheckoutRequest { pendingBookingToken: string; // short-lived JWT issued by POST /api/public/bookings // Token encodes: barberSlug, serviceId, startTime, clientEmail, depositAmount }
```

Response 200:

```
interface CreateCheckoutResponse { checkoutUrl: string; // https://checkout.stripe.com/pay/cs_live_... sessionId: string; // Stripe session ID for reference expiresAt: string; // ISO datetime }
```

Stripe session is created with:

- `mode: "payment"`
- `line_items`: service deposit line item (from service record)
- successurl: ${NEXTPUBLICBASEURL}/book/{barberSlug}/confirmation?sessionid={CHECKOUTSESSION_ID}
- cancelurl: ${NEXTPUBLICBASEURL}/b/{barberSlug}
- `metadata`:

```
{ "barber_id": "uuid", "service_id": "uuid", "start_time": "2026-04-15T10:00:00", "client_name": "John Doe", "client_email": "john@example.com", "client_phone": "+16135550101", "client_notes": "fade on sides", "deposit_amount": 1000 }
```

- `paymentintentdata.capture_method`: `"automatic"`
- `expires_at`: 30 minutes from now

Error responses:

```
// 400 — invalid or expired pending booking token { "error": "INVALID_TOKEN", "message": "Booking token is invalid or has expired." } // 409 — slot was taken before checkout creation { "error": "SLOT_UNAVAILABLE", "message": "That time slot is no longer available." }
```

POST `/api/payments/webhook`

Auth: Stripe-Signature header verification (HMAC-SHA256)

Content-Type: `application/json` (raw body required for signature verification)

Handled Stripe events:

Event	Action
`checkout.session.completed`	Create booking, create/update client, create payment record, send confirmation email
`checkout.session.expired`	Release any held slot, log the expiry
`payment_intent.payment_failed`	Log failure, notify barber if relevant

Request: Raw Stripe event payload (no custom request body).

Response 200:

```
{ "received": true }
```

Error responses:

```
// 400 — signature verification failed { "error": "INVALID_SIGNATURE" } // 500 — unexpected
processing error (Stripe will retry) { "error": "WEBHOOK_PROCESSING_ERROR", "message": "..." }
```

GET `/api/barber/bookings`

Auth: barber JWT

Query params (all optional):

```
?status=pending|confirmed|completed|cancelled|no_show &dateFrom=2026-04-01 &dateTo=2026-04-30
&clientId=uuid &page=1 &limit=20
```

Response 200:

```
interface GetBarberBookingsResponse { bookings: Array<{ id: string; confirmationCode: string;
status: "pending" | "confirmed" | "completed" | "cancelled" | "no_show"; startTime: string;
endTime: string; service: { id: string; name: string; durationMinutes: number; price: number };
client: { id: string; name: string; email: string; phone: string | null }; depositPaid: boolean;
depositAmount: number; notes: string | null; createdAt: string; }>; pagination: { page: number;
limit: number; total: number; totalPages: number; }; }
```

Error responses:

```
// 401 — missing or invalid JWT { "error": "UNAUTHORIZED" } // 403 — JWT belongs to wrong role {
"error": "FORBIDDEN" }
```

PATCH `/api/barber/bookings/[id]`

Auth: barber JWT

URL params: `id` — booking UUID

Request body:

```
interface PatchBookingRequest { status?: "confirmed" | "cancelled" | "completed" | "no_show";
notes?: string; // barber-side notes update startTime?: string; // reschedule (triggers
availability re-check) endTime?: string; // reschedule }
```

Response 200:

```
interface PatchBookingResponse { id: string; status: string; startTime: string; endTime: string;
notes: string | null; updatedAt: string; }
```

Side effects triggered by status change:

- `confirmed` → send client confirmation email
- `cancelled` → send client cancellation email; trigger deposit refund if applicable
- completed → increment client.totalvisits, set client.lastvisit_at
- noshow → increment client.noshow_count

Error responses:

```
// 404 — booking not found or belongs to another barber { "error": "BOOKING_NOT_FOUND" } // 409 —
status transition not allowed { "error": "INVALID_STATUS_TRANSITION", "message": "Cannot
transition from 'completed' to 'confirmed'." } // 409 — reschedule slot unavailable { "error":
"SLOT_UNAVAILABLE", "message": "The requested reschedule slot is not available." }
```

GET `/api/barber/clients`

Auth: barber JWT

Query params (all optional):

```
?search=john // searches name, email, phone &sortBy=last_visit_at|total_visits|name
&sortOrder=asc|desc &page=1 &limit=20
```

Response 200:

```
interface GetBarberClientsResponse { clients: Array<{ id: string; name: string; email: string;
phone: string | null; totalVisits: number; noShowCount: number; lastVisitAt: string | null;
notes: string | null; createdAt: string; }>; pagination: { page: number; limit: number; total:
number; totalPages: number; }; }
```

Error responses:

```
// 401 { "error": "UNAUTHORIZED" }
```

PUT `/api/barber/services/[id]`

Auth: barber JWT

URL params: `id` — service UUID

Request body:

```
interface PutServiceRequest { name: string; description?: string; durationMinutes: number; //
must be multiple of 15, min 15, max 480 price: number; // cents; min 0 depositAmount: number; //
cents; 0 = no deposit; must be <= price isActive: boolean; }
```

Response 200:

```
interface PutServiceResponse { id: string; name: string; description: string | null;
durationMinutes: number; price: number; depositAmount: number; isActive: boolean; updatedAt:
string; }
```

Error responses:

```
// 400 — validation failure { "error": "VALIDATION_ERROR", "details": { "durationMinutes": "Must
be a multiple of 15." } } // 403 — service belongs to a different barber { "error": "FORBIDDEN" }
// 404 — service not found { "error": "SERVICE_NOT_FOUND" }
```

---

3. Booking Creation Logic

The following algorithm executes inside `POST /api/public/bookings`.

```
FUNCTION createBooking(input: CreateBookingRequest) -> CreateBookingResponse // ── Step 1:
Validate Input ──────────────────────────────────────────────── schema = z.object({ barberSlug:
z.string().min(1), serviceId: z.string().uuid(), startTime: z.string().datetime(), client:
z.object({ name: z.string().min(1).max(100), email: z.string().email(), phone:
z.string().regex(E164_REGEX).optional(), }), notes: z.string().max(500).optional(), }) result =
schema.safeParse(input) IF result.error THEN RETURN 400 { error: "VALIDATION_ERROR", details:
result.error.flatten() } // ── Step 2: Resolve Barber and Service
──────────────────────────────────── barber = db.barbers.findOne({ slug: input.barberSlug,
isActive: true }) IF NOT barber THEN RETURN 404 BARBER_NOT_FOUND service = db.services.findOne({
id: input.serviceId, barberId: barber.id, isActive: true }) IF NOT service THEN RETURN 422
SERVICE_INACTIVE startTime = parseISO(input.startTime) endTime = addMinutes(startTime,
service.durationMinutes) IF startTime < now() THEN RETURN 422 DATE_IN_PAST // ── Step 3: Check
Slot Availability ────────────────────────────────────── available = checkAvailability({ barberId:
barber.id, date: toDateString(startTime), requestedStart: startTime, requestedEnd: endTime,
serviceDurationMinutes: service.durationMinutes, }) // (see Section 4 for full checkAvailability
algorithm) IF NOT available THEN RETURN 409 { error: "SLOT_UNAVAILABLE" } // ── Step 4: Determine
Deposit Requirement ──────────────────────────────── requiresDeposit = service.depositAmount > 0
IF requiresDeposit THEN // ── Step 5a: Deposit path — return Stripe checkout URL ────────────────
pendingBookingToken = signJWT({ barberId: barber.id, serviceId: service.id, startTime:
startTime.toISO(), clientEmail: input.client.email, depositAmount: service.depositAmount, },
expiresIn: "30m") // Store a pending slot hold in DB (status = "hold") to prevent double-booking
db.bookings.insert({ barberId: barber.id, ... [truncated for brevity]
```

---

4. Availability Validation Logic

Used by both `GET /api/public/availability/[slug]` (to list all available slots) and `checkAvailability()` called during booking creation (to validate a single requested slot).

```
FUNCTION getAvailableSlots( barberSlug: string, date: string, // "YYYY-MM-DD"
serviceDurationMinutes: number, bufferMinutes: number = 0 // configurable gap between bookings )
-> Array<{ startTime: string; endTime: string }> // ── Step 1: Resolve barber
──────────────────────────────────────────────── barber = db.barbers.findOne({ slug: barberSlug,
```

```
isActive: true }) IF NOT barber THEN THROW BARBER_NOT_FOUND dayOfWeek = getDayOfWeek(date) //
0=Sun … 6=Sat // ── Step 2: Fetch availability rule for this day ──────────────────────── rule =
db.availabilityRules.findOne({ barberId: barber.id, dayOfWeek: dayOfWeek, isActive: true, }) IF
NOT rule OR NOT rule.isWorkingDay THEN RETURN [] // barber does not work on this day workStart =
parseTime(date, rule.startTime) // e.g. 09:00 → datetime workEnd = parseTime(date, rule.endTime)
// e.g. 18:00 → datetime // ── Step 3: Fetch blocked times for this date
──────────────────────────── blockedTimes = db.blockedTimes.findAll({ barberId: barber.id,
date_range_overlaps: [workStart, workEnd], }) // blockedTimes: Array<{ startTime: datetime;
endTime: datetime }> // ── Step 4: Fetch existing confirmed/hold bookings for this date ────────
existingBookings = db.bookings.findAll({ barberId: barber.id, startTime: { gte: workStart, lt:
workEnd }, status: IN ["confirmed", "pending", "hold"], }) // ── Step 5: Build list of busy
windows ──────────────────────────────────── busyWindows = [] FOR EACH b IN existingBookings:
busyWindows.push({ start: b.startTime, end: addMinutes(b.endTime, bufferMinutes), // add
post-booking buffer }) FOR EACH bt IN blockedTimes: busyWindows.push({ start: bt.startTime, end:
bt.endTime }) busyWindows = mergeOverlappingWindows(busyWindows) // sort + merge any overlaps //
── Step 6: Generate candidate slots at service-duration intervals ──────── slotDuration =
serviceDurationMinutes cursor = workStart availableSlots = [] WHILE cursor + slotDuration <=
workEnd: ... [truncated for brevity]
```

Helper: `mergeOverlappingWindows`

```
FUNCTION mergeOverlappingWindows(windows) -> merged windows sorted = windows.sort by start
ascending merged = [sorted[0]] FOR EACH w IN sorted[1..]: last = merged[merged.length - 1] IF
w.start <= last.end: last.end = max(last.end, w.end) // extend ELSE: merged.push(w) RETURN merged
```

---

5. Payment Flow Logic

The deposit flow spans two API routes (`/api/payments/create-checkout` and `/api/payments/webhook`) and the Stripe-hosted Checkout page.

```
┌──────────────┐ POST /api/public/bookings ┌──────────────────────┐ │ Customer │
──────────────────────────────────────────► │ Next.js API │ │ (Browser) │
◄────────────────────────────────────────── │ /public/bookings │ │ │ { status: "deposit_required",
└──────────┬───────────┘ │ │ checkoutUrl, expiresAt } │ insert hold booking │ │ sign
pendingBookingToken │ │ Redirect to checkoutUrl │ │ │
──────────────────────────────────────────────────────────► │ │ │ ┌──────────▼─────────────┐ │ │ Stripe Checkout
Page │ Stripe │ │ │ ◄──────────────────────────────────────── │ (hosted page) │ │ │ ─── customer
enters card + pays ──────► │ │ └──────────────┘ └──────────┬─────────────┘ │ POST webhook event ▼
┌────────────────────┐ │ /api/payments/ │ │ webhook │ └──────────┬─────────────┘ │ verify + process ▼
┌────────────────────┐ │ DB + Email │ └────────────────────┘
```

5.1 Create Checkout Session (`POST /api/payments/create-checkout`)

```
FUNCTION createCheckoutSession(req) // 1. Verify pending booking token payload =
verifyJWT(req.body.pendingBookingToken, JWT_SECRET) IF invalid/expired THEN RETURN 400
INVALID_TOKEN // 2. Confirm hold booking still exists and has not expired holdBooking =
db.bookings.findOne({ barberId: payload.barberId, serviceId: payload.serviceId, startTime:
payload.startTime, status: "hold", expiresAt: { gt: now() }, }) IF NOT holdBooking THEN RETURN
409 SLOT_UNAVAILABLE // 3. Fetch service details for line item service = db.services.findOne({
id: payload.serviceId }) barber = db.barbers.findOne({ id: payload.barberId }) // 4. Create
Stripe Checkout Session session = stripe.checkout.sessions.create({ mode: "payment", line_items:
[{ price_data: { currency: "cad", unit_amount: service.depositAmount, // cents product_data: {
name: `Deposit — ${service.name} with ${barber.displayName}`, }, }, quantity: 1, }], success_url:
`${BASE_URL}/b/${barber.slug}/confirmation?session_id={CHECKOUT_SESSION_ID}`, cancel_url:
`${BASE_URL}/b/${barber.slug}`, customer_email: payload.clientEmail, expires_at:
Math.floor((Date.now() + 30 * 60 * 1000) / 1000), metadata: { barber_id: barber.id, service_id:
service.id, start_time: payload.startTime, client_name: payload.clientName, client_email:
```

```
payload.clientEmail, client_phone: payload.clientPhone ?? "", client_notes: payload.clientNotes
?? "", deposit_amount: String(service.depositAmount), hold_booking_id: holdBooking.id, }, }) //
5. Attach Stripe session ID to hold booking db.bookings.update({ id: holdBooking.id }, {
stripeSessionId: session.id }) RETURN 200 { checkoutUrl: session.url, sessionId: session.id,
expiresAt: new Date(session.expires_at * 1000).toISO(), }
```

5.2 Webhook Handler (`POST /api/payments/webhook`)

```
FUNCTION handleWebhook(req) // 1. Read raw body (Next.js: disable body parser for this route)
rawBody = await req.text() signature = req.headers.get("stripe-signature") // 2. Verify Stripe
webhook signature TRY event = stripe.webhooks.constructEvent(rawBody, signature,
STRIPE_WEBHOOK_SECRET) CATCH RETURN 400 { error: "INVALID_SIGNATURE" } // 3. Route by event type
SWITCH event.type: CASE "checkout.session.completed": session = event.data.object meta =
session.metadata // 4. Idempotency guard — check if booking already created for this session
existing = db.bookings.findOne({ stripeSessionId: session.id, status: NOT "hold" }) IF existing
THEN RETURN 200 { received: true } // already processed // 5. Upsert client client =
db.clients.upsert( match: { barberId: meta.barber_id, email: meta.client_email }, update: { name:
meta.client_name, phone: meta.client_phone || existing.phone }, insert: { barberId:
meta.barber_id, name: meta.client_name, email: meta.client_email, phone: meta.client_phone } ) //
6. Promote hold booking → confirmed confirmationCode = generateConfirmationCode()
db.bookings.update( { id: meta.hold_booking_id }, { clientId: client.id, status: "confirmed",
confirmationCode, depositPaid: true, depositAmount: parseInt(meta.deposit_amount), notes:
meta.client_notes || null, stripeSessionId: session.id, stripePaymentIntentId:
session.payment_intent, } ) // 7. Create payment record db.paymentRecords.insert({ bookingId:
meta.hold_booking_id, stripeSessionId: session.id, stripePaymentIntentId: session.payment_intent,
amount: parseInt(meta.deposit_amount), currency: "cad", status: "succeeded", paidAt: new
Date().toISO(), }) // 8. Send confirmation email NotificationService.send({ ... [truncated for
brevity]
```

---

6. CRM Update Logic

Client records are stored in the `clients` table, scoped per barber. The platform maintains a lightweight CRM: visit history, no-show tracking, and free-text notes.

6.1 Client Upsert on Booking Creation

```
FUNCTION upsertClient(barberId, input) -> Client // Primary match: barber_id + email existing =
db.clients.findOne({ barberId, email: input.email }) IF NOT existing AND input.phone: // Fallback
match: barber_id + phone (handles email-less repeat clients) existing = db.clients.findOne({
barberId, phone: input.phone }) IF existing THEN db.clients.update({ id: existing.id }, { name:
input.name ?? existing.name, phone: input.phone ?? existing.phone, // email is never overwritten
to prevent identity confusion }) RETURN { ...existing, ...updates } ELSE RETURN
db.clients.insert({ barberId, name: input.name, email: input.email, phone: input.phone ?? null,
totalVisits: 0, noShowCount: 0, lastVisitAt: null, notes: null, })
```

6.2 On Booking Completion

Triggered when barber sets booking status → `completed` (via `PATCH /api/barber/bookings/[id]`):

```
FUNCTION onBookingCompleted(booking) db.clients.update( { id: booking.clientId }, { totalVisits:
increment(1), lastVisitAt: booking.endTime, } )
```

6.3 On No-Show

Triggered when barber sets booking status → `no_show`:

```
FUNCTION onNoShow(booking) db.clients.update( { id: booking.clientId }, { noShowCount:
increment(1), } ) // Note: totalVisits is NOT incremented for no-shows.
```

6.4 Barber Notes on Client Record

The barber can add or update free-text notes on a client via `PATCH /api/barber/clients/[id]`:

```
// Request body interface PatchClientRequest { notes: string; // max 2000 characters; overwrites
existing notes }
```

- Notes are stored in `clients.notes` (text column).
- Notes are visible only to the barber who owns the client record.
- Notes are displayed in the barber dashboard CRM view alongside visit count and last visit.
- There is no revision history in MVP; the field is a simple overwrite.

6.5 Client Data Isolation

Every query against the `clients` table in barber-auth routes includes a `WHERE barber_id = :authenticatedBarberId` constraint enforced at the API layer. Supabase Row Level Security (RLS) policies additionally enforce this at the database level:

```
-- RLS policy on clients table CREATE POLICY "barbers_own_clients" ON clients FOR ALL USING
(barber_id = auth.uid());
```

---

7. Reminder Logic

7.1 Scheduling Reminders

Reminders are scheduled at booking confirmation time (either via the no-deposit path in `POST /api/public/bookings`, or via the webhook handler after deposit).

```
FUNCTION scheduleReminder({ bookingId, sendAt, templateId }) // Persist the job in the
reminder_jobs table db.reminderJobs.insert({ bookingId, templateId, sendAt, status: "scheduled",
}) // Option A (preferred for MVP): Vercel Cron Job polls every minute // → cron reads all
reminder_jobs WHERE sendAt <= now() AND status = "scheduled" // → for each job, call
processReminderJob(job) // Option B (future): Use a job queue (e.g. Inngest, Trigger.dev, Upstash
QStash) // → enqueue({ payload: { bookingId, templateId }, runAt: sendAt })
```

7.2 Cron Job (`/app/api/cron/reminders/route.ts`)

Runs every minute via Vercel Cron (configured in `vercel.json`):

```
{ "crons": [{ "path": "/api/cron/reminders", "schedule": "* * * * *" }] }
```

```
FUNCTION cronProcessReminders(req) // Guard: verify Vercel cron secret IF
req.headers.get("Authorization") !== `Bearer ${CRON_SECRET}` THEN RETURN 401 dueJobs =
db.reminderJobs.findAll({ status: "scheduled", sendAt: { lte: now() }, }) FOR EACH job IN dueJobs
(in parallel, max concurrency = 5): TRY await processReminderJob(job) db.reminderJobs.update({
id: job.id }, { status: "sent", sentAt: now() }) CATCH error db.reminderJobs.update({ id: job.id
}, { status: "failed", errorMessage: error.message, retryCount: increment(1) } ) // Retry up to 3
times; after 3 failures set status = "abandoned" IF job.retryCount >= 3 THEN
db.reminderJobs.update({ id: job.id }, { status: "abandoned" })
```

7.3 Reminder Job Processor

```
ASYNC FUNCTION processReminderJob(job) booking = db.bookings.findOne({ id: job.bookingId }) IF
NOT booking OR booking.status IN ["cancelled", "no_show", "expired"] THEN // Skip stale
reminders; mark as voided db.reminderJobs.update({ id: job.id }, { status: "voided" }) RETURN
client = db.clients.findOne({ id: booking.clientId }) service = db.services.findOne({ id:
booking.serviceId }) barber = db.barbers.findOne({ id: booking.barberId }) await
NotificationService.send({ channel: "email", to: client.email, templateId: job.templateId, //
"reminder_24h" or "reminder_1h" data: { clientName: client.name, barberName: barber.displayName,
serviceName: service.name, startTime: booking.startTime, durationMinutes:
service.durationMinutes, confirmationCode: booking.confirmationCode, cancelUrl:
`${BASE_URL}/b/${barber.slug}/cancel?code=${booking.confirmationCode}`, }, })
```

7.4 NotificationService Abstraction

All notification sends go through a single `NotificationService` to decouple transport from business logic. Email is implemented via Resend; SMS interface is defined for future implementation.

```
// /lib/notifications/NotificationService.ts interface SendParams { channel: "email" | "sms"; to:
string; // email address or phone number templateId: string; data: Record<string, unknown>; }
interface SendResult { success: boolean; providerMessageId?: string; error?: string; } class
NotificationService { static async send(params: SendParams): Promise<SendResult> { switch
(params.channel) { case "email": return EmailProvider.send(params); // Resend implementation case
"sms": // SMS not implemented in MVP; log intent and return gracefully
console.warn("[NotificationService] SMS channel not yet implemented.", params.to); return {
success: false, error: "SMS_NOT_IMPLEMENTED" }; } } }
```

Email provider (Resend):

```
// /lib/notifications/EmailProvider.ts import { Resend } from "resend"; const resend = new
Resend(process.env.RESEND_API_KEY); async function send({ to, templateId, data }: SendParams):
Promise<SendResult> { const template = resolveTemplate(templateId, data); // renders React Email
component const result = await resend.emails.send({ from: process.env.EMAIL_FROM, // e.g.
"BarberBook <noreply@barberbook.app>" to, subject: template.subject, react: template.component,
}); // Log every attempt in MessageLog table await db.messageLogs.insert({ channel: "email",
templateId, recipientEmail: to, providerMessageId: result.id ?? null, status: result.error ?
"failed" : "sent", errorMessage: result.error?.message ?? null, sentAt: new Date().toISO(), });
return { success: !result.error, providerMessageId: result.id, error: result.error?.message, }; }
```

SMS interface (stub for future):

```
// /lib/notifications/SmsProvider.ts // Placeholder — implement with Twilio or similar when SMS
is enabled. export async function send(params: SendParams): Promise<SendResult> { throw new
Error("SMS provider not configured."); }
```

7.5 Email Templates

`templateId`	Trigger	Subject
`booking_confirmation`	No-deposit booking created	"Your booking is confirmed!"
`booking_confirmation_deposit`	Deposit paid via Stripe	"Booking confirmed — deposit received"
`reminder_24h`	24 hours before appointment	"Your appointment is tomorrow"
`reminder_1h`	1 hour before appointment	"Your appointment starts in 1 hour"
`booking_cancellation`	Barber or customer cancels	"Your booking has been cancelled"
`booking_rescheduled`	Barber reschedules booking	"Your booking has been rescheduled"

---

8. Middleware & Auth

8.1 Overview

Authentication is handled by Next.js middleware (`/middleware.ts`) using the `@supabase/ssr` package to read and validate the Supabase session from HTTP-only cookies. The middleware runs on every matching request before the route handler executes.

```
// /middleware.ts import { createServerClient } from "@supabase/ssr"; import { NextResponse, type
NextRequest } from "next/server"; export async function middleware(request: NextRequest) { const
response = NextResponse.next({ request: { headers: request.headers }, }); // ── 1. Create
Supabase SSR client (reads/refreshes cookie) ──────────────── const supabase =
createServerClient( process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!, { cookies: { getAll: () => request.cookies.getAll(),
setAll: (cookiesToSet) => { cookiesToSet.forEach(({ name, value, options }) =>
response.cookies.set(name, value, options) ); }, }, } ); // ── 2. Refresh session (extends cookie
TTL on activity) ─────────────────── const { data: { user } } = await supabase.auth.getUser(); //
getUser() validates the JWT with Supabase auth server on every call. // Never rely solely on
getSession() for auth decisions in middleware. const pathname = request.nextUrl.pathname; // ──
3. Route-level access control ──────────────────────────────────────── // 3a. Public routes —
always allowed, no auth required if ( pathname.startsWith("/b/") ||
pathname.startsWith("/api/public/") || pathname.startsWith("/api/payments/webhook") ||
pathname.startsWith("/api/cron/") ) { return response; } // 3b. Barber dashboard and barber API
routes if (pathname.startsWith("/barber") || pathname.startsWith("/api/barber/")) { if (!user) {
return redirectToLogin(request, pathname); } const role = user.user_metadata?.role ?? ( await
fetchRoleFromDB(supabase, user.id) // fallback: query profiles table ); if (role !== "barber" &&
role !== "admin") { return NextResponse.json({ error: "FORBIDDEN" }, { status: 403 }); } return
response; } // 3c. Admin dashboard and admin API routes ... [truncated for brevity]
```

8.2 Role Storage

User roles are stored in two places for resilience:

1. `auth.users.user_metadata.role` — set when barber is provisioned via `POST /api/admin/barbers`. Fast to read; available directly from `getUser()`.
2. `public.profiles.role` — canonical source of truth in the DB. Queried as a fallback if `user_metadata.role` is absent or stale.

```
-- profiles table CREATE TABLE profiles ( id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE, role TEXT NOT NULL CHECK (role IN ('barber', 'admin')), barber_id UUID REFERENCES barbers(id) ); -- RLS: users can only read their own profile CREATE POLICY "own_profile" ON profiles FOR SELECT USING (id = auth.uid());
```

8.3 Helper: `redirectToLogin`

```
function redirectToLogin(request: NextRequest, returnPath: string) { const loginUrl = new URL("/login", request.url); loginUrl.searchParams.set("returnTo", returnPath); return NextResponse.redirect(loginUrl); }
```

After successful login, the `/login` page reads `returnTo` from query params and redirects the user back to their intended destination.

8.4 Route-Level Auth Summary

Path Pattern	Unauthenticated	`role=barber`	`role=admin`
`/b/*`	□ Allowed	□ Allowed	□ Allowed
`/api/public/*`	□ Allowed	□ Allowed	□ Allowed
`/api/payments/webhook`	□ Allowed*	□ Allowed	□ Allowed
`/api/cron/*`	□ Allowed†	□ Allowed	□ Allowed
`/barber/*`	□ → `/login`	□ Allowed	□ Allowed
`/api/barber/*`	□ 401	□ Allowed	□ Allowed
`/admin/*`	□ → `/login`	□ 403	□ Allowed
`/api/admin/*`	□ 401	□ 403	□ Allowed

** Webhook is allowed through middleware, but verifies Stripe signature internally.*

† Cron is allowed through middleware, but verifies `CRON_SECRET` header internally.

8.5 API Route-Level Auth Guard (Defence in Depth)

In addition to middleware, each barber/admin API route handler calls a utility to verify auth independently. This prevents auth bypass if middleware is accidentally misconfigured:

```
// /lib/auth/requireAuth.ts import { createClient } from "@/lib/supabase/server"; export async function requireBarber() { const supabase = createClient(); const { data: { user }, error } = await supabase.auth.getUser(); if (error || !user) throw new AuthError("UNAUTHORIZED", 401); const role = user.user_metadata?.role ?? await fetchRoleFromDB(supabase, user.id); if (role !== "barber" && role !== "admin") throw new AuthError("FORBIDDEN", 403); return user; } export async
```

```
function requireAdmin() { const supabase = createClient(); const { data: { user }, error } =
await supabase.auth.getUser(); if (error || !user) throw new AuthError("UNAUTHORIZED", 401);
const role = user.user_metadata?.role ?? await fetchRoleFromDB(supabase, user.id); if (role !==
"admin") throw new AuthError("FORBIDDEN", 403); return user; }
```

Usage in a route handler:

```
// /app/api/barber/bookings/route.ts export async function GET(req: NextRequest) { try { const
user = await requireBarber(); // ... handler logic } catch (err) { if (err instanceof AuthError)
{ return NextResponse.json({ error: err.code }, { status: err.status }); } return
NextResponse.json({ error: "INTERNAL_ERROR" }, { status: 500 }); } }
```

End of Phase 5: API / Backend Plan

Phase 6

Implementation Roadmap

Milestones · Risks · Launch

Phase 6: Implementation Roadmap

BarberBook — Single-Barber Branded Booking Micro-Platform

Tech Stack: Next.js (App Router) · Tailwind CSS · Supabase (PostgreSQL + Auth + Storage) · Stripe · Resend · Vercel

---

1. Build Order Rationale

The build sequence is governed by three principles: dependency ordering, risk-front-loading, and feedback-loop optimization.

Infrastructure before UI

Database schema, authentication, and environment configuration must be locked before any UI component is built. Schema migrations made after UI is wired to the DB cause cascading rework — column renames, null-constraint additions, and policy changes all ripple through every query and form. Starting with a stable schema means every subsequent milestone builds on a known foundation rather than a shifting one.

RLS (Row Level Security) policies in Supabase must be designed at schema time, not retrofitted. Adding security policies to a live schema risks missing edge cases; designing them alongside the tables ensures every data access path is covered from the first query.

Customer Booking Flow before Barber Dashboard

The customer booking flow is the product's core revenue path. Every other feature exists to support or manage that flow. Building it first:

- Validates the most critical user journey (book → pay → get confirmed) before investing in operational tooling.
- Establishes the canonical data model (bookings, services, time slots) that the barber dashboard simply reads and manages.
- Enables early-stage user testing with a real booking URL, generating real feedback while the dashboard is still being built.
- Forces Stripe integration to be proven end-to-end (checkout → webhook → booking confirmed) before the barber dashboard depends on it for payment status display.

Barber Dashboard before CRM

The barber dashboard's bookings list and availability settings are load-bearing: they define what a "confirmed booking" looks like to the operator and gate which availability slots are valid. CRM features (client history, message logs) are additive — they read data already produced by the booking flow. Building the dashboard core first means CRM has real booking data to display rather than mock records.

CRM + Notifications before Admin Panel

Reminder scheduling (Vercel Cron) and notification logic must be wired to real booking records. This milestone is sequenced after the booking flow and dashboard are stable so that cron jobs trigger against verified data. The admin panel is the last milestone because it configures and monitors a system that must already be running; building the config screens before there is anything to configure wastes iteration cycles.

Mock-first, replace-later

Any feature that blocks another feature but is not itself on the critical path is mocked at that milestone and replaced in a later one. This keeps each milestone independently deployable and testable. See Section 3 for the full mock inventory.

---

2. Milestone Breakdown

Team assumption: 1–2 developers (full-stack capable). Durations are working-week estimates at sustainable pace.

Milestone 0 — Foundation

Duration: Week 1

Goal: A deployable skeleton with zero functionality but a fully wired infrastructure stack. Every subsequent milestone adds features, not infrastructure.

Deliverables

Repository & Project Scaffold

- Initialize Next.js project with App Router (`npx create-next-app@latest --app --ts --tailwind`)
- Establish directory structure:

```
/app /(customer)/b/[barberSlug]/ ← public booking routes /(barber)/dashboard/ ← barber-facing
routes /(admin)/admin/ ← admin routes /api/ ← API route handlers /components /ui/ ← shared
primitives /customer/ ← customer-facing components /barber/ ← barber dashboard components /admin/
← admin components /lib /supabase/ ← client + server Supabase helpers /stripe/ ← Stripe helpers
/resend/ ← email helpers /types/ ← shared TypeScript types /supabase/migrations/ ← SQL migration
files
```

- Configure ESLint, Prettier, TypeScript strict mode
- Set up Husky pre-commit hooks (lint + type-check)
- `.env.local` template documenting every required variable (see below)

Environment Variables Template

```
# Supabase NEXT_PUBLIC_SUPABASE_URL= NEXT_PUBLIC_SUPABASE_ANON_KEY= SUPABASE_SERVICE_ROLE_KEY= #
Stripe STRIPE_SECRET_KEY= NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY= STRIPE_WEBHOOK_SECRET= # Resend
RESEND_API_KEY= RESEND_FROM_EMAIL= # App NEXT_PUBLIC_APP_URL= CRON_SECRET= # shared secret for
Vercel Cron auth # Feature Flags (optional, for mock toggling)
NEXT_PUBLIC_MOCK_AVAILABILITY=false NEXT_PUBLIC_MOCK_PAYMENTS=false
```

Supabase Project + Schema Migration

- Create Supabase project (production + staging environments)
- Write and apply initial migration covering all MVP tables:
  - `barbers` — profile, slug, branding config
  - `services` — per-barber service catalog
  - `availability_rules` — weekly recurring working hours
  - `blocked_times` — one-off blocked ranges
  - `bookings` — appointment records with status enum
  - `clients` — CRM records, auto-upserted on booking
  - `message_logs` — outbound notification audit trail
  - `payments` — Stripe payment intent / charge records
- Apply RLS policies for all tables at migration time:
  - Barbers can only read/write their own rows
  - Admins have service-role access via server-side clients only
  - Public read access scoped to published barber profiles and their services only
  - Bookings readable by the booking owner (via email match) and their barber
- Enable Supabase Storage bucket for barber avatars (public read, authenticated write)

Auth Setup

- Configure Supabase Auth: email/password for barbers and admins
- Disable social OAuth providers (not needed for MVP)
- Create two custom user metadata roles: `barber`, `admin`
- Write server-side auth helpers (`createServerClient`, `createAdminClient` using service role)
- Write client-side auth context provider

Middleware for Route Protection

```
// middleware.ts // /dashboard/* → requires barber role // /admin/* → requires admin role // /b/*
→ public (no auth required) // Redirects unauthenticated requests to /login
```

Base Layout Components

- `CustomerLayout` — public-facing, no navigation chrome; barber brand header slot
- `BarberDashboardLayout` — sidebar navigation, mobile-responsive drawer
- `AdminLayout` — minimal admin chrome, distinct visual treatment

Vercel Project Setup

- Link repo to Vercel project
- Configure environment variables in Vercel dashboard (production + preview)
- Set up preview deployments for all PRs
- Configure `vercel.json` for cron job declarations (stubs only at this stage)
- Verify deployment pipeline end-to-end with a `/health` route returning `{ status: "ok" }`

What to Mock at This Stage

Everything above the database layer. No real UI screens exist yet — routes return placeholder `<h1>` elements. Auth middleware is wired but pages behind it are stubs. The deployment pipeline is verified with a health check route only.

Exit Criteria for Milestone 0:

- [ ] `main` branch deploys to Vercel without errors
- [ ] Supabase schema migration runs cleanly on a fresh database
- [ ] `/health` returns 200
- [ ] Auth redirect works: unauthenticated visit to `/dashboard` redirects to `/login`
- [ ] `.env.local.example` committed and documented

Milestone 1 — Customer Booking Flow

Duration: Weeks 2–3

Goal: A complete end-to-end booking journey — a customer can visit a barber's page, pick a service, choose a slot, pay a deposit (if configured), and receive a confirmation email. This is the shippable core of the product.

Deliverables

Public Barber Landing Page (`/b/[barberSlug]`)

- Fetch barber profile by slug (name, bio, avatar, brand color)
- Display service list with names, durations, and prices
- "Book Now" CTA that initiates the booking wizard
- Handle 404 gracefully for unknown slugs
- OpenGraph metadata for link sharing

Multi-Step Booking Wizard

Implemented as a client-side state machine (React context or Zustand), navigating through sequential steps without full page reloads:

Step 1 — Service Selection

- List services with duration and price
- Single-select with visual highlight
- Persist selection to wizard state

Step 2 — Date/Time Picker

- Calendar UI showing available dates (dates with no available slots are greyed out)
- Time slot grid for selected date
- Availability validation logic:
  - Query `availability_rules` for the barber's working hours on that weekday
  - Query `blocked_times` for any manual blocks on that date
  - Query `bookings` for existing confirmed/pending bookings on that date
  - Subtract service duration from slot boundaries to prevent overflow bookings
  - Return available slots as 15- or 30-minute intervals (configurable per barber)
- Race condition note: Slot display is a "soft hold" — final availability is re-validated server-side at booking creation. See Section 6 (Risk Register).

Step 3 — Customer Details Form

- Fields: first name, last name, email, phone (optional)
- Client-side validation (required fields, email format, phone format)
- No account creation required — customer books as a guest

Step 4 — Review & Payment

- Summary of service, date/time, price, deposit amount
- If barber has deposit configured: redirect to Stripe Checkout
- If no deposit configured: direct booking creation on form submit

Stripe Deposit Checkout

- Server action / API route creates a Stripe Checkout Session:
  - `mode: "payment"`
  - `line_items` from service price and deposit percentage
  - metadata carries barberid, serviceid, slotdatetime, customeremail
  - successurl → /b/[barberSlug]/confirm?sessionid={CHECKOUTSESSIONID}
  - `cancel_url` → `/b/[barberSlug]?step=review`
- Booking record created with status `pending_payment` before redirect
- Booking is only confirmed via webhook (never on success URL alone)

Stripe Webhook Handler (`/api/webhooks/stripe`)

- Verify webhook signature using `STRIPEWEBHOOKSECRET`
- Handle `checkout.session.completed`:
  - Update booking status `pending_payment` → `confirmed`
  - Create `payments` record
  - Trigger confirmation email
- Handle `checkout.session.expired`:
  - Update booking status → `expired`
  - Release the held slot
- Idempotency: check if booking already confirmed before processing (Stripe may retry)
- Return `200` immediately; process asynchronously if needed

Booking Confirmation Page (`/b/[barberSlug]/confirm`)

- Display booking summary (service, barber, date/time, location if set)
- "Add to Calendar" links (Google Calendar, `.ics` download)
- Instructions / what to bring note (from barber config)

Confirmation Email (Resend)

- Triggered by webhook handler after booking confirmed
- Template: booking summary, date/time, service, barber name + contact
- "Add to Calendar" link included
- From address uses barber's configured display name (e.g., `Jay's Barbershop <bookings@barberbook.app>`)

What to Mock at This Stage

- Barber data: Seed one barber record directly in Supabase with a known slug (e.g., `/b/demo`). No barber creation UI yet.
- Availability: Seed `availability_rules` with hardcoded Mon–Fri 9am–6pm slots. No availability management UI yet.
- Services: Seed 3–4 services directly in the database. No services CRUD UI yet.
- No-deposit path: Can be tested immediately; deposit path requires Stripe test mode keys.

Exit Criteria for Milestone 1:

- [ ] End-to-end booking completes in test mode (guest → service → slot → Stripe → confirmed)
- [ ] Webhook updates booking status correctly (verified via Stripe CLI)
- [ ] Confirmation email arrives with correct content
- [ ] Expired/cancelled Stripe sessions release the slot
- [ ] 404 page shown for unknown barber slugs
- [ ] Wizard is mobile-usable (no horizontal scroll, tap targets ≥44px)

Milestone 2 — Barber Dashboard Core

Duration: Weeks 4–5

Goal: The barber can log in, see their bookings, manage their services and availability, and update booking statuses. This transforms the platform from a read-only booking page into an operable business tool.

Deliverables

Barber Login / Auth Flow

- `/login` page with email + password form
- On successful auth, redirect to `/dashboard`
- "Forgot password" triggers Supabase password reset email
- Session persistence (Supabase handles this via cookies)
- Logout clears session and redirects to `/login`

Dashboard Overview Screen (`/dashboard`)

- Today's appointments (count + list preview)
- Upcoming appointments this week
- Quick stats: bookings this week, bookings this month, revenue this month (sum of confirmed payment amounts)
- "New booking" notice badge (since last login — tracked via `lastseenat` on barber record)

Calendar / Bookings List (`/dashboard/bookings`)

- Toggle between calendar view (week/month) and list view
- Filter by status: all, confirmed, pending, cancelled, completed
- Each booking card shows: client name, service, time, status badge, deposit indicator
- Pagination or infinite scroll for list view

Booking Detail View (`/dashboard/bookings/[bookingId]`)

- Full booking information: client details, service, time, payment status
- Status update actions:
  - `confirmed` → `completed` (mark as done)
  - `confirmed` → `cancelled` (with optional cancellation note)
  - `pending_payment` → `cancelled` (no-show / expired)
- Cancellation triggers: slot release, email notification to client (see Milestone 3)
- Notes field for barber to annotate the booking

Services Management (`/dashboard/services`)

- List all services for this barber
- Create service: name, description, duration (minutes), price, deposit amount / deposit percentage toggle, active/inactive flag

- Edit service inline or via modal
- Soft-delete (set `active = false`) — existing bookings referencing the service are preserved

Availability Settings (`/dashboard/availability`)

- Weekly hours grid: for each day of the week, toggle on/off and set open/close time
- Blocked times: date range picker to add one-off blocked periods (holidays, vacations)
- Blocked time list with delete action
- Changes take effect immediately for future bookings (no retroactive slot cancellation)

Basic Client List (`/dashboard/clients`)

- Table of all clients who have booked with this barber
- Columns: name, email, phone, last booking date, total bookings count
- Search by name or email
- Click row → client detail (stub for Milestone 3 CRM expansion)

What to Mock at This Stage

- Reports / analytics: Dashboard stats show live counts but no charts, trend lines, or export. Mock UI placeholder with "Coming soon" state.
- Reminder scheduling: Notification settings UI exists but cron jobs are not wired (Milestone 3).
- Client message history: Client detail shows empty state placeholder.

Exit Criteria for Milestone 2:

- [ ] Barber can log in and see today's appointments
- [ ] Services CRUD works end-to-end (create, edit, deactivate)
- [ ] Availability changes immediately affect slot availability on the public booking page
- [ ] Booking status transitions work and are reflected in the booking list
- [ ] Client list populates from existing bookings
- [ ] RLS verified: barber A cannot see barber B's bookings

Milestone 3 — CRM + Notifications

Duration: Week 6

Goal: Automate the post-booking communication loop. Clients receive timely reminders; barbers are notified of new bookings. Client relationship data is surfaced in a useful CRM view.

Deliverables

Full Client CRM Screen (`/dashboard/clients/[clientId]`)

- Client profile: name, email, phone, notes field (barber-editable)
- Total visit count, total spend (sum of confirmed payment amounts)
- "Preferred service" derived from most-booked service
- Last visit date, next upcoming appointment
- Tags field (free-form, comma-separated — e.g., "VIP", "fade specialist")

Client History View

- Chronological list of all bookings for this client
- Each entry: service, date, status, amount paid, barber notes
- Booking entry links to full booking detail

Auto-Upsert Client on Booking

- Triggered during booking creation (server action / API route)
- Match on email address (case-insensitive)
- If client exists: increment visitcount, update lastbooking_at
- If new: create client record from booking details
- `barber_id` scoped — same email booking two barbers creates two separate client records (multi-tenant isolation)

Reminder Scheduling Logic (Vercel Cron)

- Cron job runs every hour: `0`
- Queries bookings where:
  - `status = 'confirmed'`
  - `reminder24hsent = false` AND `appointment_at` is within next 25 hours (1-hour buffer)
  - OR `reminder1hsent = false` AND `appointment_at` is within next 65 minutes
- For each qualifying booking:

1. Send reminder email (see below)
2. Update `reminder24hsent` / `reminder1hsent` flag to `true`
3. Insert row into `message_logs`

- Cron endpoint authenticated via `CRON_SECRET` header (Vercel Cron passes this automatically)
- Idempotent: flag check prevents duplicate sends on re-runs

24-Hour Reminder Email

- Template: "Your appointment is tomorrow at [time] with [barber name]"
- Includes: service, duration, address/location, cancellation link (tokenized)
- Reply-to set to barber's contact email

1-Hour Reminder Email

- Template: "Your appointment is in about an hour — see you at [time]"
- Shorter copy, same key details
- No cancellation link (too close to appointment)

Barber Notification on New Booking

- Triggered immediately after booking confirmed (webhook handler or server action)
- Email to barber: client name, service, date/time, deposit paid
- Subject: "New booking: [Client Name] — [Service] at [Time]"

Message Log Tracking

- Every outbound email (confirmation, reminder, barber notification) writes a row to `message_logs`:
  - bookingid, clientid, barberid, messagetype enum, sentat, resendmessage_id
- Message log viewable in barber dashboard under client detail (read-only audit trail)
- Enables debugging delivery issues without querying Resend directly

What to Mock at This Stage

- SMS channel: Schema includes `preferred_channel` field on clients (email/SMS) but SMS sending is a no-op. Resend handles all notifications.
- Advanced analytics: Booking trends, revenue charts — not built.
- Cancellation self-service: Tokenized cancellation link in 24h reminder routes to a confirmation page but does not yet cancel the booking automatically (manual process at this stage — deferred to post-MVP).

Exit Criteria for Milestone 3:

- [ ] 24h and 1h reminder emails send correctly in staging (tested by creating bookings with near-future times)
- [ ] Barber receives new booking notification email
- [ ] `message_logs` populated for all sent notifications
- [ ] Client CRM page displays history correctly
- [ ] Auto-upsert creates client record on first booking, updates on repeat bookings
- [ ] Cron job authenticated and rejects requests without valid `CRON_SECRET`

Milestone 4 — Admin Panel + Polish

Duration: Weeks 7–8

Goal: Operations tooling for the platform owner, mobile UI refinement, end-to-end testing, and production deploy readiness.

Deliverables

Admin Login

- Separate `/admin/login` route
- Auth checks for `admin` role in user metadata
- Admins cannot access barber dashboard routes (and vice versa)

Admin Dashboard (`/admin`)

- Platform-level stats: total barbers, total bookings (all time + this month), total revenue processed
- Recent activity feed: new barber signups, recent bookings across all barbers
- Quick-link cards to barber management, config, monitoring

Barber Creation / Setup Flow (`/admin/barbers/new`)

- Form: barber name, email, slug (auto-generated, editable), bio
- Creates Supabase Auth user with `barber` role
- Sends welcome email with temporary password (Supabase password reset flow)
- Creates `barbers` record linked to the new user
- Confirmation screen with the barber's public booking URL

Barber Management (`/admin/barbers`)

- List all barbers with status (active/inactive), booking count, last active
- Edit barber profile
- Deactivate barber (makes their booking page return 404, does not delete data)
- Impersonate barber (admin switches to barber's dashboard view — read-only)

Branding + Booking Rules Config (`/admin/barbers/[barberId]/config`)

- Upload barber avatar (Supabase Storage)
- Brand color picker (used as accent on customer booking page)
- Booking rules: minimum advance notice (hours), maximum advance booking window (days), cancellation policy text
- Deposit rules: require deposit toggle, deposit amount / percentage

Payment + Reminder Config (`/admin/barbers/[barberId]/payments`)

- Stripe account connection status (for future Connect integration — stub for MVP)
- Current platform Stripe key in use (display masked)
- Reminder enable/disable toggles (24h, 1h)
- From email / reply-to override per barber

Monitoring Screen (`/admin/monitoring`)

- Recent webhook events (last 50 `message_logs` + recent Stripe webhook receipts)
- Failed/pending bookings list (status stuck in `pending_payment` > 30 minutes)
- Cron job last-run timestamps (stored in a `cron_runs` table updated by the cron handler)
- Links to Vercel logs, Supabase dashboard, Stripe dashboard

Mobile UI Polish Pass

- Audit every screen at 375px, 390px, and 430px viewport widths
- Fix tap target sizes (minimum 44×44px for all interactive elements)
- Fix overflow/scroll issues in calendar and time picker components
- Test booking wizard on iOS Safari and Android Chrome
- Ensure modals and drawers are accessible (focus trap, escape to close)
- Review font sizes (minimum 16px for form inputs to prevent iOS zoom)

End-to-End Testing

- Write Playwright tests for the critical booking path:

1. Visit `/b/demo`
2. Select a service
3. Select an available date and time
4. Fill in customer details
5. Complete Stripe test checkout
6. Verify confirmation page
7. Verify booking appears in barber dashboard

- Write Playwright tests for barber dashboard:

1. Login
2. View today's bookings
3. Update a booking status to `completed`

- Set up test database seeding script for CI

Production Deploy Checklist

See Section 5 for the full checklist. At this milestone:

- Verify all checklist items are green
- Configure Supabase production database (separate from staging)
- Set production Stripe keys (live mode)
- Set production Resend keys
- Configure custom domain on Vercel
- Run smoke tests against production URL

What to Defer to Post-MVP

See Section 4 for the full post-MVP backlog with rationale.

Exit Criteria for Milestone 4:

- [ ] Admin can create a new barber end-to-end (signup → booking page live)
- [ ] Playwright critical path tests pass in CI
- [ ] All pages pass mobile audit at 375px
- [ ] Production environment smoke test: full booking flow completes successfully
- [ ] All items on the MVP Launch Checklist are green

---

3. What to Mock Initially

Feature	Mock Approach	When to Replace
Barber profile data	Seed 1 barber record directly in Supabase (SQL insert)	Milestone 4 — admin barber creation flow
Services catalog	Seed 3–4 services via SQL insert	Milestone 2 — services CRUD UI
Availability rules	Hardcode Mon–Fri 9am–6pm in seed migration	Milestone 2 — availability settings UI
Stripe payments (no deposit)	Booking creation skips Stripe entirely when `deposit_required = false`	Already real; enabled by config
Stripe payments (deposit)	Test mode keys in staging; full flow works with Stripe test cards	Milestone 4 — switch to live keys at production deploy
Reminder scheduling	Reminder fields exist in schema, no emails sent	Milestone 3 — Vercel Cron + Resend integration
Barber notification emails	No notification sent when booking is created	Milestone 3 — Resend trigger in webhook handler
Client CRM detail	Client list shows basic info; detail page shows empty state	Milestone 3 — full CRM implementation
Analytics / reports	Dashboard shows live counts; charts are placeholder UI	Post-MVP
SMS reminders	`preferred_channel` field exists; SMS is a no-op	Post-MVP (Twilio integration)
Cancellation self-service	Cancel link in reminder email routes to a static "contact your barber" page	Post-MVP
Admin panel	No admin UI; admin operations done via Supabase dashboard directly	Milestone 4
Branding config	Barber brand color is hardcoded in seed data	Milestone 4 — admin config screen
Cron monitoring	No monitoring UI; verify via Vercel function logs	Milestone 4 — admin monitoring screen

---

4. What to Postpone (Post-MVP Backlog)

SMS / Twilio Integration

Email reminders cover the core notification need for MVP. SMS adds meaningful complexity: Twilio account provisioning, phone number purchasing, carrier compliance (A2P 10DLC registration in the US/Canada), opt-in tracking, and unsubscribe handling. The compliance overhead alone justifies deferral until there is clear user demand.

Gallery / Style Showcase

A photo gallery on the barber's booking page adds visual appeal but does not affect conversion for users who already intend to book. It requires image upload tooling, CDN optimization, and a layout component that adds scope without proving the core booking loop. Defer until Milestone 0–4 is stable and launched.

Rebook Quick-Link

A personalized link that pre-fills a returning client's details and preferred service would improve repeat booking UX. It requires client authentication or tokenized links with expiry, adding auth complexity. Clients can complete the standard booking wizard in under 2 minutes for MVP.

Message Templates

Allowing barbers to customize the content of confirmation and reminder emails requires a template editor (rich text or variable-based), template storage, and a rendering layer. Resend's built-in template system can be adopted post-MVP. The default templates are sufficient for launch.

Basic Reports / Export

Revenue reports, booking trend charts, and CSV export require aggregation queries, chart libraries, and export formatting. This is operational nice-to-have, not a launch blocker. Barbers can view bookings list and derive counts manually for MVP.

Multi-Tenant Architecture

BarberBook MVP is scoped to a single barber per deployment (or a small number of manually-onboarded barbers). Multi-tenant SaaS architecture — self-serve signup, per-tenant billing, tenant isolation at the infrastructure level — is a major architectural expansion that belongs after product-market fit is validated.

WhatsApp Integration

WhatsApp Business API requires Meta business verification, an approved message template review process, and a phone number. Similar compliance complexity to SMS. High potential in specific markets (MENA, UK Afro-Caribbean communities) but inappropriate scope for MVP.

Loyalty / Rewards

Punch-card or points-based loyalty programs require a rewards ledger, redemption logic, and UI for both client and barber. There is no booking data history to demonstrate value to the barber at MVP. Build after the platform has enough transaction volume to make the loyalty feature meaningful.

Promo Codes

Discount codes require Stripe Coupon/Promotion Code integration, a code management UI in the admin/barber dashboard, and conditional pricing logic in the checkout flow. Non-trivial

Stripe scope. Defer until barbers request it based on real usage.

Reviews / Ratings

Post-appointment review collection requires a review submission flow (triggered by email after `completed` status), a review storage schema, a moderation layer (to prevent abuse), and display on the public booking page. Meaningful only when there is a volume of completed bookings. Build after the first cohort of barbers has real booking history.

---

5. MVP Launch Checklist

Infrastructure & Config

- [ ] Vercel project linked to production branch (`main`)
- [ ] Production environment variables set in Vercel dashboard (not `.env.local`)
- [ ] Custom domain configured and SSL certificate active
- [ ] `NEXTPUBLICAPP_URL` set to production URL (no trailing slash)
- [ ] Supabase production project separate from staging
- [ ] Supabase connection pooler (PgBouncer) enabled for production
- [ ] `CRON_SECRET` is a strong random string (min 32 chars), set in Vercel env
- [ ] Vercel Cron jobs declared in `vercel.json` and visible in Vercel dashboard
- [ ] `.env.local.example` committed with all keys listed (values empty)
- [ ] No secrets committed to Git (run `git log --all --full-history -- .env*` check)

Auth & Security

- [ ] Supabase Auth email confirmation disabled for barber accounts (admin creates accounts)
- [ ] Password reset flow tested end-to-end (barber receives email, sets new password, can log in)
- [ ] Middleware protects `/dashboard/*` routes (unauthenticated → `/login`)
- [ ] Middleware protects `/admin/*` routes (non-admin → 404 or `/login`)
- [ ] RLS policies enabled on all tables (verify via Supabase dashboard → Auth → Policies)
- [ ] Service role key (`SUPABASESERVICEROLE_KEY`) only used server-side (never in client bundles)
- [ ] `NEXTPUBLIC` prefixed variables contain no secrets
- [ ] Stripe webhook signature verification active (not skipped in production)
- [ ] CSRF: Next.js App Router server actions are used for all mutations (not plain GET routes)
- [ ] Rate limiting on booking creation endpoint (to prevent slot flooding)

Customer Booking Flow

- [ ] Barber landing page loads correctly for a valid slug
- [ ] Barber landing page returns 404 for an unknown slug
- [ ] All services are listed with correct prices and durations
- [ ] Date picker greys out dates with no available slots
- [ ] Time picker shows only slots that do not conflict with existing bookings
- [ ] Service duration overflow check: last slot of day does not allow a booking that runs past close
- [ ] Customer details form validates email format and required fields
- [ ] Stripe Checkout session creates successfully in live mode

- [ ] `success_url` redirects to confirmation page after successful payment
- [ ] `cancel_url` returns user to booking wizard with state preserved
- [ ] Booking status is `pending_payment` before Stripe completes (never pre-confirmed)
- [ ] Booking status transitions to `confirmed` only via webhook (not via success URL)
- [ ] Confirmation page displays correct booking details
- [ ] "Add to Google Calendar" link generates correct event URL
- [ ] `.ics` download contains correct VCALENDAR data

Barber Dashboard

- [ ] Login works with email + password
- [ ] "Forgot password" email sends and link is valid
- [ ] Dashboard overview shows today's bookings
- [ ] Booking list filters by status work
- [ ] Booking detail shows full client and payment info
- [ ] Status update (confirmed → completed) saves and reflects in list
- [ ] Status update (confirmed → cancelled) saves and reflects in list
- [ ] Services create, edit, and deactivate work
- [ ] Deactivated services do not appear on public booking page
- [ ] Availability rules update and immediately affect slot availability
- [ ] Blocked times prevent booking on affected dates
- [ ] Client list populates from bookings
- [ ] Client detail shows booking history

Admin Panel

- [ ] Admin login works and is restricted to users with `admin` role
- [ ] Barber creation flow creates Supabase Auth user + `barbers` record
- [ ] Welcome email sends to new barber with password reset link
- [ ] Newly created barber's booking page is immediately accessible
- [ ] Barber deactivation makes booking page return 404
- [ ] Branding config (avatar upload, brand color) reflects on public booking page
- [ ] Deposit config (enable/disable, amount) reflects in Stripe checkout
- [ ] Monitoring screen shows recent message logs and pending bookings

Payments (Stripe)

- [ ] Stripe account in live mode (not test mode) for production deploy
- [ ] Webhook endpoint registered in Stripe dashboard pointing to `/api/webhooks/stripe`
- [ ] Webhook endpoint configured to listen for `checkout.session.completed` and `checkout.session.expired`
- [ ] Webhook signature secret (`STRIPEWEBHOOKSECRET`) is the live-mode secret
- [ ] `checkout.session.completed` updates booking to `confirmed` and creates payment record
- [ ] `checkout.session.expired` updates booking to `expired` and releases slot
- [ ] Idempotency verified: processing the same event twice does not double-confirm or double-charge
- [ ] Stripe test mode end-to-end tested with test card `4242 4242 4242 4242` before go-live
- [ ] Decline scenario tested with test card `4000 0000 0000 0002`
- [ ] Payment amounts match service price × deposit percentage (no rounding errors on odd prices)

Notifications (Email)

- [ ] Resend account on a paid plan (free tier is rate-limited)
- [ ] Sending domain verified in Resend (SPF, DKIM records set)
- [ ] Confirmation email sends after booking confirmed (webhook trigger)
- [ ] 24h reminder sends via cron job
- [ ] 1h reminder sends via cron job
- [ ] Barber notification sends on new confirmed booking
- [ ] All emails render correctly on mobile (tested in email client preview or Litmus)
- [ ] Unsubscribe header present in reminder emails (CAN-SPAM / CASL compliance)
- [ ] Reply-to set to barber contact email (not a no-reply address)
- [ ] `message_logs` table records all sent emails
- [ ] No duplicate emails (idempotency flags checked before sending)

Data & Database

- [ ] All migrations run cleanly on a fresh Supabase project (tested in staging)
- [ ] RLS policies tested: barber A cannot query barber B's data
- [ ] Public anon key cannot access `payments`, `clients`, or admin-only tables
- [ ] Database backups enabled in Supabase (point-in-time recovery)
- [ ] At least one barber seeded in production with services and availability rules
- [ ] `message_logs` foreign keys referencing `bookings` and `clients` are nullable (supports system-level messages)

Testing

- [ ] Playwright critical path test suite passes against staging URL
- [ ] Playwright barber dashboard test suite passes
- [ ] Manual end-to-end test on iOS Safari (iPhone) — full booking flow
- [ ] Manual end-to-end test on Android Chrome — full booking flow
- [ ] Manual test on desktop Chrome, Firefox, Safari
- [ ] Load test: 10 concurrent booking submissions to the same slot (verify only 1 succeeds)

Deployment

- [ ] Production deploy from `main` branch succeeds with no build errors
- [ ] No TypeScript errors in production build (`tsc --noEmit` passes)
- [ ] No ESLint errors in production build
- [ ] Core Web Vitals pass on Vercel Analytics (LCP < 2.5s, CLS < 0.1)
- [ ] Vercel preview deployments working for feature branches
- [ ] Rollback procedure documented (Vercel instant rollback to previous deployment)

Legal / Compliance Basics

- [ ] Privacy Policy page exists at `/privacy` and is linked in footer
- [ ] Privacy policy covers: data collected (name, email, phone, payment intent), how it is used (booking confirmation, reminders), third-party processors (Stripe, Resend, Supabase), and contact for deletion requests
- [ ] Terms of Service page exists at `/terms` and is linked in footer
- [ ] Terms cover: booking and cancellation policy, deposit non-refundability (if applicable), service description accuracy
- [ ] Cookie notice displayed on first visit (if using analytics cookies)

- [ ] CASL compliance for Canadian users: transactional emails (confirmation, reminders) are exempt; marketing emails require explicit opt-in (no marketing emails in MVP scope)
- [ ] Stripe handles PCI compliance for payment data — verify no card numbers are logged anywhere in application code

---

6. Risk Register

Risk	Likelihood	Impact	Mitigation
Stripe webhook delivery failure — Stripe cannot reach the webhook endpoint (Vercel cold start timeout, deploy in progress, network error), leaving bookings stuck in `pending_payment`	Medium	High	1) Stripe automatically retries failed webhooks for up to 3 days with exponential backoff. 2) Admin monitoring screen surfaces bookings stuck in `pending_payment` > 30 minutes. 3) Manual resolution procedure documented: admin updates status via Supabase dashboard. 4) Webhook handler returns `200` immediately and processes asynchronously to avoid timeouts.
Double-booking race condition — Two customers select the same slot simultaneously; both see it as available and both submit; both bookings are created	Medium	High	1) Slot availability re-validated inside a Supabase database transaction at booking insert time using a `SELECT ... FOR UPDATE` lock or a unique constraint on `(barber_id, appointment_at)`. 2) If slot is taken, return a 409 response and redirect the second user to re-select. 3) Stripe Checkout is only created after the slot lock succeeds (never before). 4) Load test 10 concurrent submissions before launch.

Risk	Likelihood	Impact	Mitigation
Email deliverability failure — Confirmation or reminder emails land in spam or are not delivered	Medium	High	1) Verify sending domain in Resend with SPF + DKIM + DMARC records before launch. 2) Use a subdomain (`mail.barberbook.app`) for sending to isolate reputation. 3) Resend's `message_logs` integration captures `resend_message_id` for every send; delivery status can be verified. 4) Include a plain-text version of all emails. 5) Monitor Resend dashboard for bounce rates post-launch.
Slot availability race condition (UI desync) — User sees a slot as available on the picker, but by the time they complete the form and submit, the slot is gone	High	Medium	This is a UX issue, not a data integrity issue (the DB constraint prevents the actual double-booking). Mitigation: 1) Show a friendly "That slot was just taken — please choose another time" message on 409. 2) Return the updated available slots in the 409 response so the UI refreshes without a full page reload. 3) Add a short "slot hold" timer display on the review step ("Your slot is held for 8 minutes").

Risk	Likelihood	Impact	Mitigation
Supabase RLS misconfiguration — A policy is too permissive, allowing one barber to read or modify another barber's bookings	Low	Critical	1) Write dedicated RLS tests (SQL-level) that assert cross-barber queries return empty sets. 2) Run these tests in CI on every schema migration. 3) Use Supabase's built-in policy tester in staging before every release. 4) Principle of least privilege: start with all-deny and explicitly allow. 5) Server-side admin operations use the service role key (bypasses RLS) — keep usage auditable and minimal.
Mobile UX failure on iOS Safari — Date/time picker, modal overlays, or form inputs behave incorrectly on iOS Safari (virtual keyboard resize, 100vh bug, input zoom)	High	Medium	1) Dedicate the final week of Milestone 4 to a mobile-specific audit pass. 2) Use `dvh` (dynamic viewport height) instead of `100vh` for full-height layouts. 3) Set `font-size: 16px` minimum on all form inputs to suppress iOS zoom. 4) Test on a real iPhone device (not simulator) — Safari behavior differs from Chrome DevTools mobile emulation. 5) Avoid `position: fixed` elements near the bottom of the screen (overlapped by iOS home bar).

Risk	Likelihood	Impact	Mitigation
Vercel Cron job silent failure — Cron job runs but sends no emails, and there is no alerting	Medium	Medium	1) Cron handler writes a row to `cron_runs` table on every execution with a `success` / `error` status and count of emails sent. 2) Admin monitoring screen surfaces last run time and status. 3) If `cron_runs` has no entry within the last 2 hours, the monitoring screen shows an alert badge. 4) Set up a Vercel log drain or uptime monitor (e.g., BetterUptime ping) on the cron endpoint.
Stripe live mode misconfiguration — Production deploy uses test mode keys, resulting in real users completing test checkouts with no charge	Low	Critical	1) Production Vercel environment uses a separate variable set with live keys. 2) Add a startup check: if `STRIPE_SECRET_KEY` starts with `sk_test_` and `NODE_ENV === 'production'`, throw an error and refuse to start. 3) Pre-launch checklist item specifically verifies live mode key format.

---

7. Tech Debt to Track

The following are deliberate shortcuts acceptable for MVP speed. Each item should be revisited before the platform reaches 10+ barbers or 1,000+ bookings/month.

1. Slot availability computed at query time, not cached

Availability slots are computed on-demand by querying availabilityrules, blockedtimes, and bookings for every date picker render. At low volume this is fine, but at scale it generates significant read load. Replace with a pre-computed available_slots table maintained by database triggers or a background job.

2. No job queue for outbound emails

Confirmation emails and barber notifications are sent synchronously inside the Stripe webhook handler. If Resend is slow or unavailable, the webhook handler times out and Stripe may retry. Replace with a lightweight job queue (e.g., pg-boss on top of Supabase Postgres, or Inngest)

that decouples email sending from webhook processing.

3. Vercel Cron precision is ±1 minute

Reminders sent "1 hour before" may arrive 55–65 minutes early. For MVP this is acceptable, but for production quality it should be replaced with a proper scheduler (Inngest, Trigger.dev, or pg-boss) that schedules jobs at exact appointment-relative offsets.

4. No soft-delete for bookings

Cancelled bookings are updated in place (status change). There is no audit trail of who cancelled, when, or the reason beyond an optional cancellationnote field. Before scaling, add an auditlog table or use Supabase's Realtime replication to capture row-level change history.

5. Stripe Connect not implemented

For MVP, all payments go to a single platform Stripe account. This means the platform operator manually settles with barbers (or barbers are employees). Before onboarding independent barbers as merchants, Stripe Connect (Express accounts) must be implemented for direct-to-barber payouts and correct tax reporting.

6. Client matching is email-only

Auto-upsert matches clients by email address. If a returning client books with a different email (personal vs. work), they generate a duplicate client record. Post-MVP: add phone-based matching as a secondary key, with a manual merge tool in the barber dashboard.

7. No database migrations framework in CI

Supabase CLI migrations are applied manually. Before multiple developers work on schema changes concurrently, integrate `supabase db push` or a migration tool into the CI pipeline with a staging-to-production promotion gate.

8. Hardcoded reminder windows (24h, 1h)

Reminder intervals are constants in the cron handler code. Barbers cannot customize when reminders are sent. Post-MVP, move reminder windows to the barber config table and read them dynamically in the cron job.

9. No pagination on Supabase queries

Bookings list, client list, and message logs query without hard row limits. At low volume (< 500 records per barber) this is fine. Add `range()` pagination or cursor-based pagination before data volumes could cause response time degradation.

10. No observability beyond Vercel logs

There is no structured logging, error tracking (e.g., Sentry), or performance monitoring beyond Vercel's built-in analytics. Before production traffic scales, add Sentry (or equivalent) for error alerting and a structured log format that makes debugging webhook failures and cron issues faster.

Document version: 1.0 — Phase 6 of the BarberBook MVP Technical Specification

Prepared for a 1–2 developer team targeting an 8-week build to production.

Phase 7

Code Scaffold

50+ Files · Full TypeScript

Phase 7: Complete Code Scaffold — BarberBook MVP

Tech Stack: Next.js 14 App Router · TypeScript · Tailwind CSS · Supabase (@supabase/ssr) · Stripe · Resend · Zod

>

This document contains the complete, production-ready code scaffold for the BarberBook MVP. Every file is complete with real logic — no stubs or TODO comments.

Section 1 & 2: Folder Structure + Environment Variables

1. Complete Folder Structure

```
barberbook/ ├── .env.local.example # Environment variable template ├── .eslintrc.json # ESLint
config ├── .gitignore # Git ignore rules ├── middleware.ts # Next.js global middleware (auth +
role guard) ├── next.config.ts # Next.js configuration ├── package.json # Project dependencies
├── tailwind.config.ts # Tailwind CSS config + design tokens ├── tsconfig.json # TypeScript
compiler config ├── vercel.json # Vercel deployment + cron config │ ├── supabase/ │ └──
migrations/ │ └── 001_initial.sql # Complete DB schema, ENUMs, indexes, RLS, triggers │ ├── src/
│ ├── types/ │ │ └── index.ts # All TypeScript interfaces and enums │ │ ├── lib/ │ │ ├──
supabase/ │ │ │ ├── client.ts # Browser-side Supabase client (createBrowserClient) │ │ ├──
server.ts # Server-side Supabase client (createServerClient) │ │ └── middleware.ts # Middleware
Supabase client for session refresh │ │ │ │ └── services/ │ │ ├── availability.service.ts #
Slot calculation, conflict detection, window merging │ │ ├── booking.service.ts #
Create/update/query bookings │ │ ├── client.service.ts # Upsert client, history, stats │ │ ├──
notification.service.ts # Email via Resend, confirmation + reminders │ │ └── payment.service.ts #
Stripe checkout, webhook handler │ │ └── components/ │ ├── BookingStepIndicator.tsx # Step 1-4
progress indicator │ ├── ServiceCard.tsx # Customer-facing service group card │ ├──
TimeSlotPicker.tsx # Grid of available time slot buttons │ ├── StatusBadge.tsx # Color-coded
booking status badge │ └── LoadingSpinner.tsx # Centered animated loading spinner │ └── app/ ├──
layout.tsx # Root HTML layout (fonts, providers, metadata) ├── page.tsx # Marketing root redirect
page │ ├── b/ │ └── [barberSlug]/ │ ├── layout.tsx # Customer layout (fetches barber branding) │
├── page.tsx # Barber public landing page │ ├── book/ │ │ └── page.tsx # Multi-step booking flow
(service→time→details→deposit) │ └── confirmation/ │ └── page.tsx # Post-booking confirmation
page │ ├── barber/ │ ├── login/ │ │ └── page.tsx # Barber login with Supabase email/password auth
│ ├── dashboard/ │ │ └── page.tsx # Dashboard: stats, upcoming bookings │ ├── bookings/ │ │ └──
page.tsx # Bookings list with status/date filters │ ├── services/ ... [truncated for brevity]
```

---

2. Environment Variables Template

File: `.env.local.example`

```
# ─── Supabase ─────────────────────────────────────────────────────────────────── # Found in:
Supabase Dashboard → Settings → API NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key-here # Service role key — NEVER expose to client.
Used in server-only code. SUPABASE_SERVICE_ROLE_KEY=your-service-role-key-here # ─── Stripe
──────────────────────────────────────────────────────────────────────────────── # Found in: Stripe Dashboard →
Developers → API Keys STRIPE_SECRET_KEY=sk_live_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_... # Found in: Stripe Dashboard → Developers →
Webhooks → your endpoint → Signing secret STRIPE_WEBHOOK_SECRET=whsec_... # ─── Resend
(Transactional Email) ────────────────────────────────────────────── # Found in:
https://resend.com/api-keys RESEND_API_KEY=re_... # ─── App
──────────────────────────────────────────────────────────────────────────────── # The public URL of your
deployment (no trailing slash) NEXT_PUBLIC_APP_URL=https://barberbook.app # ─── Internal Security
──────────────────────────────────────────────────────────────── # Random secret used to authenticate
internal cron job calls # Generate with: openssl rand -hex 32
INTERNAL_CRON_SECRET=your-random-secret-here
```

Section 3: Shared Types

File: `src/types/index.ts`

```
// ─── Enums ────────────────────────────────────────────────────────────────────── export enum
BookingStatus { PENDING = 'pending', DEPOSIT_PENDING = 'deposit_pending', CONFIRMED =
'confirmed', CANCELLED = 'cancelled', NO_SHOW = 'no_show', COMPLETED = 'completed', } export enum
PaymentStatus { NONE = 'none', PENDING = 'pending', PAID = 'paid', REFUNDED = 'refunded', FAILED
= 'failed', } export enum DepositType { NONE = 'none', FIXED = 'fixed', PERCENTAGE =
'percentage', } export enum MessageType { CONFIRMATION = 'confirmation', REMINDER = 'reminder',
CANCELLATION = 'cancellation', CUSTOM = 'custom', } export enum DayOfWeek { SUNDAY = 0, MONDAY =
1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5, SATURDAY = 6, } // ─── Core Entities
──────────────────────────────────────────────────────────────────── export interface User { id: string;
email: string; role: 'barber' | 'admin'; created_at: string; } export interface Barber { id:
string; user_id: string; slug: string; name: string; bio: string | null; phone: string | null;
avatar_url: string | null; brand_color: string; // hex e.g. "#1a1a2e" ... [truncated for brevity]
```

Section 4: Supabase Client Setup

`src/lib/supabase/client.ts`

```
import { createBrowserClient } from '@supabase/ssr'; export function createClient() { return
createBrowserClient( process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY! ); }
```

---

src/lib/supabase/server.ts

```
import { createServerClient } from '@supabase/ssr'; import { cookies } from 'next/headers';
export async function createClient() { const cookieStore = await cookies(); return
createServerClient( process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!, { cookies: { getAll() { return cookieStore.getAll();
}, setAll(cookiesToSet) { try { cookiesToSet.forEach(({ name, value, options }) =>
cookieStore.set(name, value, options) ); } catch { // In Server Components, cookies are read-only
after headers are sent. // This is expected during RSC renders. } }, }, } ); } /** Server client
using service role key — bypasses RLS. Use only in trusted server code. */ export async function
createServiceClient() { const cookieStore = await cookies(); return createServerClient(
process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.SUPABASE_SERVICE_ROLE_KEY!, { cookies: {
getAll() { return cookieStore.getAll(); }, setAll(cookiesToSet) { try { cookiesToSet.forEach(({
name, value, options }) => cookieStore.set(name, value, options) ); } catch { // Expected in RSC
renders } }, }, } ); }
```

---

src/lib/supabase/middleware.ts

```
import { createServerClient } from '@supabase/ssr'; import { type NextRequest, NextResponse }
from 'next/server'; export async function updateSession(request: NextRequest) { let
supabaseResponse = NextResponse.next({ request }); const supabase = createServerClient(
process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!, { cookies: {
getAll() { return request.cookies.getAll(); }, setAll(cookiesToSet) { cookiesToSet.forEach(({
name, value }) => request.cookies.set(name, value) ); supabaseResponse = NextResponse.next({
request }); cookiesToSet.forEach(({ name, value, options }) => supabaseResponse.cookies.set(name,
value, options) ); }, }, } ); // Refresh session — must call getUser() to refresh JWT const {
data: { user }, } = await supabase.auth.getUser(); return { supabaseResponse, user, supabase }; }
```

Section 5: Next.js Middleware

File: `middleware.ts` (root of project)

```
import { type NextRequest, NextResponse } from 'next/server'; import { updateSession } from
'@/lib/supabase/middleware'; import { createServerClient } from '@supabase/ssr'; const
BARBER_PROTECTED = /^\/barber(?!\/login)/; const ADMIN_PROTECTED = /^\/admin(?!\/login)/; const
BARBER_LOGIN = '/barber/login'; const ADMIN_LOGIN = '/admin/login'; export async function
middleware(request: NextRequest) { const { pathname } = request.nextUrl; // Refresh session
tokens in cookies const { supabaseResponse, user } = await updateSession(request); // ── Barber
routes ──────────────────────────────────────────────────────────── if
(BARBER_PROTECTED.test(pathname)) { if (!user) { const loginUrl = request.nextUrl.clone();
loginUrl.pathname = BARBER_LOGIN; loginUrl.searchParams.set('redirectTo', pathname); return
NextResponse.redirect(loginUrl); } // Check role from user metadata const role =
user.user_metadata?.role as string | undefined; if (role !== 'barber' && role !== 'admin') { //
Authenticated but wrong role — redirect to barber login const loginUrl = request.nextUrl.clone();
loginUrl.pathname = BARBER_LOGIN; return NextResponse.redirect(loginUrl); } } // ── Admin routes
──────────────────────────────────────────────────────────── if (ADMIN_PROTECTED.test(pathname)) {
if (!user) { const loginUrl = request.nextUrl.clone(); loginUrl.pathname = ADMIN_LOGIN;
loginUrl.searchParams.set('redirectTo', pathname); return NextResponse.redirect(loginUrl); }
const role = user.user_metadata?.role as string | undefined; if (role !== 'admin') { const
```

```
loginUrl = request.nextUrl.clone(); loginUrl.pathname = ADMIN_LOGIN; return
NextResponse.redirect(loginUrl); } } // ── Redirect logged-in users away from login pages
──────────────────────────── if (user) { const role = user.user_metadata?.role as string |
undefined; if (pathname === BARBER_LOGIN && (role === 'barber' || role === 'admin')) { const
redirectTo = request.nextUrl.searchParams.get('redirectTo') ?? '/barber/dashboard'; const dest =
request.nextUrl.clone(); dest.pathname = redirectTo; dest.search = ''; ... [truncated for
brevity]
```

Section 6: Root Layout + Customer Layout

---

app/layout.tsx

```
import type { Metadata } from 'next'; import { Inter } from 'next/font/google'; import
'./globals.css'; const inter = Inter({ subsets: ['latin'], variable: '--font-inter', display:
'swap', }); export const metadata: Metadata = { title: { default: 'BarberBook', template: '%s |
BarberBook', }, description: 'Book your barber appointment online — fast, easy, no phone calls.',
metadataBase: new URL(process.env.NEXT_PUBLIC_APP_URL ?? 'https://barberbook.app'), }; export
default function RootLayout({ children, }: { children: React.ReactNode; }) { return ( <html
lang="en" className={inter.variable}> <body className="min-h-screen bg-white font-sans
antialiased"> {children} </body> </html> ); }
```

---

app/b/[barberSlug]/layout.tsx

```
import { notFound } from 'next/navigation'; import type { Metadata } from 'next'; import {
createClient } from '@/lib/supabase/server'; import type { Barber } from '@/types'; interface
Props { children: React.ReactNode; params: Promise<{ barberSlug: string }>; } export async
function generateMetadata({ params }: Props): Promise<Metadata> { const { barberSlug } = await
params; const supabase = await createClient(); const { data: barber } = await supabase
.from('barbers') .select('name, bio, avatar_url') .eq('slug', barberSlug) .eq('is_active', true)
.single(); if (!barber) { return { title: 'Barber Not Found' }; } return { title: `${barber.name}
— Book an Appointment`, description: barber.bio ?? `Book your next appointment with
${barber.name}.`, openGraph: { title: `${barber.name} — Book an Appointment`, description:
barber.bio ?? `Book with ${barber.name}`, ...(barber.avatar_url && { images: [{ url:
barber.avatar_url }] }), }, }; } export default async function BarberLayout({ children, params }:
Props) { const { barberSlug } = await params; const supabase = await createClient(); const {
data: barber } = await supabase .from('barbers') .select('id, name, brand_color, brand_logo_url,
avatar_url') .eq('slug', barberSlug) .eq('is_active', true) .single(); if (!barber) { notFound();
} const brandColor = (barber as Barber).brand_color ?? '#111827'; return ( <div
className="min-h-screen" style={{ '--brand-color': brandColor } as React.CSSProperties} > {/*
Minimal branded header */} <header className="border-b border-gray-100 bg-white/95 backdrop-blur
sticky top-0 z-40"> ... [truncated for brevity]
```

Section 7: Customer Booking Pages

---

app/b/[barberSlug]/page.tsx

```
import { notFound } from 'next/navigation'; import Link from 'next/link'; import { createClient }
from '@/lib/supabase/server'; import ServiceCard from '@/components/ServiceCard'; import type {
Barber, ServiceGroup } from '@/types'; interface Props { params: Promise<{ barberSlug: string }>;
} export default async function BarberLandingPage({ params }: Props) { const { barberSlug } =
await params; const supabase = await createClient(); const { data: barber } = await supabase
.from('barbers') .select('*') .eq('slug', barberSlug) .eq('is_active', true) .single<Barber>();
if (!barber) notFound(); const { data: services } = await supabase .from('service_groups')
.select('*') .eq('barber_id', barber.id) .eq('is_active', true) .order('display_order', {
ascending: true }) .returns<ServiceGroup[]>(); const activeServices = services ?? []; return (
<div className="space-y-10"> {/* Hero Section */} <section className="text-center space-y-4
pt-4"> {barber.avatar_url ? ( <img src={barber.avatar_url} alt={barber.name} className="w-24 h-24
rounded-full object-cover mx-auto border-4 border-white shadow-lg" /> ) : ( <div className="w-24
h-24 rounded-full mx-auto flex items-center justify-center text-white text-3xl font-bold
shadow-lg" style={{ backgroundColor: barber.brand_color }} >
{barber.name.charAt(0).toUpperCase()} </div> )} <div> <h1 className="text-3xl font-bold
text-gray-900">{barber.name}</h1> {barber.bio && ( <p className="mt-2 text-gray-600 max-w-md
mx-auto text-sm leading-relaxed"> {barber.bio} </p> )} </div> {barber.phone && ( ... [truncated
for brevity]
```

---

app/b/[barberSlug]/book/page.tsx

```
'use client'; import { useState, useEffect, useCallback } from 'react'; import { useParams,
useRouter, useSearchParams } from 'next/navigation'; import { z } from 'zod'; import
BookingStepIndicator from '@/components/BookingStepIndicator'; import TimeSlotPicker from
'@/components/TimeSlotPicker'; import LoadingSpinner from '@/components/LoadingSpinner'; import
type { ServiceGroup, TimeSlot, Barber } from '@/types'; const STEPS = ['Service', 'Date & Time',
'Your Details', 'Confirm'] as const; type Step = 0 | 1 | 2 | 3; const clientDetailsSchema =
z.object({ name: z.string().min(2, 'Name must be at least 2 characters'), email:
z.string().email('Enter a valid email address'), phone: z.string().optional(), notes:
z.string().optional(), }); type ClientDetails = z.infer<typeof clientDetailsSchema>; export
default function BookPage() { const params = useParams<{ barberSlug: string }>(); const router =
useRouter(); const searchParams = useSearchParams(); const [step, setStep] = useState<Step>(0);
const [barber, setBarber] = useState<Barber | null>(null); const [services, setServices] =
useState<ServiceGroup[]>([]); const [selectedService, setSelectedService] = useState<ServiceGroup
| null>(null); const [selectedDate, setSelectedDate] = useState<string>(''); const [slots,
setSlots] = useState<TimeSlot[]>([]); const [selectedSlot, setSelectedSlot] = useState<TimeSlot |
null>(null); const [details, setDetails] = useState<ClientDetails>({ name: '', email: '', phone:
'', notes: '' }); const [errors, setErrors] = useState<Partial<ClientDetails>>({}); const
[loading, setLoading] = useState(true); const [slotsLoading, setSlotsLoading] = useState(false);
const [submitting, setSubmitting] = useState(false); const [submitError, setSubmitError] =
useState<string | null>(null); // Pre-select service from query param const preselectedServiceId
= searchParams.get('service'); // Fetch barber profile and services useEffect(() => { async
function fetchProfile() { try { const res = await
fetch(`/api/public/barbers/${params.barberSlug}`); if (!res.ok) throw new Error('Not found');
const json = await res.json(); setBarber(json.data.barber); setServices(json.data.services); if
(preselectedServiceId) { const svc = json.data.services.find((s: ServiceGroup) => s.id ===
preselectedServiceId); if (svc) { setSelectedService(svc); setStep(1); } ... [truncated for
brevity]
```

---

app/b/[barberSlug]/confirmation/page.tsx

```
import Link from 'next/link'; import { notFound } from 'next/navigation'; import { createClient }
from '@/lib/supabase/server'; import type { Booking, ServiceGroup, Barber, Client } from
'@/types'; interface Props { params: Promise<{ barberSlug: string }>; searchParams: Promise<{
booking_id?: string }>; } export default async function ConfirmationPage({ params, searchParams
}: Props) { const { barberSlug } = await params; const { booking_id } = await searchParams; if
(!booking_id) notFound(); const supabase = await createClient(); // Fetch booking with joins
const { data: booking } = await supabase .from('bookings') .select(` *,
service:service_groups(*), client:clients(*), barber:barbers(*) `) .eq('id', booking_id)
.single<Booking & { service: ServiceGroup; client: Client; barber: Barber }>(); if (!booking ||
booking.barber?.slug !== barberSlug) notFound(); const formattedDate = new
Date(booking.starts_at).toLocaleDateString('en-CA', { weekday: 'long', year: 'numeric', month:
'long', day: 'numeric', }); const formattedTime = new
Date(booking.starts_at).toLocaleTimeString('en-CA', { hour: 'numeric', minute: '2-digit', });
const isPaid = booking.payment_status === 'paid'; return ( <div className="text-center space-y-8
py-8"> {/* Success Icon */} <div className="flex justify-center"> <div className="w-20 h-20
rounded-full flex items-center justify-center" style={{ backgroundColor:
`${booking.barber.brand_color}18` }} > <svg className="w-10 h-10" fill="none"
stroke={booking.barber.brand_color} viewBox="0 0 24 24" strokeWidth={2} > ... [truncated for
brevity]
```

Section 8: Barber Dashboard Pages + Admin Pages

---

app/barber/login/page.tsx

```
'use client'; import { useState } from 'react'; import { useRouter, useSearchParams } from
'next/navigation'; import { createClient } from '@/lib/supabase/client'; export default function
BarberLoginPage() { const router = useRouter(); const searchParams = useSearchParams(); const
redirectTo = searchParams.get('redirectTo') ?? '/barber/dashboard'; const [email, setEmail] =
useState(''); const [password, setPassword] = useState(''); const [error, setError] =
useState<string | null>(null); const [loading, setLoading] = useState(false); async function
handleLogin(e: React.FormEvent) { e.preventDefault(); setLoading(true); setError(null); const
supabase = createClient(); const { error: authError } = await supabase.auth.signInWithPassword({
email, password }); if (authError) { setError(authError.message); setLoading(false); return; }
router.push(redirectTo); router.refresh(); } return ( <div className="min-h-screen flex
items-center justify-center bg-gray-50 px-4"> <div className="w-full max-w-sm space-y-8"> <div
className="text-center"> <div className="w-12 h-12 bg-gray-900 rounded-xl mx-auto flex
items-center justify-center mb-4"> <svg className="w-6 h-6 text-white" fill="none" viewBox="0 0
24 24" stroke="currentColor" strokeWidth={2}> <path strokeLinecap="round" strokeLinejoin="round"
d="M16 7a4 4 0 11-8 0 4 4 0 018 0zM12 14a7 7 0 00-7 7h14a7 7 0 00-7-7z" /> </svg> </div> <h1
className="text-2xl font-bold text-gray-900">Barber Login</h1> <p className="text-sm
text-gray-500 mt-1">Sign in to your dashboard</p> </div> <form onSubmit={handleLogin}
className="space-y-4 bg-white p-8 rounded-2xl shadow-sm border border-gray-100"> {error && ( <div
className="bg-red-50 border border-red-200 text-red-700 px-4 py-3 rounded-lg text-sm"> {error}
</div> )} <div> <label className="block text-sm font-medium text-gray-700 mb-1">Email</label>
<input type="email" required value={email} ... [truncated for brevity]
```

---

app/barber/dashboard/page.tsx

```
import { redirect } from 'next/navigation'; import Link from 'next/link'; import { createClient }
from '@/lib/supabase/server'; import StatusBadge from '@/components/StatusBadge'; import type {
Booking, BarberStats } from '@/types'; export default async function BarberDashboardPage() {
const supabase = await createClient(); const { data: { user } } = await supabase.auth.getUser();
if (!user) redirect('/barber/login'); const { data: barber } = await supabase .from('barbers')
.select('*') .eq('user_id', user.id) .single(); if (!barber) redirect('/barber/login'); const
today = new Date(); const startOfDay = new Date(today.setHours(0, 0, 0, 0)).toISOString(); const
endOfDay = new Date(today.setHours(23, 59, 59, 999)).toISOString(); today.setHours(0, 0, 0, 0);
// reset // Fetch stats const [ { count: totalBookings }, { count: confirmedBookings }, { data:
revenueData }, { count: totalClients }, { data: upcomingBookings }, ] = await Promise.all([
supabase.from('bookings').select('*', { count: 'exact', head: true }).eq('barber_id', barber.id),
supabase.from('bookings').select('*', { count: 'exact', head: true }).eq('barber_id',
barber.id).eq('status', 'confirmed'),
supabase.from('payment_records').select('amount_cents').eq('barber_id', barber.id).eq('status',
'paid'), supabase.from('clients').select('*', { count: 'exact', head: true }).eq('barber_id',
barber.id), supabase .from('bookings') .select('*, service:service_groups(name,
duration_minutes), client:clients(name, email)') .eq('barber_id', barber.id) .gte('starts_at',
new Date().toISOString()) .in('status', ['confirmed', 'deposit_pending', 'pending'])
.order('starts_at', { ascending: true }) .limit(5), ]); const totalRevenue = (revenueData ??
[]).reduce((sum, r) => sum + (r.amount_cents ?? 0), 0); const stats: BarberStats = {
total_bookings: totalBookings ?? 0, confirmed_bookings: confirmedBookings ?? 0,
total_revenue_cents: totalRevenue, total_clients: totalClients ?? 0, upcoming_count:
upcomingBookings?.length ?? 0, }; const STAT_CARDS = [ { label: 'Total Bookings', value:
stats.total_bookings.toString() }, { label: 'Confirmed', value:
stats.confirmed_bookings.toString() }, ... [truncated for brevity]
```

---

app/barber/bookings/page.tsx

```
import { redirect } from 'next/navigation'; import Link from 'next/link'; import { createClient }
from '@/lib/supabase/server'; import StatusBadge from '@/components/StatusBadge'; import type {
BookingStatus } from '@/types'; interface Props { searchParams: Promise<{ status?: string; date?:
string; page?: string }>; } export default async function BarberBookingsPage({ searchParams }:
Props) { const { status, date, page: pageStr } = await searchParams; const supabase = await
createClient(); const { data: { user } } = await supabase.auth.getUser(); if (!user)
redirect('/barber/login'); const { data: barber } = await supabase .from('barbers').select('id,
slug').eq('user_id', user.id).single(); if (!barber) redirect('/barber/login'); const page =
parseInt(pageStr ?? '1', 10); const PER_PAGE = 20; const offset = (page - 1) * PER_PAGE; let
query = supabase .from('bookings') .select('*, service:service_groups(name, duration_minutes,
price_cents), client:clients(name, email, phone)', { count: 'exact' }) .eq('barber_id',
barber.id) .order('starts_at', { ascending: false }) .range(offset, offset + PER_PAGE - 1); if
(status && status !== 'all') { query = query.eq('status', status as BookingStatus); } if (date) {
const start = `${date}T00:00:00.000Z`; const end = `${date}T23:59:59.999Z`; query =
query.gte('starts_at', start).lte('starts_at', end); } const { data: bookings, count } = await
query; const totalPages = Math.ceil((count ?? 0) / PER_PAGE); const STATUS_OPTIONS = [ { value:
'all', label: 'All' }, { value: 'pending', label: 'Pending' }, { value: 'confirmed', label:
'Confirmed' }, { value: 'deposit_pending', label: 'Deposit Pending' }, { value: 'completed',
label: 'Completed' }, { value: 'cancelled', label: 'Cancelled' }, { value: 'no_show', label: 'No
Show' }, ]; return ( <div className="min-h-screen bg-gray-50"> <nav className="bg-white border-b
border-gray-200 sticky top-0 z-30"> <div className="max-w-6xl mx-auto px-4 py-3 flex items-center
gap-6 text-sm font-medium text-gray-600"> <Link href="/barber/dashboard"
className="hover:text-gray-900">Dashboard</Link> ... [truncated for brevity]
```

---

app/barber/services/page.tsx

```
'use client'; import { useState, useEffect } from 'react'; import Link from 'next/link'; import
type { ServiceGroup } from '@/types'; export default function BarberServicesPage() { const
[services, setServices] = useState<ServiceGroup[]>([]); const [loading, setLoading] =
useState(true); const [showForm, setShowForm] = useState(false); const [editingService,
setEditingService] = useState<ServiceGroup | null>(null); const [form, setForm] = useState({
name: '', description: '', duration_minutes: 30, price_cents: 0, display_order: 0 }); const
[saving, setSaving] = useState(false); const [error, setError] = useState<string | null>(null);
async function fetchServices() { const res = await fetch('/api/barber/services'); const json =
await res.json(); setServices(json.data ?? []); setLoading(false); } useEffect(() => {
fetchServices(); }, []); function openNew() { setEditingService(null); setForm({ name: '',
description: '', duration_minutes: 30, price_cents: 0, display_order: services.length });
setShowForm(true); } function openEdit(svc: ServiceGroup) { setEditingService(svc); setForm({
name: svc.name, description: svc.description ?? '', duration_minutes: svc.duration_minutes,
price_cents: svc.price_cents, display_order: svc.display_order, }); setShowForm(true); } async
function handleSave(e: React.FormEvent) { e.preventDefault(); setSaving(true); setError(null);
try { const method = editingService ? 'PUT' : 'POST'; const url = editingService ?
`/api/barber/services/${editingService.id}` : '/api/barber/services'; const res = await
fetch(url, { method, headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({
...form, price_cents: Math.round(form.price_cents), }), }); if (!res.ok) { const j = await
res.json(); ... [truncated for brevity]
```

---

app/barber/clients/page.tsx

```
import { redirect } from 'next/navigation'; import Link from 'next/link'; import { createClient }
from '@/lib/supabase/server'; import type { Client } from '@/types'; interface Props {
searchParams: Promise<{ q?: string }>; } export default async function BarberClientsPage({
searchParams }: Props) { const { q } = await searchParams; const supabase = await createClient();
const { data: { user } } = await supabase.auth.getUser(); if (!user) redirect('/barber/login');
const { data: barber } = await supabase.from('barbers').select('id').eq('user_id',
user.id).single(); if (!barber) redirect('/barber/login'); let query = supabase .from('clients')
.select('*') .eq('barber_id', barber.id) .order('last_visit_at', { ascending: false, nullsFirst:
false }); if (q) { query = query.or(`name.ilike.%${q}%,email.ilike.%${q}%`); } const { data:
clients } = await query.returns<Client[]>(); return ( <div className="min-h-screen bg-gray-50">
<nav className="bg-white border-b border-gray-200 sticky top-0 z-30"> <div className="max-w-6xl
mx-auto px-4 py-3 flex items-center gap-6 text-sm font-medium text-gray-600"> <Link
href="/barber/dashboard" className="hover:text-gray-900">Dashboard</Link> <Link
href="/barber/bookings" className="hover:text-gray-900">Bookings</Link> <Link
href="/barber/clients" className="text-gray-900">Clients</Link> <Link href="/barber/services"
className="hover:text-gray-900">Services</Link> <Link href="/barber/settings"
className="hover:text-gray-900">Settings</Link> </div> </nav> <main className="max-w-6xl mx-auto
px-4 py-8 space-y-6"> <div className="flex justify-between items-center"> <h1 className="text-2xl
font-bold text-gray-900">Clients <span className="text-gray-400 font-normal
text-base">({clients?.length ?? 0})</span></h1> </div> {/* Search */} <form method="GET"
action="/barber/clients"> <input name="q" type="search" defaultValue={q} placeholder="Search by
name or email…" className="w-full md:w-80 border border-gray-300 rounded-lg px-3 py-2 text-sm
focus:outline-none focus:ring-2 focus:ring-gray-900" /> </form> {/* Table */} ... [truncated for
brevity]
```

---

app/barber/settings/page.tsx

```
'use client'; import { useState, useEffect } from 'react'; import Link from 'next/link'; import
type { Barber, AvailabilityRule, DepositType } from '@/types'; type Section = 'profile' | 'hours'
| 'booking' | 'notifications'; export default function BarberSettingsPage() { const [section,
setSection] = useState<Section>('profile'); const [barber, setBarber] = useState<Barber |
null>(null); const [availability, setAvailability] = useState<AvailabilityRule[]>([]); const
```

```
[loading, setLoading] = useState(true); const [saving, setSaving] = useState(false); const
[saved, setSaved] = useState(false); const [error, setError] = useState<string | null>(null);
const DAYS = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'];
useEffect(() => { async function load() { const [profileRes, availRes] = await Promise.all([
fetch('/api/barber/profile'), fetch('/api/barber/availability'), ]); const profileJson = await
profileRes.json(); const availJson = await availRes.json(); setBarber(profileJson.data);
setAvailability(availJson.data ?? []); setLoading(false); } load(); }, []); async function
saveProfile(e: React.FormEvent) { e.preventDefault(); if (!barber) return; setSaving(true);
setError(null); try { const res = await fetch('/api/barber/profile', { method: 'PUT', headers: {
'Content-Type': 'application/json' }, body: JSON.stringify({ name: barber.name, bio: barber.bio,
phone: barber.phone, brand_color: barber.brand_color, notification_email:
barber.notification_email, }), }); if (!res.ok) { const j = await res.json(); throw new
Error(j.error); } setSaved(true); setTimeout(() => setSaved(false), 3000); } catch (err: any) {
setError(err.message); } finally { setSaving(false); } } async function saveBookingRules(e:
React.FormEvent) { ... [truncated for brevity]
```

---

app/admin/login/page.tsx

```
'use client'; import { useState } from 'react'; import { useRouter, useSearchParams } from
'next/navigation'; import { createClient } from '@/lib/supabase/client'; export default function
AdminLoginPage() { const router = useRouter(); const searchParams = useSearchParams(); const
redirectTo = searchParams.get('redirectTo') ?? '/admin/dashboard'; const [email, setEmail] =
useState(''); const [password, setPassword] = useState(''); const [error, setError] =
useState<string | null>(null); const [loading, setLoading] = useState(false); async function
handleLogin(e: React.FormEvent) { e.preventDefault(); setLoading(true); setError(null); const
supabase = createClient(); const { data, error: authError } = await
supabase.auth.signInWithPassword({ email, password }); if (authError) {
setError(authError.message); setLoading(false); return; } if (data.user?.user_metadata?.role !==
'admin') { await supabase.auth.signOut(); setError('Access denied. Admin credentials required.');
setLoading(false); return; } router.push(redirectTo); router.refresh(); } return ( <div
className="min-h-screen flex items-center justify-center bg-gray-950 px-4"> <div
className="w-full max-w-sm space-y-8"> <div className="text-center"> <div className="w-12 h-12
bg-white rounded-xl mx-auto flex items-center justify-center mb-4"> <svg className="w-6 h-6
text-gray-900" fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}> <path
strokeLinecap="round" strokeLinejoin="round" d="M9 12l2 2 4-4m5.618-4.016A11.955 11.955 0 0112
2.944a11.955 11.955 0 01-8.618 3.04A12.02 12.02 0 003 9c0 5.591 3.824 10.29 9 11.622 5.176-1.332
9-6.03 9-11.622 0-1.042-.133-2.052-.382-3.016z" /> </svg> </div> <h1 className="text-2xl
font-bold text-white">Admin Login</h1> <p className="text-sm text-gray-400 mt-1">BarberBook
Administration</p> </div> <form onSubmit={handleLogin} className="space-y-4 bg-gray-900 p-8
rounded-2xl border border-gray-800"> {error && <div className="bg-red-900/40 border
border-red-700 text-red-300 px-4 py-3 rounded-lg text-sm">{error}</div>} <div> <label
className="block text-sm font-medium text-gray-300 mb-1">Email</label> <input type="email"
required value={email} onChange={(e) => setEmail(e.target.value)} className="w-full bg-gray-800
border border-gray-700 text-white rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2
focus:ring-white" placeholder="admin@barberbook.app" /> </div> <div> <label className="block
text-sm font-medium text-gray-300 mb-1">Password</label> <input type="password" required
value={password} onChange={(e) => setPassword(e.target.value)} className="w-full bg-gray-800
border border-gray-700 text-white rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2
focus:ring-white" placeholder="••••••••" /> </div> <button type="submit" disabled={loading}
className="w-full bg-white text-gray-900 py-2.5 rounded-lg text-sm font-semibold
hover:bg-gray-100 disabled:opacity-50 transition-colors"> {loading ? 'Signing in…' : 'Sign In'}
</button> </form> ... [truncated for brevity]
```

---

app/admin/dashboard/page.tsx

```
import { redirect } from 'next/navigation'; import Link from 'next/link'; import { createClient }
from '@/lib/supabase/server'; export default async function AdminDashboardPage() { const supabase
= await createClient(); const { data: { user } } = await supabase.auth.getUser(); if (!user ||
user.user_metadata?.role !== 'admin') redirect('/admin/login'); const [ { count: totalBarbers },
{ count: activeBarbers }, { count: totalBookings }, { count: totalClients }, ] = await
Promise.all([ supabase.from('barbers').select('*', { count: 'exact', head: true }),
supabase.from('barbers').select('*', { count: 'exact', head: true }).eq('is_active', true),
supabase.from('bookings').select('*', { count: 'exact', head: true }),
supabase.from('clients').select('*', { count: 'exact', head: true }), ]); const { data:
recentBarbers } = await supabase .from('barbers').select('id, name, slug, is_active,
created_at').order('created_at', { ascending: false }).limit(10); return ( <div
className="min-h-screen bg-gray-950 text-white"> <nav className="border-b border-gray-800 sticky
top-0 z-30 bg-gray-950"> <div className="max-w-6xl mx-auto px-4 py-3 flex items-center
justify-between"> <span className="font-bold text-white">BarberBook Admin</span> <div
className="flex items-center gap-6 text-sm text-gray-400"> <Link href="/admin/dashboard"
className="text-white">Dashboard</Link> <Link href="/admin/barbers"
className="hover:text-white">Barbers</Link> </div> </div> </nav> <main className="max-w-6xl
mx-auto px-4 py-8 space-y-8"> <h1 className="text-2xl font-bold">Platform Overview</h1> <div
className="grid grid-cols-2 md:grid-cols-4 gap-4"> {[ { label: 'Total Barbers', value:
totalBarbers ?? 0 }, { label: 'Active Barbers', value: activeBarbers ?? 0 }, { label: 'Total
Bookings', value: totalBookings ?? 0 }, { label: 'Total Clients', value: totalClients ?? 0 },
].map((s) => ( <div key={s.label} className="bg-gray-900 rounded-xl p-5 border border-gray-800">
<p className="text-xs text-gray-500 uppercase tracking-wide">{s.label}</p> <p className="text-2xl
font-bold text-white mt-1">{s.value}</p> </div> ))} </div> <div className="bg-gray-900 rounded-xl
border border-gray-800"> <div className="px-6 py-4 border-b border-gray-800 flex justify-between
items-center"> <h2 className="font-semibold">Recent Barbers</h2> <Link href="/admin/barbers/new"
className="bg-white text-gray-900 px-3 py-1.5 rounded-lg text-xs font-semibold
hover:bg-gray-100"> + New Barber </Link> </div> ... [truncated for brevity]
```

---

app/admin/barbers/page.tsx

```
import { redirect } from 'next/navigation'; import Link from 'next/link'; import { createClient }
from '@/lib/supabase/server'; export default async function AdminBarbersPage() { const supabase =
await createClient(); const { data: { user } } = await supabase.auth.getUser(); if (!user ||
user.user_metadata?.role !== 'admin') redirect('/admin/login'); const { data: barbers } = await
supabase .from('barbers') .select('*, user:users(email)') .order('created_at', { ascending: false
}); return ( <div className="min-h-screen bg-gray-950 text-white"> <nav className="border-b
border-gray-800 sticky top-0 z-30 bg-gray-950"> <div className="max-w-6xl mx-auto px-4 py-3 flex
items-center gap-6 text-sm text-gray-400"> <span className="font-bold text-white">BarberBook
Admin</span> <Link href="/admin/dashboard" className="hover:text-white">Dashboard</Link> <Link
href="/admin/barbers" className="text-white">Barbers</Link> </div> </nav> <main
className="max-w-6xl mx-auto px-4 py-8 space-y-6"> <div className="flex justify-between
items-center"> <h1 className="text-2xl font-bold">All Barbers</h1> <Link
href="/admin/barbers/new" className="bg-white text-gray-900 px-4 py-2 rounded-lg text-sm
font-semibold hover:bg-gray-100"> + New Barber </Link> </div> <div className="bg-gray-900
rounded-xl border border-gray-800 overflow-hidden"> <table className="w-full text-sm"> <thead
className="border-b border-gray-800"> <tr> {['Name', 'Slug', 'Email', 'Status', 'Created',
'Actions'].map((h) => ( <th key={h} className="px-4 py-3 text-left text-xs font-semibold
text-gray-500 uppercase tracking-wide">{h}</th> ))} </tr> </thead> <tbody className="divide-y
divide-gray-800"> {(barbers ?? []).map((b: any) => ( <tr key={b.id}
className="hover:bg-gray-800/50 transition-colors"> <td className="px-4 py-3 font-medium
text-white">{b.name}</td> <td className="px-4 py-3 text-gray-400 font-mono text-xs">{b.slug}</td>
<td className="px-4 py-3 text-gray-400">{b.user?.email ?? '—'}</td> <td className="px-4 py-3">
<span className={`text-xs px-2 py-0.5 rounded-full ${b.is_active ? 'bg-green-900 text-green-300'
: 'bg-gray-800 text-gray-500'}`}> {b.is_active ? 'Active' : 'Inactive'} </span> </td> <td
className="px-4 py-3 text-gray-500"> {new Date(b.created_at).toLocaleDateString('en-CA')} </td>
<td className="px-4 py-3 flex gap-3"> <Link href={`/b/${b.slug}`} target="_blank"
className="text-xs text-gray-400 hover:text-white">View</Link> </td> </tr> ))} ... [truncated for
brevity]
```

---

app/admin/barbers/new/page.tsx

```
'use client'; import { useState } from 'react'; import { useRouter } from 'next/navigation';
import Link from 'next/link'; export default function AdminNewBarberPage() { const router =
useRouter(); const [form, setForm] = useState({ name: '', email: '', password: '', slug: '', bio:
'', phone: '', brand_color: '#111827', }); const [saving, setSaving] = useState(false); const
[error, setError] = useState<string | null>(null); function slugify(name: string) { return
name.toLowerCase().replace(/[^a-z0-9]+/g, '-').replace(/^-|-$/g, ''); } async function
handleSubmit(e: React.FormEvent) { e.preventDefault(); setSaving(true); setError(null); try {
const res = await fetch('/api/admin/barbers', { method: 'POST', headers: { 'Content-Type':
'application/json' }, body: JSON.stringify(form), }); const json = await res.json(); if (!res.ok)
throw new Error(json.error ?? 'Failed to create barber'); router.push('/admin/barbers'); } catch
(err: any) { setError(err.message); } finally { setSaving(false); } } return ( <div
className="min-h-screen bg-gray-950 text-white"> <nav className="border-b border-gray-800 sticky
top-0 z-30 bg-gray-950"> <div className="max-w-6xl mx-auto px-4 py-3 flex items-center gap-6
text-sm text-gray-400"> <span className="font-bold text-white">BarberBook Admin</span> <Link
href="/admin/dashboard" className="hover:text-white">Dashboard</Link> <Link href="/admin/barbers"
className="hover:text-white">Barbers</Link> </div> </nav> <main className="max-w-xl mx-auto px-4
py-8 space-y-6"> <h1 className="text-2xl font-bold">New Barber</h1> {error && <div
className="bg-red-900/40 border border-red-700 text-red-300 px-4 py-3 rounded-lg
text-sm">{error}</div>} <form onSubmit={handleSubmit} className="bg-gray-900 rounded-xl border
border-gray-800 p-6 space-y-5"> {[ { label: 'Full Name', key: 'name', type: 'text', required:
true, placeholder: 'Marcus Johnson' }, { label: 'Email', key: 'email', type: 'email', required:
true, placeholder: 'marcus@example.com' }, { label: 'Password', key: 'password', type:
'password', required: true, placeholder: '••••••••' }, { label: 'URL Slug', key: 'slug', type:
'text', required: true, placeholder: 'marcus-johnson' }, { label: 'Phone (optional)', key:
'phone', type: 'tel', required: false, placeholder: '+1 (555) 000-0000' }, ].map(({ label, key,
type, required, placeholder }) => ( ... [truncated for brevity]
```

Section 10: API Routes

app/api/public/barbers/[slug]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import type { Barber, ServiceGroup } from '@/types'; export async
function GET( _request: NextRequest, { params }: { params: Promise<{ slug: string }> } ) { const
{ slug } = await params; const supabase = await createClient(); const { data: barber, error } =
await supabase .from('barbers') .select('id, slug, name, bio, phone, avatar_url, brand_color,
brand_logo_url, booking_lead_hours, booking_window_days, cancellation_hours, deposit_type,
deposit_amount, is_active') .eq('slug', slug) .eq('is_active', true) .single<Barber>(); if (error
|| !barber) { return NextResponse.json({ error: 'Barber not found' }, { status: 404 }); } const {
data: services } = await supabase .from('service_groups') .select('*') .eq('barber_id',
barber.id) .eq('is_active', true) .order('display_order', { ascending: true })
.returns<ServiceGroup[]>(); return NextResponse.json({ data: { barber, services: services ?? [],
}, }); }
```

app/api/public/availability/[slug]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { getAvailableSlots } from '@/lib/services/availability.service';
export async function GET( request: NextRequest, { params }: { params: Promise<{ slug: string }>
} ) { const { slug } = await params; const { searchParams } = new URL(request.url); const date =
searchParams.get('date'); const service_id = searchParams.get('service_id'); if (!date ||
!service_id) { return NextResponse.json({ error: 'Missing required params: date, service_id' }, {
status: 400 }); } // Validate date format YYYY-MM-DD if (!/^\d{4}-\d{2}-\d{2}$/.test(date)) {
return NextResponse.json({ error: 'Invalid date format. Use YYYY-MM-DD' }, { status: 400 }); }
const supabase = await createClient(); const { data: barber } = await supabase .from('barbers')
.select('id, booking_lead_hours, booking_window_days') .eq('slug', slug) .eq('is_active', true)
.single(); if (!barber) { return NextResponse.json({ error: 'Barber not found' }, { status: 404
}); } const { data: service } = await supabase .from('service_groups') .select('id,
duration_minutes') .eq('id', service_id) .eq('barber_id', barber.id) .eq('is_active', true)
.single(); if (!service) { return NextResponse.json({ error: 'Service not found' }, { status: 404
}); } const slots = await getAvailableSlots({ barberId: barber.id, date, durationMinutes:
service.duration_minutes, leadHours: barber.booking_lead_hours ?? 24, supabase, }); return
NextResponse.json({ data: { slots, date, service_duration_minutes: service.duration_minutes, ...
[truncated for brevity]
```

---

app/api/public/bookings/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createServiceClient } from
'@/lib/supabase/server'; import { createBooking } from '@/lib/services/booking.service'; import {
upsertClient } from '@/lib/services/client.service'; import { createCheckoutSession } from
'@/lib/services/payment.service'; import { sendConfirmation } from
'@/lib/services/notification.service'; import { z } from 'zod'; const createBookingSchema =
z.object({ barber_slug: z.string().min(1), service_id: z.string().uuid(), starts_at:
z.string().datetime(), client_name: z.string().min(2), client_email: z.string().email(),
client_phone: z.string().optional(), notes: z.string().optional(), pay_deposit: z.boolean(), });
export async function POST(request: NextRequest) { const body = await request.json().catch(() =>
null); if (!body) return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
const parsed = createBookingSchema.safeParse(body); if (!parsed.success) { return
NextResponse.json({ error: 'Validation failed', details: parsed.error.flatten() }, { status: 422
}); } const data = parsed.data; const supabase = await createServiceClient(); // Fetch barber
const { data: barber } = await supabase .from('barbers') .select('id, deposit_type,
deposit_amount') .eq('slug', data.barber_slug) .eq('is_active', true) .single(); if (!barber) {
return NextResponse.json({ error: 'Barber not found' }, { status: 404 }); } // Fetch service
const { data: service } = await supabase .from('service_groups') .select('id, duration_minutes,
price_cents, deposit_override_type, deposit_override_amount') .eq('id', data.service_id)
.eq('barber_id', barber.id) .eq('is_active', true) .single(); if (!service) { return
NextResponse.json({ error: 'Service not found' }, { status: 404 }); } // Upsert client const
client = await upsertClient({ barberId: barber.id, email: data.client_email, ... [truncated for
brevity]
```

---

app/api/barber/profile/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { z } from 'zod'; const updateProfileSchema = z.object({ name:
z.string().min(1).optional(), bio: z.string().nullable().optional(), phone:
z.string().nullable().optional(), brand_color: z.string().regex(/^#[0-9A-Fa-f]{6}$/).optional(),
booking_lead_hours: z.number().int().min(0).optional(), booking_window_days:
z.number().int().min(1).optional(), cancellation_hours: z.number().int().min(0).optional(),
deposit_type: z.enum(['none', 'fixed', 'percentage']).optional(), deposit_amount:
z.number().int().min(0).optional(), notification_email: z.string().email().nullable().optional(),
}); async function getAuthBarber(supabase: Awaited<ReturnType<typeof createClient>>) { const {
```

```
data: { user } } = await supabase.auth.getUser(); if (!user) return null; const { data: barber }
= await supabase.from('barbers').select('*').eq('user_id', user.id).single(); return barber; }
export async function GET() { const supabase = await createClient(); const barber = await
getAuthBarber(supabase); if (!barber) return NextResponse.json({ error: 'Unauthorized' }, {
status: 401 }); return NextResponse.json({ data: barber }); } export async function PUT(request:
NextRequest) { const supabase = await createClient(); const barber = await
getAuthBarber(supabase); if (!barber) return NextResponse.json({ error: 'Unauthorized' }, {
status: 401 }); const body = await request.json().catch(() => null); if (!body) return
NextResponse.json({ error: 'Invalid JSON' }, { status: 400 }); const parsed =
updateProfileSchema.safeParse(body); if (!parsed.success) { return NextResponse.json({ error:
'Validation failed', details: parsed.error.flatten() }, { status: 422 }); } const { data:
updated, error } = await supabase .from('barbers') .update({ ...parsed.data, updated_at: new
Date().toISOString() }) .eq('id', barber.id) .select() .single(); if (error) return
NextResponse.json({ error: error.message }, { status: 500 }); return NextResponse.json({ data:
updated }); }
```

---

app/api/barber/services/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { z } from 'zod'; const createServiceSchema = z.object({ name:
z.string().min(1), description: z.string().optional(), duration_minutes: z.number().int().min(5),
price_cents: z.number().int().min(0), deposit_override_type: z.enum(['none', 'fixed',
'percentage']).nullable().optional(), deposit_override_amount:
z.number().int().min(0).nullable().optional(), display_order: z.number().int().min(0).optional(),
}); async function getAuthBarber(supabase: Awaited<ReturnType<typeof createClient>>) { const {
data: { user } } = await supabase.auth.getUser(); if (!user) return null; const { data: barber }
= await supabase.from('barbers').select('id').eq('user_id', user.id).single(); return barber; }
export async function GET() { const supabase = await createClient(); const barber = await
getAuthBarber(supabase); if (!barber) return NextResponse.json({ error: 'Unauthorized' }, {
status: 401 }); const { data: services } = await supabase .from('service_groups') .select('*')
.eq('barber_id', barber.id) .order('display_order', { ascending: true }); return
NextResponse.json({ data: services ?? [] }); } export async function POST(request: NextRequest) {
const supabase = await createClient(); const barber = await getAuthBarber(supabase); if (!barber)
return NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); const body = await
request.json().catch(() => null); const parsed = createServiceSchema.safeParse(body); if
(!parsed.success) { return NextResponse.json({ error: 'Validation failed', details:
parsed.error.flatten() }, { status: 422 }); } const { data: service, error } = await supabase
.from('service_groups') .insert({ ...parsed.data, barber_id: barber.id, is_active: true })
.select() .single(); if (error) return NextResponse.json({ error: error.message }, { status: 500
}); return NextResponse.json({ data: service }, { status: 201 }); }
```

---

app/api/barber/services/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { z } from 'zod'; const updateServiceSchema = z.object({ name:
z.string().min(1).optional(), description: z.string().nullable().optional(), duration_minutes:
z.number().int().min(5).optional(), price_cents: z.number().int().min(0).optional(),
deposit_override_type: z.enum(['none', 'fixed', 'percentage']).nullable().optional(),
deposit_override_amount: z.number().int().min(0).nullable().optional(), display_order:
z.number().int().min(0).optional(), is_active: z.boolean().optional(), }); async function
getAuthBarber(supabase: Awaited<ReturnType<typeof createClient>>) { const { data: { user } } =
await supabase.auth.getUser(); if (!user) return null; const { data: barber } = await
supabase.from('barbers').select('id').eq('user_id', user.id).single(); return barber; } export
async function PUT( request: NextRequest, { params }: { params: Promise<{ id: string }> } ) {
const { id } = await params; const supabase = await createClient(); const barber = await
```

```
getAuthBarber(supabase); if (!barber) return NextResponse.json({ error: 'Unauthorized' }, {
status: 401 }); const body = await request.json().catch(() => null); const parsed =
updateServiceSchema.safeParse(body); if (!parsed.success) { return NextResponse.json({ error:
'Validation failed', details: parsed.error.flatten() }, { status: 422 }); } // Verify ownership
const { data: existing } = await supabase .from('service_groups').select('id').eq('id',
id).eq('barber_id', barber.id).single(); if (!existing) return NextResponse.json({ error:
'Service not found' }, { status: 404 }); const { data: service, error } = await supabase
.from('service_groups') .update({ ...parsed.data, updated_at: new Date().toISOString() })
.eq('id', id) .select() .single(); if (error) return NextResponse.json({ error: error.message },
{ status: 500 }); return NextResponse.json({ data: service }); } export async function DELETE(
_request: NextRequest, { params }: { params: Promise<{ id: string }> } ) { const { id } = await
params; const supabase = await createClient(); const barber = await getAuthBarber(supabase); ...
[truncated for brevity]
```

---

app/api/barber/bookings/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; export async function GET(request: NextRequest) { const supabase = await
createClient(); const { data: { user } } = await supabase.auth.getUser(); if (!user) return
NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); const { data: barber } = await
supabase.from('barbers').select('id').eq('user_id', user.id).single(); if (!barber) return
NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); const { searchParams } = new
URL(request.url); const status = searchParams.get('status'); const date =
searchParams.get('date'); const page = parseInt(searchParams.get('page') ?? '1', 10); const
perPage = parseInt(searchParams.get('per_page') ?? '20', 10); const offset = (page - 1) *
perPage; let query = supabase .from('bookings') .select('*, service:service_groups(name,
duration_minutes, price_cents), client:clients(name, email, phone)', { count: 'exact' })
.eq('barber_id', barber.id) .order('starts_at', { ascending: false }) .range(offset, offset +
perPage - 1); if (status && status !== 'all') query = query.eq('status', status); if (date) {
query = query .gte('starts_at', `${date}T00:00:00.000Z`) .lte('starts_at',
`${date}T23:59:59.999Z`); } const { data: bookings, count, error } = await query; if (error)
return NextResponse.json({ error: error.message }, { status: 500 }); return NextResponse.json({
data: { bookings: bookings ?? [], total: count ?? 0, page, per_page: perPage } }); }
```

---

app/api/barber/bookings/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { z } from 'zod'; import { sendCancellation } from
'@/lib/services/notification.service'; const patchSchema = z.object({ status:
z.enum(['confirmed', 'cancelled', 'no_show', 'completed']), notes: z.string().optional(), });
export async function PATCH( request: NextRequest, { params }: { params: Promise<{ id: string }>
} ) { const { id } = await params; const supabase = await createClient(); const { data: { user } }
= await supabase.auth.getUser(); if (!user) return NextResponse.json({ error: 'Unauthorized' }, {
status: 401 }); const { data: barber } = await
supabase.from('barbers').select('id').eq('user_id', user.id).single(); if (!barber) return
NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); const { data: booking } = await
supabase .from('bookings').select('*').eq('id', id).eq('barber_id', barber.id).single(); if
(!booking) return NextResponse.json({ error: 'Booking not found' }, { status: 404 }); const body
= await request.json().catch(() => null); const parsed = patchSchema.safeParse(body); if
(!parsed.success) { return NextResponse.json({ error: 'Validation failed', details:
parsed.error.flatten() }, { status: 422 }); } const { data: updated, error } = await supabase
.from('bookings') .update({ status: parsed.data.status, notes: parsed.data.notes ??
booking.notes, updated_at: new Date().toISOString() }) .eq('id', id) .select() .single(); if
(error) return NextResponse.json({ error: error.message }, { status: 500 }); // Send cancellation
email if relevant if (parsed.data.status === 'cancelled') { await sendCancellation({ bookingId:
```

```
id, supabase }).catch(console.error); } return NextResponse.json({ data: updated }); }
```

---

app/api/barber/clients/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; export async function GET(request: NextRequest) { const supabase = await
createClient(); const { data: { user } } = await supabase.auth.getUser(); if (!user) return
NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); const { data: barber } = await
supabase.from('barbers').select('id').eq('user_id', user.id).single(); if (!barber) return
NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); const { searchParams } = new
URL(request.url); const q = searchParams.get('q'); let query = supabase .from('clients')
.select('*', { count: 'exact' }) .eq('barber_id', barber.id) .order('last_visit_at', { ascending:
false, nullsFirst: false }); if (q) query = query.or(`name.ilike.%${q}%,email.ilike.%${q}%`);
const { data: clients, count, error } = await query; if (error) return NextResponse.json({ error:
error.message }, { status: 500 }); return NextResponse.json({ data: { clients: clients ?? [],
total: count ?? 0 } }); }
```

---

app/api/barber/availability/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { z } from 'zod'; const ruleSchema = z.object({ day_of_week:
z.number().int().min(0).max(6), start_time: z.string().regex(/^\d{2}:\d{2}$/), end_time:
z.string().regex(/^\d{2}:\d{2}$/), is_active: z.boolean(), }); const putSchema = z.object({
rules: z.array(ruleSchema) }); async function getAuthBarber(supabase: Awaited<ReturnType<typeof
createClient>>) { const { data: { user } } = await supabase.auth.getUser(); if (!user) return
null; const { data: barber } = await supabase.from('barbers').select('id').eq('user_id',
user.id).single(); return barber; } export async function GET() { const supabase = await
createClient(); const barber = await getAuthBarber(supabase); if (!barber) return
NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); const { data: rules } = await
supabase .from('availability_rules') .select('*') .eq('barber_id', barber.id)
.order('day_of_week', { ascending: true }); return NextResponse.json({ data: rules ?? [] }); }
export async function PUT(request: NextRequest) { const supabase = await createClient(); const
barber = await getAuthBarber(supabase); if (!barber) return NextResponse.json({ error:
'Unauthorized' }, { status: 401 }); const body = await request.json().catch(() => null); const
parsed = putSchema.safeParse(body); if (!parsed.success) { return NextResponse.json({ error:
'Validation failed', details: parsed.error.flatten() }, { status: 422 }); } // Upsert all rules —
delete existing and re-insert await supabase.from('availability_rules').delete().eq('barber_id',
barber.id); if (parsed.data.rules.length > 0) { const { error } = await supabase
.from('availability_rules') .insert(parsed.data.rules.map((r) => ({ ...r, barber_id: barber.id
})())); if (error) return NextResponse.json({ error: error.message }, { status: 500 }); } const {
data: rules } = await supabase .from('availability_rules').select('*').eq('barber_id',
barber.id).order('day_of_week'); return NextResponse.json({ data: rules ?? [] }); }
```

---

app/api/barber/blocked-times/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { z } from 'zod'; const createBlockSchema = z.object({ starts_at:
z.string().datetime(), ends_at: z.string().datetime(), reason: z.string().optional(),
}).refine((d) => new Date(d.ends_at) > new Date(d.starts_at), { message: 'ends_at must be after
starts_at', path: ['ends_at'], }); async function getAuthBarber(supabase:
```

```
Awaited<ReturnType<typeof createClient>>) { const { data: { user } } = await
supabase.auth.getUser(); if (!user) return null; const { data: barber } = await
supabase.from('barbers').select('id').eq('user_id', user.id).single(); return barber; } export
async function GET(request: NextRequest) { const supabase = await createClient(); const barber =
await getAuthBarber(supabase); if (!barber) return NextResponse.json({ error: 'Unauthorized' }, {
status: 401 }); const { searchParams } = new URL(request.url); const from =
searchParams.get('from') ?? new Date().toISOString(); const to = searchParams.get('to'); let
query = supabase .from('blocked_times') .select('*') .eq('barber_id', barber.id)
.gte('starts_at', from) .order('starts_at'); if (to) query = query.lte('ends_at', to); const {
data, error } = await query; if (error) return NextResponse.json({ error: error.message }, {
status: 500 }); return NextResponse.json({ data: data ?? [] }); } export async function
POST(request: NextRequest) { const supabase = await createClient(); const barber = await
getAuthBarber(supabase); if (!barber) return NextResponse.json({ error: 'Unauthorized' }, {
status: 401 }); const body = await request.json().catch(() => null); const parsed =
createBlockSchema.safeParse(body); if (!parsed.success) { return NextResponse.json({ error:
'Validation failed', details: parsed.error.flatten() }, { status: 422 }); } const { data, error }
= await supabase .from('blocked_times') .insert({ ...parsed.data, barber_id: barber.id })
.select() .single(); ... [truncated for brevity]
```

---

app/api/admin/barbers/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient,
createServiceClient } from '@/lib/supabase/server'; import { z } from 'zod'; const
createBarberSchema = z.object({ name: z.string().min(1), email: z.string().email(), password:
z.string().min(8), slug: z.string().min(2).regex(/^[a-z0-9-]+$/, 'Slug must be lowercase letters,
numbers, and hyphens only'), bio: z.string().optional(), phone: z.string().optional(),
brand_color: z.string().regex(/^#[0-9A-Fa-f]{6}$/).optional(), }); async function
assertAdmin(supabase: Awaited<ReturnType<typeof createClient>>) { const { data: { user } } =
await supabase.auth.getUser(); return user?.user_metadata?.role === 'admin' ? user : null; }
export async function GET() { const supabase = await createClient(); const admin = await
assertAdmin(supabase); if (!admin) return NextResponse.json({ error: 'Forbidden' }, { status: 403
}); const { data: barbers, error } = await supabase .from('barbers') .select('*,
user:users(email)') .order('created_at', { ascending: false }); if (error) return
NextResponse.json({ error: error.message }, { status: 500 }); return NextResponse.json({ data:
barbers ?? [] }); } export async function POST(request: NextRequest) { const supabase = await
createClient(); const admin = await assertAdmin(supabase); if (!admin) return NextResponse.json({
error: 'Forbidden' }, { status: 403 }); const body = await request.json().catch(() => null);
const parsed = createBarberSchema.safeParse(body); if (!parsed.success) { return
NextResponse.json({ error: 'Validation failed', details: parsed.error.flatten() }, { status: 422
}); } const serviceClient = await createServiceClient(); // Check slug uniqueness const { count:
slugCount } = await serviceClient .from('barbers').select('*', { count: 'exact', head: true
}).eq('slug', parsed.data.slug); if ((slugCount ?? 0) > 0) { return NextResponse.json({ error:
'Slug is already taken' }, { status: 409 }); } // Create auth user with barber role const { data:
authData, error: authError } = await serviceClient.auth.admin.createUser({ email:
parsed.data.email, password: parsed.data.password, email_confirm: true, user_metadata: { role:
'barber', name: parsed.data.name }, }); ... [truncated for brevity]
```

---

app/api/admin/barbers/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { z } from 'zod'; const patchSchema = z.object({ name:
z.string().min(1).optional(), is_active: z.boolean().optional(), brand_color:
z.string().regex(/^#[0-9A-Fa-f]{6}$/).optional(), }); async function assertAdmin(supabase:
Awaited<ReturnType<typeof createClient>>) { const { data: { user } } = await
supabase.auth.getUser(); return user?.user_metadata?.role === 'admin' ? user : null; } export
```

```
async function GET( _req: NextRequest, { params }: { params: Promise<{ id: string }> } ) { const {
id } = await params; const supabase = await createClient(); if (!await assertAdmin(supabase))
return NextResponse.json({ error: 'Forbidden' }, { status: 403 }); const { data: barber, error }
= await supabase.from('barbers').select('*').eq('id', id).single(); if (error || !barber) return
NextResponse.json({ error: 'Not found' }, { status: 404 }); return NextResponse.json({ data:
barber }); } export async function PATCH( request: NextRequest, { params }: { params: Promise<{
id: string }> } ) { const { id } = await params; const supabase = await createClient(); if (!await
assertAdmin(supabase)) return NextResponse.json({ error: 'Forbidden' }, { status: 403 }); const
body = await request.json().catch(() => null); const parsed = patchSchema.safeParse(body); if
(!parsed.success) { return NextResponse.json({ error: 'Validation failed', details:
parsed.error.flatten() }, { status: 422 }); } const { data: barber, error } = await supabase
.from('barbers') .update({ ...parsed.data, updated_at: new Date().toISOString() }) .eq('id', id)
.select() .single(); if (error) return NextResponse.json({ error: error.message }, { status: 500
}); return NextResponse.json({ data: barber }); } export async function DELETE( _req:
NextRequest, { params }: { params: Promise<{ id: string }> } ) { const { id } = await params;
const supabase = await createClient(); if (!await assertAdmin(supabase)) return
NextResponse.json({ error: 'Forbidden' }, { status: 403 }); ... [truncated for brevity]
```

---

app/api/payments/create-checkout/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { createCheckoutSession } from '@/lib/services/payment.service';
import { z } from 'zod'; const schema = z.object({ booking_id: z.string().uuid(), success_url:
z.string().url(), cancel_url: z.string().url(), }); export async function POST(request:
NextRequest) { const body = await request.json().catch(() => null); const parsed =
schema.safeParse(body); if (!parsed.success) { return NextResponse.json({ error: 'Validation
failed', details: parsed.error.flatten() }, { status: 422 }); } const supabase = await
createClient(); // Fetch booking with client + service info const { data: booking } = await
supabase .from('bookings') .select('*, client:clients(email), service:service_groups(name,
duration_minutes)') .eq('id', parsed.data.booking_id) .single(); if (!booking) return
NextResponse.json({ error: 'Booking not found' }, { status: 404 }); if
(booking.deposit_amount_cents <= 0) { return NextResponse.json({ error: 'No deposit required for
this booking' }, { status: 400 }); } const session = await createCheckoutSession({ bookingId:
booking.id, amountCents: booking.deposit_amount_cents, currency: 'usd', clientEmail:
booking.client?.email ?? '', serviceName: `Deposit — ${booking.service?.name}`, successUrl:
parsed.data.success_url, cancelUrl: parsed.data.cancel_url, }); if (!session) return
NextResponse.json({ error: 'Failed to create checkout session' }, { status: 500 }); // Store
session ID on booking await supabase .from('bookings') .update({ stripe_checkout_session_id:
session.id }) .eq('id', booking.id); return NextResponse.json({ data: { checkout_url:
session.url, session_id: session.id } }); }
```

---

app/api/payments/webhook/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import Stripe from 'stripe'; import {
createServiceClient } from '@/lib/supabase/server'; import { sendConfirmation } from
'@/lib/services/notification.service'; import { updateClientStats } from
'@/lib/services/client.service'; const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
apiVersion: '2025-01-27.acacia' }); export const dynamic = 'force-dynamic'; export async function
POST(request: NextRequest) { const rawBody = await request.text(); const signature =
request.headers.get('stripe-signature'); if (!signature) { return NextResponse.json({ error:
'Missing stripe-signature header' }, { status: 400 }); } let event: Stripe.Event; try { event =
stripe.webhooks.constructEvent(rawBody, signature, process.env.STRIPE_WEBHOOK_SECRET!); } catch
(err: any) { console.error('[Stripe Webhook] Signature verification failed:', err.message);
return NextResponse.json({ error: `Webhook Error: ${err.message}` }, { status: 400 }); } const
supabase = await createServiceClient(); if (event.type === 'checkout.session.completed') { const
```

```
session = event.data.object as Stripe.Checkout.Session; if (session.payment_status !== 'paid') {
return NextResponse.json({ received: true }); } const bookingId = session.metadata?.booking_id;
if (!bookingId) { console.error('[Stripe Webhook] Missing booking_id in session metadata');
return NextResponse.json({ received: true }); } // Fetch booking const { data: booking } = await
supabase .from('bookings') .select('*, client:clients(id, total_visits, total_spent_cents),
service:service_groups(price_cents)') .eq('id', bookingId) .single(); if (!booking) {
console.error(`[Stripe Webhook] Booking ${bookingId} not found`); return NextResponse.json({
received: true }); } // Update booking status await supabase .from('bookings') .update({ status:
'confirmed', payment_status: 'paid', ... [truncated for brevity]
```

---

app/api/notifications/send/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createClient } from
'@/lib/supabase/server'; import { send } from '@/lib/services/notification.service'; import { z }
from 'zod'; import { MessageType } from '@/types'; const schema = z.object({ booking_id:
z.string().uuid(), type: z.nativeEnum(MessageType), }); export async function POST(request:
NextRequest) { const supabase = await createClient(); const { data: { user } } = await
supabase.auth.getUser(); if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status:
401 }); const { data: barber } = await supabase.from('barbers').select('id').eq('user_id',
user.id).single(); if (!barber) return NextResponse.json({ error: 'Unauthorized' }, { status: 401
}); const body = await request.json().catch(() => null); const parsed = schema.safeParse(body);
if (!parsed.success) { return NextResponse.json({ error: 'Validation failed', details:
parsed.error.flatten() }, { status: 422 }); } // Verify booking belongs to this barber const {
data: booking } = await supabase .from('bookings').select('id').eq('id',
parsed.data.booking_id).eq('barber_id', barber.id).single(); if (!booking) return
NextResponse.json({ error: 'Booking not found' }, { status: 404 }); const result = await send({
bookingId: parsed.data.booking_id, type: parsed.data.type, supabase }); if (!result.success) {
return NextResponse.json({ error: result.error ?? 'Failed to send notification' }, { status: 500
}); } return NextResponse.json({ data: { sent: true, message_id: result.messageId } }); }
```

---

app/api/cron/reminders/route.ts

```
import { NextRequest, NextResponse } from 'next/server'; import { createServiceClient } from
'@/lib/supabase/server'; import { sendReminder } from '@/lib/services/notification.service';
export const dynamic = 'force-dynamic'; export async function GET(request: NextRequest) { //
Verify cron secret const authHeader = request.headers.get('authorization'); const expectedSecret
= `Bearer ${process.env.INTERNAL_CRON_SECRET}`; if (authHeader !== expectedSecret) { return
NextResponse.json({ error: 'Unauthorized' }, { status: 401 }); } const supabase = await
createServiceClient(); // Find all confirmed bookings in the 23–25 hour window from now const now
= new Date(); const windowStart = new Date(now.getTime() + 23 * 60 * 60 * 1000); const windowEnd =
new Date(now.getTime() + 25 * 60 * 60 * 1000); const { data: bookings, error } = await supabase
.from('bookings') .select('id, barber_id, client_id') .eq('status', 'confirmed')
.gte('starts_at', windowStart.toISOString()) .lte('starts_at', windowEnd.toISOString()); if
(error) { console.error('[Cron Reminders] Error fetching bookings:', error.message); return
NextResponse.json({ error: error.message }, { status: 500 }); } if (!bookings || bookings.length
=== 0) { return NextResponse.json({ data: { processed: 0, message: 'No reminders to send' } }); }
// Check which ones already have a reminder sent const bookingIds = bookings.map((b) => b.id);
const { data: alreadySent } = await supabase .from('message_logs') .select('booking_id')
.in('booking_id', bookingIds) .eq('type', 'reminder') .not('sent_at', 'is', null); const
alreadySentIds = new Set((alreadySent ?? []).map((m) => m.booking_id)); const pending =
bookings.filter((b) => !alreadySentIds.has(b.id)); let successCount = 0; let failCount = 0; await
Promise.allSettled( pending.map(async (booking) => { const result = await sendReminder({
bookingId: booking.id, supabase }); if (result.success) { successCount++; } else { failCount++;
... [truncated for brevity]
```

Section 11: Service Layer

src/lib/services/availability.service.ts

```
import type { SupabaseClient } from '@supabase/supabase-js'; import type { TimeSlot } from
'@/types'; interface GetAvailableSlotsParams { barberId: string; date: string; // YYYY-MM-DD
durationMinutes: number; leadHours: number; supabase: SupabaseClient; } interface TimeWindow {
start: Date; end: Date; } /** Merge overlapping time windows into a minimal set of windows. */
export function mergeOverlappingWindows(windows: TimeWindow[]): TimeWindow[] { if (windows.length
=== 0) return []; const sorted = [...windows].sort((a, b) => a.start.getTime() -
b.start.getTime()); const merged: TimeWindow[] = [{ ...sorted[0] }]; for (let i = 1; i <
sorted.length; i++) { const last = merged[merged.length - 1]; const current = sorted[i]; if
(current.start <= last.end) { last.end = new Date(Math.max(last.end.getTime(),
current.end.getTime())); } else { merged.push({ ...current }); } } return merged; } /** * Check
whether a specific slot is available for a barber. * Accounts for existing bookings and blocked
times. */ export async function checkSlotAvailable({ barberId, startsAt, endsAt, supabase,
excludeBookingId, }: { barberId: string; startsAt: Date; endsAt: Date; supabase: SupabaseClient;
excludeBookingId?: string; }): Promise<boolean> { // Check for overlapping confirmed/pending
bookings let bookingQuery = supabase .from('bookings') .select('id', { count: 'exact', head: true
}) .eq('barber_id', barberId) .in('status', ['confirmed', 'pending', 'deposit_pending'])
.lt('starts_at', endsAt.toISOString()) ... [truncated for brevity]
```

src/lib/services/booking.service.ts

```
import type { SupabaseClient } from '@supabase/supabase-js'; import type { Booking, BookingStatus
} from '@/types'; interface CreateBookingParams { barberId: string; clientId: string; serviceId:
string; startsAt: string; endsAt: string; notes?: string; depositAmountCents: number; status:
BookingStatus | string; supabase: SupabaseClient; } export async function createBooking(params:
CreateBookingParams): Promise<Booking | null> { const { supabase, ...fields } = params; //
Double-check for conflicts before inserting (race condition guard) const { count: conflicts } =
await supabase .from('bookings') .select('id', { count: 'exact', head: true }) .eq('barber_id',
fields.barberId) .in('status', ['confirmed', 'pending', 'deposit_pending']) .lt('starts_at',
fields.endsAt) .gt('ends_at', fields.startsAt); if ((conflicts ?? 0) > 0) {
console.warn('[Booking] Slot conflict detected — rejecting booking'); return null; } const {
data, error } = await supabase .from('bookings') .insert({ barber_id: fields.barberId, client_id:
fields.clientId, service_id: fields.serviceId, starts_at: fields.startsAt, ends_at:
fields.endsAt, notes: fields.notes ?? null, deposit_amount_cents: fields.depositAmountCents,
status: fields.status, payment_status: fields.depositAmountCents > 0 ? 'pending' : 'none', })
.select() .single<Booking>(); if (error) { console.error('[Booking] Create error:',
error.message); return null; } return data; } export async function updateBookingStatus({
bookingId, status, notes, ... [truncated for brevity]
```

src/lib/services/client.service.ts

```
import type { SupabaseClient } from '@supabase/supabase-js'; import type { Client, Booking } from
'@/types'; interface UpsertClientParams { barberId: string; email: string; name: string; phone?:
string; supabase: SupabaseClient; } export async function upsertClient(params:
```

```
UpsertClientParams): Promise<Client | null> { const { supabase, barberId, email, name, phone } =
params; // Check if client exists for this barber const { data: existing } = await supabase
.from('clients') .select('*') .eq('barber_id', barberId) .eq('email', email.toLowerCase().trim())
.single<Client>(); if (existing) { // Update name/phone if changed const updates: Partial<Client>
= { updated_at: new Date().toISOString() }; if (name !== existing.name) updates.name = name; if
(phone && phone !== existing.phone) updates.phone = phone; if (Object.keys(updates).length > 1) {
const { data: updated } = await supabase .from('clients') .update(updates) .eq('id', existing.id)
.select() .single<Client>(); return updated; } return existing; } // Create new client const {
data, error } = await supabase .from('clients') .insert({ barber_id: barberId, email:
email.toLowerCase().trim(), name, phone: phone ?? null, total_visits: 0, total_spent_cents: 0, })
.select() .single<Client>(); if (error) { console.error('[Client] Upsert error:', error.message);
return null; } ... [truncated for brevity]
```

---

src/lib/services/notification.service.ts

```
import type { SupabaseClient } from '@supabase/supabase-js'; import { Resend } from 'resend';
import { MessageType } from '@/types'; const resend = new Resend(process.env.RESEND_API_KEY!);
const FROM_EMAIL = 'noreply@barberbook.app'; interface SendResult { success: boolean; messageId?:
string; error?: string; } interface BookingEmailContext { bookingId: string; supabase:
SupabaseClient; } async function fetchBookingContext(bookingId: string, supabase: SupabaseClient)
{ const { data, error } = await supabase .from('bookings') .select(` *, client:clients(id, name,
email), service:service_groups(name, duration_minutes, price_cents), barber:barbers(name,
notification_email, cancellation_hours) `) .eq('id', bookingId) .single(); if (error || !data)
return null; return data as any; } function formatDatetime(iso: string): string { return new
Date(iso).toLocaleString('en-US', { weekday: 'long', year: 'numeric', month: 'long', day:
'numeric', hour: 'numeric', minute: '2-digit', timeZone: 'America/Toronto', }); } function
confirmationHtml(booking: any): string { return ` <div style="font-family: Arial, sans-serif;
max-width: 600px; margin: 0 auto; padding: 24px;"> <h2 style="color: #111827;">Your appointment
is confirmed!</h2> <div style="background: #f9fafb; border-radius: 12px; padding: 20px; margin:
20px 0;"> <p style="margin: 8px 0;"><strong>Barber:</strong> ${booking.barber.name}</p> <p
style="margin: 8px 0;"><strong>Service:</strong> ${booking.service.name}</p> <p style="margin:
8px 0;"><strong>Date & Time:</strong> ${formatDatetime(booking.starts_at)}</p> <p style="margin:
8px 0;"><strong>Duration:</strong> ${booking.service.duration_minutes} minutes</p> <p
style="margin: 8px 0;"><strong>Price:</strong> $${(booking.service.price_cents /
100).toFixed(2)}</p> ${booking.deposit_amount_cents > 0 && booking.payment_status === 'paid' ?
`<p style="margin: 8px 0; color: #16a34a;"><strong>Deposit paid:</strong>
$${(booking.deposit_amount_cents / 100).toFixed(2)}</p>` : ''} </div>
${booking.barber.cancellation_hours ? `<p style="color: #6b7280; font-size: 14px;">Free
cancellation up to ${booking.barber.cancellation_hours} hours before your appointment.</p>` : ''}
<p style="color: #6b7280; font-size: 12px;">Booking reference: ${booking.id.slice(0,
8).toUpperCase()}</p> ... [truncated for brevity]
```

---

src/lib/services/payment.service.ts

```
import Stripe from 'stripe'; import type { SupabaseClient } from '@supabase/supabase-js'; import
type { PaymentStatus } from '@/types'; const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!,
{ apiVersion: '2025-01-27.acacia' }); interface CreateCheckoutParams { bookingId: string;
amountCents: number; currency: string; clientEmail: string; serviceName: string; successUrl:
string; cancelUrl: string; } export async function createCheckoutSession(params:
CreateCheckoutParams): Promise<Stripe.Checkout.Session | null> { try { const session = await
stripe.checkout.sessions.create({ payment_method_types: ['card'], mode: 'payment',
customer_email: params.clientEmail, line_items: [ { price_data: { currency: params.currency,
product_data: { name: params.serviceName, }, unit_amount: params.amountCents, }, quantity: 1, },
], metadata: { booking_id: params.bookingId, }, success_url: params.successUrl, cancel_url:
params.cancelUrl, expires_at: Math.floor(Date.now() / 1000) + 30 * 60, // 30 minutes }); return
```

```
session; } catch (err: any) { console.error('[Payment] Create checkout error:', err.message);
return null; } } export async function handleWebhook({ rawBody, signature, webhookSecret, }: {
rawBody: string; signature: string; webhookSecret: string; }): Promise<Stripe.Event | null> { try
{ return stripe.webhooks.constructEvent(rawBody, signature, webhookSecret); ... [truncated for
brevity]
```

Section 12: Database Schema

File: `supabase/migrations/001_initial.sql`

```
-- ─── Extensions ──────────────────────────────────────────────────────────── create extension
if not exists "uuid-ossp"; create extension if not exists "pgcrypto"; -- ─── ENUMs
──────────────────────────────────────────────────────────────────────────── create type booking_status as
enum ( 'pending', 'deposit_pending', 'confirmed', 'cancelled', 'no_show', 'completed' ); create
type payment_status as enum ( 'none', 'pending', 'paid', 'refunded', 'failed' ); create type
deposit_type as enum ( 'none', 'fixed', 'percentage' ); create type message_type as enum (
'confirmation', 'reminder', 'cancellation', 'custom' ); -- ─── Barbers
──────────────────────────────────────────────────────────────────────────── create table barbers ( id uuid
primary key default uuid_generate_v4(), user_id uuid not null references auth.users(id) on delete
cascade, slug text not null unique, name text not null, bio text, phone text, avatar_url text,
brand_color text not null default '#111827', brand_logo_url text, booking_lead_hours integer not
null default 24 check (booking_lead_hours >= 0), booking_window_days integer not null default 30
check (booking_window_days >= 1), cancellation_hours integer not null default 24 check
(cancellation_hours >= 0), deposit_type deposit_type not null default 'none', deposit_amount
integer not null default 0 check (deposit_amount >= 0), stripe_account_id text,
notification_email text, is_active boolean not null default true, created_at timestamptz not null
default now(), updated_at timestamptz not null default now(), constraint slug_format check (slug
~ '^[a-z0-9-]+$') ); ... [truncated for brevity]
```

Section 13: Reusable UI Components + Vercel Config

`src/components/BookingStepIndicator.tsx`

```
interface Props { steps: string[]; currentStep: number; brandColor: string; } export default
function BookingStepIndicator({ steps, currentStep, brandColor }: Props) { return ( <nav
aria-label="Booking progress" className="w-full"> <ol className="flex items-center w-full">
{steps.map((label, index) => { const isCompleted = index < currentStep; const isCurrent = index
=== currentStep; return ( <li key={label} className={`flex items-center ${index < steps.length -
1 ? 'flex-1' : ''}`} > {/* Step circle */} <div className="flex flex-col items-center"> <div
className={` w-8 h-8 rounded-full flex items-center justify-center text-xs font-semibold
transition-all duration-200 ${isCompleted ? 'text-white' : isCurrent ? 'text-white ring-2
ring-offset-2' : 'bg-gray-100 text-gray-400'} `} style={ isCompleted ? { backgroundColor:
brandColor } : isCurrent ? { backgroundColor: brandColor, ['--tw-ring-color' as string]:
brandColor } : undefined } aria-current={isCurrent ? 'step' : undefined} > {isCompleted ? ( <svg
className="w-4 h-4" fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={3}> <path
strokeLinecap="round" strokeLinejoin="round" d="M5 13l4 4L19 7" /> </svg> ) : ( <span>{index +
1}</span> )} </div> <span className={`mt-1.5 text-xs font-medium whitespace-nowrap ${ isCurrent ?
```

```
'text-gray-900' : isCompleted ? 'text-gray-500' : 'text-gray-400' }`} > {label} </span> </div>
{/* Connector line */} {index < steps.length - 1 && ( <div className={`h-px flex-1 mx-2 mb-5
transition-all duration-200 ${ isCompleted ? 'bg-current' : 'bg-gray-200' }`} style={isCompleted
? { color: brandColor } : undefined} ... [truncated for brevity]
```

---

src/components/ServiceCard.tsx

```
import Link from 'next/link'; import type { ServiceGroup } from '@/types'; interface Props {
service: ServiceGroup; barberSlug: string; brandColor: string; } export default function
ServiceCard({ service, barberSlug, brandColor }: Props) { const depositText = (() => { if
(!service.deposit_override_type || service.deposit_override_type === 'none') return null; if
(service.deposit_override_type === 'fixed' && service.deposit_override_amount) { return
`$${(service.deposit_override_amount / 100).toFixed(2)} deposit`; } if
(service.deposit_override_type === 'percentage' && service.deposit_override_amount) { return
`${(service.deposit_override_amount / 100).toFixed(0)}% deposit`; } return null; })(); return (
<div className="group bg-white border border-gray-200 rounded-xl p-5 hover:border-gray-300
hover:shadow-sm transition-all"> <div className="flex items-start justify-between gap-4"> {/*
Service info */} <div className="flex-1 min-w-0"> <h3 className="font-semibold text-gray-900
text-base">{service.name}</h3> {service.description && ( <p className="text-sm text-gray-500 mt-1
leading-relaxed line-clamp-2"> {service.description} </p> )} <div className="flex items-center
gap-3 mt-2 flex-wrap"> <span className="inline-flex items-center gap-1 text-xs text-gray-400">
<svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24" stroke="currentColor"
strokeWidth={2}> <path strokeLinecap="round" strokeLinejoin="round" d="M12 8v4l3 3m6-3a9 9 0
11-18 0 9 9 0 0118 0z" /> </svg> {service.duration_minutes} min </span> {depositText && ( <span
className="text-xs bg-amber-50 text-amber-700 px-2 py-0.5 rounded-full font-medium">
{depositText} </span> )} </div> </div> {/* Price + CTA */} <div className="flex flex-col
items-end gap-3 shrink-0"> <span className="text-lg font-bold text-gray-900">
${(service.price_cents / 100).toFixed(2)} </span> <Link
href={`/b/${barberSlug}/book?service=${service.id}`} className="px-4 py-2 rounded-lg text-white
text-sm font-medium transition-opacity hover:opacity-90 whitespace-nowrap" style={{
backgroundColor: brandColor }} > Book ... [truncated for brevity]
```

---

src/components/TimeSlotPicker.tsx

```
import type { TimeSlot } from '@/types'; interface Props { slots: TimeSlot[]; selectedSlot:
TimeSlot | null; onSelect: (slot: TimeSlot) => void; brandColor: string; } function
formatTime(iso: string): string { return new Date(iso).toLocaleTimeString('en-CA', { hour:
'numeric', minute: '2-digit', hour12: true, }); } export default function TimeSlotPicker({ slots,
selectedSlot, onSelect, brandColor }: Props) { const availableSlots = slots.filter((s) =>
s.available); if (slots.length === 0) { return ( <div className="py-6 text-center text-gray-400
text-sm border border-dashed border-gray-200 rounded-xl"> No availability on this date. </div> );
} if (availableSlots.length === 0) { return ( <div className="py-6 text-center text-gray-400
text-sm border border-dashed border-gray-200 rounded-xl"> All slots are booked for this date.
Please choose another date. </div> ); } return ( <div className="grid gap-2" style={{
gridTemplateColumns: 'repeat(auto-fill, minmax(90px, 1fr))' }} role="group" aria-label="Available
time slots" > {slots.map((slot) => { const isSelected = selectedSlot?.starts_at ===
slot.starts_at; const isUnavailable = !slot.available; return ( <button key={slot.starts_at}
disabled={isUnavailable} onClick={() => onSelect(slot)} aria-pressed={isSelected}
aria-label={`${formatTime(slot.starts_at)}${isUnavailable ? ' — unavailable' : ''}`} className={`
py-2.5 px-3 rounded-lg text-xs font-medium border-2 transition-all ${isUnavailable ?
'border-gray-100 bg-gray-50 text-gray-300 cursor-not-allowed' : isSelected ? 'text-white
border-transparent shadow-sm' ... [truncated for brevity]
```

---

src/components/StatusBadge.tsx

```
import type { BookingStatus } from '@/types'; interface Props { status: BookingStatus | string;
className?: string; } const STATUS_CONFIG: Record<string, { label: string; classes: string }> = {
pending: { label: 'Pending', classes: 'bg-yellow-50 text-yellow-700 border-yellow-200', },
deposit_pending: { label: 'Deposit Pending', classes: 'bg-orange-50 text-orange-700
border-orange-200', }, confirmed: { label: 'Confirmed', classes: 'bg-green-50 text-green-700
border-green-200', }, cancelled: { label: 'Cancelled', classes: 'bg-gray-50 text-gray-500
border-gray-200', }, no_show: { label: 'No Show', classes: 'bg-red-50 text-red-700
border-red-200', }, completed: { label: 'Completed', classes: 'bg-blue-50 text-blue-700
border-blue-200', }, }; export default function StatusBadge({ status, className = '' }: Props) {
const config = STATUS_CONFIG[status] ?? { label: status, classes: 'bg-gray-50 text-gray-500
border-gray-200', }; return ( <span className={`inline-flex items-center px-2.5 py-0.5
rounded-full text-xs font-medium border ${config.classes} ${className}`} > {config.label} </span>
); }
```

---

src/components/LoadingSpinner.tsx

```
interface Props { size?: 'sm' | 'md' | 'lg'; className?: string; } const SIZE_CLASSES = { sm:
'w-4 h-4', md: 'w-8 h-8', lg: 'w-12 h-12', }; export default function LoadingSpinner({ size =
'md', className = '' }: Props) { return ( <div className={`flex items-center justify-center
${className}`} role="status" aria-label="Loading"> <svg className={`animate-spin text-gray-400
${SIZE_CLASSES[size]}`} xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
aria-hidden="true" > <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor"
strokeWidth="4" /> <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373
0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042 1.135 5.824 3 7.938l3-2.647z" /> </svg>
</div> ); }
```

---

vercel.json

```
{ "crons": [ { "path": "/api/cron/reminders", "schedule": "0 10 * * *" } ] }
```

> *The cron runs daily at 10:00 UTC (6:00 AM ET). It finds all confirmed bookings in the 23–25 hour window from now and sends reminder emails to clients who haven't already received one.*

> *To authenticate the cron call, Vercel includes a special Authorization: Bearer <CRONSECRET> header automatically. Set INTERNALCRON_SECRET in your Vercel environment variables to the same value as in .env.local.*

---

15. Package Configuration

File: `package.json`

```
{ "name": "barberbook", "version": "0.1.0", "private": true, "scripts": { "dev": "next dev",
"build": "next build", "start": "next start", "lint": "next lint", "type-check": "tsc --noEmit"
}, "dependencies": { "@supabase/ssr": "^0.5.2", "@supabase/supabase-js": "^2.49.4", "next":
"14.2.29", "react": "^18", "react-dom": "^18", "resend": "^4.0.0", "stripe": "^17.7.0", "zod":
"^3.23.8" }, "devDependencies": { "@types/node": "^20", "@types/react": "^18",
"@types/react-dom": "^18", "autoprefixer": "^10.0.1", "eslint": "^8", "eslint-config-next":
"14.2.29", "postcss": "^8", "tailwindcss": "^3.4.1", "typescript": "^5" } }
```

File: `next.config.ts`

```
import type { NextConfig } from 'next'; const nextConfig: NextConfig = { images: {
remotePatterns: [ { protocol: 'https', hostname: '*.supabase.co', pathname:
'/storage/v1/object/public/**', }, ], }, // Allow Stripe webhook to receive raw body
experimental: { serverComponentsExternalPackages: ['stripe'], }, }; export default nextConfig;
```

File: `tailwind.config.ts`

```
import type { Config } from 'tailwindcss'; const config: Config = { content: [
'./src/**/*.{js,ts,jsx,tsx,mdx}', './app/**/*.{js,ts,jsx,tsx,mdx}', ], theme: { extend: {
fontFamily: { sans: ['var(--font-inter)', 'ui-sans-serif', 'system-ui', 'sans-serif'], }, colors:
{ brand: 'var(--brand-color)', }, }, }, plugins: [], }; export default config;
```

File: `app/globals.css`

```
@tailwind base; @tailwind components; @tailwind utilities; @layer base { :root { --brand-color:
#111827; } * { box-sizing: border-box; } html { -webkit-font-smoothing: antialiased;
-moz-osx-font-smoothing: grayscale; } ::selection { background-color: var(--brand-color); color:
white; } } @layer utilities { .line-clamp-2 { overflow: hidden; display: -webkit-box;
-webkit-box-orient: vertical; -webkit-line-clamp: 2; } }
```

---

16. Implementation Notes

Stripe Webhook Local Testing

Run the Stripe CLI to forward webhooks during development:

```
stripe listen --forward-to localhost:3000/api/payments/webhook
```

Copy the printed signing secret into `STRIPEWEBHOOKSECRET`.

Supabase Storage (Avatar Uploads)

Create a public `avatars` bucket in Supabase Storage. Upload avatar images from the barber settings page using `supabase.storage.from('avatars').upload(...)` and store the resulting public URL in `barbers.avatar_url`.

Barber Slug Uniqueness

The `slug` column has a `UNIQUE` constraint enforced at the DB level. The admin creation form auto-generates a slug from the barber's name and checks uniqueness before submission.

Timezone Handling

All timestamps are stored in UTC. The `formatDatetime` helper in `notification.service.ts` uses `America/Toronto` as the display timezone — update this to suit your deployment region, or make it configurable per-barber.

Cron Authentication

Vercel automatically injects Authorization: Bearer <CRONSECRET> into cron requests when CRONSECRET is set in the project environment. The INTERNALCRONSECRET env var must match exactly.

Race Condition Protection

Slot booking uses a double-check in `createBooking()` — it queries for conflicts immediately before INSERT, reducing (but not eliminating) race conditions. For high-traffic barbers, consider using a database-level exclusive lock via a Postgres advisory lock wrapped in an RPC function.

Phase 8

Test Plan

43 Tests · E2E · Unit · CI

Phase 8: Test Plan

BarberBook MVP — Technical Specification

Document version: 1.0

Date: 2026-03-30

Stack: Next.js 14, TypeScript, Supabase, Stripe, Resend

Testing tools: Vitest (unit/integration), Playwright (E2E), Supabase local dev

---

Table of Contents

1. [Testing Strategy Overview](#1-testing-strategy-overview)
2. [Customer Booking Flow Tests (E2E — Playwright)](#2-customer-booking-flow-tests-e2e--playwright)
3. [Barber Auth Tests](#3-barber-auth-tests)
4. [Admin Auth Tests](#4-admin-auth-tests)
5. [Booking Conflict Tests (Unit — Vitest)](#5-booking-conflict-tests-unit--vitest)
6. [Payment Tests (Integration — Vitest + Stripe test mode)](#6-payment-tests-integration--vitest--stripe-test-mode)
7. [CRM Tests (Unit — Vitest)](#7-crm-tests-unit--vitest)
8. [Reminder/Notification Tests (Integration — Vitest)](#8-remindernotification-tests-integration--vitest)
9. [API Route Tests (Integration — Vitest)](#9-api-route-tests-integration--vitest)
10. [Test Setup & Helpers](#10-test-setup--helpers)
11. [CI/CD Test Integration](#11-cicd-test-integration)

---

1. Testing Strategy Overview

1.1 Test Pyramid

```
/\ / \ / E2E \ ← ~10% of tests | Playwright | Slowest, highest confidence /--------\ /
Integration\ ← ~30% of tests | Vitest + Supabase local + Stripe test mode /--------------\ / Unit
Tests \ ← ~60% of tests | Vitest | Fast, isolated, deterministic /------------------\
```

The BarberBook test pyramid is deliberately weighted toward unit and integration tests. Business logic — slot generation, conflict detection, CRM matching, webhook idempotency — is best validated at lower layers where tests run in milliseconds and failures are immediately actionable. E2E tests cover the critical happy paths and core failure modes that cannot be caught lower in the stack (redirect behaviour, Stripe checkout handoff, session persistence).

1.2 What Gets Tested at Each Layer

Layer	Scope	Examples
Unit (Vitest)	Pure functions, business logic, data transformations	Availability engine, CRM deduplication logic, slot generation
Integration (Vitest)	API route handlers, DB interactions, external service mocks	Stripe webhook handler, Resend email calls, Supabase RLS policies, API routes
E2E (Playwright)	Full user flows through the browser	Booking a service, logging in as a barber, 404 pages, Stripe redirect

---

1.3 Tools

Tool	Version	Purpose
Vitest	^1.6	Unit and integration test runner
@vitest/coverage-v8	^1.6	Code coverage via V8
Playwright	^1.44	Browser-based E2E tests
@playwright/test	^1.44	Playwright test runner
Supabase CLI	latest	Local Supabase instance for integration tests
stripe (test mode)	^14	Stripe SDK in test mode; webhook signature helpers
msw	^2.3	Mock Service Worker for mocking Resend in integration tests
@faker-js/faker	^8	Generating realistic test data in factories

---

1.4 Coverage Targets

Category	Target	Enforced
Statements	≥ 80%	CI fail below threshold
Branches	≥ 75%	CI fail below threshold
Functions	≥ 80%	CI fail below threshold
Lines	≥ 80%	CI fail below threshold
Critical paths (availability engine, webhook handler, CRM)	≥ 95%	Manual review gate

Coverage is collected only from src/lib/, src/app/api/, and src/services/**. Generated files, config, and Next.js internals are excluded.

1.5 File Structure Convention

```
barberbook/ ├── src/ │ ├── lib/ │ │ ├── availability.ts │ │ ├── crm.ts │ │ └── notifications.ts │
└── app/api/ │ ├── public/ │ └── barber/ ├── tests/ │ ├── unit/ │ │ ├── availability.test.ts │ |
└── crm.test.ts │ ├── integration/ │ │ ├── api/ │ │ ├── payments/ │ │ └── notifications/ │ └──
e2e/ │ ├── booking.spec.ts │ ├── barber-auth.spec.ts │ └── admin-auth.spec.ts ├── tests/helpers/
│ ├── factories.ts │ ├── db.ts │ └── stripe-helpers.ts ├── vitest.config.ts └──
playwright.config.ts
```

---

2. Customer Booking Flow Tests (E2E — Playwright)

All E2E tests run against a local dev server (`http://localhost:3000`) backed by a local Supabase instance seeded with `test-barber` data.

TC-CUST-01: Full Booking Flow — No Deposit

Field	Value
Test ID	TC-CUST-01
Description	A customer completes the full booking flow for a service that requires no deposit payment.
Preconditions	Barber `test-barber` exists in DB with at least one active service (no deposit required). Working hours set for today. No conflicting bookings exist.
Steps	1. Navigate to `/b/test-barber` 2. View service list 3. Select a service 4. Select a date with available slots 5. Select an available time slot 6. Fill in name, phone, email 7. Submit booking 8. Assert confirmation page 9. Assert DB record

Field	Value
Expected Result	Confirmation page displays booking details. A `bookings` row exists in the DB with `status = 'confirmed'` and matching customer info.

```
// tests/e2e/booking.spec.ts import { test, expect } from '@playwright/test'; import {
createClient } from '@supabase/supabase-js'; const supabase = createClient(
process.env.SUPABASE_URL!, process.env.SUPABASE_SERVICE_ROLE_KEY! ); test.describe('TC-CUST-01:
Full booking flow — no deposit', () => { test.beforeEach(async ({ page }) => { // Ensure clean
state: remove any test bookings from today await supabase .from('bookings') .delete()
.eq('customer_email', 'playwright-test@example.com'); }); test('completes booking and shows
confirmation', async ({ page }) => { // Step 1: Navigate to barber page await
page.goto('/b/test-barber'); await expect(page).toHaveTitle(/test-barber/i); // Step 2: View
service list const serviceList = page.getByTestId('service-list'); await
expect(serviceList).toBeVisible(); // Step 3: Select first available no-deposit service const
serviceCard = page.getByTestId('service-card').first(); const serviceName = await
serviceCard.getByTestId('service-name').textContent(); await serviceCard.getByRole('button', {
name: /select|book/i }).click(); // Step 4: Select a date — pick tomorrow to avoid past-time
filtering const tomorrow = new Date(); tomorrow.setDate(tomorrow.getDate() + 1); const dateStr =
tomorrow.toISOString().split('T')[0]; // yyyy-mm-dd await
page.getByTestId('date-picker').fill(dateStr); // Step 5: Select first available time slot const
slotButtons = page.getByTestId('time-slot'); await expect(slotButtons.first()).toBeVisible();
const selectedSlot = await slotButtons.first().textContent(); await slotButtons.first().click();
// Step 6: Fill in customer details await page.getByLabel(/full name/i).fill('Playwright Test
User'); await page.getByLabel(/phone/i).fill('+16135551234'); await
page.getByLabel(/email/i).fill('playwright-test@example.com'); // Step 7: Submit await
page.getByRole('button', { name: /confirm booking/i }).click(); // Step 8: Assert confirmation
page await expect(page).toHaveURL(/\/booking\/confirmation/); await
expect(page.getByTestId('confirmation-heading')).toContainText(/booking confirmed/i); await
expect(page.getByTestId('confirmation-service')).toContainText(serviceName!.trim()); await
expect(page.getByTestId('confirmation-customer-name')).toContainText('Playwright Test User'); //
Step 9: Assert DB record const { data: booking } = await supabase ... [truncated for brevity]
```

TC-CUST-02: Full Booking Flow — With Deposit (Stripe)

Field	Value
Test ID	TC-CUST-02
Description	Customer selects a deposit-required service, is redirected to Stripe checkout, and after the simulated webhook fires, the booking is confirmed in the DB.
Preconditions	A service with `deposit_required = true` and `deposit_amount > 0` exists for `test-barber`. Stripe is in test mode. Webhook endpoint registered locally (e.g. via `stripe listen`).
Steps	1. Open booking flow 2. Select deposit-required service 3. Fill details 4. Submit 5. Assert redirect to Stripe 6. Fire `checkout.session.completed` webhook 7. Assert DB booking with `payment_status = 'paid'`

Field	Value
Expected Result	After webhook fires, a booking row exists with `payment_status = 'paid'` and `status = 'confirmed'`.

```
// tests/e2e/booking.spec.ts (continued) import { stripeWebhookHelper } from
'../helpers/stripe-helpers'; test.describe('TC-CUST-02: Full booking flow — with deposit
(Stripe)', () => { test('redirects to Stripe and booking confirmed after webhook', async ({ page,
context }) => { await page.goto('/b/test-barber'); // Select deposit-required service const
depositService = page.getByTestId('service-card').filter({ hasText: /deposit required/i,
}).first(); await depositService.getByRole('button', { name: /select|book/i }).click(); // Select
date + slot const tomorrow = new Date(); tomorrow.setDate(tomorrow.getDate() + 1); await
page.getByTestId('date-picker').fill(tomorrow.toISOString().split('T')[0]); await
page.getByTestId('time-slot').first().click(); // Fill customer details await
page.getByLabel(/full name/i).fill('Deposit Test User'); await
page.getByLabel(/phone/i).fill('+16135559999'); await
page.getByLabel(/email/i).fill('deposit-test@example.com'); // Submit — expect redirect to Stripe
await page.getByRole('button', { name: /confirm booking/i }).click(); // Assert we land on Stripe
checkout (external domain) await page.waitForURL(/checkout\.stripe\.com/, { timeout: 10000 });
expect(page.url()).toMatch(/checkout\.stripe\.com/); // Extract session ID from URL for webhook
simulation const stripeUrl = new URL(page.url()); const sessionId =
stripeUrl.pathname.split('/').pop()!; // Simulate Stripe webhook: checkout.session.completed
await stripeWebhookHelper.sendEvent('checkout.session.completed', { id: sessionId,
customer_email: 'deposit-test@example.com', metadata: { barber_slug: 'test-barber',
customer_name: 'Deposit Test User', customer_phone: '+16135559999', }, payment_status: 'paid',
}); // Wait for webhook to be processed await page.waitForTimeout(2000); // Assert DB booking
created with correct payment status const { data: booking } = await supabase .from('bookings')
.select('*') .eq('customer_email', 'deposit-test@example.com') .single();
expect(booking).not.toBeNull(); expect(booking!.payment_status).toBe('paid');
expect(booking!.status).toBe('confirmed'); ... [truncated for brevity]
```

TC-CUST-03: Unavailable Slot Selection

Field	Value
Test ID	TC-CUST-03
Description	Customer attempts to select a slot that was already booked by another customer.
Preconditions	An existing confirmed booking occupies slot 10:00 AM tomorrow for `test-barber`.
Steps	1. Seed a booking for 10:00 AM tomorrow 2. Navigate to `/b/test-barber` 3. Select date 4. Observe the 10:00 slot 5. Attempt to click (if somehow rendered) 6. Assert slot is not selectable or shows error
Expected Result	The 10:00 slot is either not rendered, visually disabled, or clicking it shows an error/unavailable state.

```
test.describe('TC-CUST-03: Unavailable slot selection', () => { test.beforeEach(async () => { //
Seed an existing booking at 10:00 AM tomorrow const tomorrow = new Date();
tomorrow.setDate(tomorrow.getDate() + 1); tomorrow.setHours(10, 0, 0, 0); await
```

```
supabase.from('bookings').insert({ barber_slug: 'test-barber', service_id: 'test-service-id',
customer_name: 'Pre-existing Customer', customer_email: 'existing@example.com', customer_phone:
'+16135550001', start_time: tomorrow.toISOString(), end_time: new Date(tomorrow.getTime() + 45 *
60000).toISOString(), status: 'confirmed', }); }); test.afterEach(async () => { await supabase
.from('bookings') .delete() .eq('customer_email', 'existing@example.com'); }); test('booked slot
is unavailable to new customer', async ({ page }) => { await page.goto('/b/test-barber'); await
page.getByTestId('service-card').first() .getByRole('button', { name: /select|book/i }).click();
const tomorrow = new Date(); tomorrow.setDate(tomorrow.getDate() + 1); await
page.getByTestId('date-picker').fill(tomorrow.toISOString().split('T')[0]); // The 10:00 slot
should either be absent or disabled const slot10am = page.getByTestId('time-slot').filter({
hasText: '10:00' }); // Option A: slot is not rendered at all const slotCount = await
slot10am.count(); if (slotCount === 0) { // Correct — slot not shown expect(slotCount).toBe(0);
return; } // Option B: slot is rendered but disabled const isDisabled = await
slot10am.first().isDisabled(); if (isDisabled) { expect(isDisabled).toBe(true); return; } //
Option C: slot can be clicked but shows error after attempting selection await
slot10am.first().click(); const errorMessage = page.getByTestId('slot-error'); await
expect(errorMessage).toContainText(/not available|already booked/i); }); });
```

TC-CUST-04: Invalid Form Submission

Field	Value
Test ID	TC-CUST-04
Description	Customer submits the booking form with required fields empty.
Preconditions	Service and slot selected. Form displayed.
Steps	1. Navigate to booking form 2. Select service and time 3. Leave name, phone, email blank 4. Click submit 5. Assert inline validation errors
Expected Result	Error messages appear beneath each required field. No navigation occurs.

```
test.describe('TC-CUST-04: Invalid form submission', () => { test('shows validation errors for
all missing required fields', async ({ page }) => { await page.goto('/b/test-barber'); // Select
service and slot to reach the form await page.getByTestId('service-card').first()
.getByRole('button', { name: /select|book/i }).click(); const tomorrow = new Date();
tomorrow.setDate(tomorrow.getDate() + 1); await
page.getByTestId('date-picker').fill(tomorrow.toISOString().split('T')[0]); await
page.getByTestId('time-slot').first().click(); // Submit without filling any fields await
page.getByRole('button', { name: /confirm booking/i }).click(); // Assert inline validation
errors await expect(page.getByText(/name is required/i)).toBeVisible(); await
expect(page.getByText(/phone is required/i)).toBeVisible(); await expect(page.getByText(/email is
required/i)).toBeVisible(); // Assert we have NOT navigated away await
expect(page).not.toHaveURL(/confirmation/); // Test individual field: invalid email format await
page.getByLabel(/email/i).fill('not-an-email'); await page.getByRole('button', { name: /confirm
booking/i }).click(); await expect(page.getByText(/invalid email/i)).toBeVisible(); // Test
individual field: invalid phone format await page.getByLabel(/phone/i).fill('abc'); await
page.getByRole('button', { name: /confirm booking/i }).click(); await
expect(page.getByText(/invalid phone/i)).toBeVisible(); }); });
```

TC-CUST-05: Barber Page Not Found

Field	Value
Test ID	TC-CUST-05
Description	Customer navigates to a barber slug that does not exist in the database.
Preconditions	No barber with slug `nonexistent-barber` exists in DB.
Steps	1. Navigate to `/b/nonexistent-barber` 2. Assert error state
Expected Result	Either a 404 HTTP response, or a friendly "page not found" / "barber not found" message is shown.

```
test.describe('TC-CUST-05: Barber page not found', () => { test('shows 404 or friendly not-found
message for unknown slug', async ({ page }) => { const response = await
page.goto('/b/nonexistent-barber'); // Assert HTTP status is 404 OR the page displays a not-found
UI const isHttp404 = response?.status() === 404; if (!isHttp404) { // If the app handles it
client-side with a custom page const notFoundEl = page.getByTestId('not-found-message')
.or(page.getByText(/not found|does not exist|no barber/i).first()); await
expect(notFoundEl).toBeVisible({ timeout: 5000 }); } else { expect(response?.status()).toBe(404);
} }); test('does not expose internal error details on not-found page', async ({ page }) => {
await page.goto('/b/nonexistent-barber'); // No stack traces or Supabase errors should be visible
await expect(page.getByText(/supabase|postgres|internal server/i)).not.toBeVisible(); }); });
```

TC-CUST-06: Double Booking Prevention

Field	Value
Test ID	TC-CUST-06
Description	Two concurrent requests attempt to book the same slot simultaneously. Only one should succeed.
Preconditions	No booking exists for target slot. Availability engine uses a DB-level unique constraint or advisory lock.
Steps	1. Prepare two identical booking payloads for the same slot 2. Fire both `POST /api/public/bookings` requests concurrently 3. Assert exactly one returns 201, the other returns 409
Expected Result	One booking is created. Second request gets a 409 Conflict or 400 with "slot not available" error. DB has exactly one booking for that slot.

```
// tests/e2e/booking.spec.ts — double booking (uses API directly, not browser UI)
test.describe('TC-CUST-06: Double booking prevention', () => { const TEST_SLOT_EMAIL_A =
'concurrent-a@example.com'; const TEST_SLOT_EMAIL_B = 'concurrent-b@example.com';
test.afterEach(async () => { await supabase.from('bookings').delete().in('customer_email', [
TEST_SLOT_EMAIL_A, TEST_SLOT_EMAIL_B, ]); }); test('only one concurrent booking for the same slot
```

```
succeeds', async ({ request }) => { const tomorrow = new Date();
tomorrow.setDate(tomorrow.getDate() + 1); tomorrow.setHours(14, 0, 0, 0); const payload = (email:
string) => ({ barber_slug: 'test-barber', service_id: 'test-service-id', start_time:
tomorrow.toISOString(), customer_name: 'Concurrent User', customer_email: email, customer_phone:
'+16135550000', }); // Fire both requests concurrently const [responseA, responseB] = await
Promise.all([ request.post('/api/public/bookings', { data: payload(TEST_SLOT_EMAIL_A) }),
request.post('/api/public/bookings', { data: payload(TEST_SLOT_EMAIL_B) }), ]); const statuses =
[responseA.status(), responseB.status()]; const successes = statuses.filter((s) => s === 201);
const conflicts = statuses.filter((s) => s === 409 || s === 400); // Exactly one should succeed,
one should fail expect(successes).toHaveLength(1); expect(conflicts).toHaveLength(1); // DB
should have exactly one booking for that time const { data: bookings } = await supabase
.from('bookings') .select('id') .eq('barber_slug', 'test-barber') .eq('start_time',
tomorrow.toISOString()); expect(bookings).toHaveLength(1); }); });
```

---

3. Barber Auth Tests

Auth tests use Playwright for the redirect and session behaviour that can only be validated in a real browser context.

TC-BARB-AUTH-01: Successful Login

Field	Value
Test ID	TC-BARB-AUTH-01
Description	A barber with valid credentials logs in and lands on the dashboard.
Preconditions	Barber account exists with email `barber@test.com` and known password.

```
// tests/e2e/barber-auth.spec.ts import { test, expect } from '@playwright/test';
test.describe('TC-BARB-AUTH-01: Successful barber login', () => { test('logs in with valid
credentials and redirects to dashboard', async ({ page }) => { await page.goto('/barber/login');
await page.getByLabel(/email/i).fill('barber@test.com'); await
page.getByLabel(/password/i).fill('TestPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); await expect(page).toHaveURL(/\/barber\/dashboard/); await
expect(page.getByTestId('dashboard-heading')).toContainText(/dashboard/i); }); });
```

TC-BARB-AUTH-02: Wrong Password

Field	Value
Test ID	TC-BARB-AUTH-02
Description	A barber enters an incorrect password and sees an error.

Field	Value
Preconditions	Barber account exists with email `barber@test.com`.

```
test.describe('TC-BARB-AUTH-02: Wrong password', () => { test('shows error for invalid credentials', async ({ page }) => { await page.goto('/barber/login'); await page.getByLabel(/email/i).fill('barber@test.com'); await page.getByLabel(/password/i).fill('WrongPassword999!'); await page.getByRole('button', { name: /sign in|log in/i }).click(); // Should NOT redirect to dashboard await expect(page).not.toHaveURL(/dashboard/); // Should show error message await expect(page.getByRole('alert').or(page.getByTestId('login-error'))).toContainText( /invalid credentials|incorrect password|sign in failed/i ); }); });
```

TC-BARB-AUTH-03: Unauthenticated Dashboard Access → Redirect

Field	Value
Test ID	TC-BARB-AUTH-03
Description	Attempting to access `/barber/dashboard` without a session redirects to `/barber/login`.
Preconditions	No active session (fresh browser context).

```
test.describe('TC-BARB-AUTH-03: Unauthenticated redirect', () => { test('redirects to login when accessing dashboard without session', async ({ browser }) => { // Use a fresh context with no cookies/session const context = await browser.newContext(); const page = await context.newPage(); await page.goto('/barber/dashboard'); await expect(page).toHaveURL(/\/barber\/login/); await context.close(); }); });
```

TC-BARB-AUTH-04: Admin Cannot Access /barber/* Routes

Field	Value
Test ID	TC-BARB-AUTH-04
Description	A logged-in admin user attempting to access barber-specific routes is denied (403 or redirect).
Preconditions	Admin account `admin@test.com` is logged in.

```
test.describe('TC-BARB-AUTH-04: Role check — admin blocked from barber routes', () => { test('admin cannot access /barber/dashboard', async ({ page }) => { // Log in as admin await page.goto('/admin/login'); await page.getByLabel(/email/i).fill('admin@test.com'); await page.getByLabel(/password/i).fill('AdminPassword123!'); await page.getByRole('button', { name: /sign in|log in/i }).click(); await expect(page).toHaveURL(/\/admin/); // Attempt to access barber route await page.goto('/barber/dashboard'); // Should be redirected or shown a forbidden state const isForbidden = page.url().includes('/admin') || page.url().includes('/barber/login') || (await page.getByText(/forbidden|not authorized|access denied/i).count()) > 0; expect(isForbidden).toBe(true); }); });
```

TC-BARB-AUTH-05: Session Persistence After Page Refresh

Field	Value
Test ID	TC-BARB-AUTH-05
Description	After logging in, refreshing the page keeps the barber authenticated.
Preconditions	Barber logged in.

```
test.describe('TC-BARB-AUTH-05: Session persistence', () => { test('stays logged in after page refresh', async ({ page }) => { // Log in await page.goto('/barber/login'); await page.getByLabel(/email/i).fill('barber@test.com'); await page.getByLabel(/password/i).fill('TestPassword123!'); await page.getByRole('button', { name: /sign in|log in/i }).click(); await expect(page).toHaveURL(/\/barber\/dashboard/); // Refresh the page await page.reload(); // Should still be on dashboard, not redirected to login await expect(page).toHaveURL(/\/barber\/dashboard/); await expect(page.getByTestId('dashboard-heading')).toBeVisible(); }); });
```

TC-BARB-AUTH-06: Logout Clears Session and Redirects

Field	Value
Test ID	TC-BARB-AUTH-06
Description	After logout, session is cleared and navigating to dashboard redirects back to login.
Preconditions	Barber logged in.

```
test.describe('TC-BARB-AUTH-06: Logout', () => { test('logout clears session and redirects', async ({ page }) => { // Log in await page.goto('/barber/login'); await page.getByLabel(/email/i).fill('barber@test.com'); await page.getByLabel(/password/i).fill('TestPassword123!'); await page.getByRole('button', { name: /sign in|log in/i }).click(); await expect(page).toHaveURL(/\/barber\/dashboard/); // Perform logout await page.getByRole('button', { name: /sign out|log out/i }).click(); // Should redirect to login await expect(page).toHaveURL(/\/barber\/login/); // Attempting to go back to dashboard should redirect to login await page.goto('/barber/dashboard'); await expect(page).toHaveURL(/\/barber\/login/); }); });
```

---

4. Admin Auth Tests

TC-ADMIN-AUTH-01: Successful Admin Login

Field	Value
Test ID	TC-ADMIN-AUTH-01
Description	Admin user logs in with valid credentials and lands on the admin panel.

```
// tests/e2e/admin-auth.spec.ts import { test, expect } from '@playwright/test';
test.describe('TC-ADMIN-AUTH-01: Successful admin login', () => { test('logs in as admin and
reaches admin panel', async ({ page }) => { await page.goto('/admin/login'); await
page.getByLabel(/email/i).fill('admin@test.com'); await
page.getByLabel(/password/i).fill('AdminPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); await expect(page).toHaveURL(/\/admin/); await
expect(page.getByTestId('admin-panel-heading')).toBeVisible(); }); });
```

TC-ADMIN-AUTH-02: Barber Cannot Access /admin/* Routes

Field	Value
Test ID	TC-ADMIN-AUTH-02
Description	A barber-role user is denied access to any `/admin/` route.

```
test.describe('TC-ADMIN-AUTH-02: Barber cannot access admin routes', () => { test('barber is
denied access to /admin/barbers', async ({ page }) => { // Log in as barber await
page.goto('/barber/login'); await page.getByLabel(/email/i).fill('barber@test.com'); await
page.getByLabel(/password/i).fill('TestPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); await expect(page).toHaveURL(/\/barber\/dashboard/); // Attempt to
access admin route await page.goto('/admin/barbers'); // Should not see admin content const
isDenied = page.url().includes('/admin/login') || page.url().includes('/barber') || (await
page.getByText(/forbidden|unauthorized|access denied/i).count()) > 0;
expect(isDenied).toBe(true); }); });
```

TC-ADMIN-AUTH-03: Admin Creates a New Barber Account

Field	Value
Test ID	TC-ADMIN-AUTH-03
Description	Logged-in admin creates a new barber account via the admin panel.

```
test.describe('TC-ADMIN-AUTH-03: Admin creates new barber account', () => { test('admin creates a
barber and it appears in the list', async ({ page }) => { // Log in as admin await
page.goto('/admin/login'); await page.getByLabel(/email/i).fill('admin@test.com'); await
page.getByLabel(/password/i).fill('AdminPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); // Navigate to barbers management await
page.goto('/admin/barbers'); await page.getByRole('button', { name: /add barber|new barber/i
}).click(); // Fill new barber form const uniqueSlug = `test-barber-${Date.now()}`; await
page.getByLabel(/name/i).fill('New Test Barber'); await
```

```
page.getByLabel(/email/i).fill(`${uniqueSlug}@example.com`); await
page.getByLabel(/slug/i).fill(uniqueSlug); await page.getByRole('button', { name: /create|save/i
}).click(); // Assert success await expect(page.getByText(/barber
created|success/i)).toBeVisible(); await expect(page.getByText('New Test Barber')).toBeVisible();
// Cleanup: delete test barber const { error } = await
supabase.from('barbers').delete().eq('slug', uniqueSlug); expect(error).toBeNull(); }); });
```

TC-ADMIN-AUTH-04: Admin Can View All Barbers

Field	Value
Test ID	TC-ADMIN-AUTH-04
Description	Admin sees a list of all barbers in the system.

```
test.describe('TC-ADMIN-AUTH-04: Admin views all barbers', () => { test('admin barber list shows
all barbers from DB', async ({ page }) => { // Seed 2 known barbers const { data: barbers } =
await supabase .from('barbers') .select('name') .order('created_at', { ascending: false })
.limit(5); // Log in and navigate await page.goto('/admin/login'); await
page.getByLabel(/email/i).fill('admin@test.com'); await
page.getByLabel(/password/i).fill('AdminPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); await page.goto('/admin/barbers'); // Each barber from DB should
appear in the list const barberRows = page.getByTestId('barber-row'); const count = await
barberRows.count(); expect(count).toBeGreaterThanOrEqual(barbers?.length ?? 1); }); });
```

TC-ADMIN-AUTH-05: Admin Can Suspend a Barber Page

Field	Value
Test ID	TC-ADMIN-AUTH-05
Description	Admin suspends a barber; the barber's public page becomes inaccessible.

```
test.describe('TC-ADMIN-AUTH-05: Admin suspends barber page', () => { const targetSlug =
'barber-to-suspend'; test.beforeEach(async () => { await supabase.from('barbers').upsert({ slug:
targetSlug, name: 'Suspend Test Barber', email: `${targetSlug}@example.com`, status: 'active',
}); }); test.afterEach(async () => { await supabase.from('barbers').delete().eq('slug',
targetSlug); }); test('suspended barber page shows suspended/unavailable state', async ({ page })
=> { // Log in as admin and suspend await page.goto('/admin/login'); await
page.getByLabel(/email/i).fill('admin@test.com'); await
page.getByLabel(/password/i).fill('AdminPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); await page.goto('/admin/barbers'); const barberRow =
page.getByTestId('barber-row').filter({ hasText: 'Suspend Test Barber' }); await
barberRow.getByRole('button', { name: /suspend/i }).click(); await page.getByRole('button', {
name: /confirm/i }).click(); await expect(barberRow.getByText(/suspended/i)).toBeVisible(); //
Now visit the barber's public page — should show suspended/unavailable await
page.goto(`/b/${targetSlug}`); const isSuspended = (await
page.getByText(/suspended|unavailable|not available/i).count()) > 0 || (await page.response())
!== null && (await page.response())!.status() >= 400; expect(isSuspended).toBe(true); }); });
```

---

5. Booking Conflict Tests (Unit — Vitest)

These tests target the availability engine at `src/lib/availability.ts`. All tests are pure — no DB or network calls.

```
// src/lib/availability.ts (interface assumed for tests) // generateSlots(params: SlotGenerationParams): TimeSlot[] // SlotGenerationParams { // date: Date; // workingHours: { start: string; end: string } | null; // null = closed // existingBookings: { startTime: Date; endTime: Date }[]; // blockedTimes: { startTime: Date; endTime: Date }[]; // serviceDurationMinutes: number; // bufferMinutes: number; // slotIntervalMinutes: number; // now: Date; // injectable for deterministic past-time filtering // } // TimeSlot { time: Date; available: boolean }
```

TC-AVAIL-01: Slot Generation — Normal Working Day

Field	Value
Test ID	TC-AVAIL-01
Description	For a standard Mon 9am–6pm working day with no bookings, generate all valid 45-min slots at 30-min intervals.
Preconditions	Pure function, no DB.

```
// tests/unit/availability.test.ts import { describe, it, expect } from 'vitest'; import { generateSlots } from '../../src/lib/availability'; const MONDAY = new Date('2026-04-06T00:00:00.000Z'); // Monday in UTC describe('TC-AVAIL-01: Slot generation — normal working day', () => { it('generates the correct number of slots for 9am–6pm, 45-min service, 30-min interval', () => { const slots = generateSlots({ date: MONDAY, workingHours: { start: '09:00', end: '18:00' }, existingBookings: [], blockedTimes: [], serviceDurationMinutes: 45, bufferMinutes: 0, slotIntervalMinutes: 30, now: new Date('2026-04-06T00:00:00.000Z'), // midnight — all slots in future }); const availableSlots = slots.filter((s) => s.available); // 9am to 6pm = 9 hours = 540 minutes // Last possible start: 6pm - 45min = 5:15pm // Slots at 30-min intervals from 9:00 to 17:15 // = 9:00, 9:30, 10:00, ..., 17:00, 17:15 → let's count: // From 9:00 to 17:00 at 30-min intervals = 17 slots // Plus 17:15 if interval allows... depends on implementation // Minimum expectation: slots at 9:00, 9:30, 10:00, 10:30 ... 17:00 = 17 slots expect(availableSlots.length).toBeGreaterThanOrEqual(16); // First slot should be 9:00am const firstSlot = availableSlots[0]; expect(firstSlot.time.getHours()).toBe(9); expect(firstSlot.time.getMinutes()).toBe(0); // Last slot start must allow service to finish by 6pm const lastSlot = availableSlots[availableSlots.length - 1]; const lastSlotEnd = new Date(lastSlot.time.getTime() + 45 * 60000); const endOfDay = new Date(MONDAY); endOfDay.setHours(18, 0, 0, 0); expect(lastSlotEnd.getTime()).toBeLessThanOrEqual(endOfDay.getTime()); // All slots should be marked available expect(slots.every((s) => s.available)).toBe(true); }); });
```

TC-AVAIL-02: Slot Blocked by Existing Booking

Field	Value
Test ID	TC-AVAIL-02

Field	Value
Description	An existing booking from 10:00–10:45 blocks the 10:00 slot; the 10:45 slot is available.

```
describe('TC-AVAIL-02: Slot blocked by existing booking', () => { it('marks 10:00 as unavailable and 10:45 as available', () => { const date = MONDAY; const booking10am = { startTime: new Date('2026-04-06T10:00:00.000Z'), endTime: new Date('2026-04-06T10:45:00.000Z'), }; const slots = generateSlots({ date, workingHours: { start: '09:00', end: '18:00' }, existingBookings: [booking10am], blockedTimes: [], serviceDurationMinutes: 45, bufferMinutes: 0, slotIntervalMinutes: 15, now: new Date('2026-04-06T00:00:00.000Z'), }); const slot10am = slots.find( (s) => s.time.getHours() === 10 && s.time.getMinutes() === 0 ); const slot10_45am = slots.find( (s) => s.time.getHours() === 10 && s.time.getMinutes() === 45 ); expect(slot10am).toBeDefined(); expect(slot10am!.available).toBe(false); expect(slot10_45am).toBeDefined(); expect(slot10_45am!.available).toBe(true); }); });
```

TC-AVAIL-03: Slot Blocked by Blocked Time

Field	Value
Test ID	TC-AVAIL-03
Description	A blocked time range of 12pm–1pm means no slots starting in that window are available.

```
describe('TC-AVAIL-03: Slot blocked by blocked time', () => { it('returns no available slots in the 12pm–1pm blocked window', () => { const slots = generateSlots({ date: MONDAY, workingHours: { start: '09:00', end: '18:00' }, existingBookings: [], blockedTimes: [ { startTime: new Date('2026-04-06T12:00:00.000Z'), endTime: new Date('2026-04-06T13:00:00.000Z'), }, ], serviceDurationMinutes: 45, bufferMinutes: 0, slotIntervalMinutes: 15, now: new Date('2026-04-06T00:00:00.000Z'), }); // Any slot that starts in or overlaps with 12:00–13:00 should be unavailable const slotsInWindow = slots.filter((s) => { const startH = s.time.getHours(); const startM = s.time.getMinutes(); const startMinutes = startH * 60 + startM; // A 45-min service starting at slot time must not overlap 12:00–13:00 return startMinutes >= 12 * 60 && startMinutes < 13 * 60; }); expect(slotsInWindow.every((s) => s.available === false)).toBe(true); // Slots before and after the block should be fine const slot11am = slots.find( (s) => s.time.getHours() === 11 && s.time.getMinutes() === 0 ); expect(slot11am?.available).toBe(true); // 1pm slot: the service starting at 1pm ends at 1:45pm — should be available const slot1pm = slots.find( (s) => s.time.getHours() === 13 && s.time.getMinutes() === 0 ); expect(slot1pm?.available).toBe(true); }); });
```

TC-AVAIL-04: Past Time Slots Not Shown

Field	Value
Test ID	TC-AVAIL-04
Description	When the current time is 2pm, no slots at or before 2pm should appear for today.

```
describe('TC-AVAIL-04: Past time slots not shown', () => { it('excludes all slots before current time (2pm) when date is today', () => { const today = new Date('2026-04-06T14:00:00.000Z'); // 2pm UTC const slots = generateSlots({ date: today, workingHours: { start: '09:00', end: '18:00' }, existingBookings: [], blockedTimes: [], serviceDurationMinutes: 45, bufferMinutes: 0, slotIntervalMinutes: 30, now: today, }); const pastSlots = slots.filter((s) => s.time.getTime() <= today.getTime() && s.available); expect(pastSlots).toHaveLength(0); // Slots at 2pm+ may still appear as available (or at least after the current moment) const futureAvailable = slots.filter( (s) => s.time.getTime() > today.getTime() && s.available ); expect(futureAvailable.length).toBeGreaterThan(0); }); });
```

TC-AVAIL-05: Off Day — No Slots

Field	Value
Test ID	TC-AVAIL-05
Description	When the barber marks Sunday as closed, no slots are returned for Sunday.

```
describe('TC-AVAIL-05: Off day — no slots', () => { it('returns an empty array for a day with null workingHours (closed)', () => { const sunday = new Date('2026-04-05T00:00:00.000Z'); // Sunday const slots = generateSlots({ date: sunday, workingHours: null, // closed existingBookings: [], blockedTimes: [], serviceDurationMinutes: 45, bufferMinutes: 0, slotIntervalMinutes: 30, now: new Date('2026-04-05T00:00:00.000Z'), }); expect(slots).toHaveLength(0); }); });
```

TC-AVAIL-06: Buffer Time Between Appointments

Field	Value
Test ID	TC-AVAIL-06
Description	With a 15-minute buffer configured, a slot immediately after an appointment ends at 11am must be blocked; 11:15 is available.

```
describe('TC-AVAIL-06: Buffer time between appointments', () => { it('marks 11:00 unavailable due to buffer, 11:15 available', () => { const booking = { startTime: new Date('2026-04-06T10:00:00.000Z'), endTime: new Date('2026-04-06T11:00:00.000Z'), // ends at 11am }; const slots = generateSlots({ date: MONDAY, workingHours: { start: '09:00', end: '18:00' }, existingBookings: [booking], blockedTimes: [], serviceDurationMinutes: 45, bufferMinutes: 15, // 15-min buffer after booking ends slotIntervalMinutes: 15, now: new Date('2026-04-06T00:00:00.000Z'), }); // 11:00 slot — within 15-min buffer of the previous booking end const slot11am = slots.find( (s) => s.time.getHours() === 11 && s.time.getMinutes() === 0 ); expect(slot11am).toBeDefined(); expect(slot11am!.available).toBe(false); // 11:15 slot — after buffer, should be available const slot11_15am = slots.find( (s) => s.time.getHours() === 11 && s.time.getMinutes() === 15 ); expect(slot11_15am).toBeDefined(); expect(slot11_15am!.available).toBe(true); }); });
```

---

6. Payment Tests (Integration — Vitest + Stripe test mode)

These tests run against the actual Stripe SDK in test mode and a local Supabase instance. They test the webhook handler at `src/app/api/webhooks/stripe/route.ts`.

```
// tests/integration/payments/stripe-webhook.test.ts import { describe, it, expect, beforeEach, afterEach, vi } from 'vitest'; import Stripe from 'stripe'; import { createTestBarber, createTestBookingPayload } from '../../helpers/factories'; import { getTestDb } from '../../helpers/db'; const stripe = new Stripe(process.env.STRIPE_SECRET_KEY_TEST!, { apiVersion: '2024-04-10' }); const db = getTestDb();
```

TC-PAY-01: Stripe Checkout Session Created with Correct Metadata

Field	Value
Test ID	TC-PAY-01
Description	When `POST /api/public/bookings` is called for a deposit-required service, the Stripe checkout session is created with correct metadata (barber_slug, customer details, booking context).

```
describe('TC-PAY-01: Stripe checkout session created with correct metadata', () => { it('creates checkout session with correct amount and metadata', async () => { const barber = await createTestBarber({ slug: 'pay-test-barber' }); // Seed a deposit-required service const { data: service } = await db.from('services').insert({ barber_id: barber.id, name: 'Deluxe Cut', duration_minutes: 60, price_cents: 5000, deposit_required: true, deposit_amount_cents: 2000, }).select().single(); const bookingPayload = { barber_slug: 'pay-test-barber', service_id: service!.id, start_time: new Date(Date.now() + 86400000).toISOString(), customer_name: 'Pay Test User', customer_email: 'paytest@example.com', customer_phone: '+16135550042', }; const response = await fetch('http://localhost:3000/api/public/bookings', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(bookingPayload), }); expect(response.status).toBe(200); const body = await response.json(); expect(body.checkoutUrl).toMatch(/checkout\.stripe\.com/); expect(body.sessionId).toBeTruthy(); // Retrieve the session from Stripe const session = await stripe.checkout.sessions.retrieve(body.sessionId); expect(session.metadata?.barber_slug).toBe('pay-test-barber'); expect(session.metadata?.customer_email).toBe('paytest@example.com'); expect(session.amount_total).toBe(2000); // deposit amount in cents }); });
```

TC-PAY-02: Webhook Handler Verifies Signature and Rejects Invalid Signatures

Field	Value
Test ID	TC-PAY-02

Field	Value
Description	The webhook route correctly verifies the Stripe-Signature header and rejects requests with an invalid or missing signature.

```
describe('TC-PAY-02: Webhook signature verification', () => { it('returns 400 for a webhook with an invalid signature', async () => { const payload = JSON.stringify({ type: 'checkout.session.completed', data: {} }); const response = await fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type': 'application/json', 'stripe-signature': 'invalid-signature-value', }, body: payload, }); expect(response.status).toBe(400); const body = await response.json(); expect(body.error).toMatch(/signature|invalid/i); }); it('returns 400 for a webhook with no signature header', async () => { const payload = JSON.stringify({ type: 'checkout.session.completed', data: {} }); const response = await fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: payload, }); expect(response.status).toBe(400); }); it('returns 200 for a correctly signed webhook event', async () => { const { generateStripeWebhookEvent } = await import('../../helpers/stripe-helpers'); const { payload, signature } = generateStripeWebhookEvent( 'checkout.session.completed', { id: 'cs_test_fake', customer_email: 'valid@example.com', metadata: { barber_slug: 'test-barber' }, payment_status: 'paid', } ); const response = await fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type': 'application/json', 'stripe-signature': signature, }, body: payload, }); expect(response.status).toBe(200); }); });
```

TC-PAY-03: Webhook Creates Booking on checkout.session.completed

Field	Value
Test ID	TC-PAY-03
Description	A valid `checkout.session.completed` webhook event results in a booking row in the DB with `payment_status = 'paid'`.

```
describe('TC-PAY-03: Webhook creates booking on checkout.session.completed', () => { it('creates a confirmed booking with payment_status=paid', async () => { const { generateStripeWebhookEvent } = await import('../../helpers/stripe-helpers'); const sessionId = `cs_test_${Date.now()}`; const { payload, signature } = generateStripeWebhookEvent( 'checkout.session.completed', { id: sessionId, customer_email: 'webhook-create@example.com', metadata: { barber_slug: 'test-barber', service_id: 'test-service-id', start_time: new Date(Date.now() + 86400000).toISOString(), customer_name: 'Webhook Create User', customer_phone: '+16135550100', }, payment_status: 'paid', } ); const response = await fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type': 'application/json', 'stripe-signature': signature, }, body: payload, }); expect(response.status).toBe(200); // Wait a beat for async DB write await new Promise((r) => setTimeout(r, 500)); const { data: booking } = await db .from('bookings') .select('*') .eq('stripe_session_id', sessionId) .single(); expect(booking).not.toBeNull(); expect(booking!.payment_status).toBe('paid'); expect(booking!.status).toBe('confirmed'); expect(booking!.customer_email).toBe('webhook-create@example.com'); }); });
```

TC-PAY-04: Webhook is Idempotent

Field	Value
Test ID	TC-PAY-04
Description	Sending the same `checkout.session.completed` event twice does not create a duplicate booking.

```
describe('TC-PAY-04: Webhook idempotency', () => { it('does not create a duplicate booking when the same event is replayed', async () => { const { generateStripeWebhookEvent } = await import('../../helpers/stripe-helpers'); const sessionId = `cs_test_idempotent_${Date.now()}`; const buildRequest = () => { const { payload, signature } = generateStripeWebhookEvent( 'checkout.session.completed', { id: sessionId, customer_email: 'idempotent@example.com', metadata: { barber_slug: 'test-barber', service_id: 'test-service-id', start_time: new Date(Date.now() + 86400000).toISOString(), customer_name: 'Idempotent User', customer_phone: '+16135550200', }, payment_status: 'paid', } ); return { payload, signature }; }; const sendWebhook = async () => { const { payload, signature } = buildRequest(); return fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type': 'application/json', 'stripe-signature': signature }, body: payload, }); }; // Send the same event twice const [r1, r2] = await Promise.all([sendWebhook(), sendWebhook()]); expect(r1.status).toBe(200); expect(r2.status).toBe(200); await new Promise((r) => setTimeout(r, 500)); const { data: bookings } = await db .from('bookings') .select('id') .eq('stripe_session_id', sessionId); expect(bookings).toHaveLength(1); }); });
```

TC-PAY-05: Expired Checkout Session Releases Held Slot

Field	Value
Test ID	TC-PAY-05
Description	When a `checkout.session.expired` event fires, any pending/held booking for that session is cancelled and the slot becomes available again.

```
describe('TC-PAY-05: Expired checkout session releases slot', () => { it('cancels a pending booking and releases the slot on session expiry', async () => { const { generateStripeWebhookEvent } = await import('../../helpers/stripe-helpers'); const sessionId = `cs_test_expired_${Date.now()}`; // Pre-create a pending booking with this session ID const startTime = new Date(Date.now() + 86400000).toISOString(); await db.from('bookings').insert({ barber_slug: 'test-barber', service_id: 'test-service-id', customer_name: 'Expire Test', customer_email: 'expire@example.com', customer_phone: '+16135550300', start_time: startTime, end_time: new Date(Date.now() + 86400000 + 45 * 60000).toISOString(), status: 'pending_payment', stripe_session_id: sessionId, payment_status: 'unpaid', }); const { payload, signature } = generateStripeWebhookEvent( 'checkout.session.expired', { id: sessionId, payment_status: 'unpaid' } ); await fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type': 'application/json', 'stripe-signature': signature }, body: payload, }); await new Promise((r) => setTimeout(r, 500)); const { data: booking } = await db .from('bookings') .select('status') .eq('stripe_session_id', sessionId) .single(); expect(booking!.status).toBe('cancelled'); }); });
```

TC-PAY-06: Payment Failure Handled Gracefully

Field	Value
Test ID	TC-PAY-06
Description	A `payment_intent.payment_failed` event does not create a confirmed booking and updates the booking to `payment_failed`.

```
describe('TC-PAY-06: Payment failure handled gracefully', () => { it('does not confirm booking on payment failure', async () => { const { generateStripeWebhookEvent } = await import('../../helpers/stripe-helpers'); const sessionId = `cs_test_failed_${Date.now()}`; await db.from('bookings').insert({ barber_slug: 'test-barber', service_id: 'test-service-id', customer_name: 'Fail Test', customer_email: 'fail@example.com', customer_phone: '+16135550400', start_time: new Date(Date.now() + 86400000).toISOString(), end_time: new Date(Date.now() + 86400000 + 45 * 60000).toISOString(), status: 'pending_payment', stripe_session_id: sessionId, payment_status: 'unpaid', }); const { payload, signature } = generateStripeWebhookEvent( 'checkout.session.async_payment_failed', { id: sessionId } ); const response = await fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type': 'application/json', 'stripe-signature': signature }, body: payload, }); expect(response.status).toBe(200); await new Promise((r) => setTimeout(r, 500)); const { data: booking } = await db .from('bookings') .select('status, payment_status') .eq('stripe_session_id', sessionId) .single(); expect(booking!.status).not.toBe('confirmed'); expect(['cancelled', 'payment_failed']).toContain(booking!.status); }); });
```

---

7. CRM Tests (Unit — Vitest)

These tests target the CRM service at `src/lib/crm.ts`. DB calls are mocked with a Supabase test client against the local instance.

```
// tests/unit/crm.test.ts import { describe, it, expect, beforeEach } from 'vitest'; import { upsertClient, incrementVisits, markNoShow, updateLastVisit } from '../../src/lib/crm'; import { getTestDb } from '../helpers/db'; import { createTestClient } from '../helpers/factories'; const db = getTestDb();
```

TC-CRM-01: New Client Created on First Booking

Field	Value
Test ID	TC-CRM-01
Description	When a booking is made for an email/phone combination that does not exist in `clients`, a new client record is created.

```
describe('TC-CRM-01: New client created on first booking', () => { it('creates a new client record for an unknown email', async () => { const email = `new-client-${Date.now()}@example.com`; const client = await upsertClient(db, { name: 'Brand New Client', email, phone: '+16135550001', }); expect(client.id).toBeTruthy(); expect(client.email).toBe(email); expect(client.total_visits).toBe(0); // Verify in DB const { data } = await db.from('clients').select('*').eq('email', email).single(); expect(data).not.toBeNull(); expect(data!.name).toBe('Brand New Client'); }); });
```

TC-CRM-02: Existing Client Matched by Email — No Duplicate

Field	Value
Test ID	TC-CRM-02
Description	When a booking comes in for an existing email, the existing client record is returned — no duplicate is inserted.

```
describe('TC-CRM-02: Existing client matched by email', () => { it('returns existing client
without creating a duplicate', async () => { const existingClient = await createTestClient({
email: 'repeat@example.com' }); const result = await upsertClient(db, { name: 'Repeat Client',
email: 'repeat@example.com', phone: '+16135550002', });
expect(result.id).toBe(existingClient.id); // Count rows with this email — should still be 1
const { count } = await db .from('clients') .select('id', { count: 'exact', head: true })
.eq('email', 'repeat@example.com'); expect(count).toBe(1); }); });
```

TC-CRM-03: Existing Client Matched by Phone When Email Differs

Field	Value
Test ID	TC-CRM-03
Description	If a booking provides a phone number matching an existing client (but with a different or no email), the same client is returned.

```
describe('TC-CRM-03: Existing client matched by phone', () => { it('matches existing client by
phone number when email differs', async () => { const existingClient = await createTestClient({
email: 'oldmail@example.com', phone: '+16135550003', }); const result = await upsertClient(db, {
name: 'Phone Match Client', email: 'newmail@example.com', // different email phone:
'+16135550003', // same phone }); expect(result.id).toBe(existingClient.id); }); });
```

TC-CRM-04: total_visits Incremented on Booking Completion

Field	Value
Test ID	TC-CRM-04
Description	When a booking is marked `completed`, the client's `total_visits` count is incremented by 1.

```
describe('TC-CRM-04: total_visits incremented on booking completion', () => { it('increments
total_visits by 1', async () => { const client = await createTestClient({ total_visits: 3 });
await incrementVisits(db, client.id); const { data: updated } = await db .from('clients')
```

```
.select('total_visits') .eq('id', client.id) .single(); expect(updated!.total_visits).toBe(4);
}); });
```

TC-CRM-05: noshowcount Incremented on No-Show Status

Field	Value
Test ID	TC-CRM-05
Description	When a booking is marked `no_show`, the client's `no_show_count` is incremented by 1.

```
describe('TC-CRM-05: no_show_count incremented', () => { it('increments no_show_count by 1',
async () => { const client = await createTestClient({ no_show_count: 1 }); await markNoShow(db,
client.id); const { data: updated } = await db .from('clients') .select('no_show_count')
.eq('id', client.id) .single(); expect(updated!.no_show_count).toBe(2); }); });
```

TC-CRM-06: lastvisitat Updated on Booking Completion

Field	Value
Test ID	TC-CRM-06
Description	When a booking completes, `last_visit_at` on the client record is updated to the booking's `start_time`.

```
describe('TC-CRM-06: last_visit_at updated on booking completion', () => { it('updates
last_visit_at to the completed booking start time', async () => { const client = await
createTestClient({ last_visit_at: null }); const bookingStartTime = new
Date('2026-03-15T10:00:00.000Z'); await updateLastVisit(db, client.id, bookingStartTime); const {
data: updated } = await db .from('clients') .select('last_visit_at') .eq('id', client.id)
.single(); expect(new Date(updated!.last_visit_at).getTime()).toBe(bookingStartTime.getTime());
}); });
```

TC-CRM-07: Client Notes Saved and Retrieved Correctly

Field	Value
Test ID	TC-CRM-07
Description	Notes added to a client record are persisted and retrievable verbatim.

```
describe('TC-CRM-07: Client notes saved and retrieved', () => { it('saves and retrieves notes
correctly', async () => { const client = await createTestClient({ notes: null }); const
noteContent = 'Prefers low fade. Sensitive scalp. Arrives 5 mins early.'; await db
```

```
.from('clients') .update({ notes: noteContent }) .eq('id', client.id); const { data: updated } =
await db .from('clients') .select('notes') .eq('id', client.id) .single();
expect(updated!.notes).toBe(noteContent); }); });
```

---

8. Reminder/Notification Tests (Integration — Vitest)

These tests mock the Resend email client using MSW (Mock Service Worker) and test the notification service at `src/lib/notifications.ts`. A real local Supabase DB is used to verify `message_log` records.

```
// tests/integration/notifications/notifications.test.ts import { describe, it, expect,
beforeAll, afterAll, vi } from 'vitest'; import { setupServer } from 'msw/node'; import { http,
HttpResponse } from 'msw'; import { sendConfirmationEmail, scheduleReminder, sendReminder, } from
'../../../src/lib/notifications'; import { getTestDb } from '../../helpers/db'; import {
createTestBooking, createTestBarber } from '../../helpers/factories'; const db = getTestDb(); //
MSW server to intercept Resend API calls const resendMockServer = setupServer(
http.post('https://api.resend.com/emails', () => { return HttpResponse.json({ id:
'mock-email-id-12345' }, { status: 200 }); }) ); beforeAll(() => resendMockServer.listen({
onUnhandledRequest: 'error' })); afterAll(() => resendMockServer.close());
```

TC-NOTIF-01: Confirmation Email Sent After Booking Creation

Field	Value
Test ID	TC-NOTIF-01
Description	After a booking is created, a confirmation email is sent to the customer via Resend.

```
describe('TC-NOTIF-01: Confirmation email sent after booking creation', () => { it('sends
confirmation email and logs the message', async () => { const barber = await createTestBarber();
const booking = await createTestBooking({ barber_id: barber.id }); const result = await
sendConfirmationEmail(db, booking.id); expect(result.success).toBe(true);
expect(result.emailId).toBe('mock-email-id-12345'); // Verify message_log record created const {
data: log } = await db .from('message_log') .select('*') .eq('booking_id', booking.id)
.eq('type', 'confirmation') .single(); expect(log).not.toBeNull();
expect(log!.status).toBe('sent'); expect(log!.provider_id).toBe('mock-email-id-12345'); }); });
```

TC-NOTIF-02: Reminder Scheduled 24h Before Appointment

Field	Value
Test ID	TC-NOTIF-02
Description	Scheduling a reminder for a booking creates a scheduled job for 24h before the appointment.

```
describe('TC-NOTIF-02: Reminder scheduled 24h before appointment', () => { it('schedules a 24h
reminder job at the correct time', async () => { const appointmentTime = new Date(Date.now() + 48
* 60 * 60 * 1000); // 48h from now const barber = await createTestBarber(); const booking = await
createTestBooking({ barber_id: barber.id, start_time: appointmentTime.toISOString(), }); await
scheduleReminder(db, booking.id, '24h'); const { data: job } = await db .from('scheduled_jobs')
.select('*') .eq('booking_id', booking.id) .eq('type', 'reminder_24h') .single();
expect(job).not.toBeNull(); const expectedTime = new Date(appointmentTime.getTime() - 24 * 60 *
60 * 1000); const actualTime = new Date(job!.scheduled_at); // Allow ±5 seconds tolerance for
test timing expect(Math.abs(actualTime.getTime() - expectedTime.getTime())).toBeLessThan(5000);
}); });
```

TC-NOTIF-03: Reminder Scheduled 1h Before Appointment

Field	Value
Test ID	TC-NOTIF-03
Description	A 1h reminder job is scheduled at the correct time relative to the appointment.

```
describe('TC-NOTIF-03: Reminder scheduled 1h before appointment', () => { it('schedules a 1h
reminder job at the correct time', async () => { const appointmentTime = new Date(Date.now() + 48
* 60 * 60 * 1000); const barber = await createTestBarber(); const booking = await
createTestBooking({ barber_id: barber.id, start_time: appointmentTime.toISOString(), }); await
scheduleReminder(db, booking.id, '1h'); const { data: job } = await db .from('scheduled_jobs')
.select('*') .eq('booking_id', booking.id) .eq('type', 'reminder_1h') .single();
expect(job).not.toBeNull(); const expectedTime = new Date(appointmentTime.getTime() - 60 * 60 *
1000); const actualTime = new Date(job!.scheduled_at); expect(Math.abs(actualTime.getTime() -
expectedTime.getTime())).toBeLessThan(5000); }); });
```

TC-NOTIF-04: Reminder Not Sent if Booking Cancelled

Field	Value
Test ID	TC-NOTIF-04
Description	The reminder sending function skips sending and returns early when the booking status is `cancelled`.

```
describe('TC-NOTIF-04: Reminder not sent for cancelled booking', () => { it('skips reminder send
and returns early for cancelled booking', async () => { const barber = await createTestBarber();
const booking = await createTestBooking({ barber_id: barber.id, status: 'cancelled', }); const
resendSpy = vi.fn(); resendMockServer.use( http.post('https://api.resend.com/emails', () => {
resendSpy(); return HttpResponse.json({ id: 'should-not-be-called' }); }) ); const result = await
sendReminder(db, booking.id, '24h'); expect(result.skipped).toBe(true);
expect(result.reason).toMatch(/cancelled/i); expect(resendSpy).not.toHaveBeenCalled(); }); });
```

TC-NOTIF-05: Reminder Not Sent if Appointment is in the Past

Field	Value
Test ID	TC-NOTIF-05
Description	If the appointment `start_time` is already in the past when the reminder fires, the send is skipped.

```
describe('TC-NOTIF-05: Reminder not sent for past appointment', () => { it('skips sending if appointment start_time is in the past', async () => { const pastTime = new Date(Date.now() - 2 * 60 * 60 * 1000); // 2h ago const barber = await createTestBarber(); const booking = await createTestBooking({ barber_id: barber.id, start_time: pastTime.toISOString(), status: 'confirmed', }); const result = await sendReminder(db, booking.id, '1h'); expect(result.skipped).toBe(true); expect(result.reason).toMatch(/past|already passed/i); }); });
```

TC-NOTIF-06: Message Log Record Created for Each Send Attempt

Field	Value
Test ID	TC-NOTIF-06
Description	Every email send attempt (success or failure) creates a record in `message_log`.

```
describe('TC-NOTIF-06: Message log record created for each send attempt', () => { it('creates a message_log record for a successful send', async () => { const barber = await createTestBarber(); const booking = await createTestBooking({ barber_id: barber.id }); await sendConfirmationEmail(db, booking.id); const { data: logs } = await db .from('message_log') .select('*') .eq('booking_id', booking.id); expect(logs!.length).toBeGreaterThanOrEqual(1); const log = logs![0]; expect(log.type).toBe('confirmation'); expect(log.status).toBe('sent'); expect(log.sent_at).not.toBeNull(); }); });
```

TC-NOTIF-07: Failed Email Send Logged with status = failed

Field	Value
Test ID	TC-NOTIF-07
Description	When the Resend API returns an error, the `message_log` record is written with `status = 'failed'` and the error message captured.

```
describe('TC-NOTIF-07: Failed email send is logged with status=failed', () => { it('logs failed status when Resend API returns an error', async () => { // Override MSW to simulate Resend failure resendMockServer.use( http.post('https://api.resend.com/emails', () => { return HttpResponse.json( { name: 'validation_error', message: 'Invalid to address' }, { status: 422 } ); }) ); const barber = await createTestBarber(); const booking = await createTestBooking({ barber_id: barber.id, customer_email: 'invalid-address', }); const result = await sendConfirmationEmail(db, booking.id); expect(result.success).toBe(false); const { data: log } = await db .from('message_log') .select('*') .eq('booking_id', booking.id) .eq('type', 'confirmation') .single(); expect(log!.status).toBe('failed');
```

```
expect(log!.error_message).toMatch(/invalid|validation/i); }); });
```

---

9. API Route Tests (Integration — Vitest)

Each route is tested at the HTTP level using `fetch` against the local dev server, with a real local Supabase DB.

Test Matrix

Route	Auth	Role guard	Validation	Happy path	Not found
`GET /api/public/barbers/[slug]`	None	N/A	Slug format	200 + barber data	404
`POST /api/public/bookings`	None	N/A	Body schema	201 + booking	—
`GET /api/public/availability/[slug]`	None	N/A	Date param	200 + slots	404
`PATCH /api/barber/bookings/[id]`	Required (barber JWT)	barber owns booking	Body schema	200 updated	404
`POST /api/admin/barbers`	Required (admin JWT)	admin role	Body schema	201 created	—

GET /api/public/barbers/[slug]

```
// tests/integration/api/public-barbers.test.ts import { describe, it, expect } from 'vitest';
import { createTestBarber } from '../../helpers/factories'; import { getTestDb } from
'../../helpers/db'; const BASE = 'http://localhost:3000'; const db = getTestDb(); describe('GET
/api/public/barbers/[slug]', () => { it('TC-API-BARBER-01: returns barber data for valid slug
(happy path)', async () => { const barber = await createTestBarber({ slug: 'api-test-barber' });
const res = await fetch(`${BASE}/api/public/barbers/api-test-barber`);
expect(res.status).toBe(200); const body = await res.json();
expect(body.barber.slug).toBe('api-test-barber'); expect(body.barber.name).toBe(barber.name); //
Sensitive fields (e.g., email) should not be exposed
expect(body.barber.password_hash).toBeUndefined(); }); it('TC-API-BARBER-02: returns 404 for
non-existent slug', async () => { const res = await
fetch(`${BASE}/api/public/barbers/does-not-exist-xyz`); expect(res.status).toBe(404); const body
= await res.json(); expect(body.error).toBeTruthy(); }); it('TC-API-BARBER-03: returns 400 for
invalid slug format', async () => { // Slug with special characters not allowed const res = await
fetch(`${BASE}/api/public/barbers/invalid slug!`); expect([400, 404]).toContain(res.status); });
});
```

POST /api/public/bookings

```
// tests/integration/api/public-bookings.test.ts import { describe, it, expect, afterEach } from
'vitest'; const BASE = 'http://localhost:3000'; describe('POST /api/public/bookings', () => {
const testEmail = `booking-api-test-${Date.now()}@example.com`; afterEach(async () => { await
db.from('bookings').delete().eq('customer_email', testEmail); }); const validPayload = () => ({
barber_slug: 'test-barber', service_id: 'test-service-id', start_time: new Date(Date.now() +
86400000).toISOString(), customer_name: 'API Test User', customer_email: testEmail,
customer_phone: '+16135550099', }); it('TC-API-BOOK-01: creates booking (happy path, no
deposit)', async () => { const res = await fetch(`${BASE}/api/public/bookings`, { method: 'POST',
headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(validPayload()), });
expect(res.status).toBe(201); const body = await res.json();
expect(body.booking.id).toBeTruthy(); expect(body.booking.status).toBe('confirmed'); });
it('TC-API-BOOK-02: returns 400 for missing required fields', async () => { const payload = {
barber_slug: 'test-barber' }; // missing all other fields const res = await
fetch(`${BASE}/api/public/bookings`, { method: 'POST', headers: { 'Content-Type':
'application/json' }, body: JSON.stringify(payload), }); expect(res.status).toBe(400); const body
= await res.json(); expect(body.errors).toBeTruthy();
expect(Array.isArray(body.errors)).toBe(true); }); it('TC-API-BOOK-03: returns 400 for invalid
email format', async () => { const payload = { ...validPayload(), customer_email: 'not-an-email'
}; const res = await fetch(`${BASE}/api/public/bookings`, { method: 'POST', headers: {
'Content-Type': 'application/json' }, body: JSON.stringify(payload), });
expect(res.status).toBe(400); }); it('TC-API-BOOK-04: returns 409 for conflicting time slot',
async () => { ... [truncated for brevity]
```

GET /api/public/availability/[slug]

```
// tests/integration/api/public-availability.test.ts describe('GET
/api/public/availability/[slug]', () => { it('TC-API-AVAIL-01: returns available slots for valid
slug and date (happy path)', async () => { const tomorrow = new Date();
tomorrow.setDate(tomorrow.getDate() + 1); const dateStr = tomorrow.toISOString().split('T')[0];
const res = await fetch(
`${BASE}/api/public/availability/test-barber?date=${dateStr}&service_id=test-service-id` );
expect(res.status).toBe(200); const body = await res.json();
expect(Array.isArray(body.slots)).toBe(true); }); it('TC-API-AVAIL-02: returns 404 for
non-existent barber slug', async () => { const res = await fetch(
`${BASE}/api/public/availability/does-not-exist-xyz?date=2026-04-06` );
expect(res.status).toBe(404); }); it('TC-API-AVAIL-03: returns 400 when date param is missing',
async () => { const res = await fetch(`${BASE}/api/public/availability/test-barber`);
expect(res.status).toBe(400); const body = await res.json();
expect(body.error).toMatch(/date|required/i); }); it('TC-API-AVAIL-04: returns 400 for invalid
date format', async () => { const res = await fetch(
`${BASE}/api/public/availability/test-barber?date=not-a-date` ); expect(res.status).toBe(400);
}); });
```

PATCH /api/barber/bookings/[id]

```
// tests/integration/api/barber-bookings.test.ts import { getBarberAuthToken } from
'../../helpers/auth'; describe('PATCH /api/barber/bookings/[id]', () => { let barberToken:
string; let bookingId: string; beforeAll(async () => { barberToken = await
getBarberAuthToken('barber@test.com', 'TestPassword123!'); const booking = await
createTestBooking({ barber_slug: 'test-barber' }); bookingId = booking.id; });
it('TC-API-BBOOK-01: updates booking status (happy path)', async () => { const res = await
fetch(`${BASE}/api/barber/bookings/${bookingId}`, { method: 'PATCH', headers: { 'Content-Type':
'application/json', Authorization: `Bearer ${barberToken}`, }, body: JSON.stringify({ status:
'completed' }), }); expect(res.status).toBe(200); const body = await res.json();
```

```
expect(body.booking.status).toBe('completed'); }); it('TC-API-BBOOK-02: returns 401 without auth
token', async () => { const res = await fetch(`${BASE}/api/barber/bookings/${bookingId}`, {
method: 'PATCH', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ status:
'completed' }), }); expect(res.status).toBe(401); }); it('TC-API-BBOOK-03: returns 403 when
barber does not own the booking', async () => { // Create a booking belonging to a different
barber const otherBooking = await createTestBooking({ barber_slug: 'other-test-barber' }); const
res = await fetch(`${BASE}/api/barber/bookings/${otherBooking.id}`, { method: 'PATCH', headers: {
'Content-Type': 'application/json', Authorization: `Bearer ${barberToken}`, // token for
test-barber, not other-test-barber }, body: JSON.stringify({ status: 'completed' }), });
expect(res.status).toBe(403); }); it('TC-API-BBOOK-04: returns 400 for invalid status value',
async () => { const res = await fetch(`${BASE}/api/barber/bookings/${bookingId}`, { method:
'PATCH', headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${barberToken}`,
}, body: JSON.stringify({ status: 'invalid-status-value' }), ... [truncated for brevity]
```

POST /api/admin/barbers

```
// tests/integration/api/admin-barbers.test.ts import { getAdminAuthToken } from
'../../helpers/auth'; import { getBarberAuthToken } from '../../helpers/auth'; describe('POST
/api/admin/barbers', () => { let adminToken: string; let barberToken: string; beforeAll(async ()
=> { adminToken = await getAdminAuthToken('admin@test.com', 'AdminPassword123!'); barberToken =
await getBarberAuthToken('barber@test.com', 'TestPassword123!'); }); const validPayload = () =>
({ name: `Test Barber ${Date.now()}`, email: `barber-${Date.now()}@example.com`, slug:
`test-barber-${Date.now()}`, password: 'SecureBarberPass123!', }); it('TC-API-ABARB-01: creates
barber (happy path, admin token)', async () => { const payload = validPayload(); const res =
await fetch(`${BASE}/api/admin/barbers`, { method: 'POST', headers: { 'Content-Type':
'application/json', Authorization: `Bearer ${adminToken}`, }, body: JSON.stringify(payload), });
expect(res.status).toBe(201); const body = await res.json();
expect(body.barber.slug).toBe(payload.slug); // Cleanup await
db.from('barbers').delete().eq('slug', payload.slug); }); it('TC-API-ABARB-02: returns 401
without auth token', async () => { const res = await fetch(`${BASE}/api/admin/barbers`, { method:
'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(validPayload()),
}); expect(res.status).toBe(401); }); it('TC-API-ABARB-03: returns 403 when barber-role token is
used', async () => { const res = await fetch(`${BASE}/api/admin/barbers`, { method: 'POST',
headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${barberToken}`, }, body:
JSON.stringify(validPayload()), }); expect(res.status).toBe(403); }); ... [truncated for brevity]
```

---

10. Test Setup & Helpers

10.1 Vitest Configuration

```
// vitest.config.ts import { defineConfig } from 'vitest/config'; import path from 'path'; export
default defineConfig({ test: { globals: true, environment: 'node', include:
['tests/unit/**/*.test.ts', 'tests/integration/**/*.test.ts'], exclude: ['tests/e2e/**',
'node_modules/**'], setupFiles: ['tests/setup/vitest-setup.ts'], coverage: { provider: 'v8',
include: ['src/lib/**', 'src/app/api/**', 'src/services/**'], exclude: [ 'src/**/*.d.ts',
'src/**/*.config.*', 'src/app/layout.tsx', 'src/app/page.tsx', ], thresholds: { statements: 80,
branches: 75, functions: 80, lines: 80, }, reporter: ['text', 'lcov', 'html'], }, testTimeout:
15000, }, resolve: { alias: { '@': path.resolve(__dirname, './src'), }, }, });
```

10.2 Vitest Setup File

```
// tests/setup/vitest-setup.ts import { beforeAll, afterAll } from 'vitest'; import { setupServer
} from 'msw/node'; // Supabase local instance must be running before tests // Run: supabase start
--local beforeAll(async () => { // Verify local Supabase is reachable const supabaseUrl =
process.env.SUPABASE_URL ?? 'http://localhost:54321'; try { const res = await
fetch(`${supabaseUrl}/health`); if (!res.ok) throw new Error('Supabase health check failed'); }
catch { throw new Error( 'Local Supabase is not running. Start it with: supabase start' ); } });
afterAll(async () => { // Clean up any stale test data const { getTestDb } = await
import('../helpers/db'); const db = getTestDb(); await
db.from('bookings').delete().like('customer_email', '%-test@example.com'); await
db.from('clients').delete().like('email', '%-test@example.com'); await
db.from('barbers').delete().like('slug', 'test-barber-%'); });
```

10.3 Test Database Helpers

```
// tests/helpers/db.ts import { createClient, SupabaseClient } from '@supabase/supabase-js'; let
_client: SupabaseClient | null = null; export function getTestDb(): SupabaseClient { if
(!_client) { const url = process.env.SUPABASE_URL ?? 'http://localhost:54321'; const key =
process.env.SUPABASE_SERVICE_ROLE_KEY ?? ''; if (!key) throw new Error('SUPABASE_SERVICE_ROLE_KEY
is required for tests'); _client = createClient(url, key, { auth: { persistSession: false }, });
} return _client; }
```

10.4 Test Factories

```
// tests/helpers/factories.ts import { faker } from '@faker-js/faker'; import { getTestDb } from
'./db'; const db = getTestDb(); // ─── Barber factory
──────────────────────────────────────────────────────────────── export interface
CreateTestBarberOptions { slug?: string; name?: string; email?: string; status?: 'active' |
'suspended'; } export async function createTestBarber(opts: CreateTestBarberOptions = {}) { const
slug = opts.slug ?? `test-barber-${faker.string.nanoid(8).toLowerCase()}`; const { data, error }
= await db .from('barbers') .insert({ slug, name: opts.name ?? faker.person.fullName(), email:
opts.email ?? faker.internet.email(), status: opts.status ?? 'active', }) .select() .single(); if
(error) throw new Error(`createTestBarber failed: ${error.message}`); return data!; } // ───
Service factory ──────────────────────────────────────────────────────────────── export interface
CreateTestServiceOptions { barber_id: string; name?: string; duration_minutes?: number;
price_cents?: number; deposit_required?: boolean; deposit_amount_cents?: number; } export async
function createTestService(opts: CreateTestServiceOptions) { const { data, error } = await db
.from('services') .insert({ barber_id: opts.barber_id, name: opts.name ??
faker.commerce.productName(), duration_minutes: opts.duration_minutes ?? 45, price_cents:
opts.price_cents ?? 3500, deposit_required: opts.deposit_required ?? false, deposit_amount_cents:
opts.deposit_amount_cents ?? 0, }) .select() .single(); if (error) throw new
Error(`createTestService failed: ${error.message}`); return data!; } ... [truncated for brevity]
```

10.5 Stripe Test Helpers

```
// tests/helpers/stripe-helpers.ts import Stripe from 'stripe'; import crypto from 'crypto';
const STRIPE_WEBHOOK_SECRET = process.env.STRIPE_WEBHOOK_SECRET_TEST!; const STRIPE_SECRET_KEY =
process.env.STRIPE_SECRET_KEY_TEST!; const stripe = new Stripe(STRIPE_SECRET_KEY, { apiVersion:
'2024-04-10' }); export interface WebhookEventData { id?: string; customer_email?: string;
metadata?: Record<string, string>; payment_status?: string; [key: string]: unknown; } export
function generateStripeWebhookEvent( type: string, dataObject: WebhookEventData ): { payload:
string; signature: string } { const timestamp = Math.floor(Date.now() / 1000); const event = {
id: `evt_test_${crypto.randomBytes(8).toString('hex')}`, object: 'event', type, created:
```

```
timestamp, data: { object: dataObject }, }; const payload = JSON.stringify(event); const
signedPayload = `${timestamp}.${payload}`; const hmac = crypto .createHmac('sha256',
STRIPE_WEBHOOK_SECRET) .update(signedPayload) .digest('hex'); const signature =
`t=${timestamp},v1=${hmac}`; return { payload, signature }; } export const stripeWebhookHelper =
{ async sendEvent(type: string, data: WebhookEventData): Promise<void> { const { payload,
signature } = generateStripeWebhookEvent(type, data); const response = await
fetch('http://localhost:3000/api/webhooks/stripe', { method: 'POST', headers: { 'Content-Type':
'application/json', 'stripe-signature': signature, }, body: payload, }); if (!response.ok) {
const body = await response.text(); throw new Error(`Webhook delivery failed
(${response.status}): ${body}`); } ... [truncated for brevity]
```

10.6 Auth Helpers

```
// tests/helpers/auth.ts import { createClient } from '@supabase/supabase-js'; const supabase =
createClient( process.env.SUPABASE_URL ?? 'http://localhost:54321', process.env.SUPABASE_ANON_KEY
?? '' ); export async function getBarberAuthToken(email: string, password: string):
Promise<string> { const { data, error } = await supabase.auth.signInWithPassword({ email,
password }); if (error || !data.session) throw new Error(`Barber auth failed:
${error?.message}`); return data.session.access_token; } export async function
getAdminAuthToken(email: string, password: string): Promise<string> { const { data, error } =
await supabase.auth.signInWithPassword({ email, password }); if (error || !data.session) throw
new Error(`Admin auth failed: ${error?.message}`); return data.session.access_token; }
```

10.7 Playwright Configuration

```
// playwright.config.ts import { defineConfig, devices } from '@playwright/test'; export default
defineConfig({ testDir: './tests/e2e', fullyParallel: false, // Serial for booking conflict tests
forbidOnly: !!process.env.CI, retries: process.env.CI ? 2 : 0, workers: process.env.CI ? 1 :
undefined, reporter: [ ['list'], ['html', { outputFolder: 'playwright-report', open: 'never' }],
['junit', { outputFile: 'playwright-results.xml' }], ], use: { baseURL:
process.env.PLAYWRIGHT_BASE_URL ?? 'http://localhost:3000', trace: 'on-first-retry', screenshot:
'only-on-failure', video: 'retain-on-failure', }, projects: [ // Auth setup project runs first {
name: 'setup', testMatch: /global\.setup\.ts/, }, // Main browser tests { name: 'chromium', use:
{ ...devices['Desktop Chrome'], storageState: 'playwright/.auth/user.json', }, dependencies:
['setup'], }, // Mobile viewport smoke test { name: 'mobile-chrome', use: { ...devices['Pixel 5']
}, testMatch: /booking\.spec\.ts/, }, ], // Start Next.js dev server before tests webServer: {
command: 'npm run dev', url: 'http://localhost:3000', reuseExistingServer: !process.env.CI,
timeout: 120000, }, });
```

10.8 Playwright Global Auth Setup

```
// tests/e2e/global.setup.ts import { test as setup, expect } from '@playwright/test'; import
path from 'path'; const barberAuthFile = path.join(__dirname,
'../../playwright/.auth/barber.json'); const adminAuthFile = path.join(__dirname,
'../../playwright/.auth/admin.json'); setup('authenticate as barber', async ({ page }) => { await
page.goto('/barber/login'); await page.getByLabel(/email/i).fill('barber@test.com'); await
page.getByLabel(/password/i).fill('TestPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); await expect(page).toHaveURL(/\/barber\/dashboard/); await
page.context().storageState({ path: barberAuthFile }); }); setup('authenticate as admin', async
({ page }) => { await page.goto('/admin/login'); await
page.getByLabel(/email/i).fill('admin@test.com'); await
page.getByLabel(/password/i).fill('AdminPassword123!'); await page.getByRole('button', { name:
/sign in|log in/i }).click(); await expect(page).toHaveURL(/\/admin/); await
```

```
page.context().storageState({ path: adminAuthFile }); });
```

10.9 Environment Variables for Tests

```
# .env.test SUPABASE_URL=http://localhost:54321
SUPABASE_ANON_KEY=<local-anon-key-from-supabase-start>
SUPABASE_SERVICE_ROLE_KEY=<local-service-role-key-from-supabase-start>
STRIPE_SECRET_KEY_TEST=sk_test_... STRIPE_WEBHOOK_SECRET_TEST=whsec_...
PLAYWRIGHT_BASE_URL=http://localhost:3000 RESEND_API_KEY=re_test_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

---

11. CI/CD Test Integration

11.1 GitHub Actions Workflow

```
# .github/workflows/test.yml name: BarberBook Test Suite on: pull_request: branches: [main,
develop] push: branches: [main] concurrency: group: ${{ github.workflow }}-${{ github.ref }}
cancel-in-progress: true jobs: # ─── Unit + Integration Tests (every PR)
──────────────────────────────────── unit-integration: name: Unit & Integration Tests runs-on:
ubuntu-latest timeout-minutes: 20 services: # Supabase local is started via CLI, not
docker-compose service # We use supabase CLI in the steps below steps: - name: Checkout uses:
actions/checkout@v4 - name: Setup Node.js uses: actions/setup-node@v4 with: node-version: '20'
cache: 'npm' - name: Install dependencies run: npm ci - name: Setup Supabase CLI uses:
supabase/setup-cli@v1 with: version: latest - name: Start Supabase local run: supabase start env:
SUPABASE_ACCESS_TOKEN: ${{ secrets.SUPABASE_ACCESS_TOKEN }} - name: Run DB migrations run:
supabase db push - name: Seed test data run: npx tsx tests/setup/seed-test-data.ts env:
SUPABASE_URL: http://localhost:54321 SUPABASE_SERVICE_ROLE_KEY: ${{ env.SUPABASE_SERVICE_ROLE_KEY
}} - name: Run unit tests run: npm run test:unit env: SUPABASE_URL: http://localhost:54321 ...
[truncated for brevity]
```

11.2 package.json Test Scripts

```
{ "scripts": { "test": "vitest run", "test:unit": "vitest run tests/unit", "test:integration":
"vitest run tests/integration", "test:watch": "vitest", "test:coverage": "vitest run --coverage",
"test:coverage-check": "vitest run --coverage --reporter=verbose", "test:e2e": "playwright test",
"test:e2e:ui": "playwright test --ui", "test:e2e:headed": "playwright test --headed" } }
```

11.3 Branch Protection Rules

Configure in GitHub → Settings → Branches → Branch protection rules for `main`:

Setting	Value
Require status checks to pass before merging	Enabled

Setting	Value
Required status checks	`Unit & Integration Tests`, `All Required Tests Passed`
Require branches to be up to date	Enabled
Require pull request reviews	1 approval minimum
Block force pushes	Enabled
E2E test suite	Run on merge (not a required PR gate — runs post-merge to main)

Note: E2E tests run post-merge to `main` rather than as a PR gate to prevent Stripe API rate limiting and long CI feedback loops. If E2E fails on `main`, the team is alerted via GitHub Actions failure notification and a fix must be immediately merged.

11.4 Supabase Local Dev Setup

```
# Start local Supabase stack supabase start # Output provides: # API URL: http://localhost:54321 # anon key: eyJ... # service_role: eyJ... # Apply all migrations supabase db push # Reset and re-seed for a clean test run supabase db reset # Inspect local DB supabase db diff # see schema drift
```

11.5 Seed Script for E2E Tests

```
// tests/setup/seed-e2e-data.ts import { createClient } from '@supabase/supabase-js'; const db = createClient( process.env.SUPABASE_URL!, process.env.SUPABASE_SERVICE_ROLE_KEY! ); async function seed() { console.log('Seeding E2E test data...'); // Upsert test barber await db.from('barbers').upsert({ slug: 'test-barber', name: 'Test Barber', email: 'test-barber@example.com', status: 'active', }, { onConflict: 'slug' }); // Upsert working hours (Mon–Sat, 9am–6pm) const days = [1, 2, 3, 4, 5, 6]; // Mon=1 ... Sat=6 for (const day of days) { await db.from('working_hours').upsert({ barber_slug: 'test-barber', day_of_week: day, start_time: '09:00', end_time: '18:00', }, { onConflict: 'barber_slug,day_of_week' }); } // Upsert test services await db.from('services').upsert([ { id: 'test-service-id', barber_slug: 'test-barber', name: 'Classic Cut', duration_minutes: 45, price_cents: 3500, deposit_required: false, deposit_amount_cents: 0, }, { id: 'test-deposit-service-id', barber_slug: 'test-barber', name: 'Deluxe Package (Deposit Required)', duration_minutes: 90, price_cents: 8000, deposit_required: true, deposit_amount_cents: 2000, }, ], { onConflict: 'id' }); // Create test barber auth user (if not exists) try { await db.auth.admin.createUser({ email: 'barber@test.com', password: 'TestPassword123!', user_metadata: { role: 'barber', barber_slug: 'test-barber' }, }); } catch {
... [truncated for brevity]
```

End of Phase 8: Test Plan

Document control

Version	Date	Author	Notes
1.0	2026-03-30	QA Engineering	Initial draft — MVP test coverage

Phase 9 **Documentation** Setup · Deploy · ADRs

BarberBook — Developer Documentation

Phase 9: MVP Technical Reference

Version: 1.0.0

Last Updated: March 30, 2026

Maintainer: BarberBook Engineering

Table of Contents

1. [Project Overview](#1-project-overview)
2. [Prerequisites](#2-prerequisites)
3. [Local Development Setup](#3-local-development-setup)
4. [Environment Variables Reference Table](#4-environment-variables-reference-table)
5. [Project Structure Guide](#5-project-structure-guide)
6. [Key Development Workflows](#6-key-development-workflows)
7. [Deployment Guide](#7-deployment-guide)
8. [Architecture Decision Records (ADRs)](#8-architecture-decision-records-adrs)
9. [Post-MVP Upgrade Notes](#9-post-mvp-upgrade-notes)
10. [Troubleshooting Guide](#10-troubleshooting-guide)

---

1. Project Overview

What Is BarberBook?

BarberBook is a single-barber branded booking micro-platform. It gives an independent barber a fully owned, white-label booking experience without relying on third-party marketplace platforms. Clients visit the barber's custom URL, browse available time slots, choose a service, and pay a deposit — all in one flow. The barber manages their schedule, services, availability, and bookings through a private admin dashboard.

Core Features (MVP)

Feature	Description
Public booking flow	Clients select a service, pick a slot, and pay a deposit
Stripe deposit collection	Stripe Checkout handles payment; webhook confirms booking
Automated email notifications	Booking confirmation, reminders (24 h + 2 h), and cancellation notices via Resend
Admin dashboard	Barber manages calendar, services, availability, and bookings
Slot availability engine	Real-time slot generation respecting existing bookings and blocked times
Scheduled reminders	Vercel Cron fires a daily job to queue upcoming appointment reminders
No customer accounts	Bookings are guest-based; clients are identified by email only

Tech Stack Summary

Layer	Technology
Framework	Next.js 14 (App Router)
Language	TypeScript 5
Styling	Tailwind CSS 3
Database + Auth	Supabase (PostgreSQL 15, GoTrue auth)
Payments	Stripe (Checkout Sessions, Webhooks)
Email	Resend (React Email templates)
Hosting	Vercel (Edge + Node.js runtimes)
Scheduled Jobs	Vercel Cron
Testing	Vitest (unit/integration), Playwright (E2E)
Package Manager	pnpm

Repository Structure Overview

```
barberbook/ ├── app/ # Next.js App Router — pages, layouts, API routes │ ├── (public)/ #
Public-facing booking pages │ ├── barber/ # Barber admin dashboard (protected) │ ├── api/ # API
route handlers │ └── layout.tsx # Root layout ├── src/ │ ├── components/ # Shared React UI
components │ ├── lib/ # Service layer, clients, utilities │ └── types/ # Shared TypeScript type
definitions ├── supabase/ │ ├── migrations/ # Ordered SQL migration files │ └── seed.sql # Test
data seed ├── public/ # Static assets (images, icons, fonts) ├── tests/ # Playwright E2E test
suites ├── __tests__/ # Vitest unit and integration tests ├── vercel.json # Vercel config
including cron schedule ├── .env.local.example # Environment variable template ├── next.config.ts
# Next.js configuration ├── tailwind.config.ts # Tailwind configuration └── tsconfig.json #
```

```
TypeScript configuration
```

---

2. Prerequisites

Before cloning and running the project, ensure the following are installed and configured on your machine.

Required Software

Tool	Minimum Version	Install
Node.js	20.x LTS	https://nodejs.org or `nvm install 20`
pnpm	9.x	`npm install -g pnpm`
Git	2.x	https://git-scm.com
Supabase CLI	1.x	`brew install supabase/tap/supabase` or see below
Stripe CLI	1.x	`brew install stripe/stripe-cli/stripe` or see below
Docker Desktop	Latest stable	https://www.docker.com/products/docker-desktop (required by Supabase local)

Install Supabase CLI (non-Homebrew):

```
# npm npm install -g supabase # or download binary # https://github.com/supabase/cli/releases
```

Install Stripe CLI (non-Homebrew):

```
# Download from https://stripe.com/docs/stripe-cli # or via scoop on Windows: scoop bucket add stripe https://github.com/stripe/scoop-stripe-cli.git scoop install stripe
```

Required Accounts

You must have active accounts on all four services before running in production. For local development, test/free tiers are sufficient.

Service	Purpose	Sign-up URL
Supabase	Database, Auth, Storage	https://supabase.com
Stripe	Payment processing	https://stripe.com
Resend	Transactional email	https://resend.com

Service	Purpose	Sign-up URL
Vercel	Hosting and cron	https://vercel.com

Node Version Management (Recommended)

Use `nvm` to pin the Node version:

```
# Install nvm curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash # Install and use Node 20 nvm install 20 nvm use 20 # Verify node -v # should print v20.x.x
```

---

3. Local Development Setup

Follow these steps in order from a clean machine or a fresh clone.

3.1 Clone the Repository

```
git clone https://github.com/your-org/barberbook.git cd barberbook pnpm install
```

`pnpm install` reads `pnpm-lock.yaml` and installs exact dependency versions. Do not use `npm install` — it will generate a conflicting lockfile.

Verify the install:

```
pnpm run build --dry-run # should exit 0 with no errors
```

3.2 Supabase Local Setup

BarberBook uses Supabase's local development stack, which runs Postgres, GoTrue, and the Supabase Studio UI inside Docker containers.

Step 1 — Verify Docker is running:

```
docker info # should print engine details, not an error
```

Step 2 — Initialize Supabase (first time only):

```
supabase init
```

This creates a `supabase/` directory with a default `config.toml`. If the directory already exists (it does in this repo), skip this step.

Step 3 — Start the local Supabase stack:

```
supabase start
```

On first run this pulls ~2 GB of Docker images. Subsequent starts are fast. When complete, the CLI prints output similar to:

```
Started supabase local development setup. API URL: http://localhost:54321 GraphQL URL:
http://localhost:54321/graphql/v1 DB URL: postgresql://postgres:postgres@localhost:54322/postgres
Studio URL: http://localhost:54323 Inbucket URL: http://localhost:54324 ← local email testing
inbox JWT secret: super-secret-jwt-token-with-at-least-32-characters-long anon key: eyJh...
service_role key: eyJh...
```

Copy the `API URL`, `anon key`, and `service_role key` — you will need them in `.env.local`.

Step 4 — Run database migrations:

```
supabase db reset
```

`db reset` drops and recreates the local database, then applies every file in `supabase/migrations/` in chronological order, and finally runs `supabase/seed.sql`.

To apply only new migrations without resetting:

```
supabase migration up
```

Step 5 — Verify migrations applied:

```
supabase status
```

Open Supabase Studio at http://localhost:54323 and confirm that the bookings, services, availabilityrules, blockedtimes, and barbers tables exist under the Table Editor.

Step 6 — Seed the database:

The seed file is applied automatically by `supabase db reset`. To re-run it manually:

```
psql postgresql://postgres:postgres@localhost:54322/postgres -f supabase/seed.sql
```

The seed creates:

- One barber account (`barber@barberbook.local` / `testpassword123`)
- Four sample services (Fade, Beard Trim, Full Cut, Shape-Up)
- Weekly availability rules (Mon–Sat, 9 AM–6 PM)
- Three sample bookings at various states (`pending`, `confirmed`, `cancelled`)

3.3 Environment Variables

Copy the example file and fill in each value:

```
cp .env.local.example .env.local
```

Open `.env.local` and populate every variable. The full reference is in [Section 4](#4-environment-variables-reference-table). Below is a detailed explanation of each variable and where to find it.

`NEXTPUBLICSUPABASE_URL`

The base URL for your Supabase project's REST/Realtime API.

- Local: `http://localhost:54321` (printed by `supabase start`)
- Production: Found in Supabase Dashboard → Project Settings → API → Project URL

This value is prefixed `NEXTPUBLIC` and is therefore exposed in browser bundles. It does not need to be kept secret.

`NEXTPUBLICSUPABASEANONKEY`

The anonymous JWT used by the client-side Supabase SDK. It is subject to Row Level Security (RLS) policies and does not bypass them.

- Local: Printed by `supabase start` as `anon key`
- Production: Found in Supabase Dashboard → Project Settings → API → `anon public`

This value is safe to expose in the browser, but access is governed by RLS rules defined in your migrations.

`SUPABASESERVICEROLE_KEY`

The service role JWT that bypasses all RLS policies. This key has full read/write access to your database.

- Local: Printed by `supabase start` as `service_role key`
- Production: Found in Supabase Dashboard → Project Settings → API → `service_role secret`

> *Critical: This key must NEVER be included in client-side code or committed to version control. It is only used in Next.js API routes (server-side), where it is passed to the admin Supabase client via* `createClient(url, serviceRoleKey)`.

`STRIPESECRETKEY`

The Stripe secret key used server-side to create Checkout Sessions, retrieve payment intents, and call the Stripe API.

- Test mode: Stripe Dashboard → Developers → API keys → Secret key (begins with `sktest`)
- Live mode: Same location, toggled to Live mode (begins with `sklive`)

Never expose this in browser-side code. Use only in API routes.

`NEXTPUBLICSTRIPEPUBLISHABLEKEY`

The Stripe publishable key used client-side to initialize Stripe.js.

- Test mode: Stripe Dashboard → Developers → API keys → Publishable key (begins with `pktest`)
- Live mode: Same location, toggled to Live mode (begins with `pklive`)

`STRIPEWEBHOOKSECRET`

The signing secret Stripe uses to sign webhook event payloads. Your API route verifies this signature to reject forged requests.

- Local development: Generated automatically by the Stripe CLI listener (see [Section 3.4](#34-stripe-local-webhook-setup)). Begins with `whsec_`.
- Production: Created when you register a webhook endpoint in the Stripe Dashboard → Developers → Webhooks → Add endpoint.

`RESENDAPIKEY`

The API key for the Resend email service, used by `NotificationService` to send all transactional emails.

- Resend Dashboard → API Keys → Create API Key
- Begins with `re_`
- Sandbox mode note: In Resend's free tier, emails can only be sent to verified addresses unless your sending domain is verified. During local development, all test emails are visible in the local Supabase Inbucket inbox at http://localhost:54324. You do not need a real Resend key for purely local development if you mock the email client.

`NEXTPUBLICAPP_URL`

The canonical base URL of the application, used to construct absolute URLs in emails (booking confirmation links, cancellation links) and in Stripe success/cancel redirect URLs.

- Local: `http://localhost:3000`
- Production: `https://yourdomain.com`

`INTERNALCRONSECRET`

A random secret string used to authenticate calls to the `/api/cron/send-reminders` route. This prevents unauthorized external parties from triggering the cron endpoint.

Generate a suitable value:

```
openssl rand -hex 32
```

The value must match exactly between `vercel.json` (or your cron trigger) and this environment variable.

3.4 Stripe Local Webhook Setup

Stripe webhooks cannot reach `localhost` directly. The Stripe CLI provides a local tunnel that forwards webhook events from Stripe's servers to your running dev server.

Step 1 — Log in to Stripe CLI:

```
stripe login
```

A browser window opens; authorize the CLI with your Stripe account.

Step 2 — Start the webhook forwarder:

```
stripe listen --forward-to localhost:3000/api/payments/webhook
```

The CLI prints a webhook signing secret immediately:

```
> Ready! Your webhook signing secret is whsec_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Step 3 — Copy the signing secret into `.env.local`:

```
STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Step 4 — Keep the listener running in a dedicated terminal tab while developing. Every time you restart it, a new `whsec_` is generated — update `.env.local` accordingly.

Testing a webhook manually:

```
stripe trigger checkout.session.completed
```

This sends a synthetic `checkout.session.completed` event through the forwarder to your local route.

3.5 Run the App

With Supabase running, `.env.local` populated, and the Stripe CLI listener active:

```
pnpm dev
```

The development server starts with Turbopack:

```
▲ Next.js 14.x (Turbopack) - Local: http://localhost:3000 - Network: http://192.168.x.x:3000
```

Open http://localhost:3000 in your browser.

URL	Description
http://localhost:3000	Public booking homepage
http://localhost:3000/book	Booking flow
http://localhost:3000/barber/login	Barber admin login

URL	Description
http://localhost:3000/barber/dashboard	Admin dashboard (auth required)
http://localhost:54323	Supabase Studio (local)
http://localhost:54324	Inbucket email inbox (local)

3.6 First-Time Seed — End-to-End Local Test

Use the seeded data to walk through the full booking flow locally.

Step 1 — Log in as the barber admin:

1. Navigate to http://localhost:3000/barber/login
2. Sign in with:

- Email: `barber@barberbook.local`
- Password: `testpassword123`

3. You are redirected to `/barber/dashboard`

Step 2 — Verify services exist:

1. In the dashboard sidebar, click Services
2. Confirm the four seeded services appear (Fade, Beard Trim, Full Cut, Shape-Up)
3. Optionally create a new service: click Add Service, fill in name/duration/price/deposit, and save

Step 3 — Book an appointment as a client:

1. Open a new incognito window and navigate to http://localhost:3000/book
2. Select any available service
3. Pick a date (any upcoming weekday/Saturday within the seeded availability window)
4. Select an available time slot
5. Fill in your name and email
6. On the Stripe payment page, use Stripe's test card:

- Card number: `4242 4242 4242 4242`
- Expiry: any future date (e.g., `12/30`)
- CVC: any 3 digits (e.g., `123`)
- ZIP: any 5 digits (e.g., `10001`)

7. Complete the payment

Step 4 — Verify the booking was created:

1. Check the Stripe CLI terminal — you should see a `checkout.session.completed` event processed with a `200` response
2. In Supabase Studio, open the `bookings` table — the new booking should appear with status `confirmed`
3. Check Inbucket at http://localhost:54324 — a booking confirmation email should have arrived

Step 5 — Verify the admin dashboard reflects the new booking:

1. Switch back to the barber dashboard window
2. Navigate to Bookings — the new booking should appear
3. Navigate to the Calendar — the slot should be marked occupied

---

4. Environment Variables Reference Table

Complete reference for all environment variables used by BarberBook. All variables belong in `.env.local` for local development and in the Vercel Environment Variables dashboard for production.

Variable	Required	Description	Example Value	Where to Get It
`NEXT_PUBLIC_SUPABASE_URL`	Yes	Supabase project API base URL	`http://localhost:54321`	`supabase start` output / Supabase Dashboard → API
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	Supabase anonymous JWT (client-side, RLS-enforced)	`eyJhbGc...`	`supabase start` output / Supabase Dashboard → API
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Supabase service role JWT — bypasses RLS, server-only	`eyJhbGc...`	`supabase start` output / Supabase Dashboard → API
`STRIPE_SECRET_KEY`	Yes	Stripe server-side secret key	`sk_test_51...`	Stripe Dashboard → Developers → API Keys
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Yes	Stripe client-side publishable key	`pk_test_51...`	Stripe Dashboard → Developers → API Keys
`STRIPE_WEBHOOK_SECRET`	Yes	Webhook signature secret for verifying Stripe events	`whsec_abc123...`	`stripe listen` output (local) / Stripe Dashboard → Webhooks (prod)
`RESEND_API_KEY`	Yes	Resend API key for sending transactional emails	`re_abcXYZ...`	Resend Dashboard → API Keys
`NEXT_PUBLIC_APP_URL`	Yes	Canonical app base URL (no trailing slash)	`http://localhost:3000`	Set manually
`INTERNAL_CRON_SECRET`	Yes	Secret to authenticate internal cron job requests	`f3a9d2...` (32-char hex)	Generate: `openssl rand -hex 32`

> *Note: Variables prefixed `NEXTPUBLIC` are embedded into browser bundles at build time and visible to end users. Never put secrets in `NEXTPUBLIC` variables.*

---

5. Project Structure Guide

`app/` — Next.js App Router

The root of all pages, layouts, and API routes, following Next.js 14 App Router conventions.

```
app/ ├── (public)/ │ ├── page.tsx # Landing / booking homepage │ ├── book/ │ │ ├── page.tsx # Step 1: service selection │ │ ├── [serviceId]/ │ │ │ └── page.tsx # Step 2: date/slot picker │ │ └── confirm/ │ │ └── page.tsx # Step 3: client details + payment redirect │ ├── booking/ │ │ ├── success/page.tsx # Post-payment confirmation page │ │ └── cancel/page.tsx # Cancelled/failed payment page │ └── cancel/[token]/page.tsx # Client self-cancellation via email link ├── barber/ │ ├── login/page.tsx # Barber login │ ├── layout.tsx # Auth-guarded layout for all /barber/* routes │ ├── dashboard/page.tsx # Overview stats │ ├── bookings/page.tsx # Booking list + management │ ├── calendar/page.tsx # Weekly calendar view │ ├── services/page.tsx # Service CRUD │ ├── availability/page.tsx # Availability rules management │ └── settings/page.tsx # Barber profile settings └── api/ ├── availability/route.ts # GET available slots for a date+service ├── bookings/ │ ├── route.ts # POST create pending booking │ └── [id]/ │ ├── route.ts # GET, PATCH, DELETE a booking │ └── cancel/route.ts # POST cancel with token auth ├── payments/ │ ├── checkout/route.ts # POST create Stripe Checkout Session │ └── webhook/route.ts # POST handle Stripe webhook events ├── services/ │ └── route.ts # GET public service list ├── notifications/ │ └── route.ts # POST trigger manual notification (admin) └── cron/ └── send-reminders/route.ts # GET cron-triggered reminder dispatch
```

Route group `(public)`: The parentheses mean this segment is not part of the URL. Pages inside render with the public layout (no auth required).

`barber/layout.tsx`: Wraps all barber admin routes. On mount it checks for a valid Supabase session; if none exists, it redirects to `/barber/login`.

`api/payments/webhook/route.ts`: Must export `config = { api: { bodyParser: false } }` (handled automatically in App Router via `req.text()`) to read the raw request body needed for Stripe signature verification.

`src/components/` — Shared UI Components

```
src/components/ ├── ui/ # Primitive components (Button, Input, Modal, Badge, etc.) ├── booking/ # Booking flow-specific components │ ├── ServiceCard.tsx │ ├── DatePicker.tsx │ ├── SlotGrid.tsx │ └── BookingForm.tsx ├── dashboard/ # Admin dashboard components │ ├── BookingTable.tsx │ ├── CalendarView.tsx │ ├── StatsCard.tsx │ └── ServiceForm.tsx ├── email/ # React Email templates │ ├── BookingConfirmation.tsx │ ├── BookingReminder.tsx │ └── BookingCancellation.tsx └── layout/ ├── PublicHeader.tsx ├── DashboardSidebar.tsx └── DashboardHeader.tsx
```

All primitive `ui/` components are built on top of Tailwind utility classes and accept standard HTML props via spread. They do not import any third-party component library.

React Email templates in `email/` are rendered server-side to HTML strings using `@react-email/render` and passed to `NotificationService`. They are not rendered in the browser.

`src/lib/` — Service Layer, Clients, and Utilities

```
src/lib/ ├── supabase/ │ ├── client.ts # Browser-side Supabase client (anon key) │ ├── server.ts
# Server-side Supabase client (service role key) │ └── middleware.ts # Supabase auth session
refresh middleware ├── stripe/ │ └── client.ts # Stripe SDK instance (server-only) ├── services/
│ ├── AvailabilityService.ts # Slot generation and conflict checking │ ├── BookingService.ts #
Booking CRUD and state transitions │ ├── NotificationService.ts # Email dispatch via Resend │ ├──
PaymentService.ts # Stripe Checkout Session creation │ └── ReminderService.ts # Cron-triggered
reminder logic ├── utils/ │ ├── dateUtils.ts # Date/time helpers (timezone-safe) │ ├──
tokenUtils.ts # Cancellation token generation/verification │ └── validators.ts # Zod schemas for
API input validation └── constants.ts # App-wide constants (slot duration, deposit %, etc.)
```

`supabase/client.ts` vs `supabase/server.ts`:

- `client.ts` uses the `NEXTPUBLICSUPABASEANONKEY` and is imported in client components and client-side hooks.
- `server.ts` uses `SUPABASESERVICEROLE_KEY` and is imported only in API routes and server components. It must never be bundled into client code.

`AvailabilityService.ts`:

The most complex service. It reads availabilityrules and blockedtimes from the database, generates all theoretical slots for a given date, subtracts any slots overlapping with existing bookings, and returns the remaining open slots. All time arithmetic uses UTC internally.

`src/types/` — Shared TypeScript Types

```
src/types/ ├── database.types.ts # Auto-generated from Supabase schema (do not edit manually) ├──
booking.ts # Booking domain types and enums ├── service.ts # Service domain types ├──
availability.ts # Slot and availability rule types └── api.ts # API request/response shapes
```

`database.types.ts` is generated by the Supabase CLI and should not be manually edited:

```
supabase gen types typescript --local > src/types/database.types.ts
```

Re-run this command after every schema migration to keep types in sync.

`supabase/` — Migrations and Seed

```
supabase/ ├── config.toml # Local Supabase stack configuration ├── seed.sql # Test data inserted
after migrations on db reset └── migrations/ ├── 20240101000000_create_barbers.sql ├──
20240101000001_create_services.sql ├── 20240101000002_create_availability_rules.sql ├──
20240101000003_create_blocked_times.sql ├── 20240101000004_create_bookings.sql ├──
20240101000005_enable_rls.sql └── 20240101000006_create_rls_policies.sql
```

Migration filenames follow the format `YYYYMMDDHHMMSS_description.sql`. Supabase applies them in lexicographic (chronological) order.

`public/` — Static Assets

```
public/ ├── favicon.ico ├── og-image.png # Open Graph preview image └── barber/ # Barber-uploaded photos (dev placeholder only)
```

In production, barber profile photos are stored in Supabase Storage, not the `public/` directory.

`tests/` — Playwright E2E Tests

```
tests/ ├── booking-flow.spec.ts # Full client booking journey ├── admin-auth.spec.ts # Login, redirect, session expiry ├── cancellation.spec.ts # Cancel via email token └── fixtures/ └── test-data.ts # Shared test data factories
```

Playwright tests run against a live local stack (`pnpm dev` + `supabase start`). See [Section 6.4](#64-running-tests).

`_tests_/` — Vitest Unit and Integration Tests

```
__tests__/ ├── services/ │ ├── AvailabilityService.test.ts │ ├── BookingService.test.ts │ └── NotificationService.test.ts ├── utils/ │ ├── dateUtils.test.ts │ └── tokenUtils.test.ts └── api/ ├── availability.test.ts └── bookings.test.ts
```

Unit tests mock Supabase and Stripe clients using `vi.mock()`. Integration tests use a test database instance.

---

6. Key Development Workflows

6.1 Adding a New Service Field

Use this workflow whenever you need to add a new column to the `services` table (example: adding a `color_code` field for calendar display).

Step 1 — Write a new SQL migration:

```
# Create a new migration file (use current timestamp) touch supabase/migrations/$(date +%Y%m%d%H%M%S)_add_color_code_to_services.sql
```

Edit the file:

```
-- supabase/migrations/YYYYMMDDHHMMSS_add_color_code_to_services.sql ALTER TABLE services ADD COLUMN color_code TEXT NOT NULL DEFAULT '#6B7280'; COMMENT ON COLUMN services.color_code IS 'Hex color for calendar display';
```

Apply locally:

```
supabase migration up
```

Step 2 — Regenerate TypeScript types:

```
supabase gen types typescript --local > src/types/database.types.ts
```

The `Services` row type in `database.types.ts` will now include `color_code: string`.

Step 3 — Update domain types (if applicable):

If you have a manual type in `src/types/service.ts`, add the field:

```
// src/types/service.ts export interface Service { id: string; name: string; duration_minutes: number; price_cents: number; deposit_cents: number; color_code: string; // ← new field is_active: boolean; }
```

Step 4 — Update the API route:

In `app/api/services/route.ts`, ensure the `select()` query returns the new column (or use `select('*')`). If you have an explicit column list, add `color_code`.

In the POST/PATCH handler, add `color_code` to the insert/update body and validate it with the Zod schema in `src/lib/utils/validators.ts`:

```
// src/lib/utils/validators.ts export const serviceSchema = z.object({ name: z.string().min(1), duration_minutes: z.number().int().positive(), price_cents: z.number().int().nonnegative(), deposit_cents: z.number().int().nonnegative(), color_code: z.string().regex(/^#[0-9A-Fa-f]{6}$/).optional().default('#6B7280'), // ← new });
```

Step 5 — Update the admin form:

In `src/components/dashboard/ServiceForm.tsx`, add a color picker input bound to the `color_code` field.

Step 6 — Update tests:

Add the new field to test fixtures in `_tests_/services/` and verify the updated schema validation passes.

6.2 Adding a New Notification Type

Use this workflow to add a new transactional email (example: a "booking rescheduled" notice).

Step 1 — Create the email template:

```
// src/components/email/BookingRescheduled.tsx import { Html, Body, Heading, Text, Button } from '@react-email/components'; interface BookingRescheduledProps { clientName: string; serviceName: string; oldDateTime: string; newDateTime: string; cancellationUrl: string; } export function BookingRescheduled({ clientName, serviceName, oldDateTime, newDateTime, cancellationUrl, }: BookingRescheduledProps) { return ( <Html> <Body> <Heading>Your appointment has been rescheduled</Heading> <Text>Hi {clientName},</Text> <Text> Your {serviceName} appointment has been moved from {oldDateTime} to {newDateTime}. </Text> <Button href={cancellationUrl}>Cancel Appointment</Button> </Body> </Html> ); }
```

Step 2 — Add the send method to `NotificationService`:

```
// src/lib/services/NotificationService.ts import { render } from '@react-email/render'; import {
BookingRescheduled } from '@/components/email/BookingRescheduled'; export class
NotificationService { // ... existing methods async sendRescheduledNotification(booking: Booking,
oldDateTime: Date): Promise<void> { const html = render( <BookingRescheduled
clientName={booking.client_name} serviceName={booking.service.name}
oldDateTime={formatDateTime(oldDateTime)} newDateTime={formatDateTime(new
Date(booking.start_time))}
cancellationUrl={`${process.env.NEXT_PUBLIC_APP_URL}/cancel/${booking.cancellation_token}`} /> );
await this.resend.emails.send({ from: 'BarberBook <noreply@yourdomain.com>', to:
booking.client_email, subject: `Your appointment has been rescheduled — ${booking.service.name}`,
html, }); } }
```

Step 3 — Trigger the notification from the relevant API route:

In `app/api/bookings/[id]/route.ts`, after a successful PATCH that changes the `start_time`, call:

```
await notificationService.sendRescheduledNotification(updatedBooking, previousStartTime);
```

Step 4 — Test locally:

Trigger a reschedule via the admin dashboard. Check Inbucket at http://localhost:54324 to confirm the email arrived and renders correctly.

6.3 Creating a New Barber (Admin Flow)

> *In the MVP, BarberBook supports a single barber. This flow documents the current admin setup process and is the foundation for the multi-barber upgrade path described in [Section 9](#9-post-mvp-upgrade-notes).*

Step 1 — Create the Supabase auth user:

In Supabase Studio (http://localhost:54323 locally, or the production dashboard):

1. Go to Authentication → Users
2. Click Invite user or Add user
3. Enter the barber's email address and a temporary password
4. Click Create user

Note the generated `user.id` (UUID) — you will need it in the next step.

Step 2 — Insert a barber record:

In Supabase Studio → SQL Editor, run:

```
INSERT INTO barbers (id, user_id, name, bio, phone, slug) VALUES ( gen_random_uuid(),
'<paste-user-id-here>', 'Marcus Johnson', 'Master barber with 10 years of experience.',
'+1-555-000-0001', 'marcus-johnson' );
```

The `slug` value becomes the URL path for this barber's public booking page.

Step 3 — Set availability rules:

Navigate to http://localhost:3000/barber/availability. Add weekly recurring availability rules for the new barber.

Step 4 — Add services:

Navigate to http://localhost:3000/barber/services. Add at least one service with a name, duration, price, and deposit amount.

Step 5 — Test the booking flow:

In an incognito window, go to http://localhost:3000/book. The new services and availability should appear immediately.

6.4 Running Tests

Unit and integration tests (Vitest):

```
pnpm test
```

Runs all files matching `_tests_/*/.test.ts` in watch mode. To run once (CI mode):

```
pnpm test --run
```

E2E tests (Playwright):

E2E tests require a running dev server and local Supabase stack.

Terminal 1:

```
supabase start supabase db reset # fresh seed
```

Terminal 2:

```
pnpm dev
```

Terminal 3:

```
pnpm test:e2e
```

To run a specific E2E test file:

```
pnpm test:e2e tests/booking-flow.spec.ts
```

To run E2E tests with the Playwright UI (interactive debugging):

```
pnpm test:e2e --ui
```

Coverage report:

```
pnpm test:coverage
```

Generates an HTML coverage report at `coverage/index.html`. Open it in a browser to see line-by-line coverage.

Recommended coverage targets (MVP):

Module	Target
`AvailabilityService`	≥ 90%
`BookingService`	≥ 85%
`NotificationService`	≥ 75%
API routes	≥ 70%

6.5 Database Migrations

Writing a new migration:

1. Generate a timestamped filename:

```
touch supabase/migrations/$(date +%Y%m%d%H%M%S)_your_description.sql
```

2. Write idempotent SQL. Prefer `IF NOT EXISTS` and `IF EXISTS` guards:

```
ALTER TABLE bookings ADD COLUMN IF NOT EXISTS notes TEXT;
```

3. Never use `DROP TABLE` or `TRUNCATE` in a migration without explicit approval from the team — these are destructive and cannot be rolled back automatically.

Apply locally:

```
supabase migration up
```

Reset local database and re-apply all migrations:

```
supabase db reset
```

Regenerate types after migration:

```
supabase gen types typescript --local > src/types/database.types.ts
```

Apply to production:

After your migration is merged to `main` and you are ready to deploy:

```
# Link to your production Supabase project (first time only) supabase link --project-ref
<your-project-ref> # Push all pending migrations to production supabase db push
```

> *Warning:* `supabase db push` *runs migrations against the live production database. Always test the migration locally and in a staging environment first. Take a manual database backup from the Supabase dashboard before pushing to production.*

Verify production migration:

```
supabase migration list
```

This shows which migrations have been applied and which are pending for the linked project.

---

7. Deployment Guide

7.1 Vercel Deployment

Step 1 — Push code to GitHub:

Ensure your repository is pushed to GitHub (or GitLab / Bitbucket):

```
git remote add origin https://github.com/your-org/barberbook.git git push -u origin main
```

Step 2 — Import project into Vercel:

1. Go to https://vercel.com/new
2. Click Import Git Repository and authorize Vercel to access your GitHub account
3. Select the `barberbook` repository
4. Vercel auto-detects Next.js. Confirm:

- Framework Preset: Next.js

- Build Command: `pnpm build`

- Output Directory: `.next`

- Install Command: `pnpm install`

Step 3 — Set environment variables:

Before deploying, set all variables from [Section 4](#4-environment-variables-reference-table) in Vercel Dashboard → Project → Settings → Environment Variables.

For each variable, set it for the Production environment (and optionally Preview and Development environments with test values).

Key production values to set differently from local:

Variable	Production Value
`NEXT_PUBLIC_SUPABASE_URL`	Your Supabase production project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Production anon key
`SUPABASE_SERVICE_ROLE_KEY`	Production service role key
`STRIPE_SECRET_KEY`	Live mode secret key (`sk_live_...`)
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Live mode publishable key (`pk_live_...`)
`STRIPE_WEBHOOK_SECRET`	Production webhook secret (`whsec_...`)

Variable	Production Value
`RESEND_API_KEY`	Production Resend key
`NEXT_PUBLIC_APP_URL`	`https://yourdomain.com`

Step 4 — Deploy:

Click Deploy. Vercel builds and deploys the app. The first deploy takes 2–4 minutes. Subsequent deploys triggered by `git push` to `main` are typically 60–90 seconds.

Step 5 — Verify deployment:

1. Visit the Vercel-provided deployment URL (e.g., `barberbook.vercel.app`)
2. Complete a test booking using Stripe test mode keys
3. Check Vercel's Functions tab for any runtime errors

7.2 Supabase Production Setup

Step 1 — Create a production Supabase project:

1. Go to https://supabase.com/dashboard
2. Click New Project
3. Choose a region geographically close to your Vercel deployment region (e.g., US East if using Vercel's `iad1` region)
4. Set a strong database password and save it securely

Step 2 — Link CLI to production project:

```
supabase link --project-ref <your-project-ref>
```

Find `<your-project-ref>` in Supabase Dashboard → Project Settings → General → Reference ID.

Step 3 — Push migrations to production:

```
supabase db push
```

All migrations are applied in order. Confirm success in the Supabase Dashboard → Database → Migrations.

Step 4 — Confirm RLS is enabled:

Row Level Security should be enabled by the `enable_rls.sql` migration. Verify in Supabase Dashboard → Database → Tables: every table should show a shield icon with RLS enabled.

To manually verify:

```
SELECT tablename, rowsecurity FROM pg_tables WHERE schemaname = 'public';
```

All tables should show `rowsecurity = true`.

> *Never disable RLS on the `bookings`, `barbers`, or `services` tables in production. Doing so would expose all customer data to unauthenticated API calls.*

Step 5 — Set up Storage bucket for barber photos:

1. Supabase Dashboard → Storage → Create Bucket
2. Name: `barber-photos`
3. Set as Public bucket (photos are displayed on the public booking page)
4. Add a storage policy to allow authenticated barbers to upload to their own folder:

```
-- Allow authenticated users to upload to their own folder CREATE POLICY "Barbers can upload
their own photos" ON storage.objects FOR INSERT TO authenticated WITH CHECK (bucket_id =
'barber-photos' AND auth.uid()::text = (storage.foldername(name))[1]); -- Allow public read
access CREATE POLICY "Public can view barber photos" ON storage.objects FOR SELECT TO public
USING (bucket_id = 'barber-photos');
```

7.3 Stripe Production Setup

Step 1 — Switch to live mode:

In the Stripe Dashboard, toggle from Test mode to Live mode using the switch in the top-left.

Step 2 — Get live API keys:

Go to Developers → API Keys:

- Copy the Publishable key (`pklive...`) → `NEXTPUBLICSTRIPEPUBLISHABLEKEY`
- Reveal and copy the Secret key (`sklive...`) → `STRIPESECRETKEY`

Update both values in Vercel's Environment Variables dashboard.

Step 3 — Register the production webhook endpoint:

1. Go to Stripe Dashboard → Developers → Webhooks → Add endpoint
2. Endpoint URL: `https://yourdomain.com/api/payments/webhook`
3. Select events to listen for:

- `checkout.session.completed`

- `checkout.session.expired`

- `paymentintent.paymentfailed`

4. Click Add endpoint
5. Click Reveal on the Signing secret → copy the `whsec_...` value
6. Update `STRIPEWEBHOOKSECRET` in Vercel Environment Variables

Step 4 — Redeploy Vercel:

After updating environment variables, trigger a redeploy from the Vercel Dashboard → Deployments → Redeploy, or push an empty commit:

```
git commit --allow-empty -m "chore: trigger redeploy for stripe live keys" git push
```

7.4 Resend Production Setup

Step 1 — Verify your sending domain:

1. Resend Dashboard → Domains → Add Domain
2. Enter your domain (e.g., `yourdomain.com`)
3. Resend provides DNS records (MX, TXT, DKIM) to add to your DNS provider
4. Add all records and click Verify. Verification typically takes 5–30 minutes.

Until your domain is verified, Resend will only send to email addresses you have verified under Email Addresses in the Resend dashboard.

Step 2 — Update the `from` address in `NotificationService`:

```
// src/lib/services/NotificationService.ts from: 'BarberBook <noreply@yourdomain.com>',
```

Replace `yourdomain.com` with your verified sending domain.

Step 3 — Get your production API key:

Resend Dashboard → API Keys → Create API Key (name it barberbook-production). Copy the re... key and update RESENDAPI_KEY in Vercel Environment Variables.

Step 4 — Test a live send:

Complete a real booking on the production site (using a real card in live Stripe mode) and confirm the confirmation email arrives in your inbox. Check Resend Dashboard → Logs for delivery status.

7.5 Vercel Cron Setup

How it works:

`vercel.json` defines a cron schedule for the reminder job:

```
{ "crons": [ { "path": "/api/cron/send-reminders", "schedule": "0 10 * * *" } ] }
```

This fires the `GET /api/cron/send-reminders` route every day at 10:00 AM UTC. The route is authenticated via the `INTERNALCRONSECRET` header.

Authentication check in the route:

```
// app/api/cron/send-reminders/route.ts export async function GET(request: Request) { const
authHeader = request.headers.get('authorization'); if (authHeader !== `Bearer
${process.env.INTERNAL_CRON_SECRET}`) { return new Response('Unauthorized', { status: 401 }); }
// ... reminder logic }
```

Vercel automatically sets the Authorization: Bearer <CRONSECRET> header when invoking cron routes, matching the INTERNALCRON_SECRET environment variable.

Monitoring:

Vercel Dashboard → Project → Logs → filter by `/api/cron/send-reminders`. Each daily invocation logs the number of reminders dispatched.

Triggering manually (testing production):

```
curl -X GET https://yourdomain.com/api/cron/send-reminders \ -H "Authorization: Bearer
<your-INTERNAL_CRON_SECRET>"
```

Cron in local development:

Vercel Cron does not run in `pnpm dev`. To test the reminder logic locally, trigger it manually:

```
curl -X GET http://localhost:3000/api/cron/send-reminders \ -H "Authorization: Bearer
<your-INTERNAL_CRON_SECRET>"
```

7.6 Custom Domain Setup

Step 1 — Add domain in Vercel:

1. Vercel Dashboard → Project → Settings → Domains
2. Click Add Domain
3. Enter your domain (e.g., `book.yourbarbershop.com`)

Step 2 — Configure DNS:

Vercel provides DNS records to add at your domain registrar:

Record Type	Name	Value
`A`	`@` (or `book`)	`76.76.21.21`
`CNAME`	`www` (or subdomain)	`cname.vercel-dns.com`

For a subdomain (`book.yourbarbershop.com`), add only the `CNAME` record pointing to `cname.vercel-dns.com`.

Step 3 — Wait for DNS propagation:

Propagation typically takes 5–30 minutes. Vercel automatically provisions an SSL certificate (Let's Encrypt) once it verifies DNS.

Step 4 — Update `NEXTPUBLICAPP_URL`:

In Vercel Environment Variables, update `NEXTPUBLICAPP_URL` to `https://book.yourbarbershop.com`.

Redeploy for the change to take effect. Verify that email confirmation links and Stripe redirect URLs now use the new domain.

---

8. Architecture Decision Records (ADRs)

ADRs document significant technical decisions, the context around them, and why alternatives were rejected. They serve as institutional memory for the team.

ADR-001: Next.js App Router over Pages Router

Status: Accepted

Date: 2024-01-01

Context:

BarberBook requires server-side data fetching for SEO (the public booking page must be indexable), colocated API routes, and a streaming-friendly architecture for the calendar and availability views.

Decision:

Use Next.js 14 App Router (`app/` directory) rather than the legacy Pages Router (`pages/` directory).

Rationale:

- React Server Components allow data fetching at the component level with zero client-side JavaScript overhead for static content
- Route Groups (`(public)/`) allow clean URL organization without polluting URL structure
- Server Actions provide a type-safe alternative to manual API routes for simple form mutations
- The App Router is Next.js's current recommended architecture; the Pages Router is in maintenance mode
- Colocation of layouts, loading states, and error boundaries reduces boilerplate

Trade-offs:

The App Router has a steeper learning curve. The mental model of "server vs. client components" requires discipline. Caching behavior is more complex than Pages Router. These costs are acceptable given the long-term architectural benefits.

ADR-002: Supabase over Custom PostgreSQL + Auth

Status: Accepted

Date: 2024-01-01

Context:

The project needs a relational database, row-level access control, authentication, and file storage. Options considered: self-hosted PostgreSQL on a VPS, Neon serverless Postgres + custom auth, and Supabase.

Decision:

Use Supabase as a fully managed backend-as-a-service providing Postgres, GoTrue auth, and Storage.

Rationale:

- Supabase provides all three required services (DB, Auth, Storage) under a single subscription and unified SDK
- Row Level Security is a first-class feature in Supabase; the dashboard makes policies visible and auditable
- The Supabase CLI enables local development with a Dockerized stack that mirrors production exactly
- Auto-generated TypeScript types (`supabase gen types`) eliminate a significant category of runtime type errors
- Generous free tier supports MVP launch with zero infrastructure cost

Trade-offs:

Supabase introduces a vendor dependency. If pricing changes significantly at scale, migrating to self-hosted Postgres requires extracting auth logic. For the MVP, this risk is acceptable.

ADR-003: Grouped Services Instead of Itemized Service Catalog

Status: Accepted

Date: 2024-01-01

Context:

Some booking platforms model services as a deep catalog (category → subcategory → add-ons → pricing tiers). BarberBook could have followed this pattern.

Decision:

Services are flat, first-class records: each service has a name, duration, price, and deposit. There are no categories, subcategories, or add-ons in the MVP.

Rationale:

- A single barber typically offers 4–8 distinct services; a catalog hierarchy adds complexity with no user-facing benefit at this scale
- Flat services simplify the booking UI: clients see a single list, tap a card, and proceed — no nested navigation
- The service schema is simple enough that adding categories or add-ons as a post-MVP enhancement is straightforward (add a `category` column and a `service_addons` join table)
- Reduces development time and testing surface area for the MVP

Trade-offs:

Power users who want upsell add-ons (e.g., "Hot Towel Shave + Beard Trim") cannot model them in the current schema. This is an accepted MVP limitation.

ADR-004: No Customer Accounts

Status: Accepted

Date: 2024-01-01

Context:

Many booking platforms require clients to create accounts before booking. This increases security and enables features like booking history and loyalty programs.

Decision:

Clients book as guests. They provide their name and email. No password or account is created. Booking management (cancellation) is handled via signed tokens in emails.

Rationale:

- Account creation friction reduces conversion. For a one-barber shop, the client experience must be frictionless
- The barber's client base is typically repeat customers who already have a personal relationship; a formal account system is unnecessary at MVP scale
- Guest bookings with email-based token cancellation are simpler to build, test, and maintain
- GDPR/privacy compliance is simpler without persistent customer accounts

Trade-offs:

Clients cannot view their booking history without calling the barber. Repeat clients must re-enter their details on each visit. Loyalty programs are not possible without accounts. These are accepted MVP trade-offs; see [Section 9](#9-post-mvp-upgrade-notes) for the customer account upgrade path.

ADR-005: Resend over SendGrid for MVP

Status: Accepted

Date: 2024-01-01

Context:

Transactional email providers considered: SendGrid, Mailgun, Postmark, and Resend.

Decision:

Use Resend with React Email templates.

Rationale:

- Resend's API is designed for developers: a simple REST API with a JavaScript/TypeScript SDK
- React Email allows building and previewing email templates as React components, eliminating context-switching between HTML email markup and application code
- The Resend free tier (3,000 emails/month) is sufficient for MVP launch and initial growth
- Email templates colocate with application code and receive TypeScript type-checking like any other component
- Resend has superior deliverability out of the box compared to the free tiers of legacy providers

Trade-offs:

Resend is a newer provider than SendGrid or Mailgun. For mission-critical email at enterprise scale, a more established provider might be preferred. For the MVP, Resend's simplicity outweighs the maturity advantage of legacy providers.

ADR-006: Vercel Cron over a Separate Queue Worker for Reminders

Status: Accepted

Date: 2024-01-01

Context:

Appointment reminders must be sent at scheduled intervals (24 hours and 2 hours before appointments). Options considered: BullMQ + Redis on a separate server, a dedicated cron server (Railway/Fly.io), and Vercel Cron.

Decision:

Use Vercel Cron to trigger a daily HTTP call to an API route that dispatches all pending reminders.

Rationale:

- Vercel Cron requires no additional infrastructure or services. It is configured in `vercel.json` with a single cron entry
- For a single-barber operation, the total daily email volume is low (typically < 20 reminder emails/day). A full queue worker is over-engineered for this scale
- The daily cron approach is transparent and debuggable: each run logs its output, which is visible in Vercel's function logs
- The same cron route can be triggered manually via `curl` for testing and backfill scenarios

Trade-offs:

Vercel Cron has minute-level granularity (not second-level) and cannot fire more than once per minute. If reminder timing precision is critical (e.g., sending at exactly T-2h), a queue-based approach would be more reliable. For the MVP, reminders are batched once daily at 10 AM, querying for all appointments in the next 26 hours, which mitigates precision concerns.

---

9. Post-MVP Upgrade Notes

These notes explain what specific changes would be required in the codebase to implement each post-MVP feature. They are intended to help future engineers plan the work accurately.

Adding SMS (Twilio)

Why: Some clients prefer SMS reminders over email. SMS has significantly higher open rates.

Changes required:

1. Install Twilio SDK:

```
pnpm add twilio
```

2. Add environment variables:

```
TWILIO_ACCOUNT_SID=ACxxxxxxxxx TWILIO_AUTH_TOKEN=xxxxxxxxx TWILIO_PHONE_NUMBER=+15550001234
```

3. Add `phone` column to bookings:

```
ALTER TABLE bookings ADD COLUMN client_phone TEXT;
```

4. Update `NotificationService`:

Add a `sendSms()` method and a `channel` parameter to all `send*()` methods:

```
async sendConfirmation(booking: Booking, channel: 'email' | 'sms' | 'both' = 'email')
```

5. Add channel routing logic:

Respect the client's preferred contact method. Store `notification_preference` on the booking (`email` | `sms` | `both`).

6. Update booking form:

Add an optional phone number field and a notification preference selector.

7. Update E2E tests:

Mock the Twilio client in tests using `vi.mock('twilio')`.

Adding Multi-Barber Admin

Why: The platform could expand to serve barbershops with multiple staff members.

Changes required:

1. Schema changes:

- Add a `barbershop` table (the top-level organization)
- Add a `role` column to `barbers` (`owner` | `barber`)
- Scope all availabilityrules, services, and bookings to a barberid
- Update all RLS policies to check `barber_id = auth.uid()` instead of a single barber assumption

2. Stripe Connect:

Each barber (or the barbershop owner) needs a Stripe Connect account to split revenue. This requires implementing OAuth with Stripe Connect and storing `stripeaccountid` on the `barbers` table.

3. URL routing:

Public booking URLs must include a barber identifier: `/book/[barberSlug]` or `/[shopSlug]/[barberSlug]`.

4. Admin dashboard:

- Add a shop-level admin view for the owner (sees all barbers' bookings)
- Add a barber-level view (sees only their own bookings)
- Implement an invitation flow for the owner to add new barbers

5. RLS changes:

The most complex change. Every table policy must be audited to ensure barbers can only access their own data, while shop owners can access all data within their shop.

Adding Customer Accounts

Why: Enables booking history, loyalty features, and reduced friction for repeat clients.

Changes required:

1. New auth flow:

- Add a client-facing login/signup page
- Use Supabase's magic link or password auth for customers
- Separate the `customers` role from the `barbers` role using Supabase custom claims

2. Schema changes:

```
CREATE TABLE customers ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID REFERENCES auth.users(id), name TEXT NOT NULL, email TEXT UNIQUE NOT NULL, phone TEXT, created_at TIMESTAMPTZ DEFAULT now() ); ALTER TABLE bookings ADD COLUMN customer_id UUID REFERENCES customers(id);
```

3. Booking flow update:

If a session exists, pre-fill the booking form with the customer's details and link the booking to their account.

4. Customer portal:

Add `/account/bookings` page showing booking history and allowing self-service cancellation without requiring the token from email.

5. RLS updates:

Add policies allowing `customers` to read and cancel their own bookings.

Adding Analytics

Why: The barber needs insights into revenue trends, popular services, and peak booking times.

Changes required:

1. Extend the Reporting Module:

Implement server-side aggregation queries in `src/lib/services/ReportingService.ts`:

```
getRevenueByPeriod(startDate: Date, endDate: Date): Promise<RevenueReport>
getBookingsByService(): Promise<ServiceBreakdown[]> getPeakHours(): Promise<HourlyDistribution[]>
```

2. Add reporting dashboard pages:

- `/barber/reports/revenue` — monthly revenue chart
- `/barber/reports/services` — service popularity pie chart
- `/barber/reports/traffic` — bookings-per-day heatmap

3. Consider a dedicated analytics table:

For high-frequency events, add an analyticsevents table with indexed eventtype and created_at columns rather than deriving all metrics from the bookings table at query time.

4. Optional: third-party analytics:
Integrate Posthog or Plausible for client-side funnel analytics (booking drop-off rates, step completion). Add the tracking snippet to `app/layout.tsx`.

Self-Serve Barber Registration

Why: Removes the need for manual admin setup when onboarding new barbers.

Changes required:

1. New public registration route:
Add `/register` with a multi-step form:

- Step 1: Business details (name, slug, bio, phone)

- Step 2: Account credentials (email, password)

- Step 3: Stripe Connect onboarding (redirect to Stripe's hosted onboarding)

- Step 4: Availability setup

2. Slug uniqueness check:
Add a `UNIQUE` constraint on `barbers.slug` and an API route to check availability in real-time as the barber types.

3. Email verification:
Trigger Supabase's built-in email verification before activating the barber account. Set `REDIRECT_TO` to `/register/verify`.

4. Admin approval gate (optional):
Add a `status` column to `barbers` (`pending` | `active` | `suspended`). Require manual approval by a platform admin before the barber's booking page goes live.

---

10. Troubleshooting Guide

Supabase RLS Blocking Queries Unexpectedly

Symptom: API route returns an empty array or a 400/401 error even though rows exist in the database. The Supabase client returns `{ data: [], error: null }` when you expect results.

Diagnosis:

1. Open Supabase Studio → SQL Editor and run the query directly as the superuser. If results appear, RLS is the issue.
2. Check which RLS policies apply to the affected table: Dashboard → Database → Tables → [table] → Policies.
3. Verify the Supabase client being used in the failing route. API routes that should access all data must use `supabase/server.ts` (service role), not `supabase/client.ts` (anon key).

Common causes and fixes:

Cause	Fix
Using anon client in an API route that needs unrestricted access	Import from `src/lib/supabase/server.ts` instead
RLS policy references `auth.uid()` but no session exists	Ensure the user is authenticated before calling the protected route
Policy uses `USING` clause that excludes the current row's state	Review the policy logic; use `supabase.rpc()` with a security definer function if needed
New table created without an explicit RLS policy	Add a policy, or run `ALTER TABLE your_table ENABLE ROW LEVEL SECURITY` and add the appropriate policy

---

Stripe Webhook Signature Verification Failing Locally

Symptom: `POST /api/payments/webhook` returns `400 Bad Request`. The server logs show `StripeSignatureVerificationError: No signatures found matching the expected signature for payload`.

Diagnosis and fixes:

1. Wrong `STRIPEWEBHOOKSECRET`: The `whsec_` value printed by `stripe listen` changes every time you restart the CLI. Copy the new value into `.env.local` and restart `pnpm dev`.
2. Body parser consuming the raw body: Stripe signature verification requires the raw request body as a `Buffer` or `string`. In Next.js App Router, read it with:

```
const rawBody = await request.text(); stripe.webhooks.constructEvent(rawBody, sig,
process.env.STRIPE_WEBHOOK_SECRET!);
```

Never pass a parsed JSON object to `constructEvent`.

3. Stripe CLI not running: Confirm the `stripe listen` process is active in a terminal. Events sent by Stripe do not reach `localhost` without it.
4. Port mismatch: Confirm the Stripe CLI is forwarding to `localhost:3000`, not `localhost:3001` or another port.

Reminder Cron Not Firing in Development

Symptom: Appointment reminders are not sent during local development.

Explanation:

Vercel Cron only runs on Vercel's infrastructure. It does not execute during `pnpm dev`.

Fix:

Trigger the cron endpoint manually during local testing:

```
curl -X GET http://localhost:3000/api/cron/send-reminders \ -H "Authorization: Bearer $(grep
INTERNAL_CRON_SECRET .env.local | cut -d '=' -f2)"
```

Check Inbucket at http://localhost:54324 for reminder emails.

If the endpoint returns `401 Unauthorized`, confirm that `INTERNALCRONSECRET` in `.env.local` matches the value passed in the `Authorization` header.

Booking Slot Not Appearing (Availability Edge Cases)

Symptom: A time slot that should be available does not appear in the slot picker, or an already-booked slot still appears.

Diagnosis steps:

1. Check availabilityrules: In Supabase Studio, confirm a rule exists for the day of week you are testing. Rules use dayof_week integers (0 = Sunday, 6 = Saturday).
2. Check blockedtimes: Confirm no blockedtimes record overlaps the expected slot.
3. Timezone issues: `AvailabilityService` does all calculations in UTC. If the barber's timezone is not UTC, confirm that `availability_rules` times are stored as UTC equivalents, and that `dateUtils.toUTC()` is called correctly when interpreting the rules.
4. Off-by-one in slot generation: Slots are generated from starttime to endtime - service.durationminutes. A service that is 60 minutes long cannot start at 17:30 if the barber's availability ends at 18:00 — only at 17:00 or earlier. Verify the service durationminutes is correct.
5. Stale cache: Next.js may cache the availability API response. In development, force a fresh fetch:

```
fetch('/api/availability?...', { cache: 'no-store' })
```

Auth Redirect Loop on `/barber/dashboard`

Symptom: Navigating to `/barber/dashboard` continuously redirects back to `/barber/login`, even after entering valid credentials.

Diagnosis and fixes:

1. Session not persisting: Confirm `supabase/middleware.ts` is configured correctly in `middleware.ts` at the project root. The middleware must call `supabase.auth.getSession()` and set the refreshed session cookies on every request to `barber/*` routes.
2. Cookie domain mismatch: On localhost, ensure cookies are set for `localhost` not `127.0.0.1`. Use `http://localhost:3000` consistently in your browser (not `http://127.0.0.1:3000`).
3. Missing NEXTPUBLICSUPABASEURL or NEXTPUBLICSUPABASEANON_KEY: If these are undefined, the Supabase client cannot validate the session. Check the browser console for supabase: missing url errors.
4. `barber/layout.tsx` redirect logic: Review the auth check:

```
const { data: { session } } = await supabase.auth.getSession(); if (!session)
redirect('/barber/login');
```

Add a `console.log(session)` temporarily to confirm the session object is populated after login.

5. Supabase local stack not running: If `supabase start` is not running, auth requests fail silently and the session check always returns null.

Supabase Storage Image Not Loading

Symptom: Barber profile photos uploaded to Supabase Storage return a 404 or 403 error when displayed on the booking page.

Diagnosis and fixes:

1. Bucket is private, not public: In Supabase Dashboard → Storage, check the `barber-photos` bucket. If it shows a lock icon, it is private. Either set it to public or generate a signed URL for the image.

- To make it public: Dashboard → Storage → [bucket] → Make Public

- Or update the storage policy (see [Section 7.2](#72-supabase-production-setup))

2. Wrong storage URL format:

The public URL for a Supabase Storage file is:

```
https://<project-ref>.supabase.co/storage/v1/object/public/barber-photos/<filename>
```

Confirm the URL stored in the `barbers.photo_url` column matches this format exactly.

3. Local development — using production URL:

If the app is running locally but `NEXTPUBLICSUPABASE_URL` points to production, local uploads go to production Storage. Use the local Supabase URL (`http://localhost:54321`) consistently in local development.

4. `next/image` domain not configured:

If using `<Image>` from `next/image`, the Supabase storage domain must be added to `next.config.ts`:

```
images: { remotePatterns: [ { protocol: 'https', hostname: '*.supabase.co', pathname:
'/storage/v1/object/public/**', }, ], },
```

5. CORS error: If images load in the Supabase dashboard but not in the browser, check for CORS errors in the browser console. In Supabase Dashboard → Storage → Policies, ensure the public read policy is active.

End of Phase 9 Developer Documentation — BarberBook MVP

Document version 1.0.0 — March 30, 2026